FarmVendor

A Role- Based Digital Platform for Farmer-Vendor Coordination

with Demand Forecasting and vendor optimization

Student Name : Maneesha Eeshwara

Student ID : 300392759

Course : CSIS 4495 Applied Research Project

Section : 03

GitHub Repository : https://github.com/maneeshaprs-rgb/W26_4495_S3_ManeeshaE.git

Contents

1. Introduction

1.1 Background and Domain Context

Local and small-scale agricultural supply chains continue to struggle with challenges such as demand uncertainty, inventory mismatch, food waste, overproduction, under-supply, communication gap between producer and vendor. These challenges are basically visible in British Columbia as many farmers and buyers still depend on informal communication like phone calls, text messages or word of mouth to coordinate with their business partners. This lack of communication may result in delays, inefficient dispatch planning, and poor visibility of demand and supply status.

With increasing interest in digital transformation, there is a clear opportunity to develop a lightweight web-based platform that supports farmers and vendors through structured workflows, shared information, and decision-support features. Many currently available systems are either too enterprise-focused, too expensive, or not designed for the needs of small and medium-scale agricultural stakeholders. This project addresses that gap by designing and implementing FarmVendor, a role-based digital platform that supports farmer–vendor coordination, demand forecasting, vendor recommendation, and real-time messaging.

1.2 Problem Definition

This research addresses the lack of a structured, secure, and intelligent coordination platform for small agricultural stakeholders. The problem includes the following limitations:

- farmers do not have a clear way to view vendor demand in one place
- vendors do not have a structured method to submit requests and track deliveries
- dispatch planning is not supported by demand forecasting
- decision-making for farmers lacks data-driven vendor recommendations
- communication between farmers and vendors is often informal and fragmented

1.3 Research Questions

Following research questions are supposed to addressed by the project:

- How can a role-based digital platform improve coordination between farmers and vendors?
- How can simulated and user-generated data support early-stage demand forecasting?
- How can machine-learning-based forecasting improve dispatch planning?
- How can farmers be supported with vendor recommendations based on predicted demand and available inventory?
- Can a real-time chat feature improve coordination and usability within the supply chain workflow?

1.4 Literature Overview and Research Gap

Literature Overview

Research in agriculture supply chain management highlights the importance of demand forecasting, inventory visibility, and coordination between farmers and vendors. Prior studies show that even basic forecasting techniques prioritised with time and structured communication should reduce food waste, improve supply planning and increase efficiency in agricultural operations. Digital platforms are increasingly used to support these goals by enabling data sharing, record keeping and making decisions on data.

There are several farm management tools and other platforms that partially support those challenges. Here is background research and limitations on existing applications.

- **AgriWebb**: Farm management mobile application that focus on helping farmers manage production records, operational planning and inventory. offer limited direct coordination with vendors such as grocery stores and restaurants
- **FarmLogs**: largely farmer-centric mobile application with good internal visibility. limited vendor supportive.
- **CropIn**: web based Agri-intelligence and AI driven platform for enterprises across the agriculture value chain. Designed only for enterprise level Agri businesses
- **AgriDigital**: Provide cloud-based grain supply chain and management platform. Use blockchain technology to secure settlement while digitize grain storage,

trading, logistics and financing. Less suitable for small and medium scale farmers where developed models need extensive historical data and long-term adoption.

A key limitation across both academic literature and existing mobile applications and platforms is the high dependability on large historic datasets. While many optimization and forecasting models assume the availability of long-term, real world transactional data newly deployed systems and small – scale agricultural networks face a cold-start problem, where little or no historical data exists.

Most of systems ignore human decision making while fully automated recommendations prioritized where agriculture grow with trust, long-term relationships and farmer experience is vital in decision making and planning.

Research Gap

Below gap remains based on existing research and platforms,

- Limited availability of lightweight, role-based platforms designed specifically for local farmers and vendors.
- Insufficient support for incremental data collection, where forecasting improves gradually over time.
- Lack of optimization models that combines distance, requested quantity and historical relationships.
- Minimal focus on human-in-the-loop decision support rather than full automation.
- Overdependence on real-world, large-scale datasets that are impractical for early-stage systems.

1.5 Hypotheses and Expected Benefits

The project is based on the following assumptions:

- structured demand communication reduces uncertainty for farmers
- forecasting can improve dispatch planning
- recommendation logic can support better vendor selection
- real-time communication can reduce coordination friction

- simulated and user-generated data can support meaningful early-stage research and evaluation

Expected benefits include:

- better visibility of demand and supply
- improved dispatch planning
- reduced communication barriers
- better farmer decision support

2. Summary Of Research Project

The FarmVendor project is a full-stack, role-based web application designed to enhance coordination between farmers and vendors by integrating demand management, forecasting, and intelligent decision support into a unified platform. The system enables farmers to manage inventory, monitor product availability, and make dispatch decisions, while vendors can submit demand requests specifying required quantities and delivery timelines.

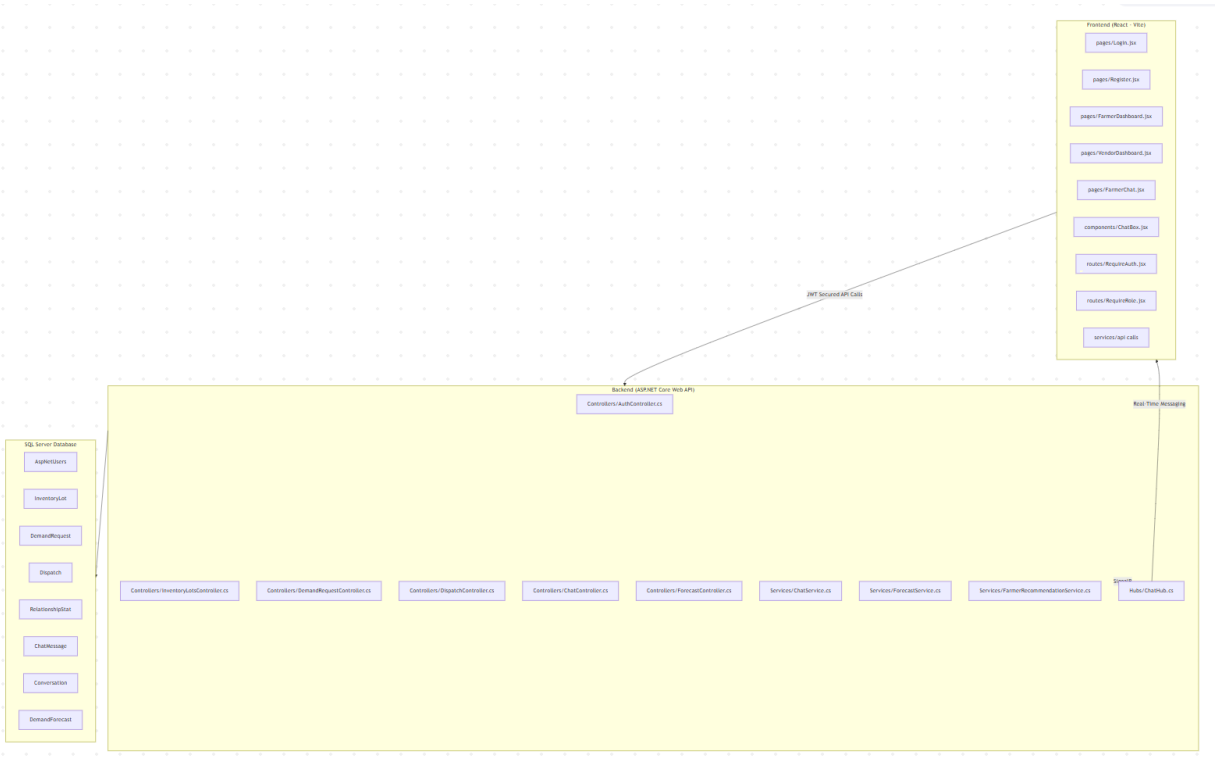
A central feature of the project is the integration of machine learning–based demand forecasting using the ML.NET Singular Spectrum Analysis (SSA) algorithm. This forecasting component analyzes historical demand request data to generate future demand predictions for specific vendor–product combinations. These predictions support proactive planning by helping farmers anticipate demand, reduce produce waste, and optimize inventory and dispatch strategies.

Building on the forecasting capability, the system incorporates an intelligent vendor recommendation module that identifies the most suitable vendors for dispatching. This module evaluates multiple factors, including forecasted demand, available farmer inventory, geographical distance between farmers and vendors, and historical relationship scores derived from past interactions. A weighted scoring mechanism is applied to rank vendors, allowing farmers to make informed, data-driven decisions when selecting vendors for fulfillment.

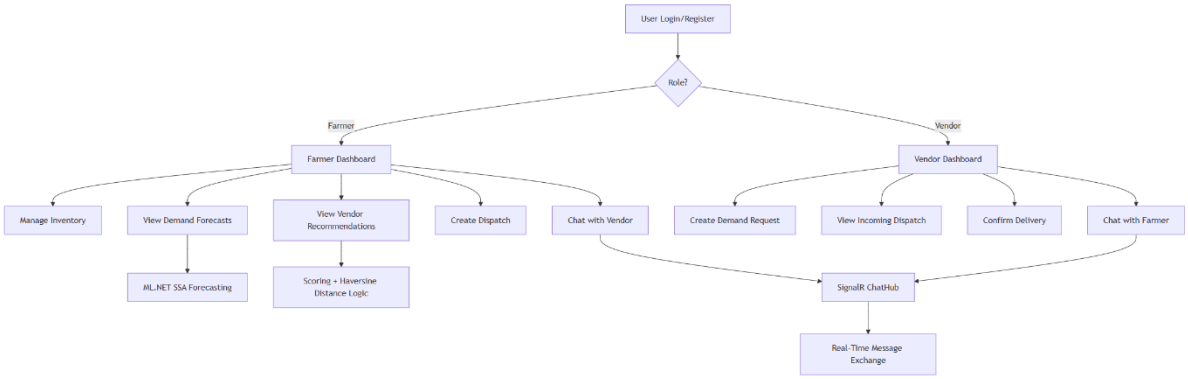
To enhance collaboration and operational efficiency, the platform also includes a real-time communication module implemented using SignalR**,** enabling direct and instant messaging between farmers and vendors. This feature supports timely coordination, negotiation, and clarification during the dispatch process.

The application is developed using a modern and scalable technology stack, with React (Vite) for the frontend**,** ASP.NET Core (.NET 8) for backend services, and SQL Server for data persistence. The system follows a layered architectural approach, clearly separating the presentation layer, business logic, and data access components to improve modularity, maintainability, and scalability.

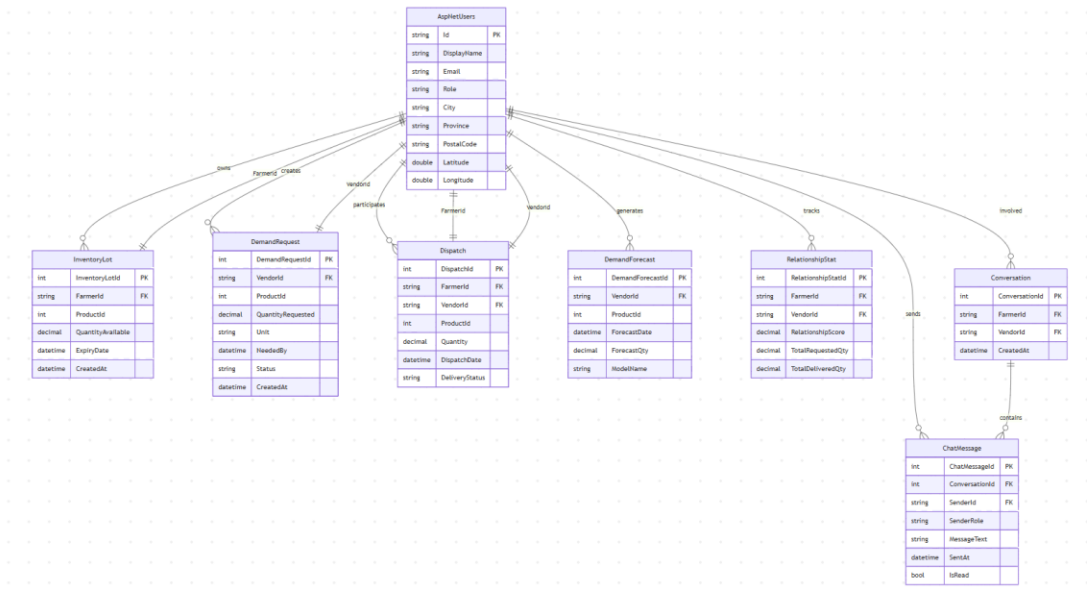
Overall, the FarmVendor project transforms a traditional transactional system into an intelligent, data-driven agricultural supply chain platform by integrating forecasting, recommendation algorithms, and real-time communication. The project demonstrates how emerging technologies can be applied to improve efficiency, reduce uncertainty, and support informed decision-making in agricultural supply chain management.



Architecture Diagram



Database Structure



3. Changes to the Proposal

During the implementation phase of the FarmVendor project, several modifications were introduced to enhance system functionality, usability, and alignment with real-world agricultural requirements. These changes were informed by iterative testing, technical feasibility analysis, and the need to support intelligent decision-making for non-technical users. Overall, the refinements strengthened the original proposal by improving scalability, interpretability, and practical applicability.

Change Area	Description	Justification
Forecast Model	The initially proposed simple logic-based demand estimation and Moving Average algorithm was replaced with a machine learning-based forecasting model using ML.NET Singular Spectrum Analysis (SSA).	The SSA model provides more accurate, data-driven, and scalable demand forecasts by analyzing historical time-series data, enabling better production planning and dispatch decisions.
Forecasting Integration (New Addition)	ML-based demand forecasting was fully embedded into the operational workflow and used as a core input for vendor recommendations.	Integrating forecasting directly into decision support improved system intelligence and ensured that recommendations were driven by future demand rather than static historical data.
Dashboard Architecture	The dashboard was upgraded from a static UI design to a fully dynamic, API-driven architecture supporting real-time data updates.	This change improved system responsiveness and enabled real-time visibility of inventory, requests, forecasts, and recommendations, enhancing user decision-making.
Authentication and Security	Role-based authentication was implemented using ASP.NET Identity with JWT-based authorization.	Secure role separation was required to protect sensitive data and enforce access control between farmer and vendor functionalities.

Recommendation Logic	The original optimization-based approach was simplified into a weighted scoring-based recommendation system.	The scoring approach improved interpretability and usability, making recommendation logic easier for non-technical agricultural users to understand and trust by prioritizing demand match, distance and relationship score which calculated score by weighted formula
Relationship Tracking Enhancement	A new relationship-tracking entity was introduced to record historical farmer–vendor interactions and compute a relationship score.	Including relationship history improved recommendation accuracy by accounting for prior collaboration performance and trust which mainly focused on Total Requested Quantity, Total Delivered Quantity, Relationship Score and finally calculating relationship Stats using EF Core aggregation queries
Location Feature	Latitude and longitude fields were added to user profiles to support geospatial calculations.	Geolocation data is required to compute distances between farmers and vendors, which is essential for distance-based ranking and logistics optimization. Here I used Haversine formula with fields Longitude and Latitude
Data Generation Strategy	The initial simulated dataset approach was enhanced to include multi-period historical demand, inventory, and dispatch data.	More realistic historical data was necessary to support reliable machine learning forecasting and to reflect real-world agricultural scenarios. Here I used SQL scripts to generate multi-week historical data. Data seeded into DemandRequest, InventoryLot, Dispatch tables
User Experience Improvements	Searchable dropdowns, improved navigation through a shared sidebar layout, hover-based details, and enhanced input validation were introduced.	These changes significantly improved usability, reduced user errors, and enhanced overall interaction efficiency for both farmer and vendor roles.

Chat Module	A real-time communication module was added using SignalR.	Real-time messaging was required to support timely coordination, negotiations, and clarification during dispatch and fulfillment processes. Here Implementation use SignalR Hub with WebSocket transport. Client uses @microsoft/signalr to establish connection
Image Integration	Product images were integrated using the Unsplash API to dynamically display images in dropdowns and dashboards.	This enhancement improves visual clarity, user engagement, and usability. It eliminates the need for manual image uploads while providing high-quality, scalable visual content. Integrated via Unsplash REST API

4. Project Completion Timeline

Activity	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7	Week 8	Week 9	Week 10	Week 11	Week 12	Week 13
Requirements & Literature	■	■	■										
System Design & Architecture		■	■	■									
Backend Development		■	■	■	■	■	■	■	■	■	■		
API Development & Integration		■	■	■	■	■	■	■	■	■	■		
Milestone: progress report 1					■	■							
Frontend Integration		■	■	■	■	■	■	■	■	■	■		
Data Modeling & Simulation			■	■	■	■	■	■	■	■	■		
Milestone: Midterm Report and Implementation				■	■	■	■						
ML Model Training & Evaluation						■	■	■	■	■	■		
Dispatch module implementation							■	■					
Milestone: progress report 2							■	■					
Forecasting logic improvement								■	■	■	■		
Integration testing								■	■	■	■	■	
UI/UX polish								■	■	■	■	■	■
Milestone: progress report 3								■	■				
Bug fixing & performance tuning									■	■	■	■	
Chat Module									■	■	■	■	
Milestone: progress report 4									■	■			
Milestone: progress report 5										■	■		
Image Integration and UI polish											■	■	
Research analysis & evaluation											■	■	■
Evaluation & Documentation											■	■	■
Milestone: Final Report and Implementation												■	■

Description of timeline

The project was executed over a 13-week period, following a structured and incremental development approach that combined research, system design, implementation, testing, and evaluation.

The development process began with requirements gathering and literature review (Weeks 1–2), where the problem domain, research objectives, and system requirements were defined. This was followed by system design and architecture planning (Weeks 2–3), where the overall system structure, database schema, and API design were finalized.

The backend development and API integration phase (Weeks 2–10) formed the core of the system implementation. During this phase, key modules such as authentication, demand request management, inventory tracking, dispatch handling, and forecasting services were developed using ASP.NET Core and Entity Framework.

Parallel to backend work, frontend integration (Weeks 2–10) was carried out using React, enabling dynamic interaction with backend APIs and real-time data visualization.

The data modeling and simulation phase (Weeks 3–9) focused on generating realistic historical datasets for demand, inventory, and dispatch operations. This was essential to support machine learning forecasting.

The machine learning phase (Weeks 6–10) involved implementing and evaluating forecasting models. Initially, a Moving Average approach was used, which was later enhanced by integrating the ML.NET SSA model to improve prediction accuracy.

Several key milestones were achieved during the project:

- Progress Report 1 (Week 4) – Initial design and backend setup
- Midterm Report & Implementation (Week 5–7) – Core system functionality completed
- Progress Report 2 (Week 7–8) – Forecasting and dispatch modules
- Progress Report 3 (Week 9) – UI/UX and integration progress
- Progress Report 4 (Week 10) – Chat module and performance improvements
- Progress Report 5 (Week 11) – Final system enhancements

Advanced features such as the chat module using SignalR (Weeks 9–11) and image integration using the Unsplash API (Weeks 11–12) were implemented in later stages to enhance usability and system interaction.

The final stages of the project included:

- Integration testing (Weeks 9–12)
- UI/UX improvements (Weeks 9–13)
- Bug fixing and performance tuning (Weeks 9–12)
- Research analysis and evaluation (Weeks 11–13)
- Final documentation and report preparation (Weeks 12–13)

Milestones and Deliverables

Milestone	Deliverable
Progress Report 1	Requirements, system design, initial backend setup
Midterm Report	Functional backend + partial frontend
Progress Report 2	Dispatch + forecasting modules
Progress Report 3	UI improvements + integration
Progress Report 4	Chat module + performance optimization
Progress Report 5	Final system features completed
Final Report	Full system + evaluation + documentation

Team Responsibilities

This project was completed individually by **Maneesha Eeshwara**, who was responsible for all aspects of the system, including:

- Backend development using ASP.NET Core
- Database design and Entity Framework integration
- Machine learning model implementation (ML.NET SSA)
- Frontend development using React
- API integration and authentication (JWT)
- Real-time chat implementation using SignalR
- UI/UX design improvements
- Testing, debugging, and performance optimization
- Final report preparation and documentation

5. Implemented Feature

Feature 1: Demand Forecasting using ML.NET SSA

This is one of the most significant features of the final system. It predicts future vendor demand using historical demand data and supports proactive dispatch planning.

This feature includes:

Forecast generation using Moving Average

Forecast generation using ML.NET SSA

Forecast results table

Forecast line charts

Multi-vendor comparison chart

Filtered vendor/product selection

The design consists of:

- Auto-loaded chart view
- Product and vendor filters
- Forecast generation controls
- CSV export
- Top vendor comparison visualization

Implementation

Frontend

- FarmerForecast.jsx
- ForecastLineChart.jsx

- MultiVendorForecastLineChart.jsx
- forecastApi.js

Backend

- DemandForecastController.cs
- DemandForecastService.cs
- MLDemandForecastingEngine.cs
- filtered vendor/product API endpoints for the logged-in farmer

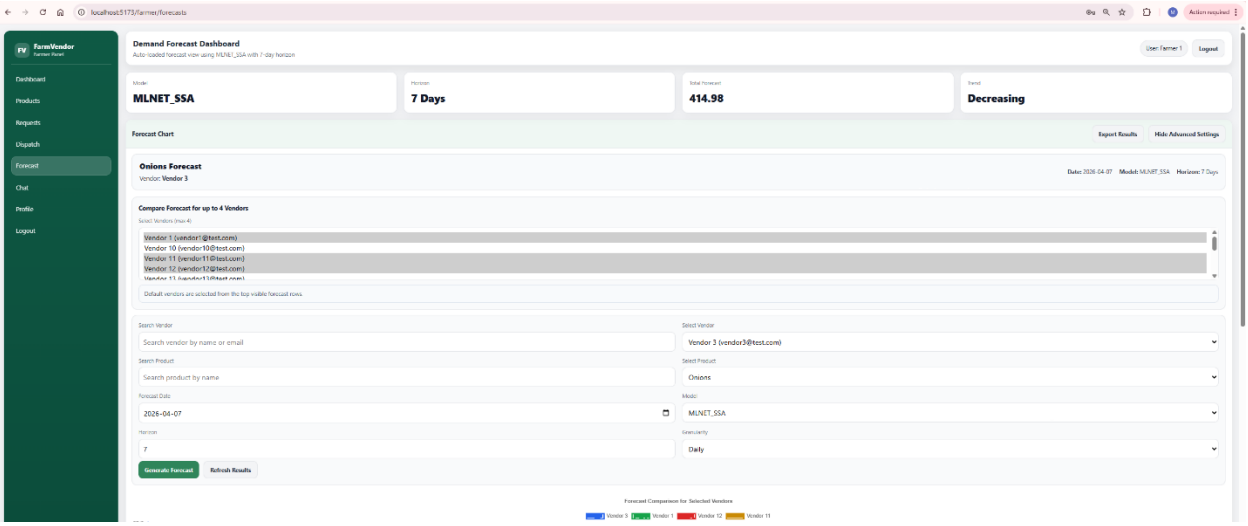
Database

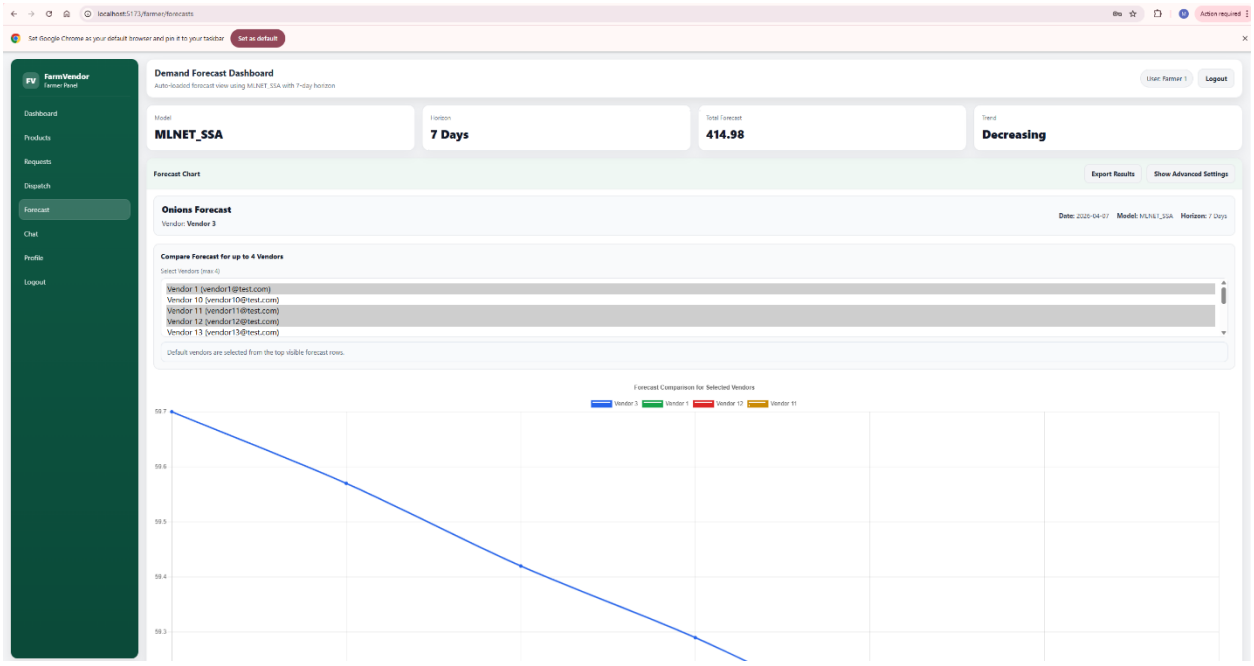
- DemandForecast table storing:
    - VendorId
    - ProductId
    - ForecastDate
    - ForecastQty
    - ModelName
    - LookbackPeriods
    - CreatedAt

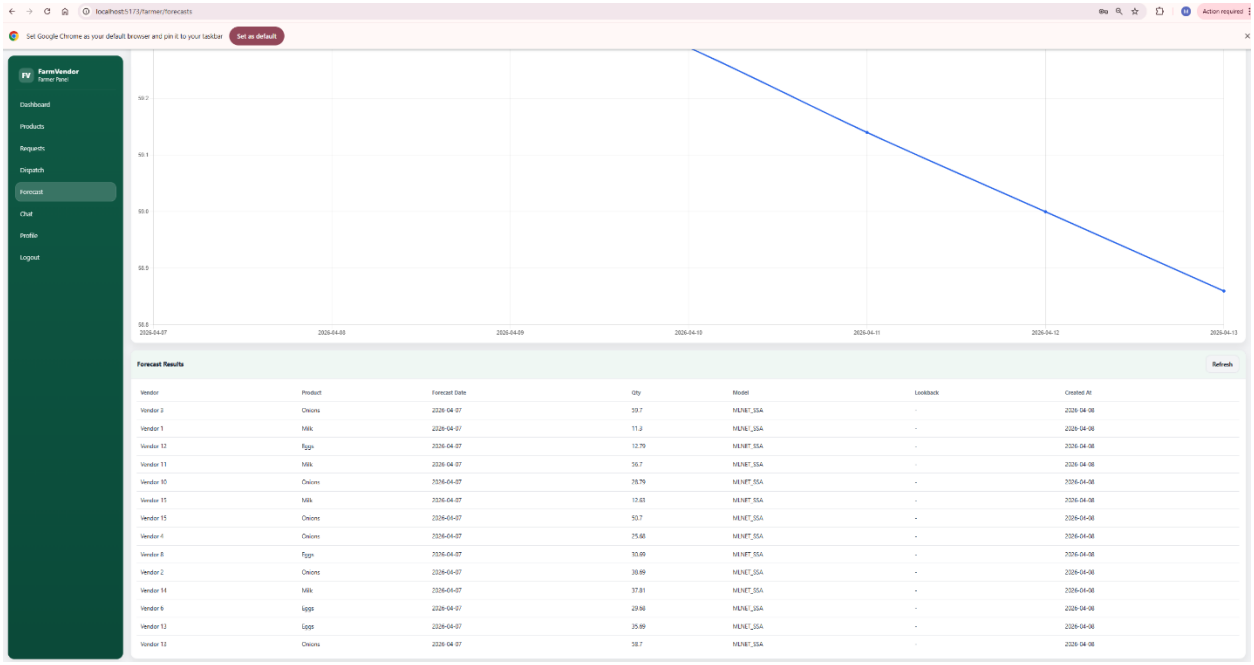
Technical Methods

- Moving Average based on recent demand history
- ML.NET SSA time-series forecasting
- historical demand grouped by VendorId + ProductId
- chart comparison using shared time axis

App

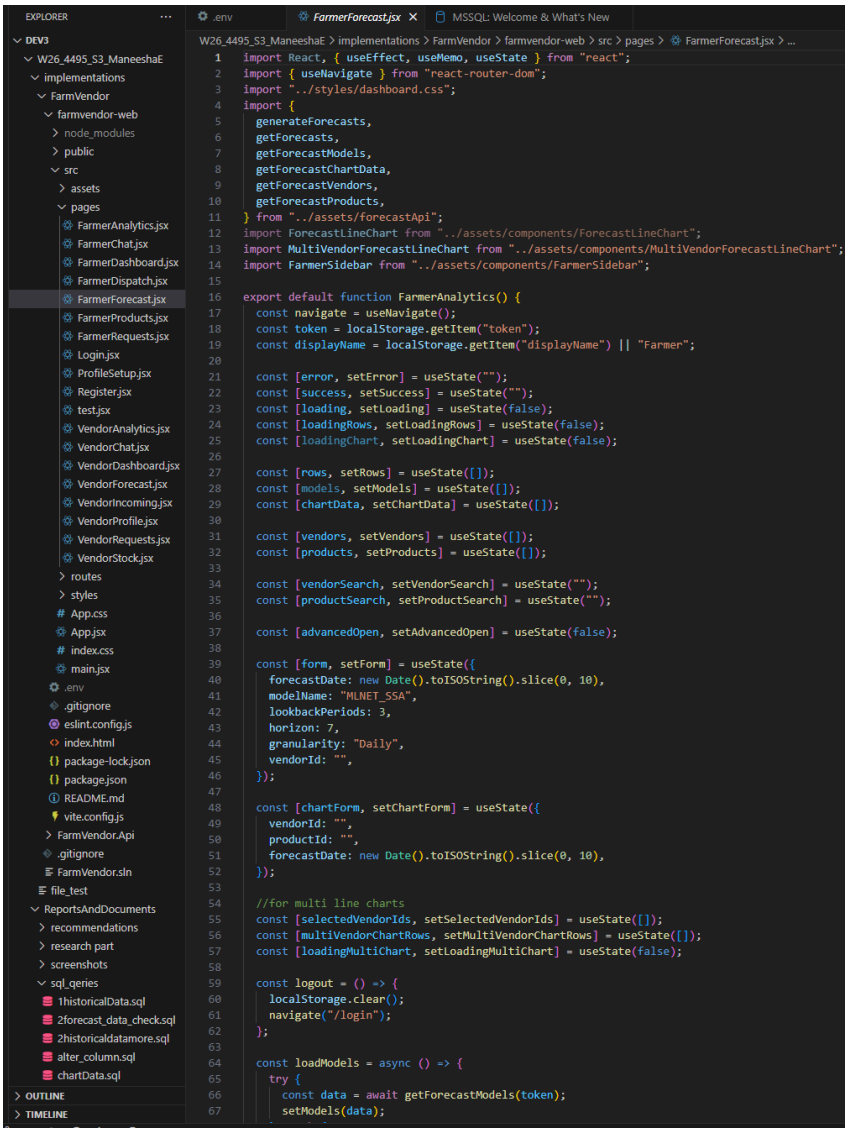

FarmVendor
Farmer Panel
Dashboard
Products
Requests
Dispatch
Forecast
Chat
Profile
Logout
Demand Forecast Dashboard
Auto-loaded forecast view using MLNET_SSA with 7-day horizon
User: Farmer 1
Logout
Model
MLNET_SSA
Horizon
7 Days
Total Forecast
414.98
Trend
Decreasing
Forecast Chart
Export Results
Hide Advanced Settings
Onions Forecast
Vendor: Vendor 3
Date: 2026-04-07
Model: MLNET_SSA
Horizon: 7 Days
Compare Forecast for up to 4 Vendors
Select Vendors (max 4)
Vendor 1 (vendor1@test.com)
Vendor 10 (vendor10@test.com)
Vendor 11 (vendor11@test.com)
Vendor 12 (vendor12@test.com)
Default vendors are selected from the top visible forecast rows.
Search Vendor
Search vendor by name or email
Select Vendor
Vendor 3 (vendor3@test.com)
Search Product
Search product by name
Select Product
Onions
Forecast Date
2026-04-07
Model
MLNET_SSA
Horizon
7
Granularity
Daily
Generate Forecast
Refresh Results
Forecast Comparison for Selected Vendors
Vendor 3
Vendor 1
Vendor 12
Vendor 11


Set Google Chrome as your default browser and pin it to your taskbar
Set as default
FarmVendor
Farmer Panel
Dashboard
Products
Requests
Dispatch
Forecast
Chat
Profile
Logout
Demand Forecast Dashboard
Auto-loaded forecast view using MLNET_SSA with 7-day horizon
User: Farmer 1
Logout
Model
MLNET_SSA
Horizon
7 Days
Total Forecast
414.98
Trend
Decreasing
Forecast Chart
Export Results
Show Advanced Settings
Onions Forecast
Vendor: Vendor 3
Date: 2026-04-07
Model: MLNET_SSA
Horizon: 7 Days
Compare Forecast for up to 4 Vendors
Select Vendors (max 4)
Vendor 1 (vendor1@test.com)
Vendor 10 (vendor10@test.com)
Vendor 11 (vendor11@test.com)
Vendor 12 (vendor12@test.com)
Vendor 13 (vendor13@test.com)
Default vendors are selected from the top visible forecast rows.
Forecast Comparison for Selected Vendors
Vendor 3
Vendor 1
Vendor 12
Vendor 11
59.7
59.6
59.5
59.4
59.3


localhost:5173/farmer/forecasts
Set Google Chrome as your default browser and pin it to your taskbar
Set as default
Action required
FarmVendor
Farmer Panel
Dashboard
Products
Requests
Dispatch
Forecast
Chat
Profile
Logout
59.2
59.1
59.0
58.9
58.8
2026-04-07
2026-04-08
2026-04-09
2026-04-10
2026-04-11
2026-04-12
2026-04-13
Forecast Results
Refresh
Vendor
Product
Forecast Date
Qty
Model
Lookback
Created At
Vendor 3 Onions 2026-04-07 59.7 MLNET_SSA - 2026-04-08
Vendor 1 Milk 2026-04-07 11.3 MLNET_SSA - 2026-04-08
Vendor 13 Eggs 2026-04-07 12.79 MLNET_SSA - 2026-04-08
Vendor 11 Milk 2026-04-07 56.7 MLNET_SSA - 2026-04-08
Vendor 10 Onions 2026-04-07 78.79 MLNET_SSA - 2026-04-08
Vendor 11 Milk 2026-04-07 13.63 MLNET_SSA - 2026-04-08
Vendor 15 Onions 2026-04-07 50.7 MLNET_SSA - 2026-04-08
Vendor 4 Onions 2026-04-07 25.68 MLNET_SSA - 2026-04-08
Vendor 8 Eggs 2026-04-07 10.89 MLNET_SSA - 2026-04-08
Vendor 2 Onions 2026-04-07 39.89 MLNET_SSA - 2026-04-08
Vendor 14 Milk 2026-04-07 37.81 MLNET_SSA - 2026-04-08
Vendor 6 Eggs 2026-04-07 29.86 MLNET_SSA - 2026-04-08
Vendor 13 Eggs 2026-04-07 25.89 MLNET_SSA - 2026-04-08
Vendor 13 Onions 2026-04-07 58.7 MLNET_SSA - 2026-04-08

Frontend

- *FarmerForecast.jsx*



FarmerForecast.jsx MSSQL: Welcome & What's New

4495_S3_ManeeshaE > implementations > FarmVendor > farmvendor-web > src > pages >

```
export default function FarmerAnalytics() {
  const logout = () => {
    navigate("/login");
  };

  const loadModels = async () => {
    try {
      const data = await getForecastModels(token);
      setModels(data);
    } catch {
      setModels([]);
    }
  };

  const loadVendors = async (search = "") => {
    try {
      const data = await getForecastVendors(token, search);
      setVendors(data);
    } catch {
      setVendors([]);
    }
  };

  const loadProducts = async (search = "") => {
    try {
      const data = await getForecastProducts(token, search);
      setProducts(data);
    } catch {
      setProducts([]);
    }
  };

  const loadChart = async (override = null) => {
    const payload = override || chartForm;

    setError("");
    setLoadingChart(true);
    setChartData([]);

    if (!payload.vendorId || !payload.productId) {
      setLoadingChart(false);
      return;
    }

    try {
      const data = await getForecastChartData(
        {
          vendorId: payload.vendorId,
          productId: Number(payload.productId),
          forecastDate: payload.forecastDate,
          modelName: "MLNET_SSA",
        },
        token
      );
      setChartData(data);
    } catch (e) {
      setError(e?.message || "Failed to load chart");
    } finally {
      setLoadingChart(false);
    }
  };

  const applyFirstValidForecastRowToChart = (forecastRows) => {
    if (!forecastRows || forecastRows.length === 0) return;

    const first = forecastRows[0];
```

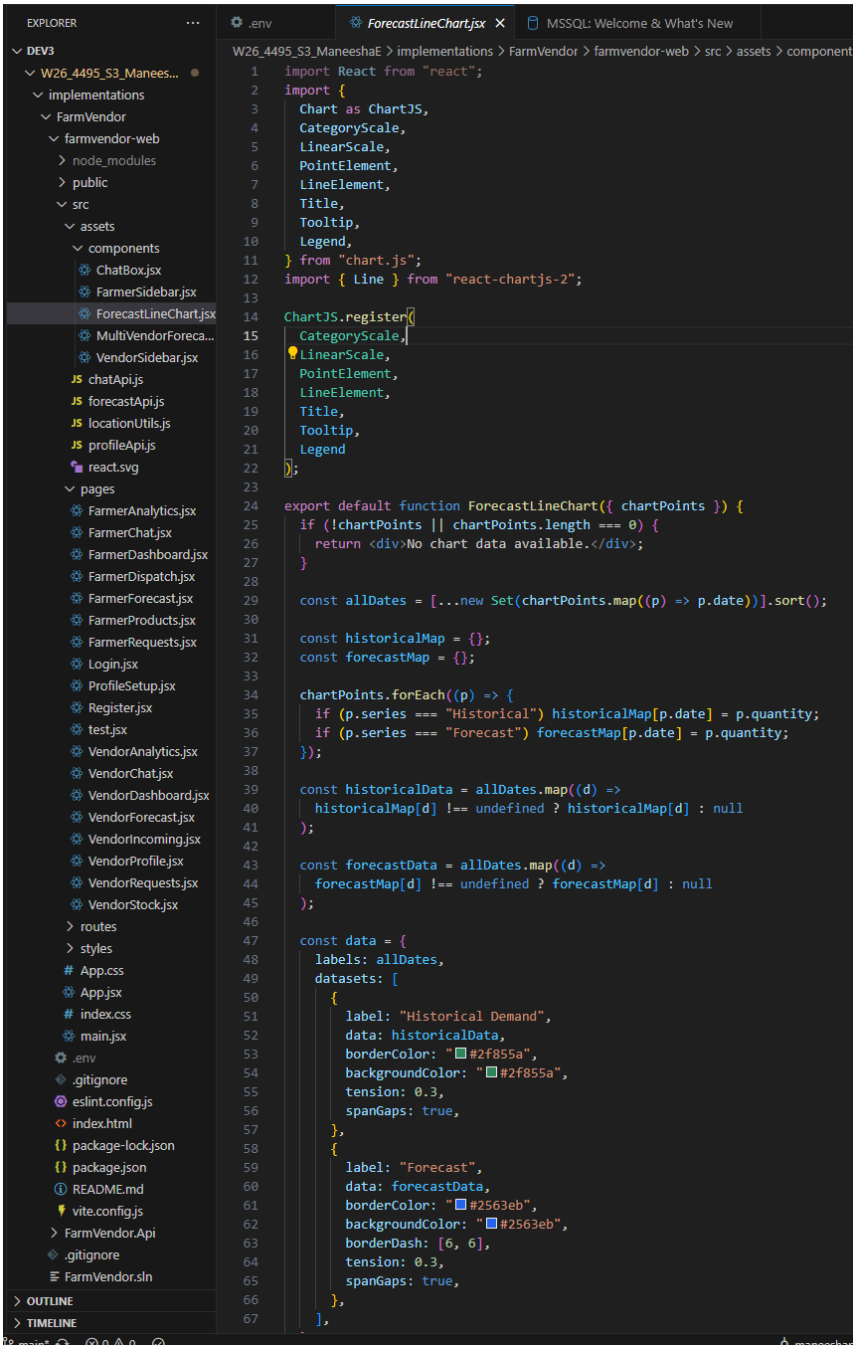
FarmerForecast.jsx MSSQL: Welcome & What's New

495_S3_ManeeshaE > implementations > FarmVendor > farmvendor-web > src > pages > FarmerForeca

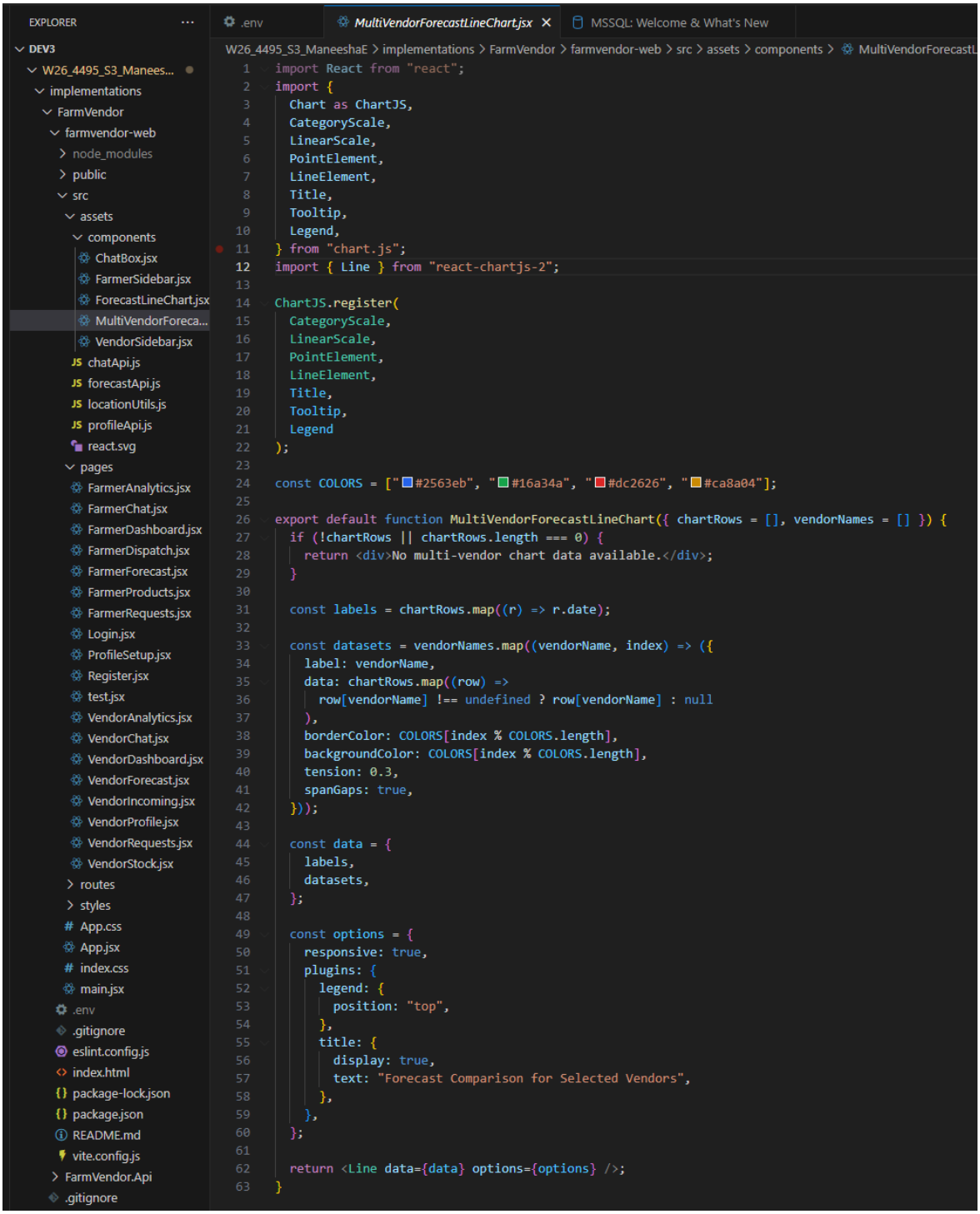
```
export default function FarmerAnalytics() {
  const loadChart = async (override = null) => {
  };

  const applyFirstValidForecastRowToChart = (forecastRows) => {
    if (!forecastRows || forecastRows.length === 0) return;

    const first = forecastRows[0];

    if (!first?.vendorId || !first?.productId || !first?.forecastDate) return;

    const nextChartForm = {
      vendorId: first.vendorId,
      productId: String(first.productId),
      forecastDate: new Date(first.forecastDate).toISOString().slice(0, 10),
    };

    setChartForm(nextChartForm);
    setForm((prev) => ({
      ...prev,
      vendorId: first.vendorId,
      forecastDate: new Date(first.forecastDate).toISOString().slice(0, 10),
    }));

    loadChart(nextChartForm);
  };

  const loadForecastRows = async (forecastDateArg, modelNameArg) => {
    setLoadingRows(true);
    setError("");
    try {
      const requestedDate = forecastDateArg || form.forecastDate;
      const requestedModel = modelNameArg || form.modelName;

      const data = await getForecasts(
        {
          forecastDate: requestedDate,
          modelName: requestedModel,
        },
        token
      );

      setRows(data);

      if (requestedModel === "MLNET_SSA" && data.length > 0) {
        applyFirstValidForecastRowToChart(data);
      } else if (data.length === 0) {
        setChartData([]);
      }
    } catch (e) {
      setError(e?.message || "Failed to load forecasts");
    } finally {
      setLoadingRows(false);
    }
  };

  const handleGenerate = async () => {
    setLoading(true);
    setError("");
    setSuccess("");

    try {
      const payload =
        form.modelName === "MovingAverage"
          ? {
              forecastDate: form.forecastDate,
              modelName: form.modelName,
```


FarmerForecast.jsx
MSSQL: Welcome & What's New
4495_S3_ManeeshaE > implementations > FarmVendor > farmvendor-web > src > pages > FarmerForecast.jsx > FarmerAnalytics
export default function FarmerAnalytics() {
const handleGenerate = async () => {
try {
const payload =
form.modelName === "MovingAverage"
? {
forecastDate: form.forecastDate,
modelName: form.modelName,
lookbackPeriods: Number(form.lookbackPeriods),
vendorId: form.vendorId || null,
}
: {
forecastDate: form.forecastDate,
modelName: form.modelName,
horizon: Number(form.horizon),
granularity: form.granularity,
vendorId: form.vendorId || null,
};
const result = await generateForecasts(payload, token);
setSuccess(
`Forecast generation completed. Model: ${result.modelName}, Rows: ${result.forecastCount}`
);
await loadModels();
await loadForecastRows(form.forecastDate, form.modelName);
} catch (e) {
setError(e?.message || "Failed to generate forecasts");
} finally {
setLoading(false);
}
};
const escapeCsv = (value) => {
if (value === null || value === undefined) return "";
const str = String(value);
if (str.includes('"') || str.includes(",") || str.includes("\n")) {
return `"${str.replace(/"/g, '""')}"`;
}
return str;
};
const exportForecastResultsCsv = () => {
if (!rows || rows.length === 0) {
setError("No forecast rows available to export.");
return;
}
const header = [
"Vendor",
"Product",
"Forecast Date",
"Qty",
"Model",
"Lookback",
"Created At",
];
const csvRows = rows.map((r) => [
r.vendorName || r.vendorId,
r.productName,
new Date(r.forecastDate).toISOString().slice(0, 10),
r.forecastQty,

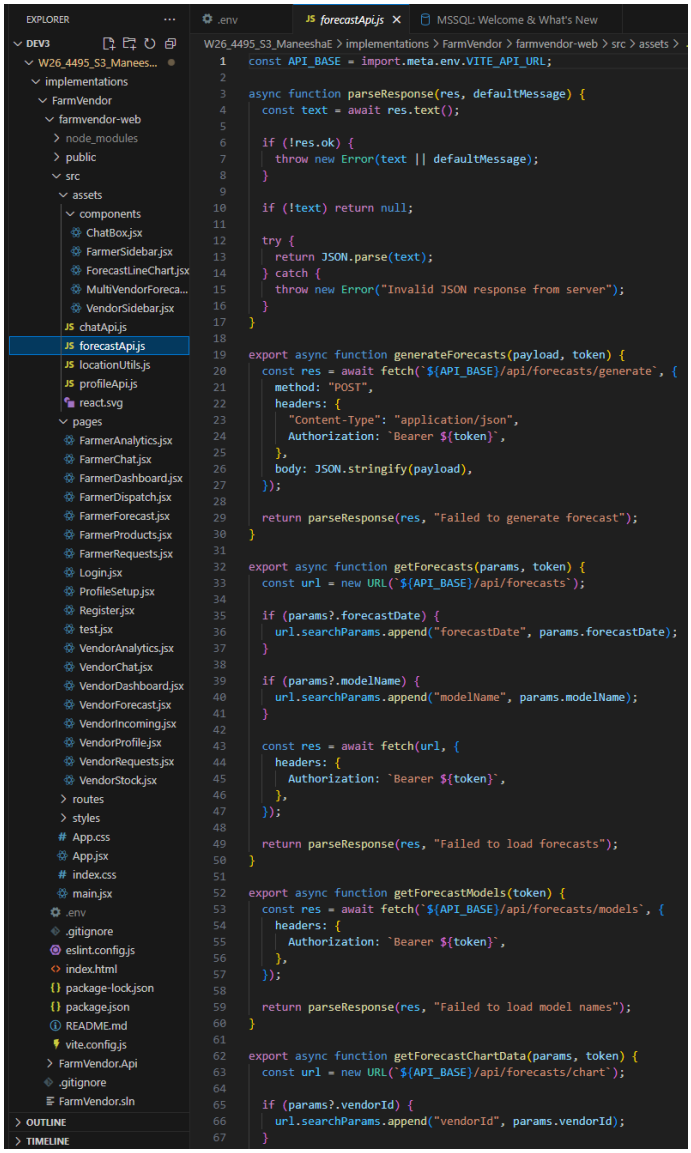
- *ForecastLineChart.jsx*



- *MultiVendorForecastLineChart.jsx*



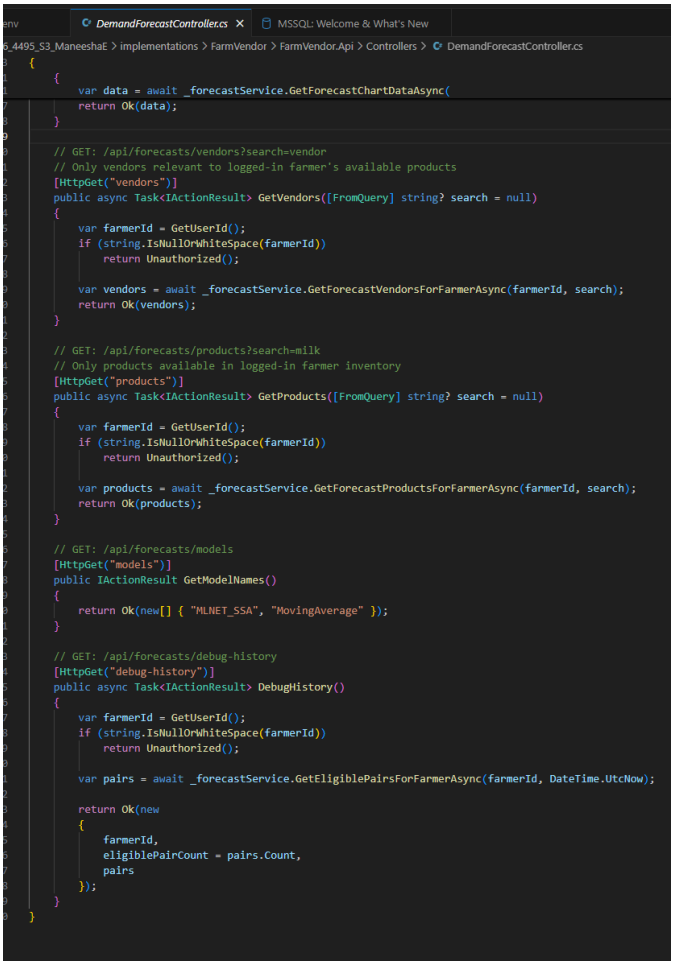
- *ForecastApi.js*



Backend

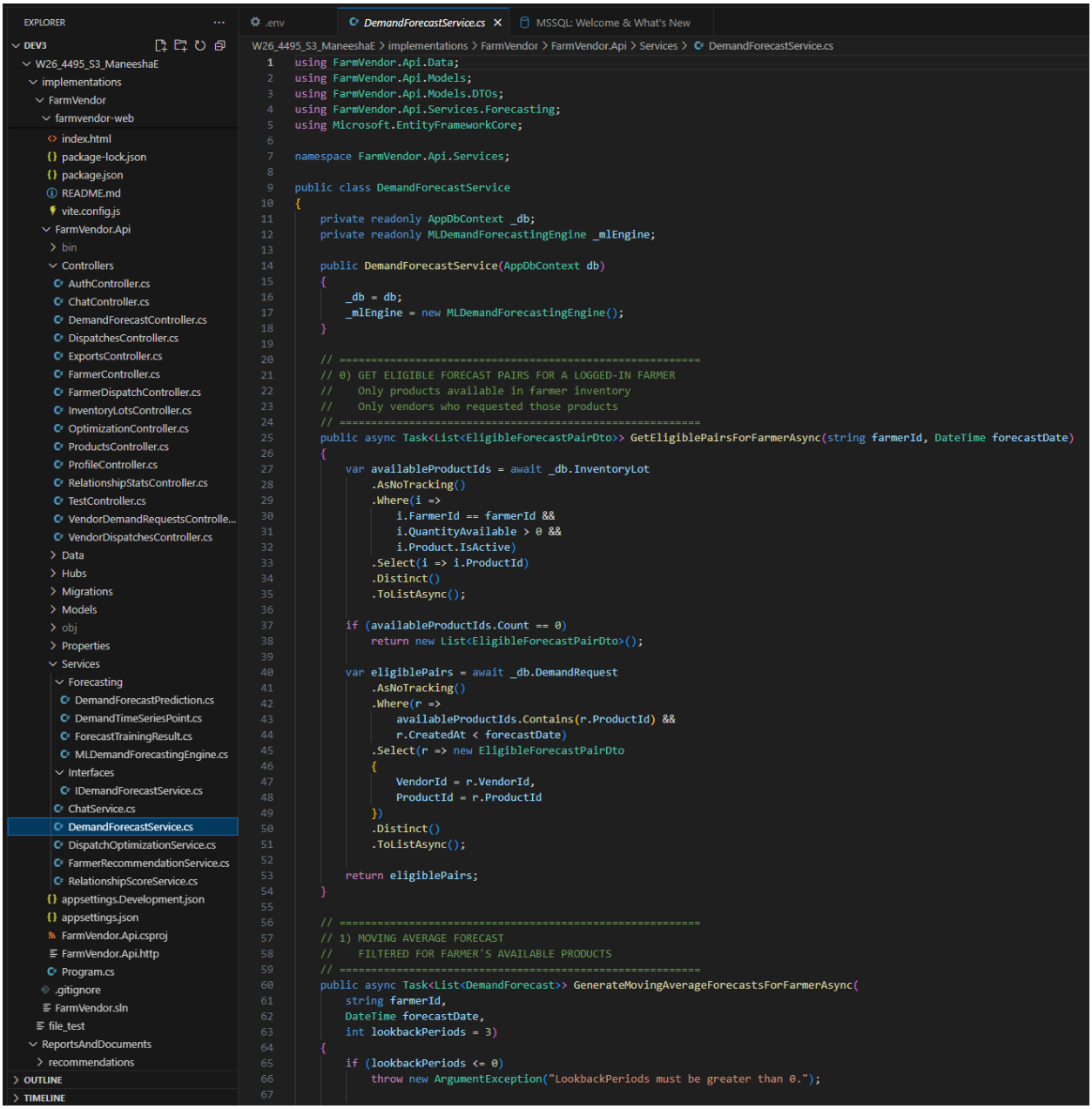
- *DemandForecastController*

```
DemandForecastController.cs  ×  MSSQL: Welcome & What's New
495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Controllers > DemandForecastController.cs
{
    {
    }

    var saved = await _forecastService.SaveForecastsAsync(forecasts);

    return Ok(new
    {
        message = "Forecast generation completed.",
        modelName = dto.ModelName,
        forecastCount = forecasts.Count,
        savedRows = saved
    });
}

// GET: /api/forecasts?forecastDate=2026-04-05&modelName=MLNET_SSA
[HttpGet]
public async Task<ActionResult<IEnumerable<DemandForecastRowDto>>> GetForecasts(
    [FromQuery] DateTime? forecastDate,
    [FromQuery] string? modelName)
{
    var requestedDate = forecastDate ?? DateTime.UtcNow.Date;
    var requestedModel = string.IsNullOrWhiteSpace(modelName)
        ? "MLNET_SSA"
        : modelName.Trim();

    var rows = await _forecastService.GetForecastRowsAsync(requestedDate, requestedModel);
    return Ok(rows);
}

// GET: /api/forecasts/chart?vendorId=abc&productId=1&forecastDate=2026-04-05&modelName=MLNET_SSA
[HttpGet("chart")]
public async Task<IActionResult> GetForecastChartData(
    [FromQuery] string vendorId,
    [FromQuery] int productId,
    [FromQuery] DateTime forecastDate,
    [FromQuery] string modelName = "MLNET_SSA")
{
    if (string.IsNullOrWhiteSpace(vendorId))
        return BadRequest("VendorId is required.");

    if (productId <= 0)
        return BadRequest("ProductId is required.");

    if (forecastDate == default)
        return BadRequest("ForecastDate is required.");

    var data = await _forecastService.GetForecastChartDataAsync(
        vendorId,
        productId,
        forecastDate,
        modelName);

    return Ok(data);
}

// GET: /api/forecasts/vendors?search=vendor
// Only vendors relevant to logged-in farmer's available products
[HttpGet("vendors")]
public async Task<IActionResult> GetVendors([FromQuery] string? search = null)
{
    var farmerId = GetUserId();
    if (string.IsNullOrWhiteSpace(farmerId))
        return Unauthorized();

    var vendors = await _forecastService.GetForecastVendorsForFarmerAsync(farmerId, search);
    return Ok(vendors);
}
```



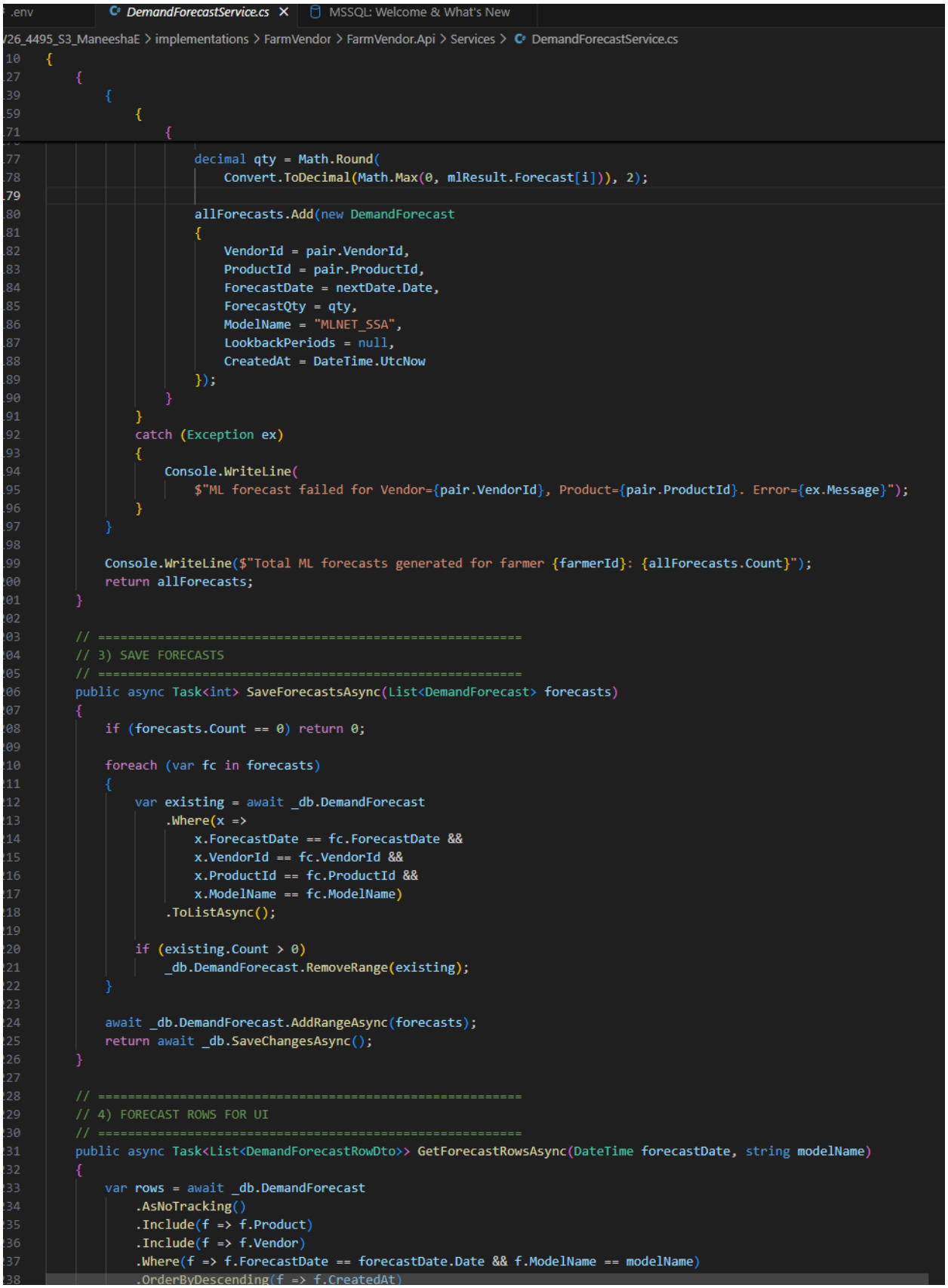
```
DemandForecastController.cs  ×  MSSQL: Welcome & What's New
env
5_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Controllers > DemandForecastController.cs
{
    {
        var data = await _forecastService.GetForecastChartDataAsync(
        return Ok(data);
    }

    // GET: /api/forecasts/vendors?search=vendor
    // Only vendors relevant to logged-in farmer's available products
    [HttpGet("vendors")]
    public async Task<IActionResult> GetVendors([FromQuery] string? search = null)
    {
        var farmerId = GetUserId();
        if (string.IsNullOrWhiteSpace(farmerId))
            return Unauthorized();

        var vendors = await _forecastService.GetForecastVendorsForFarmerAsync(farmerId, search);
        return Ok(vendors);
    }

    // GET: /api/forecasts/products?search=milk
    // Only products available in logged-in farmer inventory
    [HttpGet("products")]
    public async Task<IActionResult> GetProducts([FromQuery] string? search = null)
    {
        var farmerId = GetUserId();
        if (string.IsNullOrWhiteSpace(farmerId))
            return Unauthorized();

        var products = await _forecastService.GetForecastProductsForFarmerAsync(farmerId, search);
        return Ok(products);
    }

    // GET: /api/forecasts/models
    [HttpGet("models")]
    public IActionResult GetModelNames()
    {
        return Ok(new[] { "MLNET_SSA", "MovingAverage" });
    }

    // GET: /api/forecasts/debug-history
    [HttpGet("debug-history")]
    public async Task<IActionResult> DebugHistory()
    {
        var farmerId = GetUserId();
        if (string.IsNullOrWhiteSpace(farmerId))
            return Unauthorized();

        var pairs = await _forecastService.GetEligiblePairsForFarmerAsync(farmerId, DateTime.UtcNow);

        return Ok(new
        {
            farmerId,
            eligiblePairCount = pairs.Count,
            pairs
        });
    }
}
```

- *DemandForecastService.cs*



```
    // ========================================================
    // 1) MOVING AVERAGE FORECAST
    //    FILTERED FOR FARMER'S AVAILABLE PRODUCTS
    // ========================================================
    public async Task<List<DemandForecast>> GenerateMovingAverageForecastsForFarmerAsync(
        string farmerId,
        DateTime forecastDate,
        int lookbackPeriods = 3)
    {
        if (lookbackPeriods <= 0)
            throw new ArgumentException("LookbackPeriods must be greater than 0.");

        var eligiblePairs = await GetEligiblePairsForFarmerAsync(farmerId, forecastDate);

        if (eligiblePairs.Count == 0)
            return new List<DemandForecast>();

        var groupedRequests = await _db.DemandRequest
            .AsNoTracking()
            .Where(r => r.CreatedAt < forecastDate)
            .OrderByDescending(r => r.CreatedAt)
            .GroupBy(r => new { r.VendorId, r.ProductId })
            .Select(g => new
            {
                g.Key.VendorId,
                g.Key.ProductId,
                Requests = g.OrderByDescending(x => x.CreatedAt)
                            .Take(lookbackPeriods)
                            .Select(x => x.QuantityRequested)
                            .ToList()
            })
            .ToListAsync();

        var eligibleSet = eligiblePairs
            .Select(x => $"{x.VendorId}__{x.ProductId}")
            .ToHashSet();

        var forecasts = new List<DemandForecast>();

        foreach (var group in groupedRequests)
        {
            var key = $"{group.VendorId}__{group.ProductId}";
            if (!eligibleSet.Contains(key)) continue;
            if (group.Requests.Count == 0) continue;

            var avgQty = group.Requests.Average();

            forecasts.Add(new DemandForecast
            {
                VendorId = group.VendorId,
                ProductId = group.ProductId,
                ForecastDate = forecastDate.Date,
                ForecastQty = Math.Round(avgQty, 2),
                ModelName = "MovingAverage",
                LookbackPeriods = lookbackPeriods,
                CreatedAt = DateTime.UtcNow
            });
        }

        return forecasts;
    }
```

```
    // ============================================================
    // 2) ML.NET FORECAST
    //    FILTERED FOR FARMER'S AVAILABLE PRODUCTS
    // ============================================================
    public async Task<List<DemandForecast>> GenerateMlForecastsForFarmerAsync(
        string farmerId,
        DateTime forecastStartDate,
        int horizon = 7,
        string granularity = "Daily")
    {
        if (horizon <= 0)
            throw new ArgumentException("Horizon must be greater than 0.");

        var eligiblePairs = await GetEligiblePairsForFarmerAsync(farmerId, forecastStartDate);

        if (eligiblePairs.Count == 0)
            return new List<DemandForecast>();

        var allForecasts = new List<DemandForecast>();

        foreach (var pair in eligiblePairs)
        {
            var series = await LoadAggregatedDemandSeriesAsync(
                pair.ProductId,
                pair.VendorId,
                forecastStartDate,
                granularity);

            if (series.Count < 3)
            {
                Console.WriteLine(
                    $"Skipped ML forecast for Vendor={pair.VendorId}, Product={pair.ProductId}, Reason=Not enough history, SeriesCount={
                continue;
            }

            var orderedValues = series
                .OrderBy(x => x.PeriodDate)
                .Select(x => (float)x.TotalQuantity)
                .ToList();

            try
            {
                var safeHorizon = Math.Min(horizon, Math.Max(1, orderedValues.Count / 2));
                var mlResult = _mlEngine.TrainAndForecast(orderedValues, safeHorizon);

                if (mlResult.Forecast == null || mlResult.Forecast.Count == 0)
                {
                    Console.WriteLine(
                        $"ML forecast returned no results for Vendor={pair.VendorId}, Product={pair.ProductId}");
                    continue;
                }

                for (int i = 0; i < mlResult.Forecast.Count; i++)
                {
                    DateTime nextDate =
                        granularity.Equals("Weekly", StringComparison.OrdinalIgnoreCase)
                            ? forecastStartDate.Date.AddDays(7 * i)
                            : forecastStartDate.Date.AddDays(i);

                    decimal qty = Math.Round(
                        Convert.ToDecimal(Math.Max(0, mlResult.Forecast[i])), 2);

                    allForecasts.Add(new DemandForecast
                    {
                        VendorId = pair.VendorId,
                        ProductId = pair.ProductId,
```


.env
DemandForecastService.cs
MSSQL: Welcome & What's New
/26_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Services > DemandForecastService.cs
{
{
{
{
decimal qty = Math.Round(
Convert.ToDecimal(Math.Max(0, mlResult.Forecast[i])), 2);
allForecasts.Add(new DemandForecast
{
VendorId = pair.VendorId,
ProductId = pair.ProductId,
ForecastDate = nextDate.Date,
ForecastQty = qty,
ModelName = "MLNET_SSA",
LookbackPeriods = null,
CreatedAt = DateTime.UtcNow
});
}
}
catch (Exception ex)
{
Console.WriteLine(
$"ML forecast failed for Vendor={pair.VendorId}, Product={pair.ProductId}. Error={ex.Message}");
}
}
Console.WriteLine($"Total ML forecasts generated for farmer {farmerId}: {allForecasts.Count}");
return allForecasts;
}
// ==
// 3) SAVE FORECASTS
// ==
public async Task<int> SaveForecastsAsync(List<DemandForecast> forecasts)
{
if (forecasts.Count == 0) return 0;
foreach (var fc in forecasts)
{
var existing = await _db.DemandForecast
.Where(x =>
x.ForecastDate == fc.ForecastDate &&
x.VendorId == fc.VendorId &&
x.ProductId == fc.ProductId &&
x.ModelName == fc.ModelName)
.ToListAsync();
if (existing.Count > 0)
_db.DemandForecast.RemoveRange(existing);
}
await _db.DemandForecast.AddRangeAsync(forecasts);
return await _db.SaveChangesAsync();
}
// ==
// 4) FORECAST ROWS FOR UI
// ==
public async Task<List<DemandForecastRowDto>> GetForecastRowsAsync(DateTime forecastDate, string modelName)
{
var rows = await _db.DemandForecast
.AsNoTracking()
.Include(f => f.Product)
.Include(f => f.Vendor)
.Where(f => f.ForecastDate == forecastDate.Date && f.ModelName == modelName)
.OrderByDescending(f => f.CreatedAt)

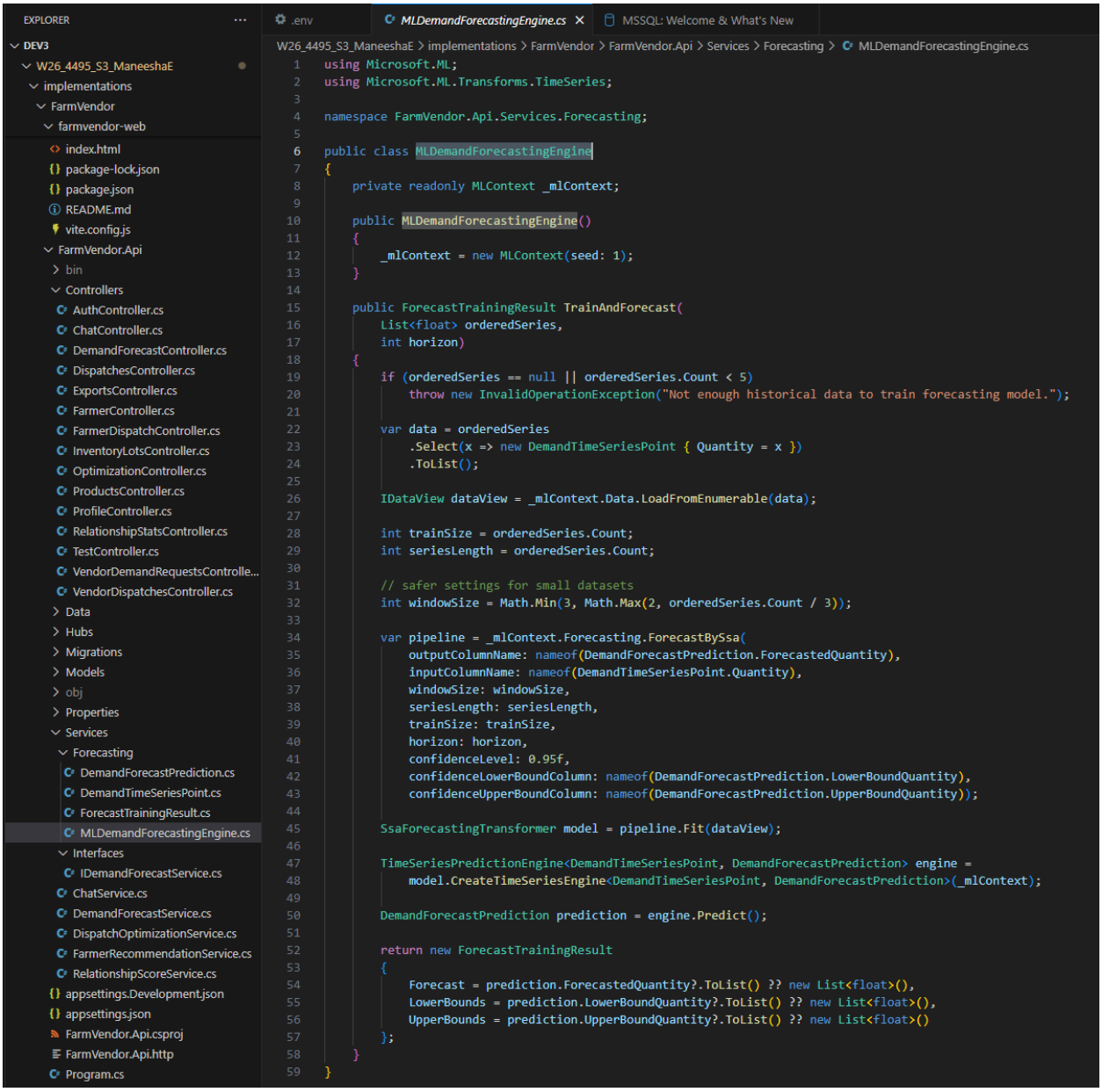

.env
DemandForecastService.cs
MSSQL: Welcome & What's New
26_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Services > DemandForecastServic
{
{
var rows = await _db.DemandForecast
{
VendorId = f.VendorId,
VendorName = f.Vendor.DisplayName,
ProductId = f.ProductId,
ProductName = f.Product.Name,
ForecastDate = f.ForecastDate,
ForecastQty = f.ForecastQty,
ModelName = f.ModelName,
LookbackPeriods = f.LookbackPeriods,
CreatedAt = f.CreatedAt
})
.ToListAsync();
return rows;
}
// ==
// 5) CHART DATA
// ==
public async Task<List<ForecastChartPointDto>> GetForecastChartDataAsync(
string vendorId,
int productId,
DateTime forecastDate,
string modelName = "MLNET_SSA")
{
var history = await _db.DemandRequest
.AsNoTracking()
.Where(r => r.VendorId == vendorId &&
r.ProductId == productId &&
r.CreatedAt < forecastDate)
.OrderByDescending(r => r.CreatedAt)
.Take(10)
.Select(r => new ForecastChartPointDto
{
Date = r.CreatedAt.ToString("yyyy-MM-dd"),
Quantity = r.QuantityRequested,
Series = "Historical"
})
.ToListAsync();
history = history.OrderBy(x => x.Date).ToList();
var forecasts = await _db.DemandForecast
.AsNoTracking()
.Where(f => f.VendorId == vendorId &&
f.ProductId == productId &&
f.ModelName == modelName &&
f.ForecastDate >= forecastDate.Date)
.OrderBy(f => f.ForecastDate)
.Take(10)
.Select(f => new ForecastChartPointDto
{
Date = f.ForecastDate.ToString("yyyy-MM-dd"),
Quantity = f.ForecastQty,
Series = "Forecast"
})
.ToListAsync();
return history.Concat(forecasts).ToList();
}
// ==
// 6) FILTERED VENDOR OPTIONS FOR FARMER

W26_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Services > DemandForecastService.cs

```
10  {
302     // ========================================================
303     // 6) FILTERED VENDOR OPTIONS FOR FARMER
304     // ========================================================
305     public async Task<List<ForecastVendorOptionDto>> GetForecastVendorsForFarmerAsync(string farmerId, string? search = null)
306     {
307         var availableProductIds = await _db.InventoryLot
308             .AsNoTracking()
309             .Where(i =>
310                 i.FarmerId == farmerId &&
311                 i.QuantityAvailable > 0 &&
312                 i.Product.IsActive)
313             .Select(i => i.ProductId)
314             .Distinct()
315             .ToListAsync();
316
317         if (availableProductIds.Count == 0)
318             return new List<ForecastVendorOptionDto>();
319
320         var query = _db.DemandRequest
321             .AsNoTracking()
322             .Where(r => availableProductIds.Contains(r.ProductId))
323             .Select(r => r.Vendor)
324             .Distinct();
325
326         if (!string.IsNullOrWhiteSpace(search))
327         {
328             var term = search.Trim().ToLower();
329             query = query.Where(v =>
330                 (v.DisplayName != null && v.DisplayName.ToLower().Contains(term)) ||
331                 (v.Email != null && v.Email.ToLower().Contains(term)));
332         }
333
334         return await query
335             .OrderBy(v => v.DisplayName)
336             .Select(v => new ForecastVendorOptionDto
337             {
338                 Id = v.Id,
339                 DisplayName = v.DisplayName ?? "",
340                 Email = v.Email ?? ""
341             })
342             .ToListAsync();
343     }
344
345     // ========================================================
346     // 7) FILTERED PRODUCT OPTIONS FOR FARMER
347     // ========================================================
348     public async Task<List<ForecastProductOptionDto>> GetForecastProductsForFarmerAsync(string farmerId, string? search = null)
349     {
350         var query = _db.InventoryLot
351             .AsNoTracking()
352             .Where(i =>
353                 i.FarmerId == farmerId &&
354                 i.QuantityAvailable > 0 &&
355                 i.Product.IsActive)
356             .Select(i => i.Product)
357             .Distinct();
358
359         if (!string.IsNullOrWhiteSpace(search))
360         {
361             var term = search.Trim().ToLower();
362             query = query.Where(p => p.Name.ToLower().Contains(term));
363         }
364
365         return await query
366             .OrderBy(p => p.Name)
367             .Select(p => new ForecastProductOptionDto
```


DemandForecastService.cs
MSSQL: Welcome & What's New
5_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Services > DemandForecastService.cs
{
{
return await query
})
.ToListAsync();
}
// ==
// 8) PRIVATE HELPER FOR DAILY / WEEKLY SERIES
// ==
private async Task<List<AggregatedDemandPoint>> LoadAggregatedDemandSeriesAsync(
int productId,
string vendorId,
DateTime beforeDate,
string granularity)
{
var raw = await _db.DemandRequest
.AsNoTracking()
.Where(r => r.ProductId == productId &&
r.VendorId == vendorId &&
r.CreatedAt < beforeDate)
.OrderBy(r => r.CreatedAt)
.Select(r => new
{
r.CreatedAt,
r.QuantityRequested
})
.ToListAsync();
if (granularity.Equals("Weekly", StringComparison.OrdinalIgnoreCase))
{
return raw
.GroupBy(x => StartOfWeek(x.CreatedAt.Date))
.Select(g => new AggregatedDemandPoint
{
PeriodDate = g.Key,
TotalQuantity = g.Sum(x => x.QuantityRequested)
})
.OrderBy(x => x.PeriodDate)
.ToList();
}
return raw
.GroupBy(x => x.CreatedAt.Date)
.Select(g => new AggregatedDemandPoint
{
PeriodDate = g.Key,
TotalQuantity = g.Sum(x => x.QuantityRequested)
})
.OrderBy(x => x.PeriodDate)
.ToList();
}
private static DateTime StartOfWeek(DateTime date)
{
int diff = (7 + (date.DayOfWeek - DayOfWeek.Monday)) % 7;
return date.AddDays(-diff).Date;
}
private class AggregatedDemandPoint
{
public DateTime PeriodDate { get; set; }
public decimal TotalQuantity { get; set; }
}
}

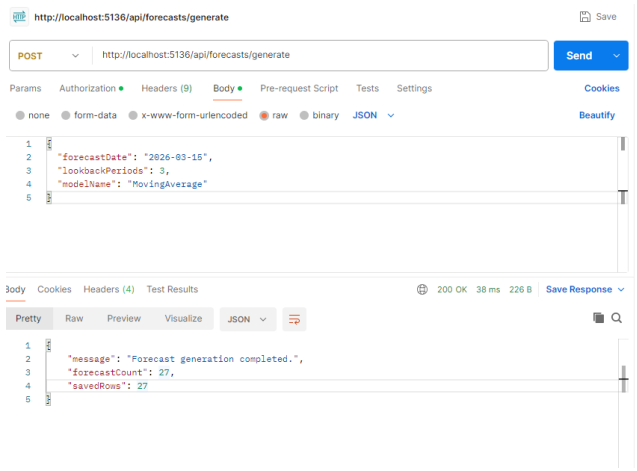
- *MLDemandForecastingEngine.cs*



- *Postman Testing*

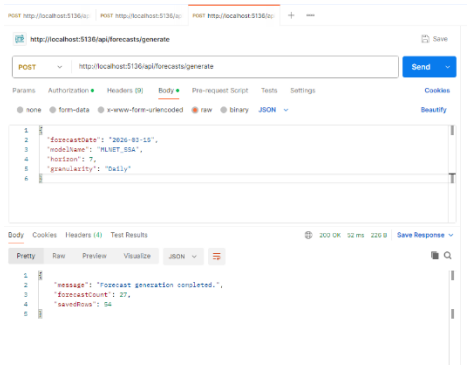
1. Moving Average request : POST : http://localhost:5136/api/forecasts/generate

{
"forecastDate": "2026-03-15",
"lookbackPeriods": 3,
"modelName": "MovingAverage"
}



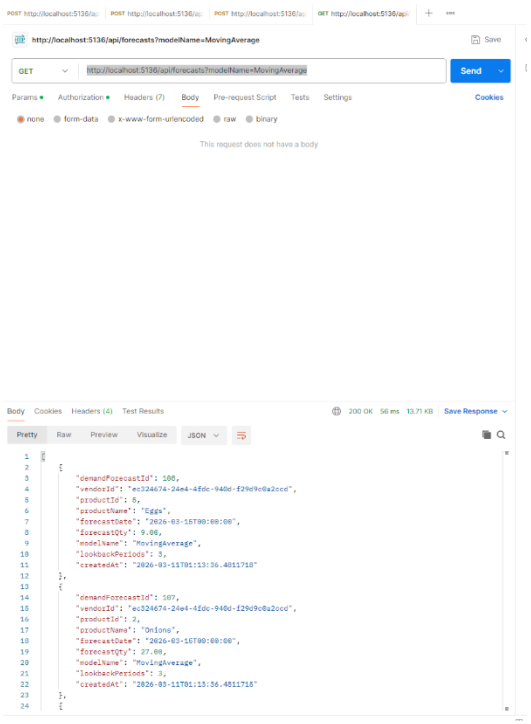
2. ML.NET request : POST : http://localhost:5136/api/forecasts/generate

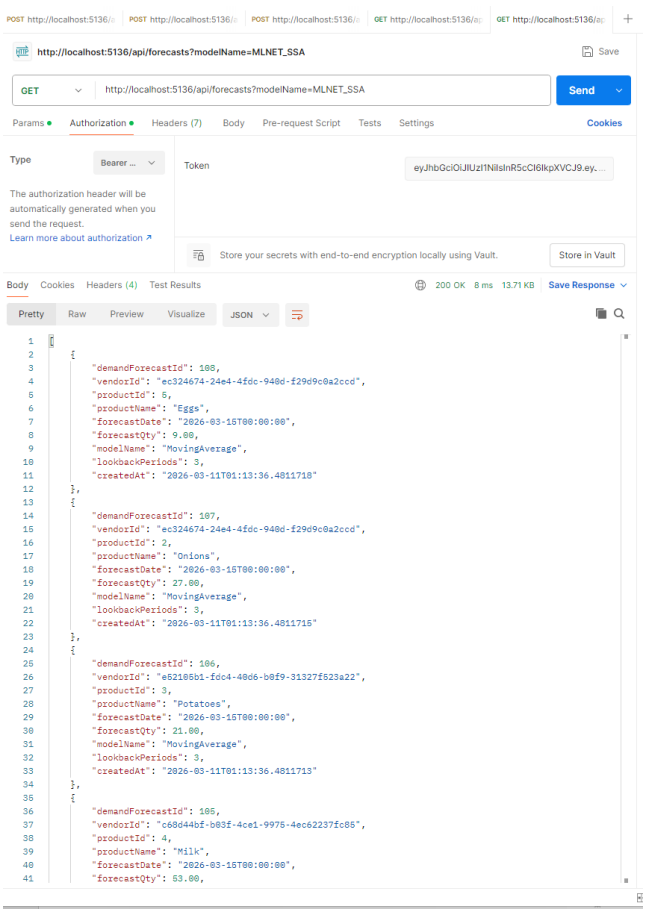
{
"forecastDate": "2026-03-15",
"modelName": "MLNET_SSA",
"horizon": 7,
"granularity": "Daily"
}



3. GET /api/forecasts?modelName=MovingAverage : http://localhost:5136/api/forecasts?modelName=MovingAverage

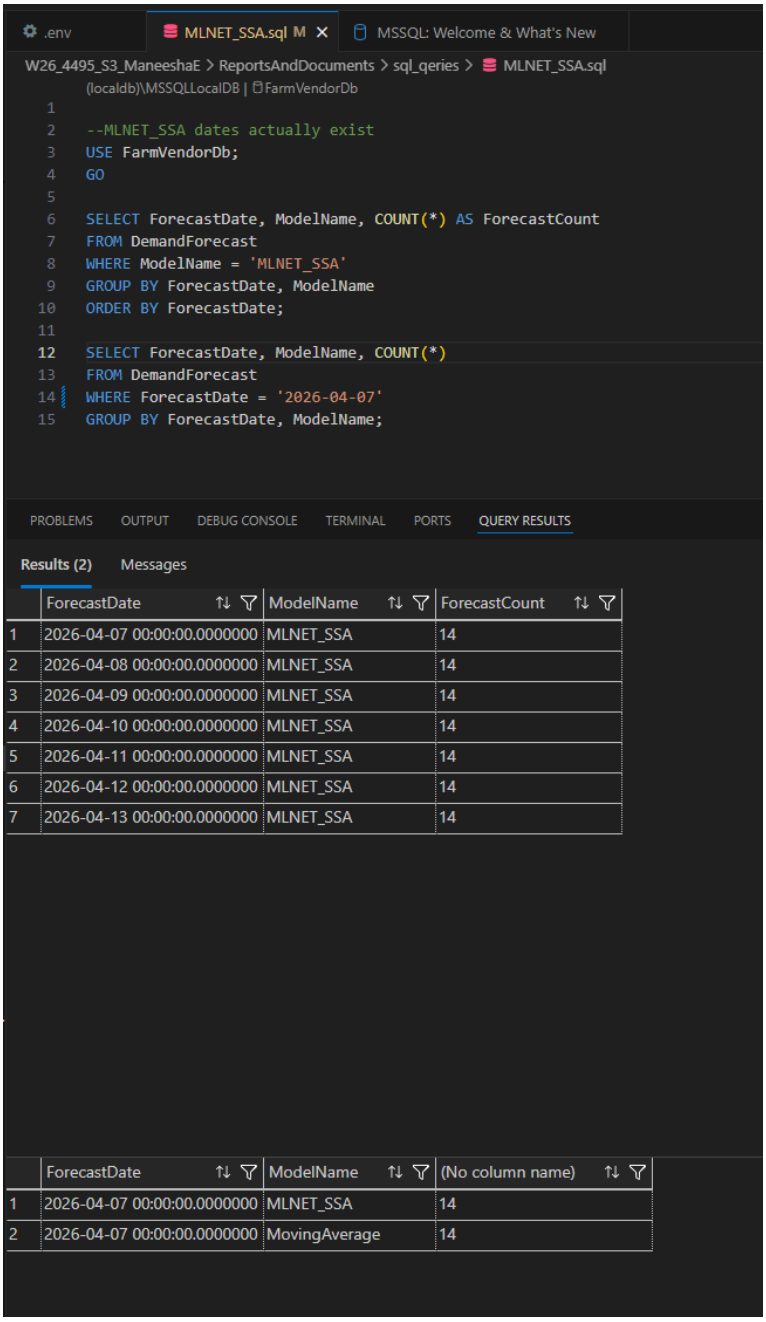
4. GET /api/forecasts?modelName=MLNET _SSA ：
   http://localhost:5136/api/forecasts?modelName=MLNET_SSA



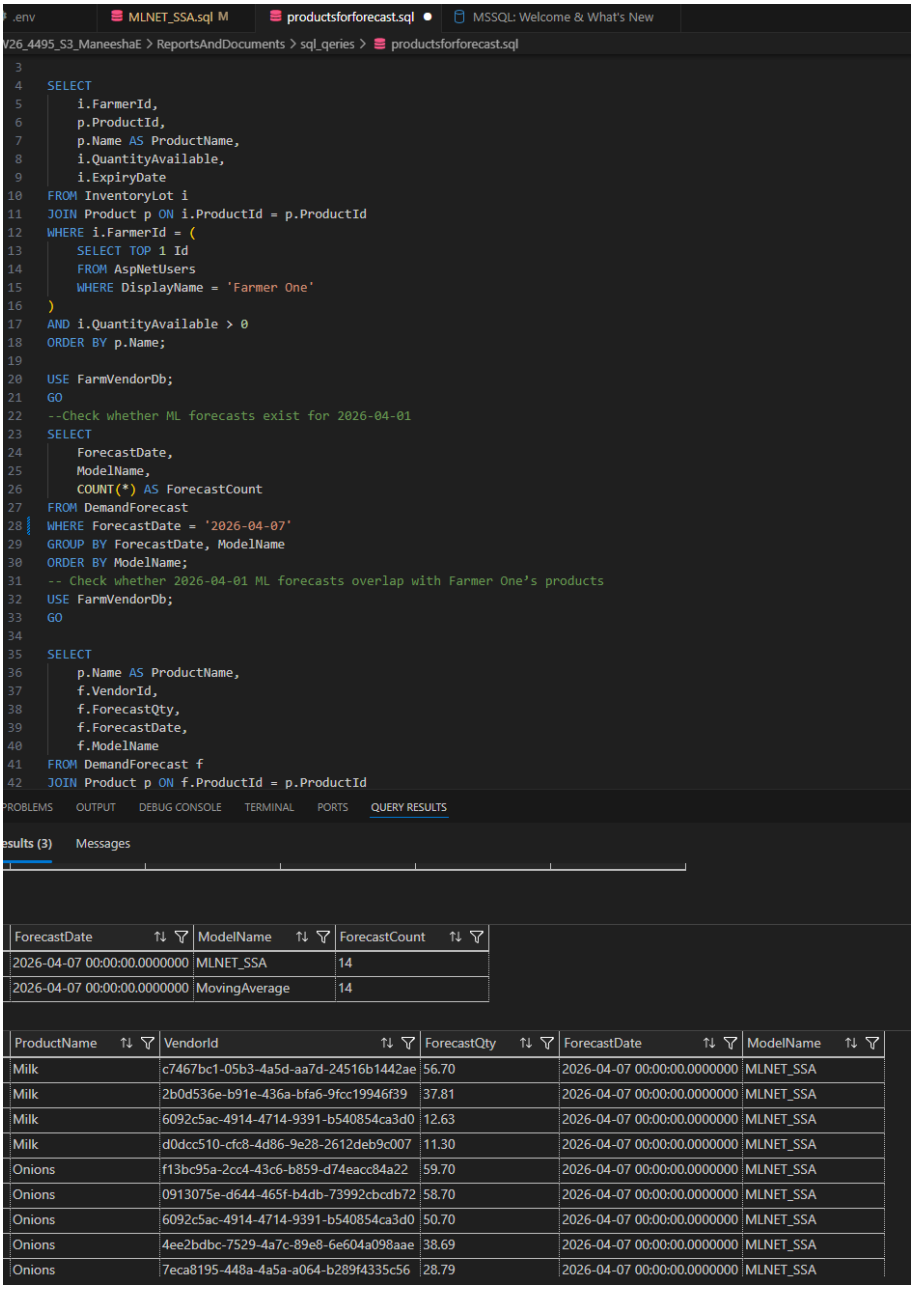


- *Database queries*

  - Forecast dates exists



- MLNET_SSA forecast data existence check



```
.env   MLNET_SSA.sql M   productsforforecast.sql ●   MSSQL: Welcome & What's New
V26_4495_S3_ManeeshaE > ReportsAndDocuments > sql_qeries > productsforforecast.sql

SELECT
    i.FarmerId,
    p.ProductId,
    p.Name AS ProductName,
    i.QuantityAvailable,
    i.ExpiryDate
FROM InventoryLot i
JOIN Product p ON i.ProductId = p.ProductId
WHERE i.FarmerId = (
    SELECT TOP 1 Id
    FROM AspNetUsers
    WHERE DisplayName = 'Farmer One'
)
AND i.QuantityAvailable > 0
ORDER BY p.Name;

USE FarmVendorDb;
GO
--Check whether ML forecasts exist for 2026-04-01
SELECT
    ForecastDate,
    ModelName,
    COUNT(*) AS ForecastCount
FROM DemandForecast
WHERE ForecastDate = '2026-04-07'
GROUP BY ForecastDate, ModelName
ORDER BY ModelName;
-- Check whether 2026-04-01 ML forecasts overlap with Farmer One's products
USE FarmVendorDb;
GO

SELECT
    p.Name AS ProductName,
    f.VendorId,
    f.ForecastQty,
    f.ForecastDate,
    f.ModelName
FROM DemandForecast f
JOIN Product p ON f.ProductId = p.ProductId
```

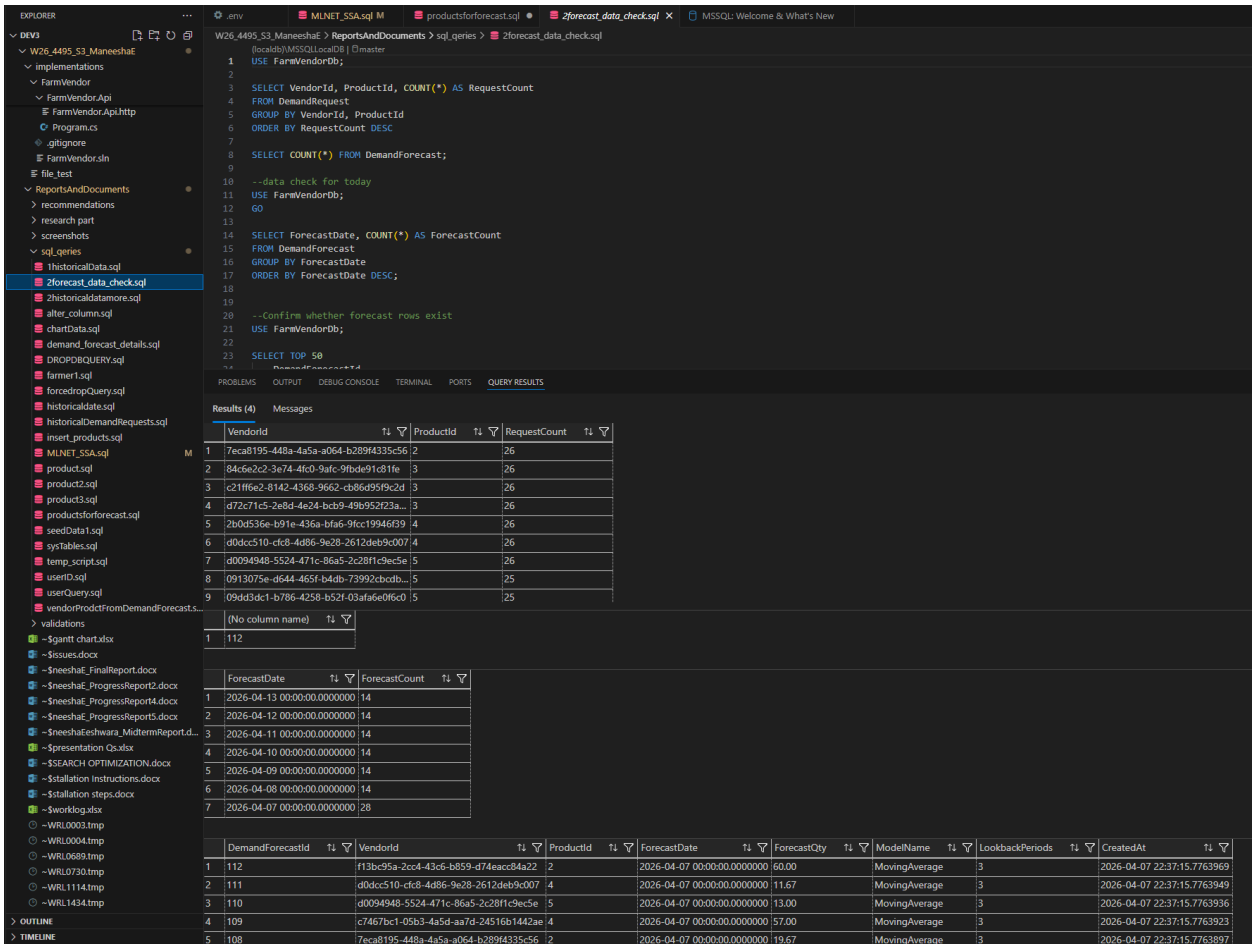
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

esults (3) Messages

ForecastDate	ModelName	ForecastCount
2026-04-07 00:00:00.0000000	MLNET_SSA	14
2026-04-07 00:00:00.0000000	MovingAverage	14

ProductName	VendorId	ForecastQty	ForecastDate	ModelName
Milk	c7467bc1-05b3-4a5d-aa7d-24516b1442ae	56.70	2026-04-07 00:00:00.0000000	MLNET_SSA
Milk	2b0d536e-b91e-436a-bfa6-9fcc19946f39	37.81	2026-04-07 00:00:00.0000000	MLNET_SSA
Milk	6092c5ac-4914-4714-9391-b540854ca3d0	12.63	2026-04-07 00:00:00.0000000	MLNET_SSA
Milk	d0dcc510-cfc8-4d86-9e28-2612deb9c007	11.30	2026-04-07 00:00:00.0000000	MLNET_SSA
Onions	f13bc95a-2cc4-43c6-b859-d74eacc84a22	59.70	2026-04-07 00:00:00.0000000	MLNET_SSA
Onions	0913075e-d644-465f-b4db-73992cbcdb72	58.70	2026-04-07 00:00:00.0000000	MLNET_SSA
Onions	6092c5ac-4914-4714-9391-b540854ca3d0	50.70	2026-04-07 00:00:00.0000000	MLNET_SSA
Onions	4ee2bdbc-7529-4a7c-89e8-6e604a098aae	38.69	2026-04-07 00:00:00.0000000	MLNET_SSA
Onions	7eca8195-448a-4a5a-a064-b289f4335c56	28.79	2026-04-07 00:00:00.0000000	MLNET_SSA

- Forecast data check



Feature 2: Vendor Recommendation, Relationship Score, and Distance-Based Matching

This feature supports dispatch decision-making by identifying suitable vendors based on demand relevance, past interactions, and physical distance.

This feature includes:

- Recommended vendors section
- Distance calculation between farmer and vendor
- Relationship score logic
- Hover popup for vendor details
- Request-specific vendor information

The design consists of:

- Score-based ranking
- Location-aware prioritization
- Historical relationship analysis

Implementation

Frontend

- FarmerDashboard.jsx
- vendor hover popup UI
- recommendation table rendering

Backend

- FarmerRecommendationService.cs
- FarmerController.cs
- RelationshipScoreService.cs
- Haversine distance calculation logic

Database

RelationshipStat table storing:

- FarmerId
- VendorId
- TotalRequestedQty

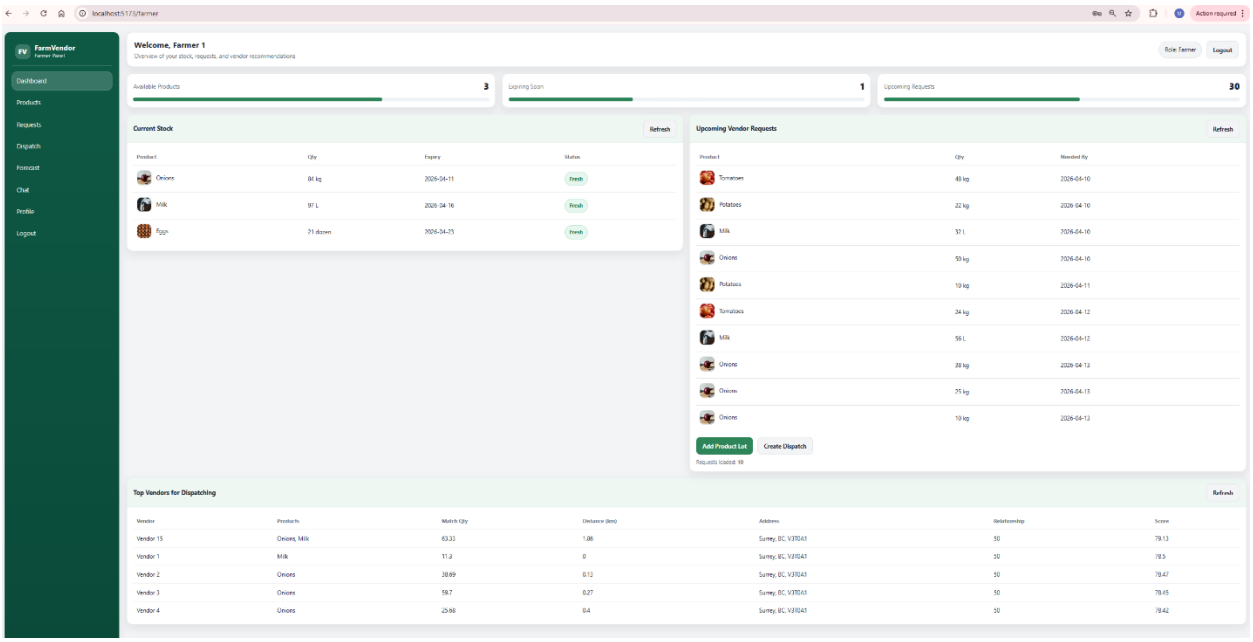
- TotalDeliveredQty
- RelationshipScore

Technical Methods

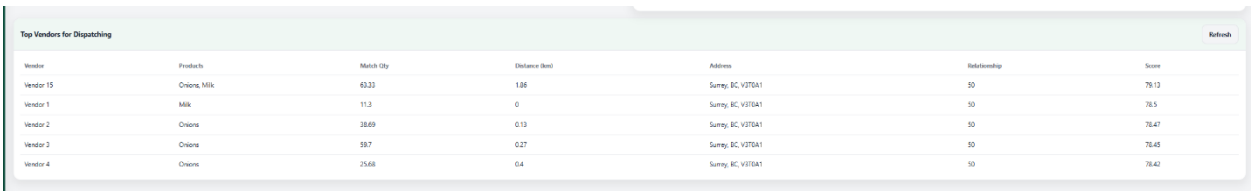
- weighted score formula using demand, distance, and relationship
- geospatial calculation using latitude/longitude
- ranking through EF Core + LINQ queries

App

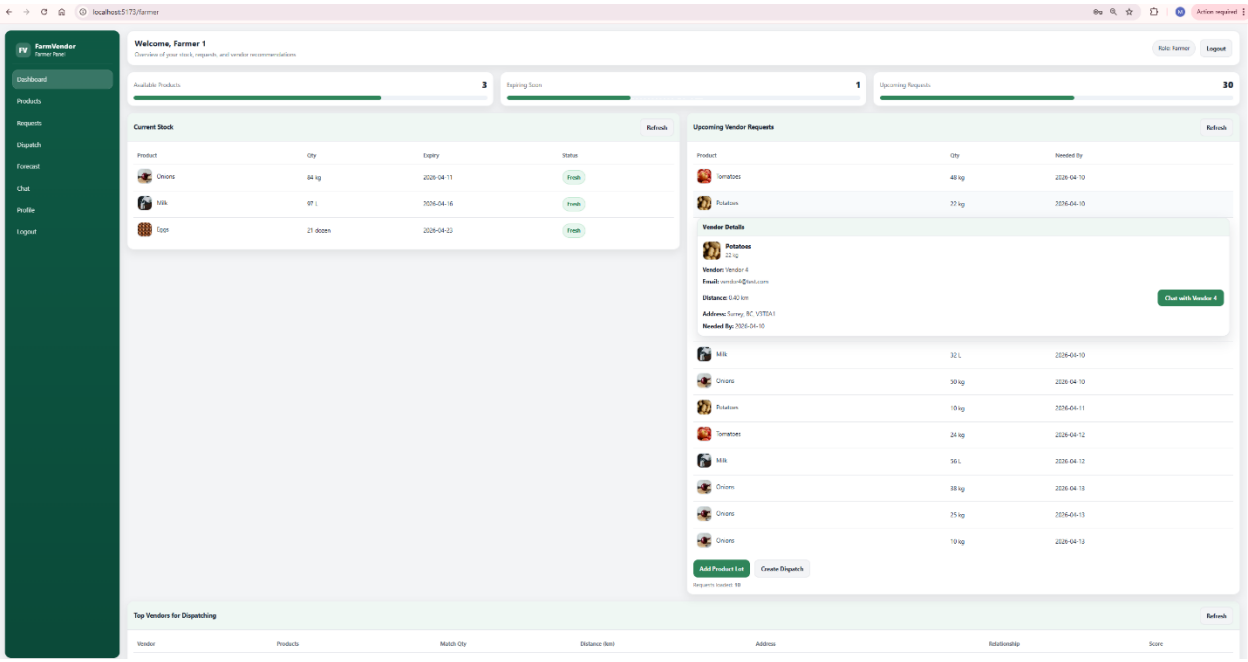
FarmerDashboard



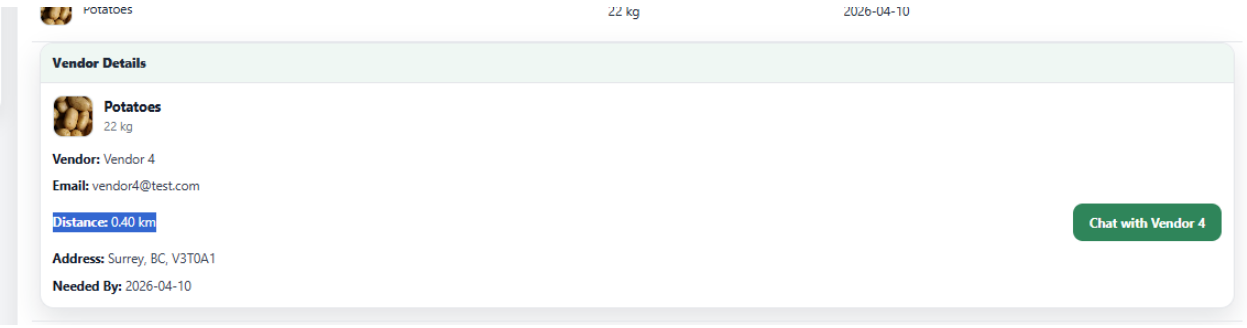
- Top Vendors for Dispatching section



- Vendor specific hover popup

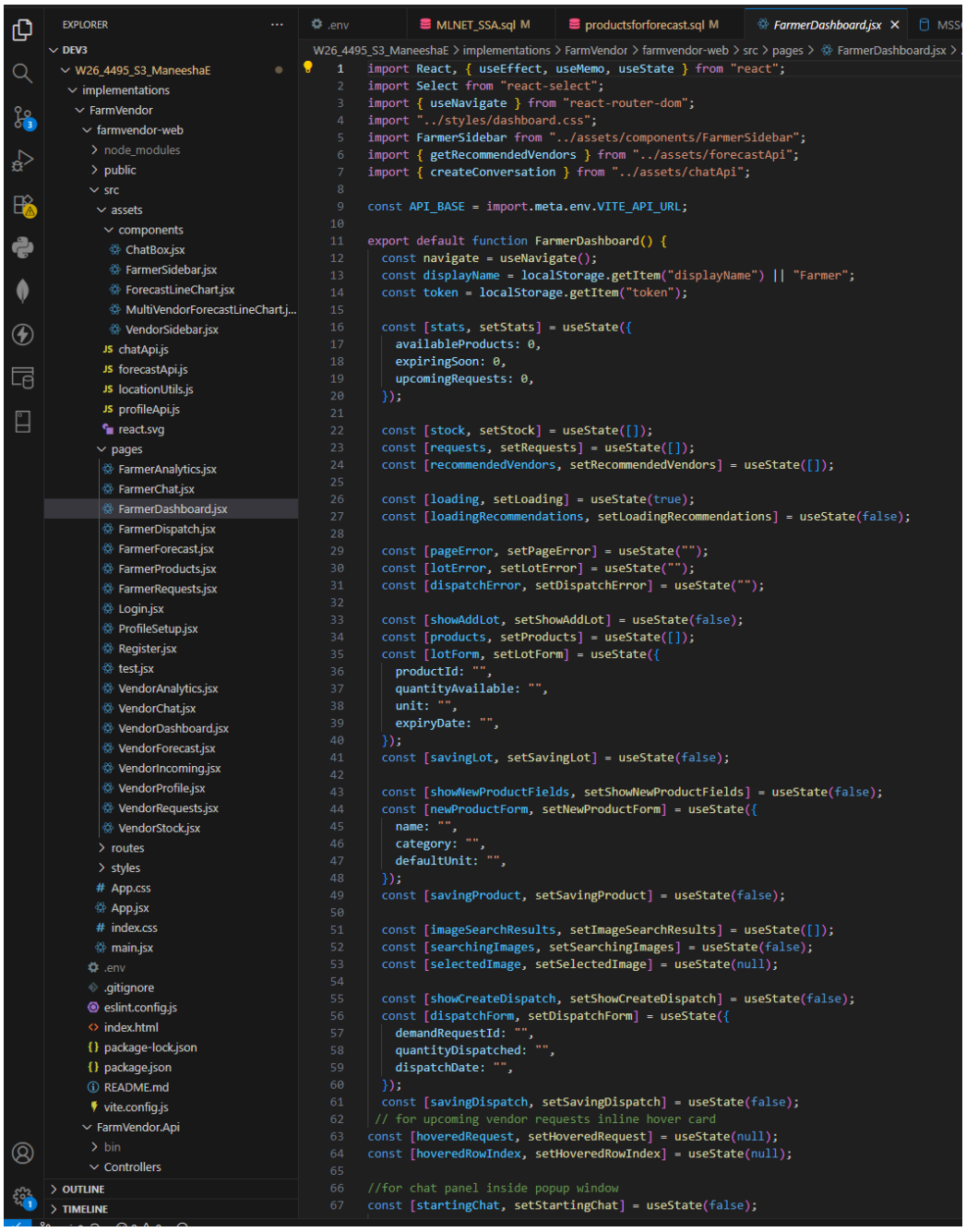


- Distance shown in request details



Frontend

- *FarmerDashboard.jsx*




MLNET_SSA.sql M
productsforforecast.sql M
FarmerDashboard.jsx
MSSQL: Welc
4495_S3_ManeeshaE > implementations > FarmVendor > farmvendor-web > src > pages > FarmerDashboard.jsx > Farme
export default function FarmerDashboard() {
const loadDashboard = async () => {
setStats(JSON.parse(statsText));
setStock(JSON.parse(stockText));
setRequests(JSON.parse(reqText));
};
const loadRecommendedVendors = async () => {
setLoadingRecommendations(true);
try {
const data = await getRecommendedVendors(
new Date().toISOString().slice(0, 10),
token
);
setRecommendedVendors(data);
} catch (e) {
setPageError(e?.message || "Failed to load recommended vendors");
} finally {
setLoadingRecommendations(false);
}
};
const searchProductImages = async () => {
setLotError("");
if (!newProductForm.name.trim()) {
return setLotError("Enter product name before searching images.");
}
setSearchingImages(true);
try {
const res = await fetch(
`${API_BASE}/api/products/search-images?query=${encodeURIComponent(
newProductForm.name.trim()
)}`,
{ headers: authHeaders }
);
const text = await res.text();
if (!res.ok) throw new Error(text || "Failed to search images.");
const data = JSON.parse(text);
setImageSearchResults(Array.isArray(data) ? data : []);
} catch (err) {
setLotError(err?.message || "Failed to search images.");
} finally {
setSearchingImages(false);
}
};


env
MLNET_SSA.sql M
productsforforecast.sql M
FarmerDashboard.jsx
MSSQL: Welcon
6_4495_S3_ManeeshaE > implementations > FarmVendor > farmvendor-web > src > pages > FarmerDashboard.jsx > FarmerD
export default function FarmerDashboard() {
<div style={{ marginTop: 8, fontSize: 12, color: "#6b7280" }}>
Requests loaded: {requests.length}

</section>
<section style={{ marginTop: 14 }}>

<h2>Top Vendors for Dispatching</h2>
<button
className="btn btn-secondary"
onClick={loadRecommendedVendors}
disabled={loadingRecommendations}
>
{loadingRecommendations ? "Loading..." : "Refresh"}
</button>

<table className="table">
<thead>
<tr>
<th>Vendor</th>
<th>Products</th>
<th>Match Qty</th>
<th>Distance (km)</th>
<th>Address</th>
<th>Relationship</th>
<th>Score</th>
</tr>
</thead>
<tbody>
{recommendedVendors.length === 0 ? (
<tr>
<td colSpan="6">No recommended vendors found.</td>
</tr>
) : (
recommendedVendors.map((v) => (
<tr key={v.vendorId}>
<td>{v.vendorName}</td>
<td>{v.matchedProducts?.join(", ")}</td>
<td>{v.matchableQuantity}</td>
<td>{v.distanceKm}</td>
<td>
{[v.vendorCity, v.vendorProvince, v.vendorPostalCode]
.filter(Boolean)
.join(", ") || "N/A"}
</td>
<td>{v.relationshipScore}</td>
<td>{v.finalScore}</td>
</tr>
))
)}
</tbody>
</table>

</section>

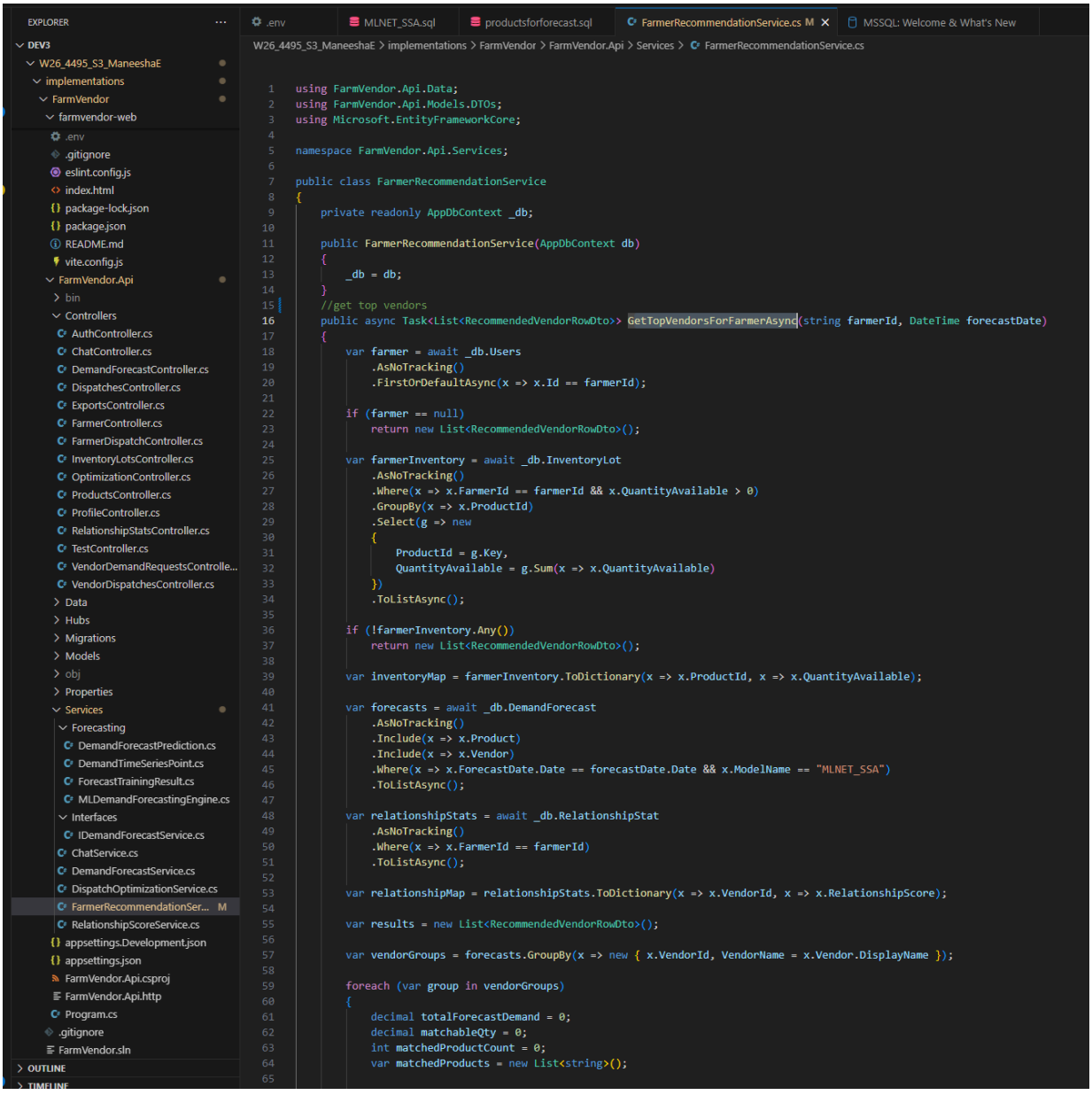
</main>

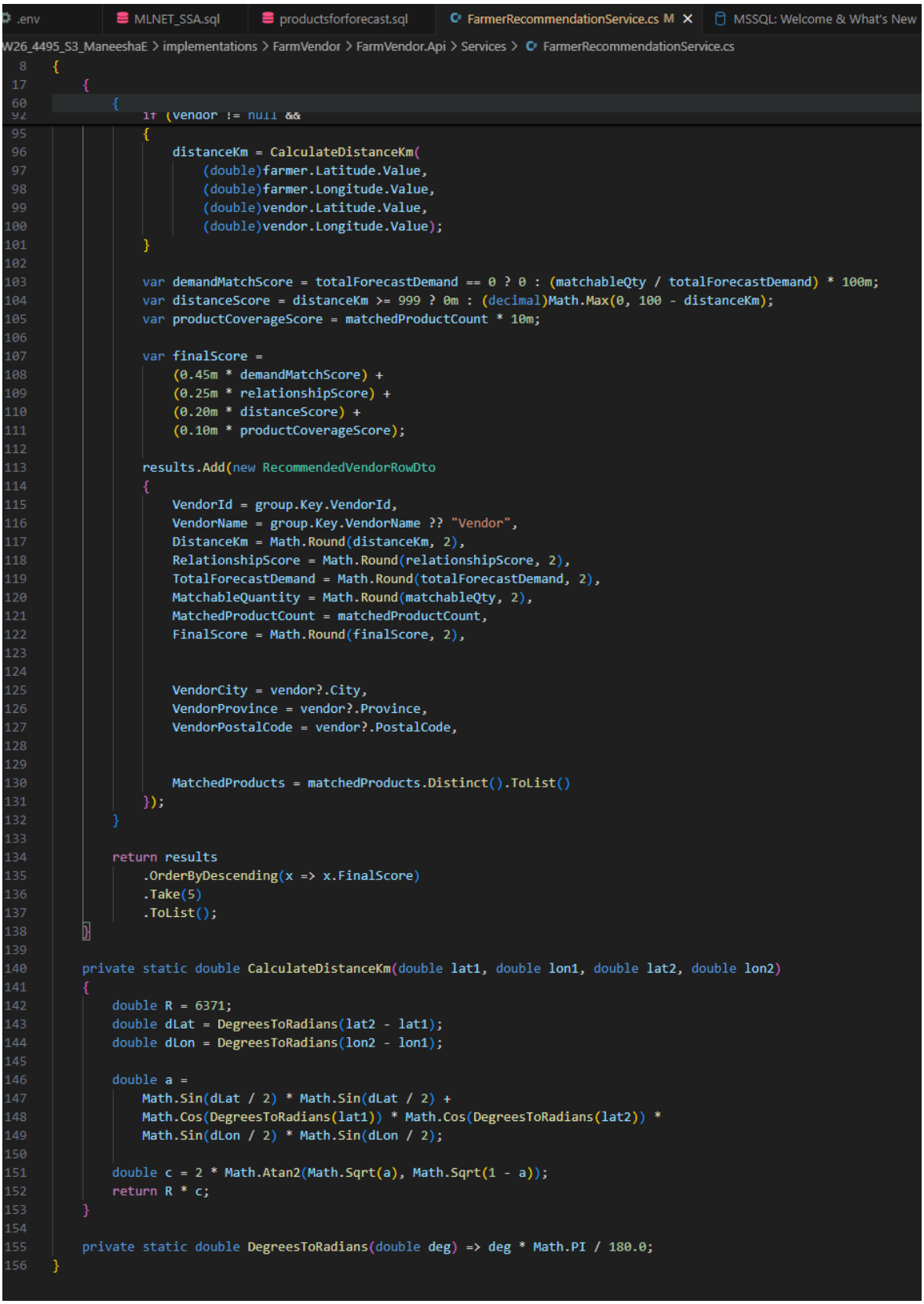
- upcoming vendor requests inline hover card

```jsx
<div className="card">
  <div className="card-header">
    <h2>Upcoming Vendor Requests</h2>
    <button className="btn btn-secondary" onClick={loadDashboard}>
      Refresh
    </button>
  </div>

  <div className="card-body">
    <table className="table">
      <thead>
        <tr>
          <th>Product</th>
          <th>Qty</th>
          <th>Needed By</th>
        </tr>
      </thead>
      <tbody>
        {requests.length === 0 ? (
          <tr>
            <td colSpan="3">No open demand requests found.</td>
          </tr>
        ) : (
          requests.map((row, idx) => (
            <React.Fragment key={row.demandRequestId ?? idx}>
              <tr
                className="request-hover-row"
                onMouseEnter={() => handleRequestMouseEnter(row, idx)}
                onMouseLeave={handleRequestMouseLeave}
              >
                <td>{renderProductCell(row)}</td>
                <td>
                  {row.qty} {row.unit}
                </td>
                <td>{new Date(row.neededBy).toISOString().slice(0, 10)}</td>
              </tr>

              {hoveredRowIndex === idx && hoveredRequest && (
                <tr
                  className="request-hover-inline-row"
                  onMouseEnter={() => handleRequestMouseEnter(row, idx)}
                  onMouseLeave={handleRequestMouseLeave}
                >
                  <td colSpan="3">
                    <div className="request-hover-inline-card">
                      <div className="request-hover-card-header">
                        <div className="request-hover-card-title">Vendor Details</div>
                      </div>

                      <div className="request-hover-card-body">
                        <div className="request-hover-product">
                          {hoveredRequest.imageThumbUrl || hoveredRequest.imageUrl ? (
                            <img
                              src={hoveredRequest.imageThumbUrl || hoveredRequest.imageUrl}
                              alt={hoveredRequest.product}
                              className="request-hover-thumb"
                            />
                          ) : (
                            <div className="request-hover-placeholder">
                              {hoveredRequest.product?.charAt(0)?.toUpperCase() || "P"}
                            </div>
                          )}

                          <div>
                            <div className="request-hover-product-name">
```

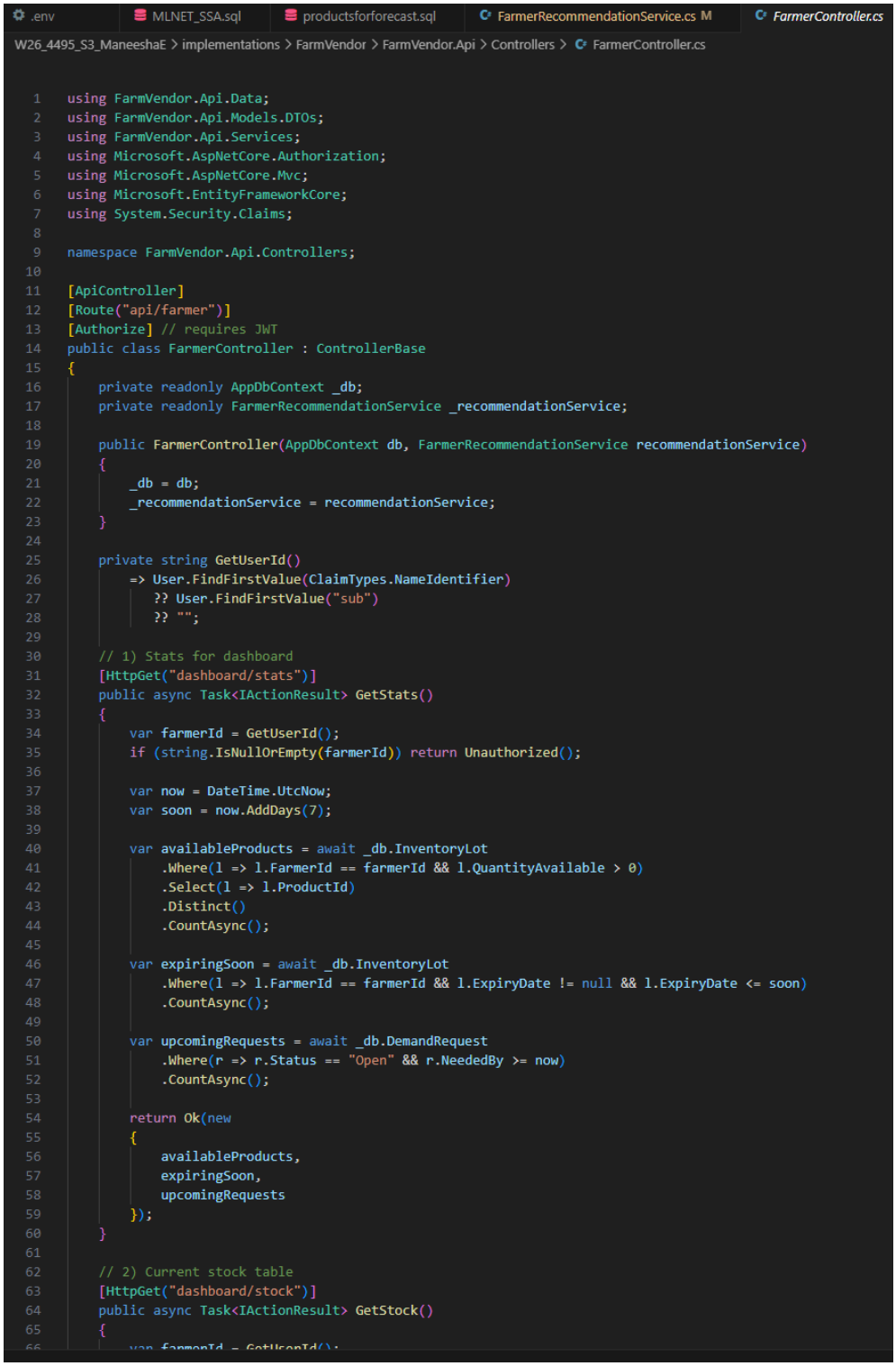
Backend

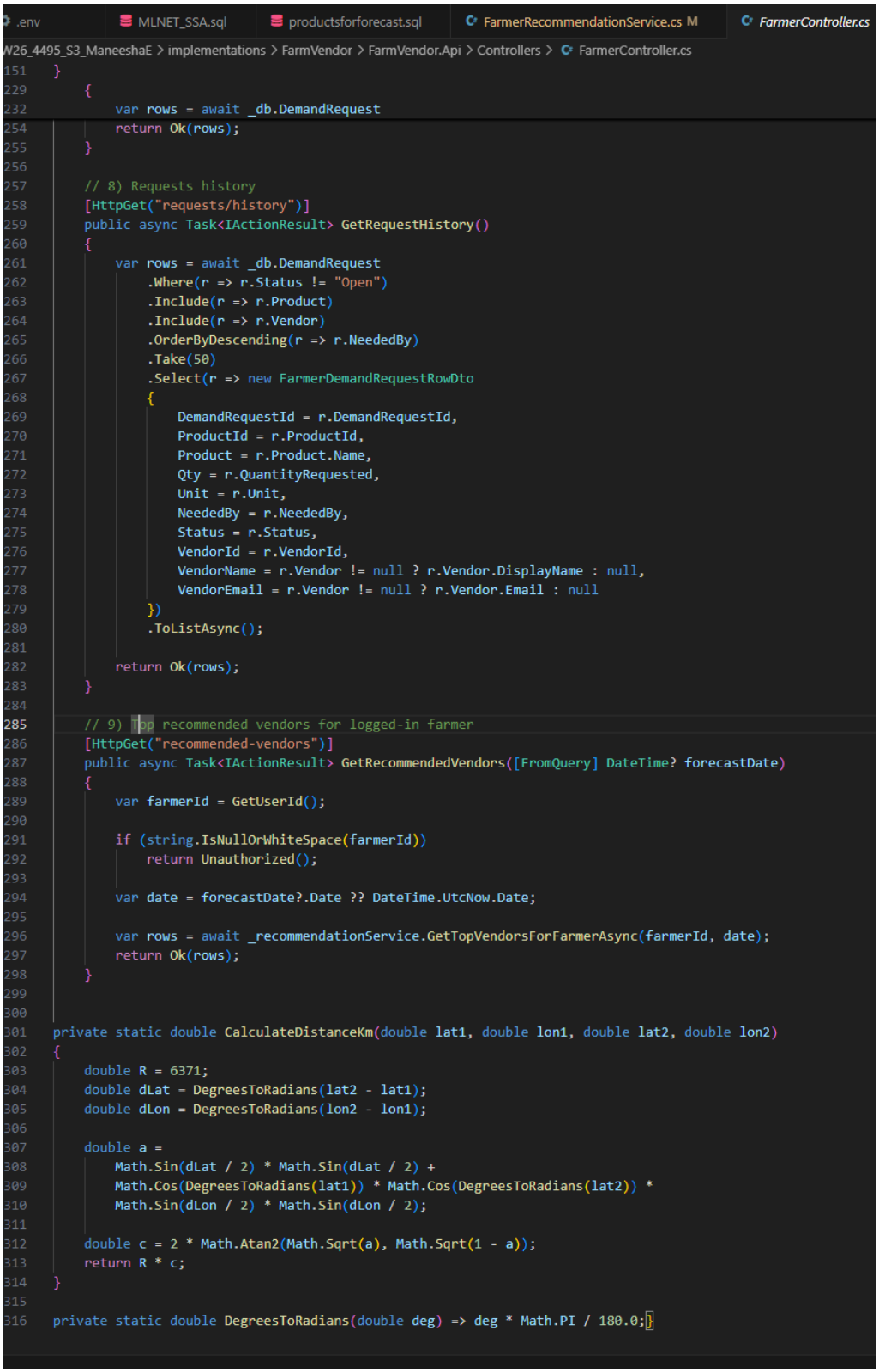
- *FarmerRecommendationService.cs*



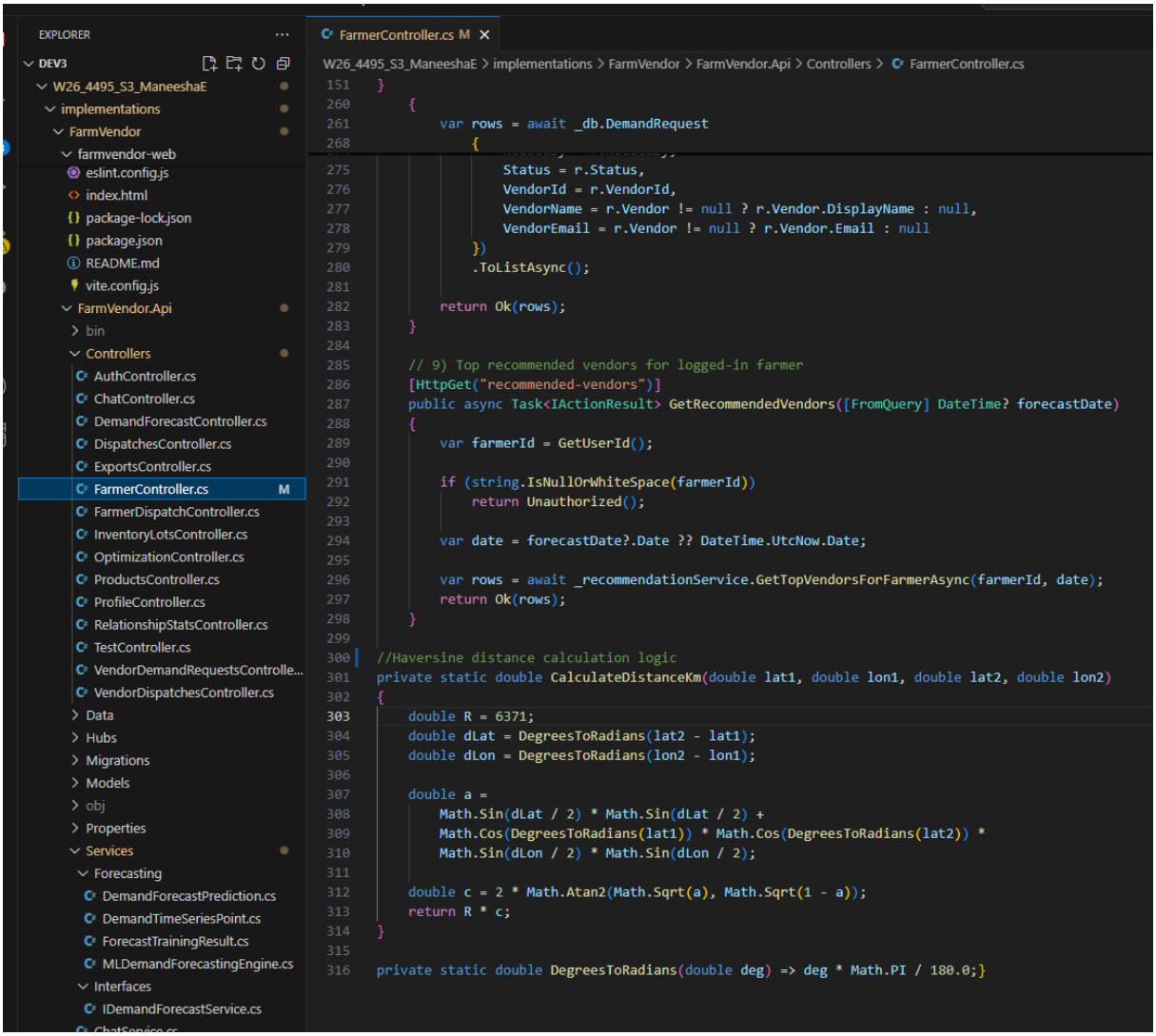

.env
MLNET_SSA.sql
productsforforecast.sql
FarmerRecommendationService.cs M
MSSQL: Welcome & What's New
W26_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Services > FarmerRecommendationService.cs
{
{
{
if (vendor != null &&
{
distanceKm = CalculateDistanceKm(
(double)farmer.Latitude.Value,
(double)farmer.Longitude.Value,
(double)vendor.Latitude.Value,
(double)vendor.Longitude.Value);
}
var demandMatchScore = totalForecastDemand == 0 ? 0 : (matchableQty / totalForecastDemand) * 100m;
var distanceScore = distanceKm >= 999 ? 0m : (decimal)Math.Max(0, 100 - distanceKm);
var productCoverageScore = matchedProductCount * 10m;
var finalScore =
(0.45m * demandMatchScore) +
(0.25m * relationshipScore) +
(0.20m * distanceScore) +
(0.10m * productCoverageScore);
results.Add(new RecommendedVendorRowDto
{
VendorId = group.Key.VendorId,
VendorName = group.Key.VendorName ?? "Vendor",
DistanceKm = Math.Round(distanceKm, 2),
RelationshipScore = Math.Round(relationshipScore, 2),
TotalForecastDemand = Math.Round(totalForecastDemand, 2),
MatchableQuantity = Math.Round(matchableQty, 2),
MatchedProductCount = matchedProductCount,
FinalScore = Math.Round(finalScore, 2),
VendorCity = vendor?.City,
VendorProvince = vendor?.Province,
VendorPostalCode = vendor?.PostalCode,
MatchedProducts = matchedProducts.Distinct().ToList()
});
}
return results
.OrderByDescending(x => x.FinalScore)
.Take(5)
.ToList();
}
private static double CalculateDistanceKm(double lat1, double lon1, double lat2, double lon2)
{
double R = 6371;
double dLat = DegreesToRadians(lat2 - lat1);
double dLon = DegreesToRadians(lon2 - lon1);
double a =
Math.Sin(dLat / 2) * Math.Sin(dLat / 2) +
Math.Cos(DegreesToRadians(lat1)) * Math.Cos(DegreesToRadians(lat2)) *
Math.Sin(dLon / 2) * Math.Sin(dLon / 2);
double c = 2 * Math.Atan2(Math.Sqrt(a), Math.Sqrt(1 - a));
return R * c;
}
private static double DegreesToRadians(double deg) => deg * Math.PI / 180.0;
}

- *FarmerController.cs*




.env
MLNET_SSA.sql
productsforforecast.sql
FarmerRecommendationService.cs M
FarmerController.cs
W26_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Controllers > FarmerController.cs
151 }
229 {
232 var rows = await _db.DemandRequest
254 return Ok(rows);
255 }
256
257 // 8) Requests history
258 [HttpGet("requests/history")]
259 public async Task<IActionResult> GetRequestHistory()
260 {
261 var rows = await _db.DemandRequest
262 .Where(r => r.Status != "Open")
263 .Include(r => r.Product)
264 .Include(r => r.Vendor)
265 .OrderByDescending(r => r.NeededBy)
266 .Take(50)
267 .Select(r => new FarmerDemandRequestRowDto
268 {
269 DemandRequestId = r.DemandRequestId,
270 ProductId = r.ProductId,
271 Product = r.Product.Name,
272 Qty = r.QuantityRequested,
273 Unit = r.Unit,
274 NeededBy = r.NeededBy,
275 Status = r.Status,
276 VendorId = r.VendorId,
277 VendorName = r.Vendor != null ? r.Vendor.DisplayName : null,
278 VendorEmail = r.Vendor != null ? r.Vendor.Email : null
279 })
280 .ToListAsync();
281
282 return Ok(rows);
283 }
284
285 // 9) Top recommended vendors for logged-in farmer
286 [HttpGet("recommended-vendors")]
287 public async Task<IActionResult> GetRecommendedVendors([FromQuery] DateTime? forecastDate)
288 {
289 var farmerId = GetUserId();
290
291 if (string.IsNullOrWhiteSpace(farmerId))
292 return Unauthorized();
293
294 var date = forecastDate?.Date ?? DateTime.UtcNow.Date;
295
296 var rows = await _recommendationService.GetTopVendorsForFarmerAsync(farmerId, date);
297 return Ok(rows);
298 }
299
300
301 private static double CalculateDistanceKm(double lat1, double lon1, double lat2, double lon2)
302 {
303 double R = 6371;
304 double dLat = DegreesToRadians(lat2 - lat1);
305 double dLon = DegreesToRadians(lon2 - lon1);
306
307 double a =
308 Math.Sin(dLat / 2) * Math.Sin(dLat / 2) +
309 Math.Cos(DegreesToRadians(lat1)) * Math.Cos(DegreesToRadians(lat2)) *
310 Math.Sin(dLon / 2) * Math.Sin(dLon / 2);
311
312 double c = 2 * Math.Atan2(Math.Sqrt(a), Math.Sqrt(1 - a));
313 return R * c;
314 }
315
316 private static double DegreesToRadians(double deg) => deg * Math.PI / 180.0;}

- Haversine formula applied in FarmerController.cs



- *RelationshipScoreService.cs*

RelationshipScoreService.cs

W26_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Services > RelationshipScoreService.cs

```
using FarmVendor.Api.Data;
using FarmVendor.Api.Models;
using Microsoft.EntityFrameworkCore;

namespace FarmVendor.Api.Services;

public class RelationshipScoreService
{
    private readonly AppDbContext _db;

    public RelationshipScoreService(AppDbContext db)
    {
        _db = db;
    }

    public async Task UpdateRelationshipScoreAsync(string farmerId, string vendorId)
    {
        if (string.IsNullOrWhiteSpace(farmerId) || string.IsNullOrWhiteSpace(vendorId))
            return;

        // All dispatches between this farmer and vendor
        var dispatches = await _db.Dispatch
            .AsNoTracking()
            .Include(d => d.DemandRequest)
            .Where(d => d.FarmerId == farmerId && d.VendorId == vendorId)
            .ToListAsync();

        if (dispatches.Count == 0)
            return;

        int totalDispatches = dispatches.Count;
        int deliveredCount = dispatches.Count(d => d.DeliveryStatus == "Delivered");
        int cancelledCount = dispatches.Count(d => d.DeliveryStatus == "Cancelled");

        int onTimeDeliveredCount = dispatches.Count(d =>
            d.DeliveryStatus == "Delivered" &&
            d.DemandRequest != null &&
            d.DispatchDate <= d.DemandRequest.NeededBy);

        decimal totalRequestedQty = dispatches
            .Where(d => d.DemandRequest != null)
            .Sum(d => d.DemandRequest!.QuantityRequested);

        decimal totalDeliveredQty = dispatches
            .Where(d => d.DeliveryStatus == "Delivered")
            .Sum(d => d.QuantityDispatched);

        decimal onTimeRate = deliveredCount == 0
            ? 0
            : (decimal)onTimeDeliveredCount / deliveredCount;

        decimal fulfillmentRate = totalRequestedQty == 0
            ? 0
            : totalDeliveredQty / totalRequestedQty;

        decimal cancellationRate = totalDispatches == 0
            ? 0
            : (decimal)cancelledCount / totalDispatches;

        // Weighted score out of 100
        // need to adjust weights later for experiments
        decimal score =
            (onTimeRate * 50m) +         // 50%
            (fulfillmentRate * 40m) +    // 40%
            ((1 - cancellationRate) * 10m); // 10%
```

RelationshipScoreService.cs

W26_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Services > RelationshipScoreService.cs

```
8   {
17      {
35          int onTimeDeliveredCount = dispatches.Count(d =>
37              d.DemandRequest != null &&
38              d.DispatchDate <= d.DemandRequest.NeededBy);
39
40          decimal totalRequestedQty = dispatches
41              .Where(d => d.DemandRequest != null)
42              .Sum(d => d.DemandRequest!.QuantityRequested);
43
44          decimal totalDeliveredQty = dispatches
45              .Where(d => d.DeliveryStatus == "Delivered")
46              .Sum(d => d.QuantityDispatched);
47
48          decimal onTimeRate = deliveredCount == 0
49              ? 0
50              : (decimal)onTimeDeliveredCount / deliveredCount;
51
52          decimal fulfillmentRate = totalRequestedQty == 0
53              ? 0
54              : totalDeliveredQty / totalRequestedQty;
55
56          decimal cancellationRate = totalDispatches == 0
57              ? 0
58              : (decimal)cancelledCount / totalDispatches;
59
60          // Weighted score out of 100
61          // need to adjust weights later for experiments
62          decimal score =
63              (onTimeRate * 50m) +         // 50%
64              (fulfillmentRate * 40m) +    // 40%
65              ((1 - cancellationRate) * 10m); // 10%
66
67          score = Math.Round(score, 2);
68
69          var existing = await _db.RelationshipStat
70              .FirstOrDefaultAsync(r => r.FarmerId == farmerId && r.VendorId == vendorId);
71
72          if (existing == null)
73          {
74              existing = new RelationshipStat
75              {
76                  FarmerId = farmerId,
77                  VendorId = vendorId
78              };
79              _db.RelationshipStat.Add(existing);
80          }
81
82          existing.TotalDispatches = totalDispatches;
83          existing.DeliveredCount = deliveredCount;
84          existing.CancelledCount = cancelledCount;
85          existing.OnTimeDeliveredCount = onTimeDeliveredCount;
86          existing.TotalRequestedQty = Math.Round(totalRequestedQty, 2);
87          existing.TotalDeliveredQty = Math.Round(totalDeliveredQty, 2);
88          existing.RelationshipScore = score;
89          existing.LastUpdatedAt = DateTime.UtcNow;
90
91          await _db.SaveChangesAsync();
92      }
93  }
```

Database queries

- RelationshipStat table

```
5
6   SELECT COLUMN_NAME
7   FROM INFORMATION_SCHEMA.COLUMNS
8   WHERE TABLE_NAME = 'RelationshipStat';
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

esults (1) Messages

COLUMN_NAME
RelationshipStatId
FarmerId
VendorId
SuccessfulDispatchCount
CancelledDispatchCount
LastDispatchDate
UpdatedAt
CancelledCount
DeliveredCount
LastUpdatedAt
OnTimeDeliveredCount
RelationshipScore
TotalDeliveredQty
TotalDispatches
TotalRequestedQty

Feature 3: Realtime chatmodule

This feature enables real-time communication between farmers and vendors, supporting quick coordination, clarification, and negotiation during dispatch and fulfillment.

This feature includes:

- Create conversation
- Conversation list
- Real-time message sending
- Persistent message history
- Farmer and Vendor chat pages

The design consists of:

- SignalR-based live communication
- conversation-based organization
- stored message history
- authenticated hub connection

Implementation

Frontend

- FarmerDashboard.jsx
- VendorChat.jsx
- FarmerChat.jsx
- ChatBox.jsx
- chatApi.js

Backend

- ChatController.cs
- ChatHub.cs
- ChatService.cs

Database

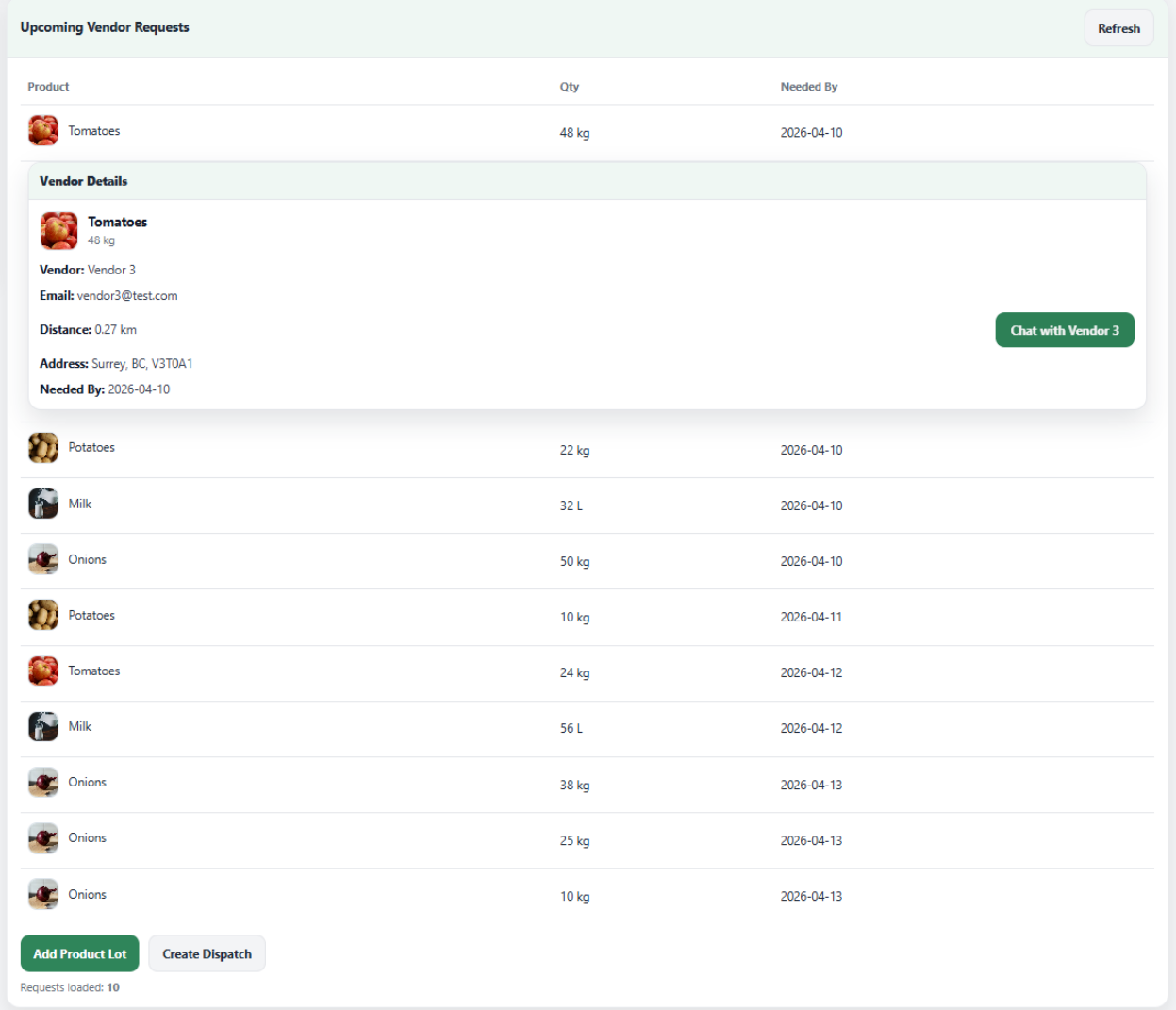
- Conversation table storing:
    - FarmerId
    - VendorId
    - CreatedAt
- ChatMessage table storing:
    - ConversationId
    - SenderId
    - SenderRole
    - MessageText
    - SentAt
    - IsRead

Technical Methods

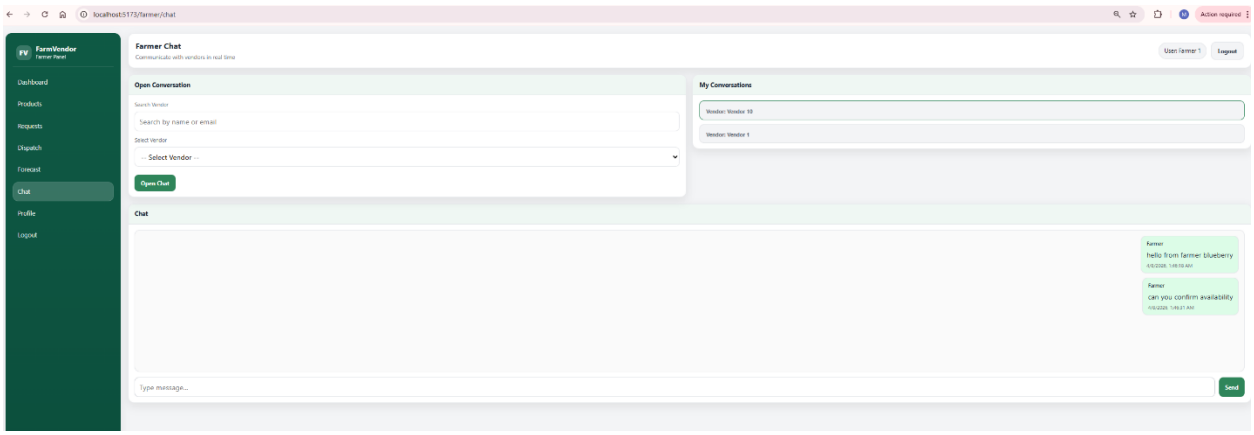
- SignalR WebSocket / fallback transport
- JWT-secured hub authentication
- real-time broadcast using conversation groups

App

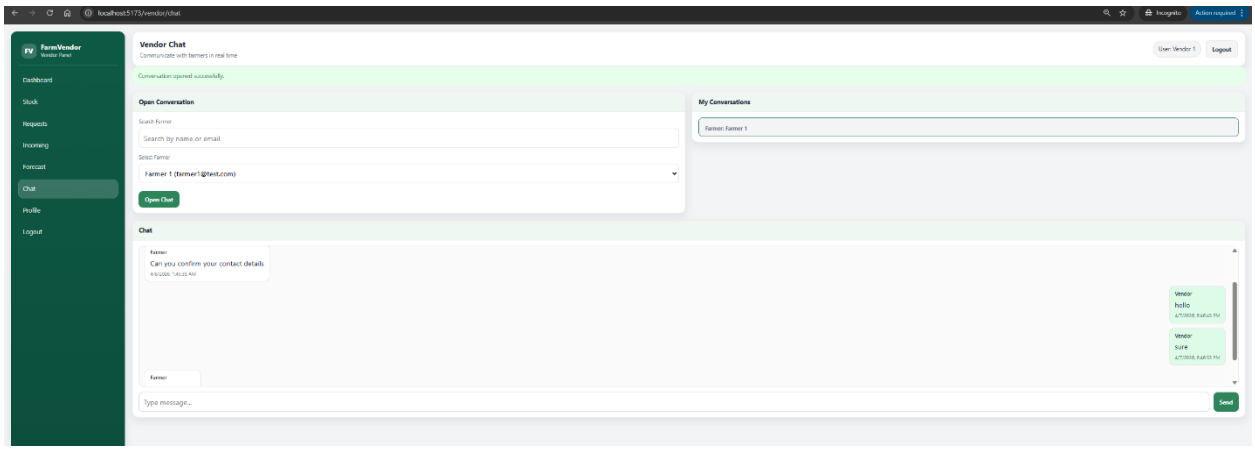
- farmerDashboard



- Farmer chat panel

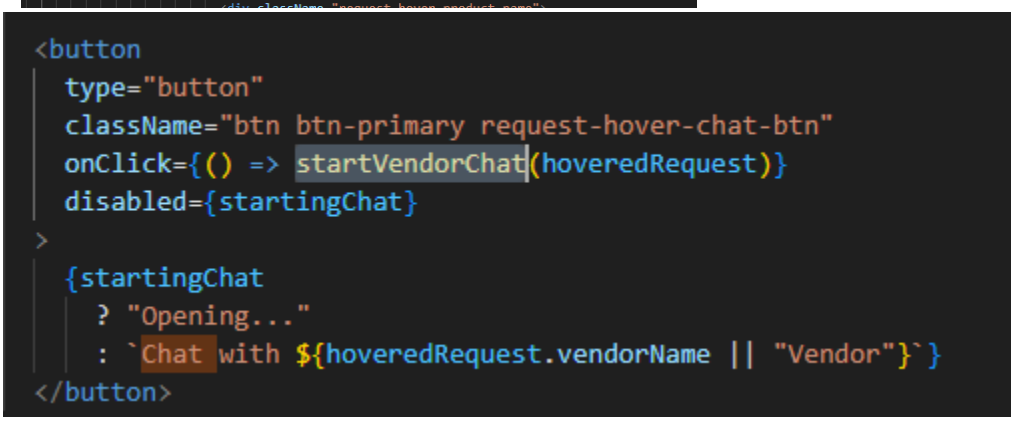


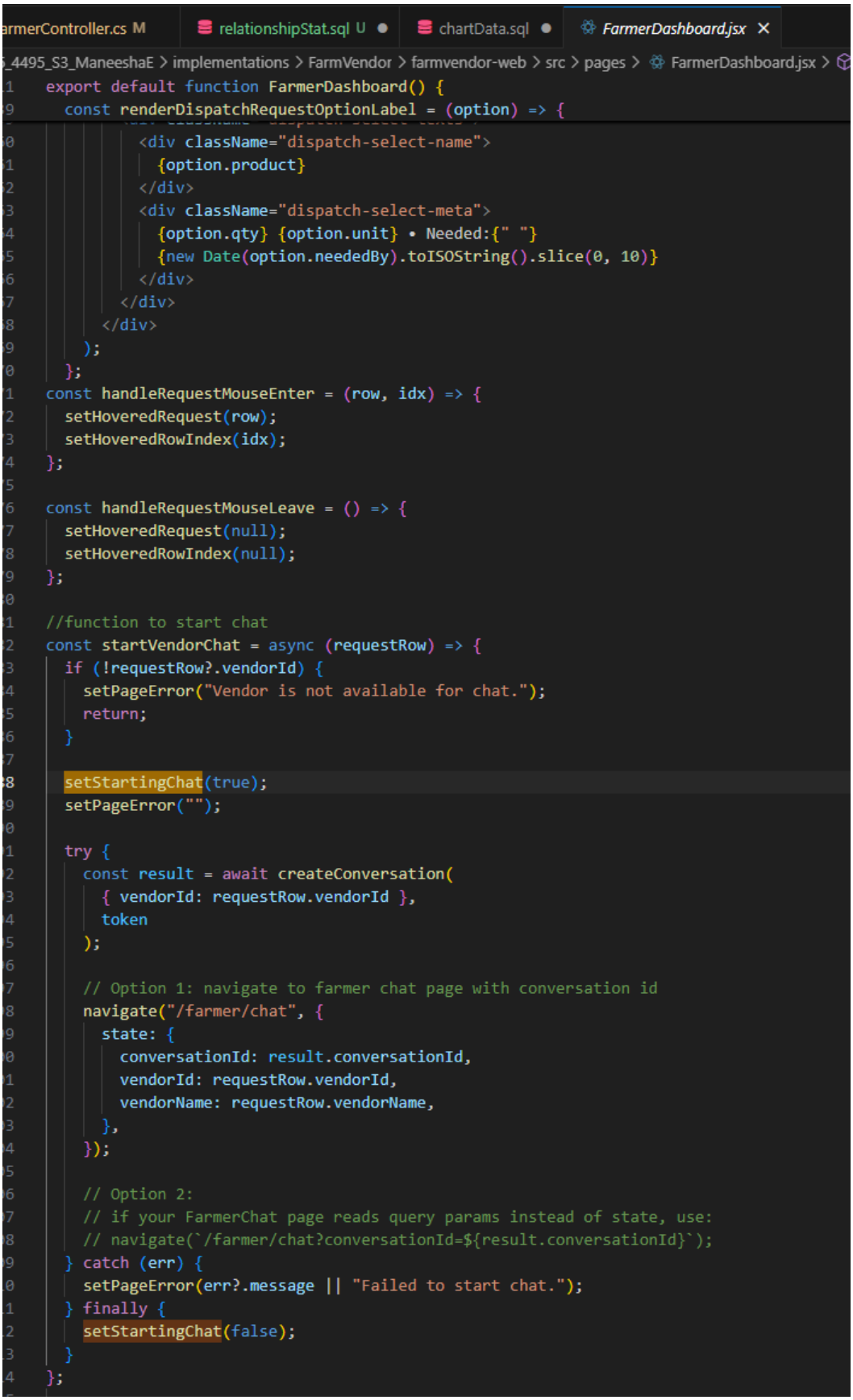
- Vendor chat panel



Frontend

- FarmerDashboard.jsx




armerController.cs M
relationshipStat.sql U
chartData.sql
FarmerDashboard.jsx
_4495_S3_ManeeshaE > implementations > FarmVendor > farmvendor-web > src > pages > FarmerDashboard.jsx
export default function FarmerDashboard() {
const renderDispatchRequestOptionLabel = (option) => {

{option.product}

{option.qty} {option.unit} • Needed:{" "}
{new Date(option.neededBy).toISOString().slice(0, 10)}

);
};
const handleRequestMouseEnter = (row, idx) => {
setHoveredRequest(row);
setHoveredRowIndex(idx);
};
const handleRequestMouseLeave = () => {
setHoveredRequest(null);
setHoveredRowIndex(null);
};
//function to start chat
const startVendorChat = async (requestRow) => {
if (!requestRow?.vendorId) {
setPageError("Vendor is not available for chat.");
return;
}
setStartingChat(true);
setPageError("");
try {
const result = await createConversation(
{ vendorId: requestRow.vendorId },
token
);
// Option 1: navigate to farmer chat page with conversation id
navigate("/farmer/chat", {
state: {
conversationId: result.conversationId,
vendorId: requestRow.vendorId,
vendorName: requestRow.vendorName,
},
});
// Option 2:
// if your FarmerChat page reads query params instead of state, use:
// navigate(`/farmer/chat?conversationId=${result.conversationId}`);
} catch (err) {
setPageError(err?.message || "Failed to start chat.");
} finally {
setStartingChat(false);
}
};

- FarmerChat.jsx

FarmerController.cs M | relationshipStat.sql U | chartData.sql M | FarmerChat.jsx

/26_4495_S3_ManeeshaE > implementations > FarmVendor > farmvendor-web > src > pages > FarmerChat.jsx > ...

```jsx
import React, { useEffect, useState } from "react";
import { useNavigate } from "react-router-dom";
import "../styles/dashboard.css";
import ChatBox from "../assets/components/ChatBox";
import FarmerSidebar from "../assets/components/FarmerSidebar";
import {
  createConversation,
  getMyConversations,
  getChatUsers,
} from "../assets/chatApi";

export default function FarmerChat() {
  const navigate = useNavigate();
  const token = localStorage.getItem("token");
  const displayName = localStorage.getItem("displayName") || "Farmer";
  const currentUserId = localStorage.getItem("userId") || "";

  const [error, setError] = useState("");
  const [success, setSuccess] = useState("");
  const [loading, setLoading] = useState(true);

  const [conversations, setConversations] = useState([]);
  const [selectedConversationId, setSelectedConversationId] = useState(null);

  const [vendors, setVendors] = useState([]);
  const [vendorSearch, setVendorSearch] = useState("");
  const [selectedVendorId, setSelectedVendorId] = useState("");

  const logout = () => {
    localStorage.clear();
    navigate("/login");
  };

  const loadConversations = async () => {
    const data = await getMyConversations(token);
    setConversations(data);
    if (data.length > 0 && !selectedConversationId) {
      setSelectedConversationId(data[0].conversationId);
    }
  };

  const loadVendors = async (search = "") => {
    const data = await getChatUsers("Vendor", token, search);
    setVendors(data);
  };

  const handleCreateConversation = async (e) => {
    e.preventDefault();
    setError("");
    setSuccess("");

    if (!selectedVendorId) {
      setError("Please select a vendor.");
      return;
    }

    try {
      const result = await createConversation(
        { vendorId: selectedVendorId },
        token
      );
```

- VendorChat.jsx

FarmerController.cs M | relationshipStat.sql U | chartData.sql M | VendorChat.jsx

26_4495_S3_ManeeshaE > implementations > FarmVendor > farmvendor-web > src > pages > VendorCh

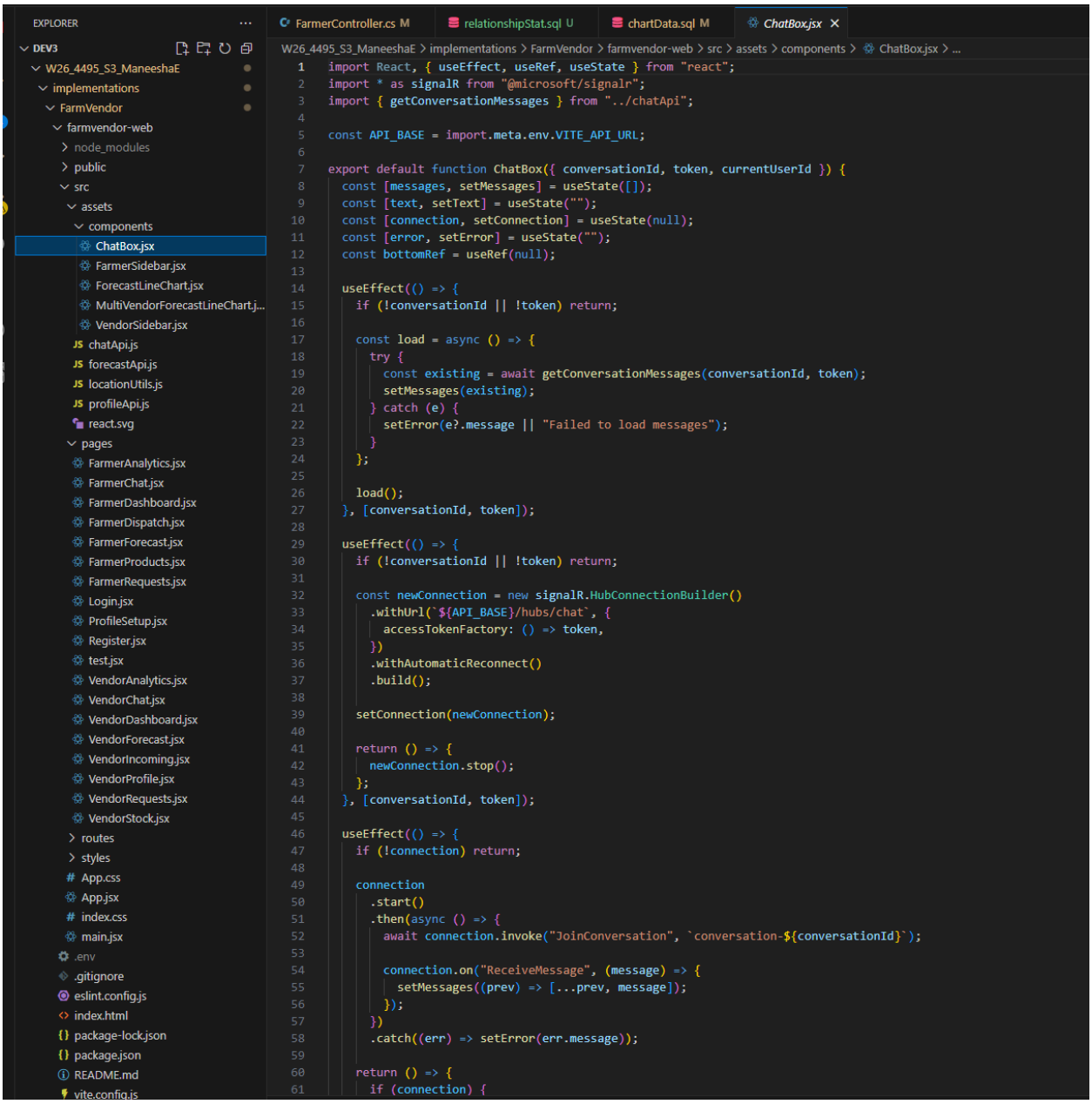
```jsx
import React, { useEffect, useState } from "react";
import { useNavigate } from "react-router-dom";
import "../styles/dashboard.css";
import ChatBox from "../assets/components/ChatBox";
import VendorSidebar from "../assets/components/VendorSidebar";
import {
  createConversation,
  getMyConversations,
  getChatUsers,
} from "../assets/chatApi";

export default function VendorChat() {
  const navigate = useNavigate();
  const token = localStorage.getItem("token");
  const displayName = localStorage.getItem("displayName") || "Vendor";
  const currentUserId = localStorage.getItem("userId") || "";

  const [error, setError] = useState("");
  const [success, setSuccess] = useState("");
  const [loading, setLoading] = useState(true);

  const [conversations, setConversations] = useState([]);
  const [selectedConversationId, setSelectedConversationId] = useState(null)

  const [farmers, setFarmers] = useState([]);
  const [farmerSearch, setFarmerSearch] = useState("");
  const [selectedFarmerId, setSelectedFarmerId] = useState("");

  const logout = () => {
    localStorage.clear();
    navigate("/login");
  };

  const loadConversations = async () => {
    const data = await getMyConversations(token);
    setConversations(data);

    if (data.length > 0 && !selectedConversationId) {
      setSelectedConversationId(data[0].conversationId);
    }
  };

  const loadFarmers = async (search = "") => {
    const data = await getChatUsers("Farmer", token, search);
    setFarmers(data);
  };

  const handleCreateConversation = async (e) => {
    e.preventDefault();
    setError("");
    setSuccess("");

    if (!selectedFarmerId) {
      setError("Please select a farmer.");
      return;
    }

    try {
      const result = await createConversation(
```

FarmerController.cs M | relationshipStat.sql U | chartData.sql M | chatApi.js

26_4495_S3_ManeeshaE > implementations > FarmVendor > farmvendor-web > src > assets > chatApi.js > ...

```
const API_BASE = import.meta.env.VITE_API_URL;

export async function createConversation(payload, token) {
  const res = await fetch(`${API_BASE}/api/chat/conversation`, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      Authorization: `Bearer ${token}`,
    },
    body: JSON.stringify(payload),
  });

  const text = await res.text();
  if (!res.ok) throw new Error(text || "Failed to create conversation");
  return JSON.parse(text);
}

export async function getMyConversations(token) {
  const res = await fetch(`${API_BASE}/api/chat/conversations`, {
    headers: {
      Authorization: `Bearer ${token}`,
    },
  });

  const text = await res.text();
  if (!res.ok) throw new Error(text || "Failed to load conversations");
  return JSON.parse(text);
}

export async function getConversationMessages(conversationId, token) {
  const res = await fetch(`${API_BASE}/api/chat/messages/${conversationId}`, {
    headers: {
      Authorization: `Bearer ${token}`,
    },
  });

  const text = await res.text();
  if (!res.ok) throw new Error(text || "Failed to load messages");
  return JSON.parse(text);
}

export async function getChatUsers(role, token, search = "") {
  const url = new URL(`${API_BASE}/api/chat/users`);
  url.searchParams.append("role", role);
  if (search.trim()) {
    url.searchParams.append("search", search.trim());
  }

  const res = await fetch(url, {
    headers: {
      Authorization: `Bearer ${token}`,
    },
  });

  const text = await res.text();
  if (!res.ok) throw new Error(text || "Failed to load users");
  return JSON.parse(text);
}
```

- ChatBox.jsx


EXPLORER
DEV3
W26_4495_S3_ManeeshaE
implementations
FarmVendor
farmvendor-web
node_modules
public
src
assets
components
ChatBox.jsx
FarmerSidebar.jsx
ForecastLineChart.jsx
MultiVendorForecastLineChart.j...
VendorSidebar.jsx
chatApi.js
forecastApi.js
locationUtils.js
profileApi.js
react.svg
pages
FarmerAnalytics.jsx
FarmerChat.jsx
FarmerDashboard.jsx
FarmerDispatch.jsx
FarmerForecast.jsx
FarmerProducts.jsx
FarmerRequests.jsx
Login.jsx
ProfileSetup.jsx
Register.jsx
test.jsx
VendorAnalytics.jsx
VendorChat.jsx
VendorDashboard.jsx
VendorForecast.jsx
VendorIncoming.jsx
VendorProfile.jsx
VendorRequests.jsx
VendorStock.jsx
routes
styles
App.css
App.jsx
index.css
main.jsx
.env
.gitignore
eslint.config.js
index.html
package-lock.json
package.json
README.md
vite.config.js
FarmerController.cs M
relationshipStat.sql U
chartData.sql M
ChatBox.jsx
W26_4495_S3_ManeeshaE > implementations > FarmVendor > farmvendor-web > src > assets > components > ChatBox.jsx > ...
import React, { useEffect, useRef, useState } from "react";
import * as signalR from "@microsoft/signalr";
import { getConversationMessages } from "../chatApi";
const API_BASE = import.meta.env.VITE_API_URL;
export default function ChatBox({ conversationId, token, currentUserId }) {
const [messages, setMessages] = useState([]);
const [text, setText] = useState("");
const [connection, setConnection] = useState(null);
const [error, setError] = useState("");
const bottomRef = useRef(null);
useEffect(() => {
if (!conversationId || !token) return;
const load = async () => {
try {
const existing = await getConversationMessages(conversationId, token);
setMessages(existing);
} catch (e) {
setError(e?.message || "Failed to load messages");
}
};
load();
}, [conversationId, token]);
useEffect(() => {
if (!conversationId || !token) return;
const newConnection = new signalR.HubConnectionBuilder()
.withUrl(`${API_BASE}/hubs/chat`, {
accessTokenFactory: () => token,
})
.withAutomaticReconnect()
.build();
setConnection(newConnection);
return () => {
newConnection.stop();
};
}, [conversationId, token]);
useEffect(() => {
if (!connection) return;
connection
.start()
.then(async () => {
await connection.invoke("JoinConversation", `conversation-${conversationId}`);
connection.on("ReceiveMessage", (message) => {
setMessages((prev) => [...prev, message]);
});
})
.catch((err) => setError(err.message));
return () => {
if (connection) {

Backend

- ChatController.cs

ChatController.cs

W26_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Controllers > ChatController.cs

```
using FarmVendor.Api.Data;
using FarmVendor.Api.Models;
using FarmVendor.Api.Models.DTOs;
using FarmVendor.Api.Services;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using System.Security.Claims;

namespace FarmVendor.Api.Controllers;

[ApiController]
[Route("api/chat")]
[Authorize]
public class ChatController : ControllerBase
{
    private readonly ChatService _chatService;
    private readonly AppDbContext _db;
    private readonly UserManager<ApplicationUser> _userManager;

    public ChatController(
        ChatService chatService,
        AppDbContext db,
        UserManager<ApplicationUser> userManager)
    {
        _chatService = chatService;
        _db = db;
        _userManager = userManager;
    }

    [HttpGet("users")]
    public async Task<IActionResult> GetUsersByRole([FromQuery] string role, [FromQuery] string? search = null)
    {
        if (string.IsNullOrWhiteSpace(role))
            return BadRequest("Role is required.");

        var usersInRole = await _userManager.GetUsersInRoleAsync(role);

        var query = usersInRole.AsQueryable();

        if (!string.IsNullOrWhiteSpace(search))
        {
            var term = search.Trim().ToLower();
            query = query.Where(u =>
                (u.DisplayName != null && u.DisplayName.ToLower().Contains(term)) ||
                (u.Email != null && u.Email.ToLower().Contains(term)));
        }

        var result = query
            .OrderBy(u => u.DisplayName)
            .Select(u => new ChatUserLookupDto
            {
                Id = u.Id,
                DisplayName = u.DisplayName ?? "",
                Email = u.Email ?? "",
                Role = role
            })
            .ToList();
```

ChatController.cs

W26_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Controllers > ChatController.cs

```
17  {
34      {
62      }
63
64      [HttpPost("conversation")]
65      public async Task<IActionResult> CreateConversation([FromBody] CreateConversationDto dto)
66      {
67          var currentUserId =
68              User.FindFirstValue(ClaimTypes.NameIdentifier) ??
69              User.FindFirstValue("sub");
70
71          var currentRole =
72              User.FindFirstValue(ClaimTypes.Role) ??
73              User.FindFirst("http://schemas.microsoft.com/ws/2008/06/identity/claims/role")?.Value;
74
75          if (string.IsNullOrWhiteSpace(currentUserId))
76              return Unauthorized("User ID not found in token.");
77
78          string farmerId;
79          string vendorId;
80
81          if (string.Equals(currentRole, "Farmer", StringComparison.OrdinalIgnoreCase))
82          {
83              if (string.IsNullOrWhiteSpace(dto.VendorId))
84                  return BadRequest("VendorId is required.");
85
86              farmerId = currentUserId;
87              vendorId = dto.VendorId.Trim();
88          }
89          else if (string.Equals(currentRole, "Vendor", StringComparison.OrdinalIgnoreCase))
90          {
91              if (string.IsNullOrWhiteSpace(dto.FarmerId))
92                  return BadRequest("FarmerId is required.");
93
94              farmerId = dto.FarmerId.Trim();
95              vendorId = currentUserId;
96          }
97          else
98          {
99              return BadRequest("Unsupported role.");
100         }
101
102         var farmerExists = await _db.Users.AnyAsync(u => u.Id == farmerId);
103         var vendorExists = await _db.Users.AnyAsync(u => u.Id == vendorId);
104
105         if (!farmerExists)
106             return BadRequest($"FarmerId '{farmerId}' does not exist.");
107         if (!vendorExists)
108             return BadRequest($"VendorId '{vendorId}' does not exist.");
109
110         var conversation = await _chatService.CreateOrGetConversationAsync(farmerId, vendorId);
111
112         return Ok(new
113         {
114             conversationId = conversation.ConversationId,
115             farmerId = conversation.FarmerId,
116             vendorId = conversation.VendorId
117         });
118     }
119
120     // [HttpGet("conversations")]
```

ChatController.cs

W26_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Controllers > ChatController.cs

```
17  {
129     //     {
130     //         conversationId = c.ConversationId,
131     //         farmerId = c.FarmerId,
132     //         vendorId = c.VendorId,
133     //         createdAt = c.CreatedAt
134     //     }));
135     // }
136
137 [HttpGet("conversations")]
138 public async Task<IActionResult> GetMyConversations()
139 {
140     var userId = User.FindFirstValue(ClaimTypes.NameIdentifier) ?? User.FindFirstValue("sub");
141     if (string.IsNullOrWhiteSpace(userId))
142         return Unauthorized();
143
144     var conversations = await _db.Conversation
145         .Include(c => c.Farmer)
146         .Include(c => c.Vendor)
147         .OrderByDescending(c => c.CreatedAt)
148         .ToListAsync();
149
150     var result = conversations
151         .Where(c => c.FarmerId == userId || c.VendorId == userId)
152         .Select(c => new
153         {
154             conversationId = c.ConversationId,
155             farmerId = c.FarmerId,
156             vendorId = c.VendorId,
157
158             farmerName = c.Farmer.DisplayName,
159             vendorName = c.Vendor.DisplayName,
160
161             createdAt = c.CreatedAt
162         });
163
164     return Ok(result);
165 }
166
167     [HttpGet("messages/{conversationId:int}")]
168     public async Task<IActionResult> GetMessages(int conversationId)
169     {
170         var messages = await _chatService.GetMessagesAsync(conversationId);
171         return Ok(messages);
172     }
173 }
```

- ChatHub.cs

ChatHub.cs

W26_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Hubs > ChatHub.cs

```
using FarmVendor.Api.Services;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.SignalR;
using System.Security.Claims;

namespace FarmVendor.Api.Hubs;

[Authorize]
public class ChatHub : Hub
{
    private readonly ChatService _chatService;

    public ChatHub(ChatService chatService)
    {
        _chatService = chatService;
    }

    public async Task JoinConversation(string conversationKey)
    {
        await Groups.AddToGroupAsync(Context.ConnectionId, conversationKey);
    }

    public async Task LeaveConversation(string conversationKey)
    {
        await Groups.RemoveFromGroupAsync(Context.ConnectionId, conversationKey);
    }

    public async Task SendMessage(int conversationId, string messageText)
    {
        var userId = Context.User?.FindFirstValue(ClaimTypes.NameIdentifier)
                     ?? Context.User?.FindFirstValue("sub");

        var role = Context.User?.FindFirstValue(ClaimTypes.Role) ?? "";

        if (string.IsNullOrWhiteSpace(userId))
            throw new HubException("Unauthorized");

        if (string.IsNullOrWhiteSpace(messageText))
            throw new HubException("Message cannot be empty.");

        var saved = await _chatService.SaveMessageAsync(conversationId, userId, role, messageText);

        var conversationKey = $"conversation-{conversationId}";

        await Clients.Group(conversationKey).SendAsync("ReceiveMessage", new
        {
            chatMessageId = saved.ChatMessageId,
            conversationId = saved.ConversationId,
            senderId = saved.SenderId,
            senderRole = saved.SenderRole,
            messageText = saved.MessageText,
            sentAt = saved.SentAt,
            isRead = saved.IsRead
        });
    }
}
```

- ChatService.cs

ChatService.cs

W26_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Services > ChatService.cs

```
using FarmVendor.Api.Data;
using FarmVendor.Api.Models;
using FarmVendor.Api.Models.DTOs;
using Microsoft.EntityFrameworkCore;

namespace FarmVendor.Api.Services;

public class ChatService
{
    private readonly AppDbContext _db;

    public ChatService(AppDbContext db)
    {
        _db = db;
    }

    public async Task<Conversation> CreateOrGetConversationAsync(string farmerId, string vendorId)
    {
        var existing = await _db.Conversation
            .FirstOrDefaultAsync(c => c.FarmerId == farmerId && c.VendorId == vendorId);

        if (existing != null) return existing;

        var conversation = new Conversation
        {
            FarmerId = farmerId,
            VendorId = vendorId,
            CreatedAt = DateTime.UtcNow
        };

        _db.Conversation.Add(conversation);
        await _db.SaveChangesAsync();

        return conversation;
    }

    public async Task<List<Conversation>> GetMyConversationsAsync(string userId)
    {
        return await _db.Conversation
            .Include(c => c.Messages)
            .Where(c => c.FarmerId == userId || c.VendorId == userId)
            .OrderByDescending(c => c.CreatedAt)
            .ToListAsync();
    }

    public async Task<List<ChatMessageRowDto>> GetMessagesAsync(int conversationId)
    {
        return await _db.ChatMessage
            .AsNoTracking()
            .Where(m => m.ConversationId == conversationId)
            .OrderBy(m => m.SentAt)
            .Select(m => new ChatMessageRowDto
            {
                ChatMessageId = m.ChatMessageId,
                ConversationId = m.ConversationId,
                SenderId = m.SenderId,
                SenderRole = m.SenderRole,
                MessageText = m.MessageText,
                SentAt = m.SentAt,
```

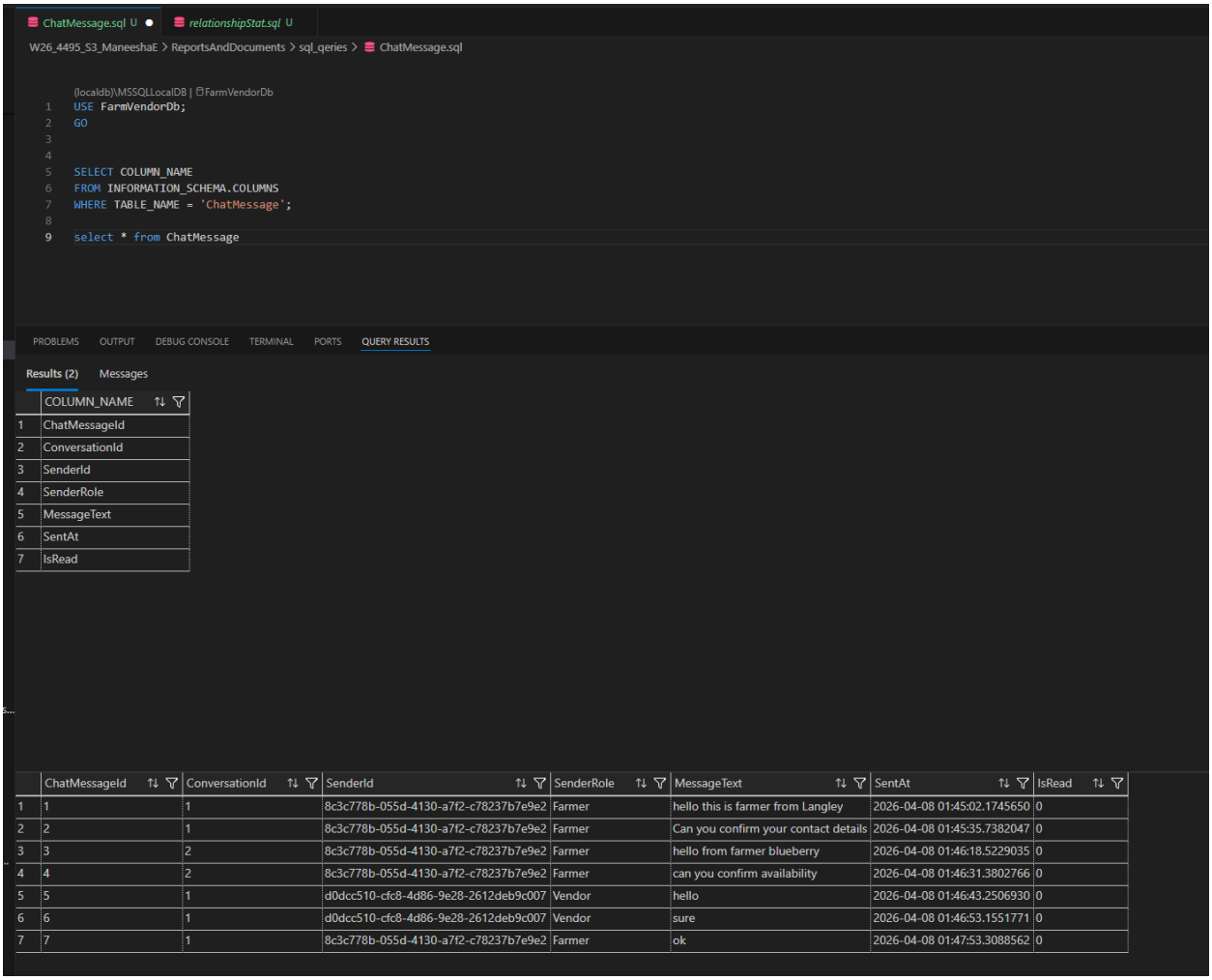
ChatService.cs

W26_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Services > ChatService.cs

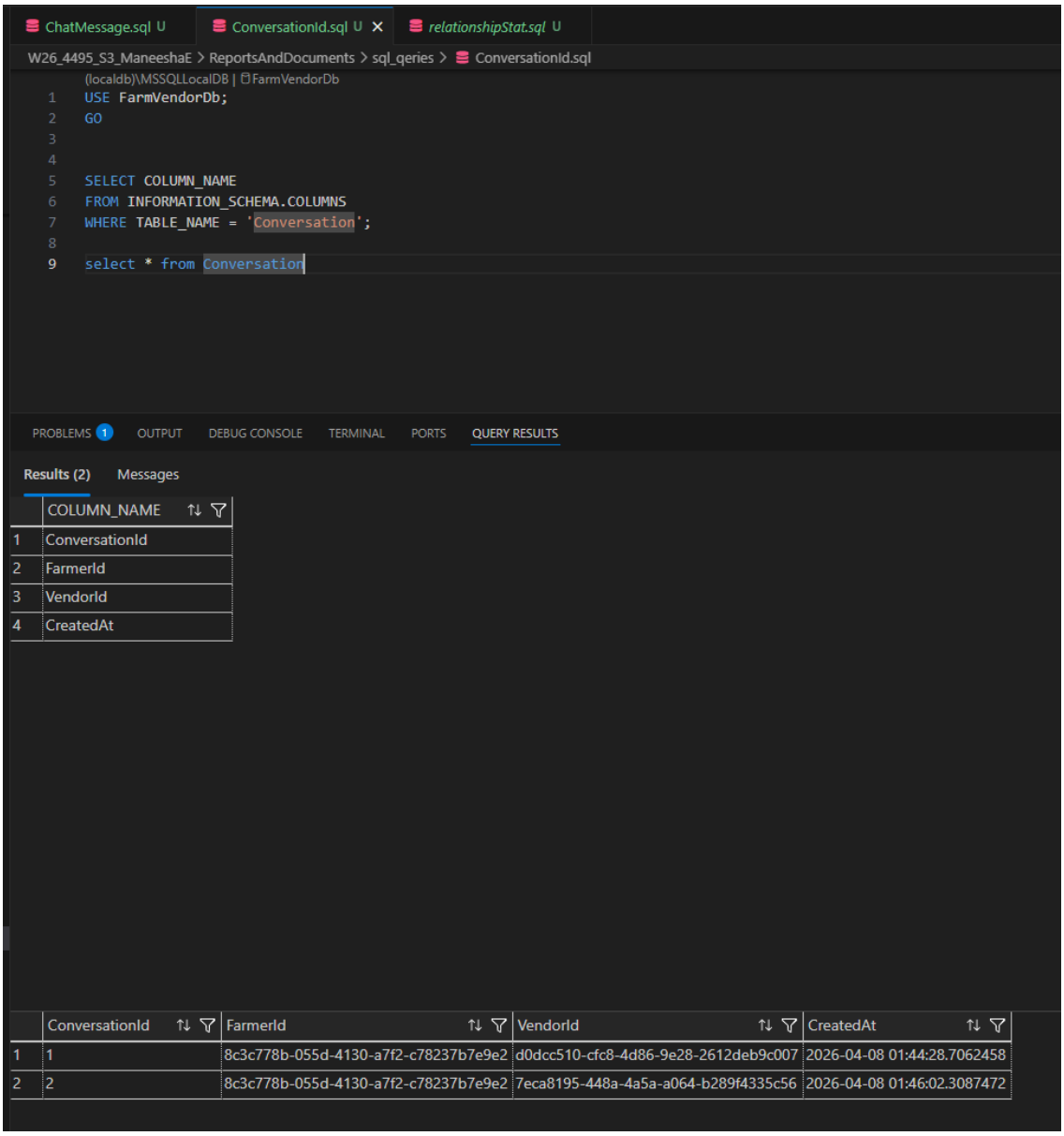
```
9   {
18      {
30
31          _db.Conversation.Add(conversation);
32          await _db.SaveChangesAsync();
33
34          return conversation;
35      }
36
37      public async Task<List<Conversation>> GetMyConversationsAsync(string userId)
38      {
39          return await _db.Conversation
40              .Include(c => c.Messages)
41              .Where(c => c.FarmerId == userId || c.VendorId == userId)
42              .OrderByDescending(c => c.CreatedAt)
43              .ToListAsync();
44      }
45
46      public async Task<List<ChatMessageRowDto>> GetMessagesAsync(int conversationId)
47      {
48          return await _db.ChatMessage
49              .AsNoTracking()
50              .Where(m => m.ConversationId == conversationId)
51              .OrderBy(m => m.SentAt)
52              .Select(m => new ChatMessageRowDto
53              {
54                  ChatMessageId = m.ChatMessageId,
55                  ConversationId = m.ConversationId,
56                  SenderId = m.SenderId,
57                  SenderRole = m.SenderRole,
58                  MessageText = m.MessageText,
59                  SentAt = m.SentAt,
60                  IsRead = m.IsRead
61              })
62              .ToListAsync();
63      }
64
65      public async Task<ChatMessage> SaveMessageAsync(int conversationId, string senderId, string senderRole, string messageText)
66      {
67          var message = new ChatMessage
68          {
69              ConversationId = conversationId,
70              SenderId = senderId,
71              SenderRole = senderRole,
72              MessageText = messageText,
73              SentAt = DateTime.UtcNow,
74              IsRead = false
75          };
76
77          _db.ChatMessage.Add(message);
78          await _db.SaveChangesAsync();
79
80          return message;
81      }
82  }
```

Database

- ChatMessage table



- Conversation Table



Feature 3: Forecast Export Functionality

This feature allows users to export forecast results in CSV format for external analysis, reporting, and presentation.

This feature includes:

- Export Results button
- CSV generation from forecast table
- Downloadable file with forecast details

Implementation

Frontend

- FarmerForecast.jsx
- CSV blob generation using JavaScript
- dynamic file download handling

Backend

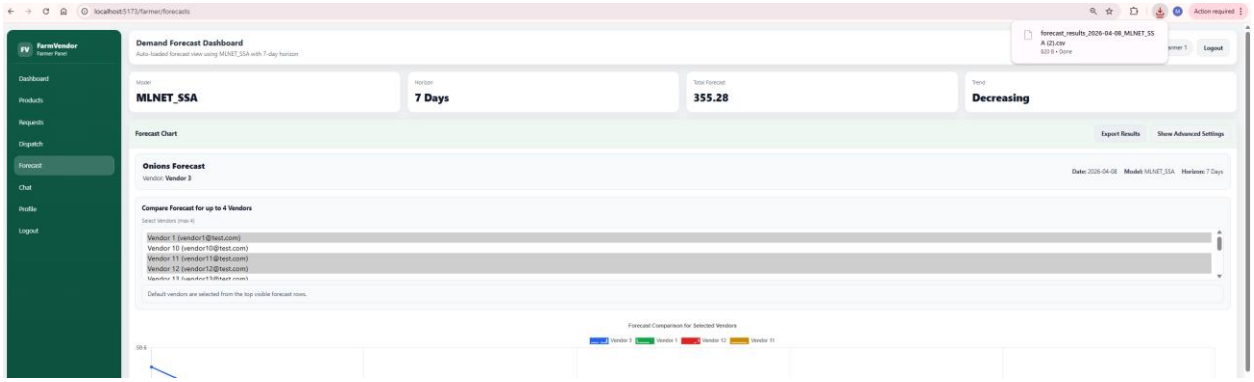
- No dedicated backend export endpoint required for current version
- data taken from forecast rows already loaded through API

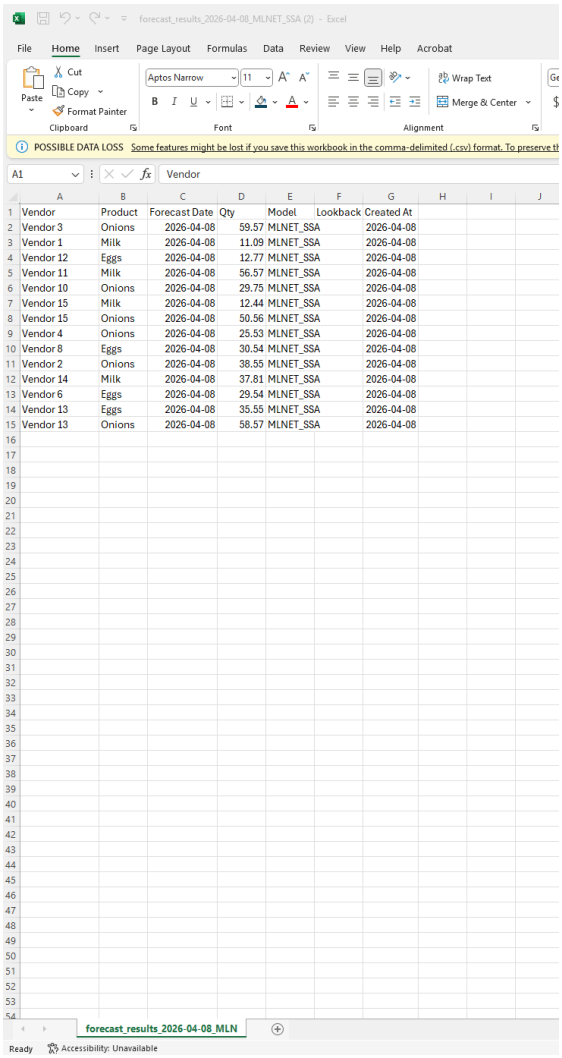
Exported Fields

- Vendor Name
- Product
- Forecast Date
- Quantity
- Model
- Lookback
- Created At

App

- Farmer forecast dashboard

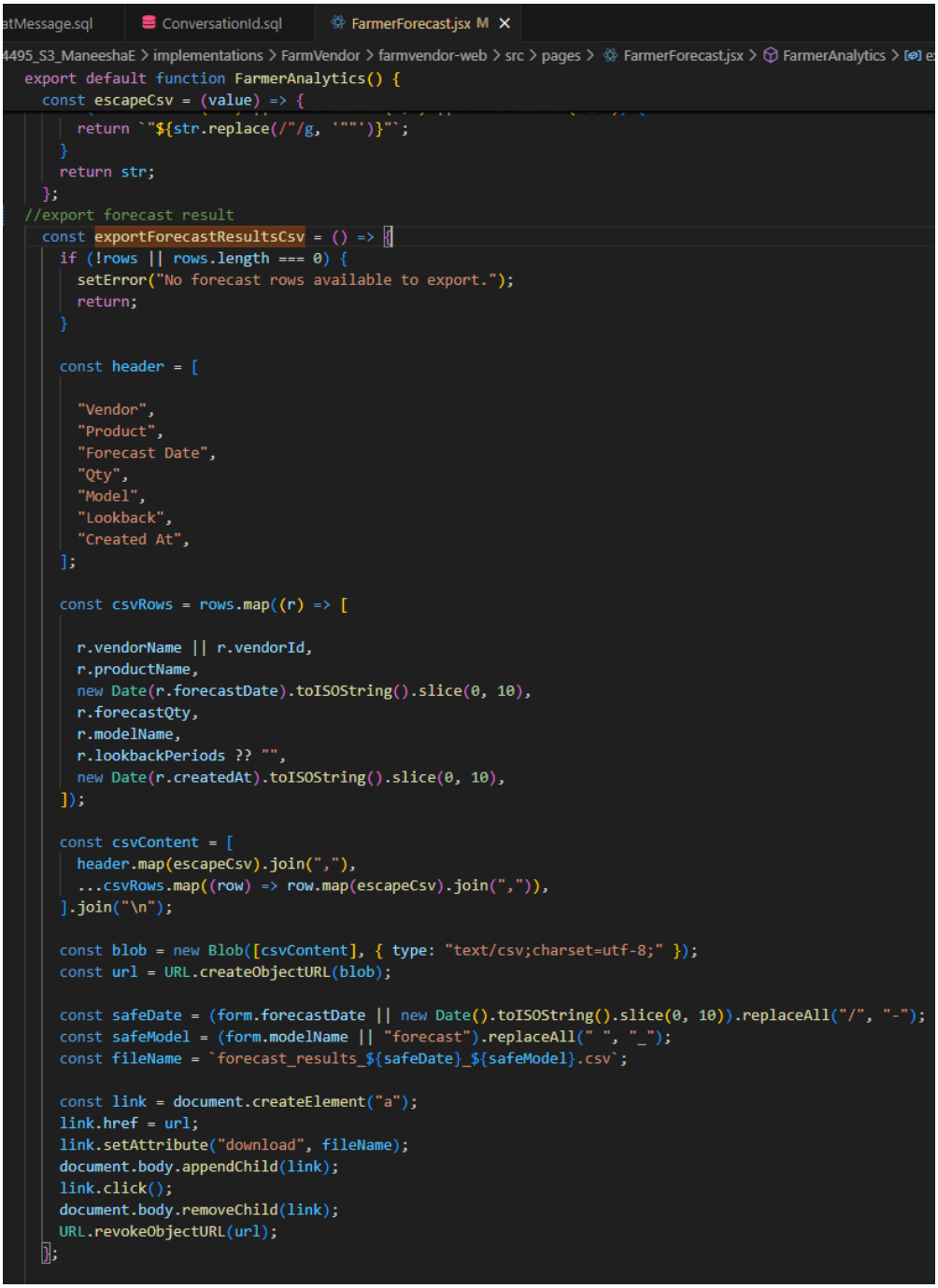


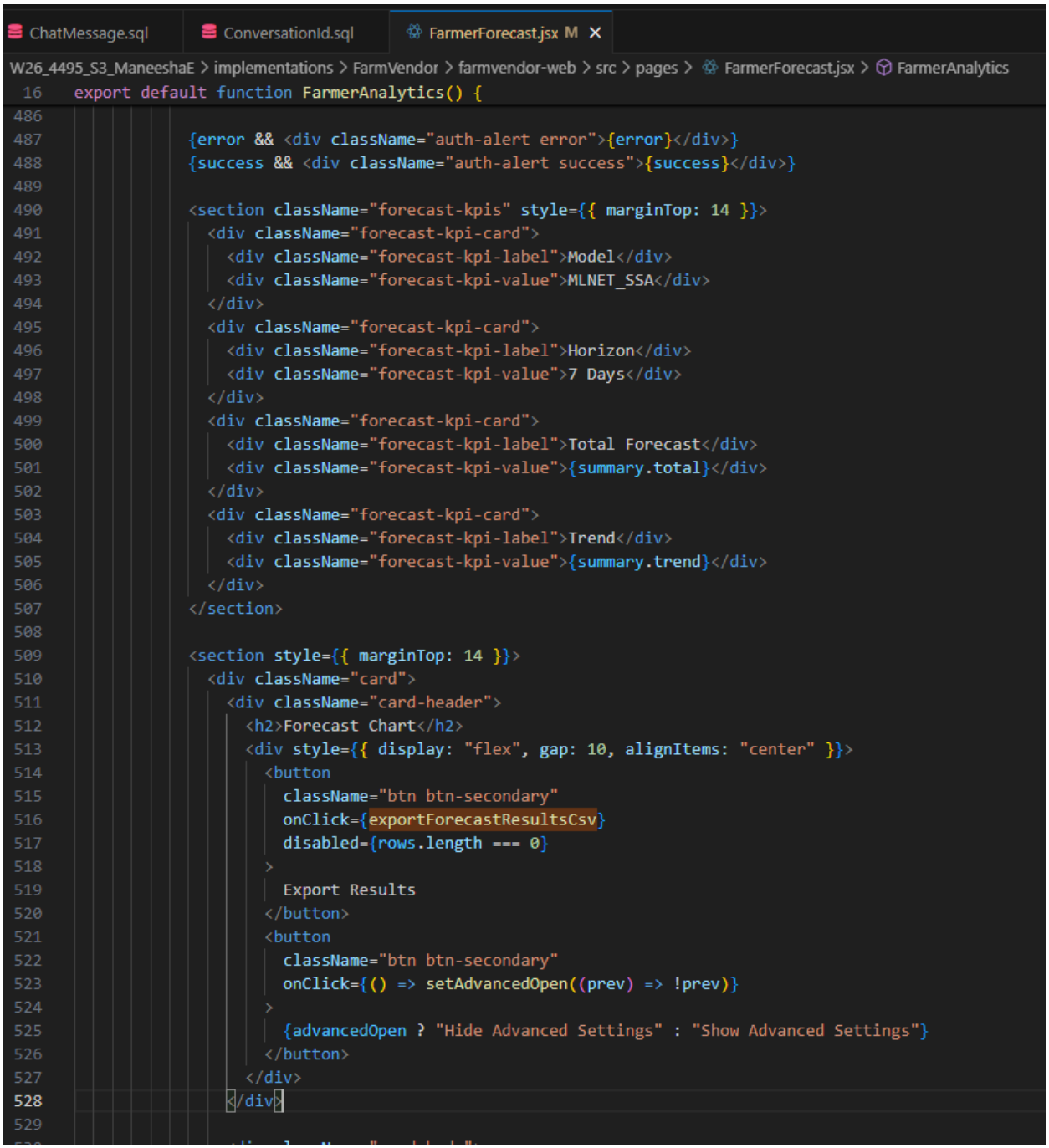


	A	B	C	D	E	F	G
1	Vendor	Product	Forecast Date	Qty	Model	Lookback	Created At
2	Vendor 3	Onions	2026-04-08	59.57	MLNET_SSA		2026-04-08
3	Vendor 1	Milk	2026-04-08	11.09	MLNET_SSA		2026-04-08
4	Vendor 12	Eggs	2026-04-08	12.77	MLNET_SSA		2026-04-08
5	Vendor 11	Milk	2026-04-08	56.57	MLNET_SSA		2026-04-08
6	Vendor 10	Onions	2026-04-08	29.75	MLNET_SSA		2026-04-08
7	Vendor 15	Milk	2026-04-08	12.44	MLNET_SSA		2026-04-08
8	Vendor 15	Onions	2026-04-08	50.56	MLNET_SSA		2026-04-08
9	Vendor 4	Onions	2026-04-08	25.53	MLNET_SSA		2026-04-08
10	Vendor 8	Eggs	2026-04-08	30.54	MLNET_SSA		2026-04-08
11	Vendor 2	Onions	2026-04-08	38.55	MLNET_SSA		2026-04-08
12	Vendor 14	Milk	2026-04-08	37.81	MLNET_SSA		2026-04-08
13	Vendor 6	Eggs	2026-04-08	29.54	MLNET_SSA		2026-04-08
14	Vendor 13	Eggs	2026-04-08	35.55	MLNET_SSA		2026-04-08
15	Vendor 13	Onions	2026-04-08	58.57	MLNET_SSA		2026-04-08

Frontend

- FarmerForecast.jsx




ChatMessage.sql
ConversationId.sql
FarmerForecast.jsx M
W26_4495_S3_ManeeshaE > implementations > FarmVendor > farmvendor-web > src > pages > FarmerForecast.jsx > FarmerAnalytics
16 export default function FarmerAnalytics() {
{error && {error} }
{success && {success} }
<section className="forecast-kpis" style={{ marginTop: 14 }}>

Model
MLNET_SSA

Horizon
7 Days

Total Forecast
{summary.total}

Trend
{summary.trend}

</section>
<section style={{ marginTop: 14 }}>

<h2>Forecast Chart</h2>
<div style={{ display: "flex", gap: 10, alignItems: "center" }}>
<button
className="btn btn-secondary"
onClick={exportForecastResultsCsv}
disabled={rows.length === 0}
>
Export Results
</button>
<button
className="btn btn-secondary"
onClick={() => setAdvancedOpen((prev) => !prev)}
>
{advancedOpen ? "Hide Advanced Settings" : "Show Advanced Settings"}
</button>

Feature 4: Role-Based Authentication (Registration & Login)

This feature allows users to access the system as either a Farmer or a Vendor. Authentication is implemented using ASP.NET Identity with JWT-based authorization, ensuring secure access, protected API calls, and role-based navigation.

This feature includes:

- User registration with role selection
- Secure login with JWT token generation
- Role-based dashboard access
- Route protection in the frontend

The design consists of:

- Registration page with role selection
- Login page with credential validation
- Backend JWT token generation
- Role-based route protection using React guards

Implementation

Frontend

- Register.jsx – handles registration form, role selection, and API call
- Login.jsx – handles login, stores JWT in localStorage, redirects user dashboard
- RequireAuth.jsx – protects authenticated routes
- RequireRole.jsx – restricts role access based on user type

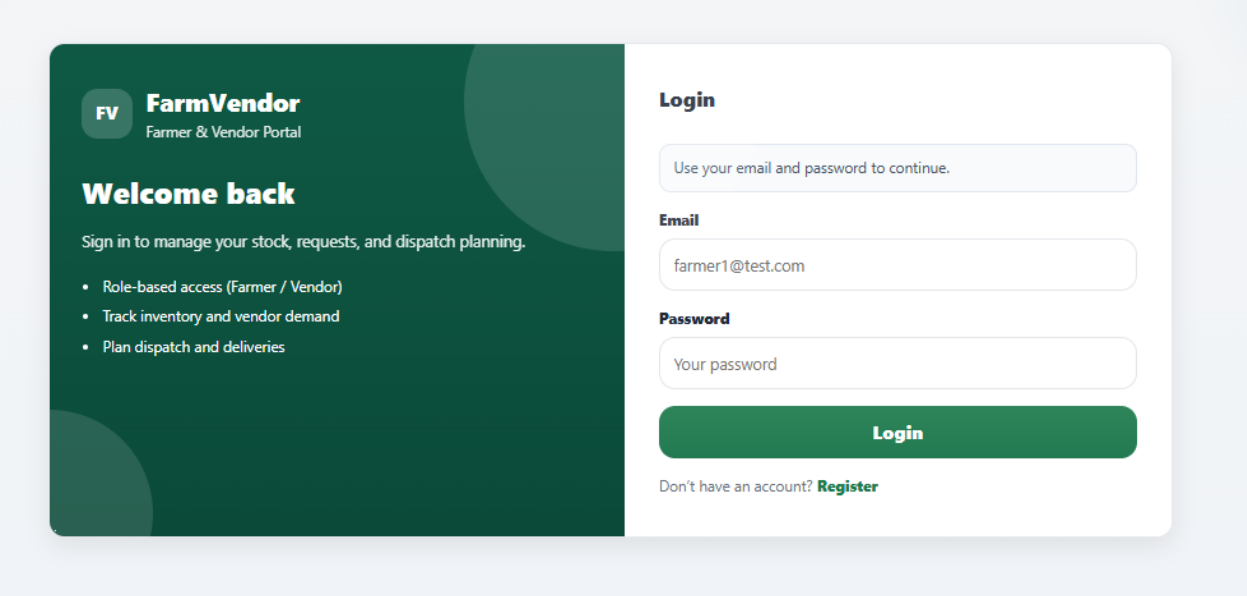
Backend

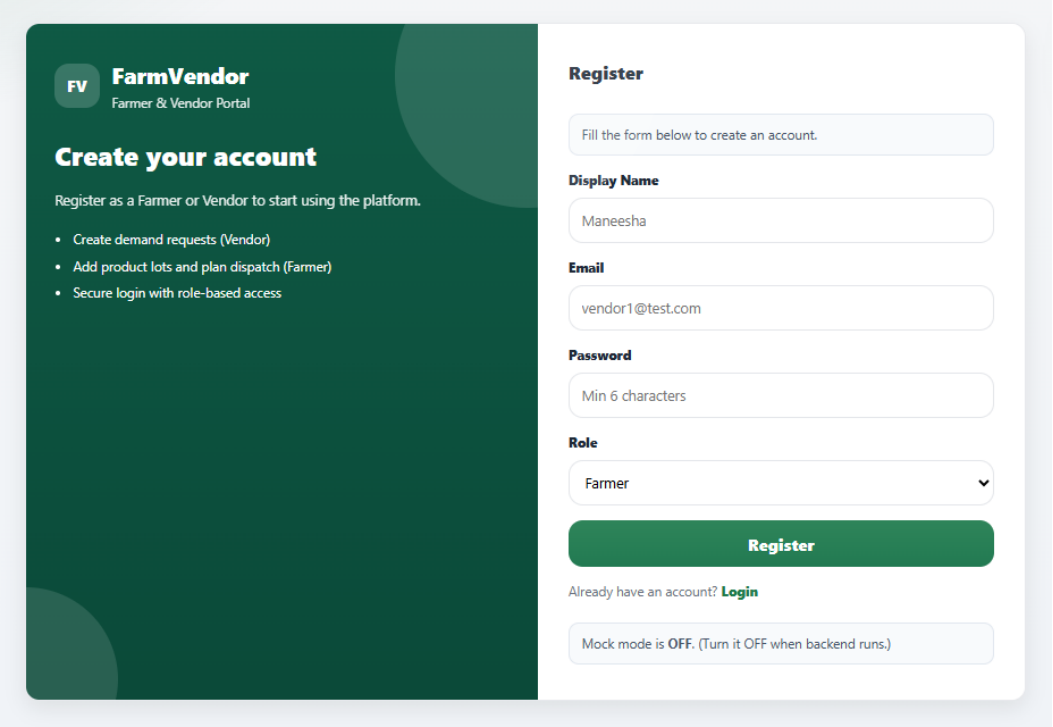
- AuthController.cs – login and register endpoints
- ApplicationUser.cs – extended Identity user with custom fields
- Program.cs – JWT authentication and authorization middleware
- JWT token generation logic using issuer, audience, and signing key

Database

- AspNetUsers table stores user accounts
- Identity role tables map users to Farmer / Vendor roles

App





- Register.jsx

Register.jsx

```jsx
import React, { useState } from "react";
import { Link, useNavigate } from "react-router-dom";
import "../styles/auth.css";

const API_BASE = import.meta.env.VITE_API_URL;
const USE_MOCK = false; //backend is running

export default function Register() {
  const navigate = useNavigate();

  const [form, setForm] = useState({
    displayName: "",
    email: "",
    password: "",
    role: "Farmer",
  });

  const [loading, setLoading] = useState(false);
  const [error, setError] = useState("");
  const [success, setSuccess] = useState("");

  const onChange = (e) => {
    setForm((p) => ({ ...p, [e.target.name]: e.target.value }));
  };

  const handleRegister = async (e) => {
    e.preventDefault();
    setError("");
    setSuccess("");
    setLoading(true);

    try {
      if (USE_MOCK) {
        setSuccess("Registered successfully (mock). You can login now.");
        setTimeout(() => navigate("/login"), 800);
        return;
      }

      const res = await fetch(`${API_BASE}/api/Auth/register`, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(form),
      });

      if (!res.ok) { //checking for strong password
        const text = await res.text();
        throw new Error(`Registration failed (${res.status}): ${text}`);
      }

      setSuccess("Registered successfully. Please login.");
      setTimeout(() => navigate("/login"), 900);
    } catch (err) {
      setError(err?.message || "Something went wrong");
    } finally {
      setLoading(false);
    }
  };

  return (
    <div className="auth-page">
      <div className="auth-card">
        {/* Left panel */}
        <div className="auth-header">
          <div className="auth-brand">
            <div className="auth-logo">FV</div>
            <div>
              <div className="auth-title">FarmVendor</div>
```

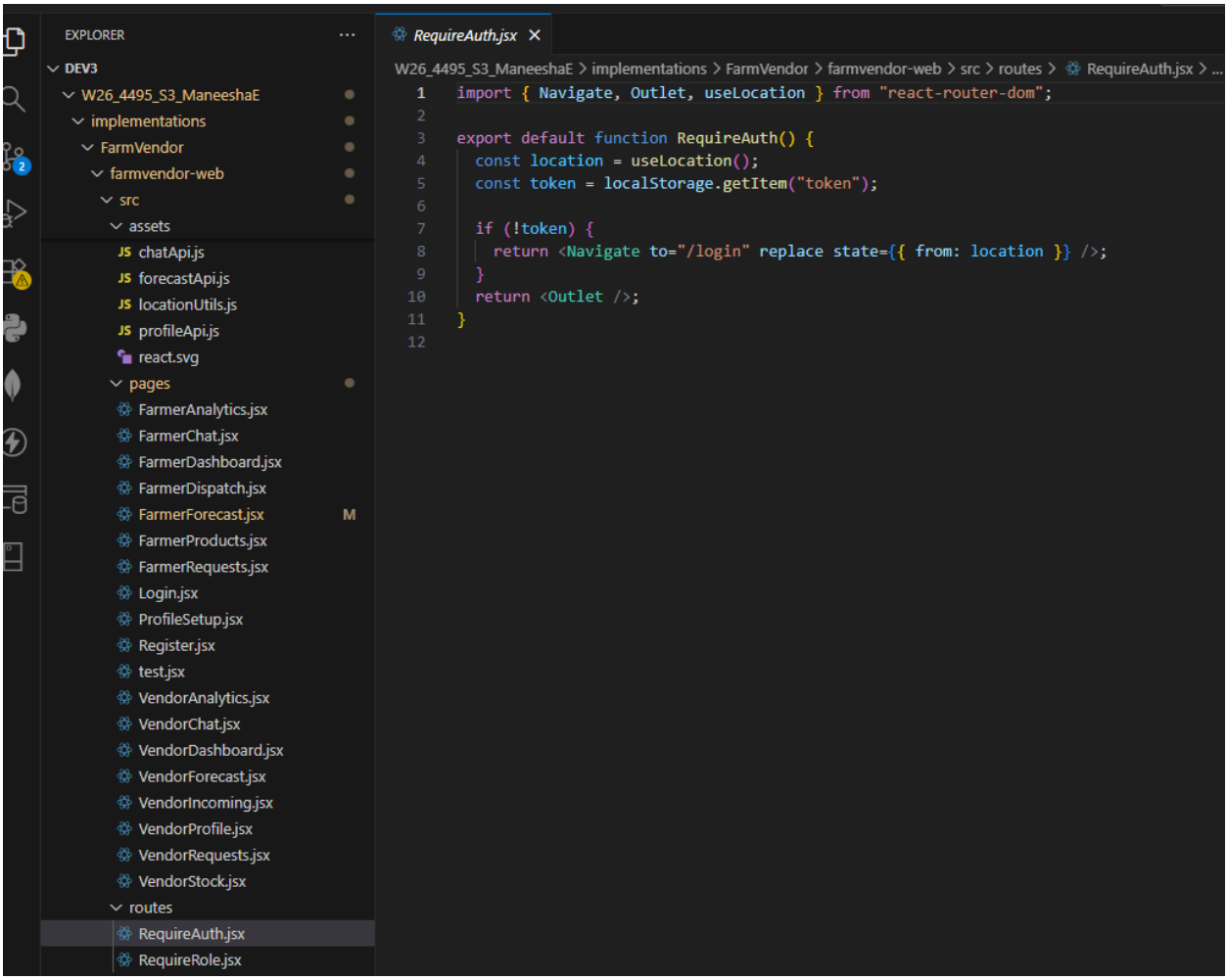
- Login.jsx

Login.jsx

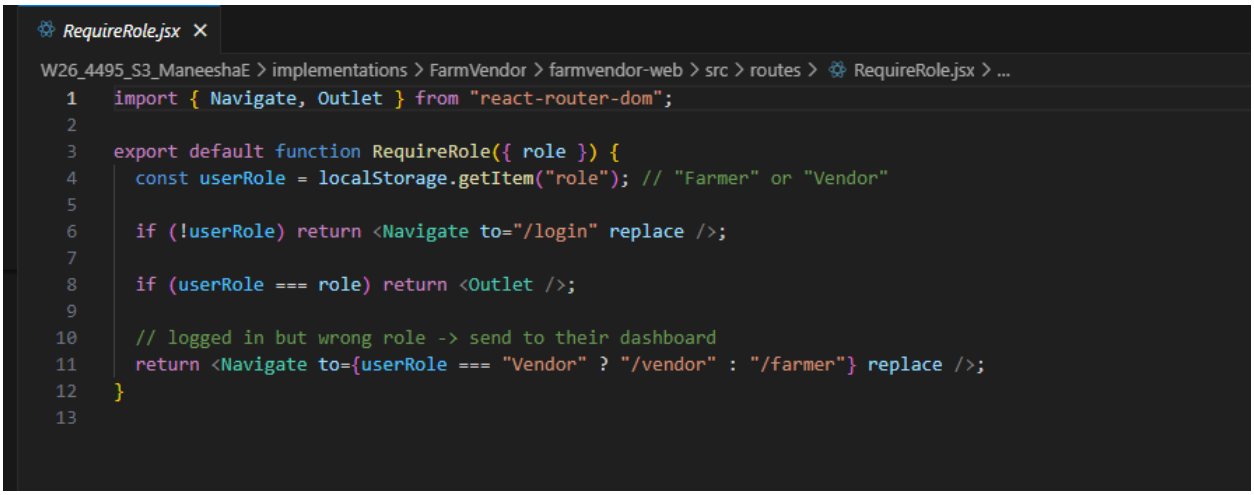
W26_4495_S3_ManeeshaE > implementations > FarmVendor > farmvendor-web > src > pages > Login.jsx > ...

```jsx
import React, { useState } from "react";
import { useNavigate, Link } from "react-router-dom";
import "../styles/auth.css";

const API_BASE = import.meta.env.VITE_API_URL;
console.log("API_BASE =", API_BASE);
const USE_MOCK = false;

export default function Login() {
  const navigate = useNavigate();
  const [form, setForm] = useState({ email: "", password: "" });
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState("");

  const onChange = (e) => {
    setForm((p) => ({ ...p, [e.target.name]: e.target.value }));
  };

  const handleLogin = async (e) => {
    e.preventDefault();
    setError("");
    setLoading(true);

    try {
      if (USE_MOCK) {
        const role = form.email.toLowerCase().includes("vendor") ? "Vendor" : "Farmer";

        localStorage.setItem("token", "mock-jwt-token");
        localStorage.setItem("role", role);
        localStorage.setItem("displayName", role === "Vendor" ? "Vendor User" : "Farmer User");
        localStorage.setItem("userId", role === "Vendor" ? "mock-vendor-id" : "mock-farmer-id");
        localStorage.setItem("profileComplete", "false");

        navigate("/profile/setup");
        return;
      }

      const res = await fetch(`${API_BASE}/api/Auth/login`, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({
          email: form.email,
          password: form.password,
        }),
      });

      if (!res.ok) {
        const text = await res.text();
        throw new Error(text || "Login failed");
      }

      const data = await res.json();

      localStorage.setItem("token", data.token);
      localStorage.setItem("role", data.role);
      localStorage.setItem("displayName", data.displayName);
      localStorage.setItem("userId", data.userId);
      localStorage.setItem("profileComplete", String(data.profileComplete));

      if (!data.profileComplete) {
        navigate("/profile/setup");
      } else {
        navigate(data.role === "Vendor" ? "/vendor" : "/farmer");
      }
    } catch (err) {
      setError(err?.message || "Something went wrong");
    } finally {
```

- RequireAuth.jsx



- RequireRole.jsx



Backend

- AuthController.cs

AuthController.cs

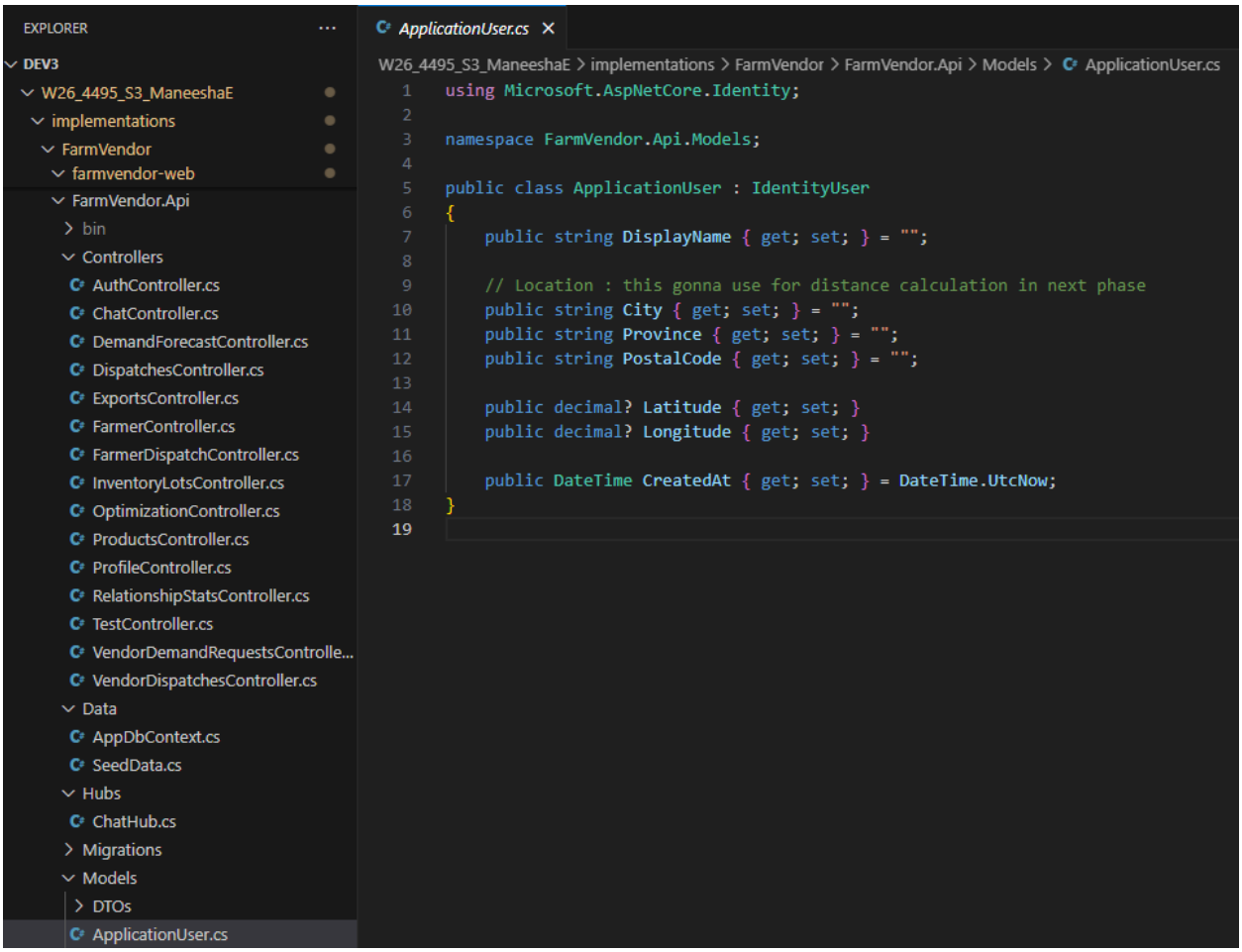
W26_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Controllers > AuthController.cs

```
using FarmVendor.Api.Models;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;
using Microsoft.IdentityModel.Tokens;
using System.IdentityModel.Tokens.Jwt;
using System.Security.Claims;
using System.Text;

namespace FarmVendor.Api.Controllers;

[ApiController]
[Route("api/[controller]")]
public class AuthController : ControllerBase
{
    private readonly UserManager<ApplicationUser> _userManager;
    private readonly IConfiguration _config;

    public AuthController(UserManager<ApplicationUser> userManager, IConfiguration config)
    {
        _userManager = userManager;
        _config = config;
    }

    public record RegisterDto(string Email, string Password, string DisplayName, string Role);
    public record LoginDto(string Email, string Password);

    [HttpPost("register")]
    public async Task<IActionResult> Register(RegisterDto dto)
    {
        if (dto.Role != "Farmer" && dto.Role != "Vendor")
            return BadRequest("Role must be Farmer or Vendor");

        var user = new ApplicationUser
        {
            UserName = dto.Email,
            Email = dto.Email,
            DisplayName = dto.DisplayName
        };

        var result = await _userManager.CreateAsync(user, dto.Password);
        if (!result.Succeeded) return BadRequest(result.Errors);

        await _userManager.AddToRoleAsync(user, dto.Role);

        return Ok(new { message = "Registered successfully" });
    }

```

AuthController.cs

W26_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Controllers > AuthController.cs

```
14  {
29      {
40          var result = await _userManager.CreateAsync(user, dto.Password);
41          if (!result.Succeeded) return BadRequest(result.Errors);
42
43          await _userManager.AddToRoleAsync(user, dto.Role);
44
45          return Ok(new { message = "Registered successfully" });
46      }
47
48      [HttpPost("login")]
49      public async Task<IActionResult> Login(LoginDto dto)
50      {
51          var user = await _userManager.FindByEmailAsync(dto.Email);
52          if (user == null) return Unauthorized("Invalid credentials");
53
54          var valid = await _userManager.CheckPasswordAsync(user, dto.Password);
55          if (!valid) return Unauthorized("Invalid credentials");
56
57          var roles = await _userManager.GetRolesAsync(user);
58
59          var claims = new List<Claim>
60          {
61              new Claim(JwtRegisteredClaimNames.Sub, user.Id),
62              new Claim(JwtRegisteredClaimNames.Email, user.Email ?? ""),
63              new Claim("displayName", user.DisplayName)
64          };
65
66          foreach (var role in roles)
67              claims.Add(new Claim(ClaimTypes.Role, role));
68
69          var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_config["Jwt:Key"]!));
70          var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);
71
72          var token = new JwtSecurityToken(
73              issuer: _config["Jwt:Issuer"],
74              audience: _config["Jwt:Audience"],
75              claims: claims,
76              expires: DateTime.UtcNow.AddHours(6),
77              signingCredentials: creds
78          );
79
80          return Ok(new
81          {
82              token = new JwtSecurityTokenHandler().WriteToken(token),
83              role = roles.FirstOrDefault(),
84              displayName = user.DisplayName,
85              userId = user.Id,
86              city = user.City,
87              province = user.Province,
88              postalCode = user.PostalCode,
89              latitude = user.Latitude,
90              longitude = user.Longitude,
91              profileComplete =
92                  !string.IsNullOrWhiteSpace(user.DisplayName) &&
93                  !string.IsNullOrWhiteSpace(user.City) &&
94                  !string.IsNullOrWhiteSpace(user.Province) &&
95                  !string.IsNullOrWhiteSpace(user.PostalCode) &&
96                  user.Latitude.HasValue &&
97                  user.Longitude.HasValue
98          });
99      }
100 }
101
```


EXPLORER
DEV3
W26_4495_S3_ManeeshaE
implementations
FarmVendor
farmvendor-web
FarmVendor.Api
bin
Controllers
AuthController.cs
ChatController.cs
DemandForecastController.cs
DispatchesController.cs
ExportsController.cs
FarmerController.cs
FarmerDispatchController.cs
InventoryLotsController.cs
OptimizationController.cs
ProductsController.cs
ProfileController.cs
RelationshipStatsController.cs
TestController.cs
VendorDemandRequestsControlle...
VendorDispatchesController.cs
Data
AppDbContext.cs
SeedData.cs
Hubs
ChatHub.cs
Migrations
Models
DTOs
ApplicationUser.cs
ApplicationUser.cs
W26_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Models > ApplicationUser.cs
using Microsoft.AspNetCore.Identity;
namespace FarmVendor.Api.Models;
public class ApplicationUser : IdentityUser
{
public string DisplayName { get; set; } = "";
// Location : this gonna use for distance calculation in next phase
public string City { get; set; } = "";
public string Province { get; set; } = "";
public string PostalCode { get; set; } = "";
public decimal? Latitude { get; set; }
public decimal? Longitude { get; set; }
public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
}

- Program.cs

Program.cs

V26_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Program.cs

```
using FarmVendor.Api.Data;
using FarmVendor.Api.Models;
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.AspNetCore.Identity;
using Microsoft.EntityFrameworkCore;
using Microsoft.IdentityModel.Tokens;
using FarmVendor.Api.Services;
using System.Text;
using FarmVendor.Api.Hubs;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();

// Swagger + JWT Authorize button
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

// DB
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("Default")));

// Identity
builder.Services.AddIdentity<ApplicationUser, IdentityRole>(options =>
{
    options.Password.RequiredLength = 6;
    options.User.RequireUniqueEmail = true;
})
.AddEntityFrameworkStores<AppDbContext>()
.AddDefaultTokenProviders();

// JWT
var jwtKey = builder.Configuration["Jwt:Key"]!;
var jwtIssuer = builder.Configuration["Jwt:Issuer"]!;
var jwtAudience = builder.Configuration["Jwt:Audience"]!;

builder.Services.AddAuthentication(options =>
{
    options.DefaultAuthenticateScheme = JwtBearerDefaults.AuthenticationScheme;
    options.DefaultChallengeScheme = JwtBearerDefaults.AuthenticationScheme;
})
.AddJwtBearer(options =>
{
    options.TokenValidationParameters = new TokenValidationParameters
    {
        ValidateIssuer = true,
        ValidateAudience = true,
        ValidateLifetime = true,
        ValidateIssuerSigningKey = true,
        ValidIssuer = jwtIssuer,
        ValidAudience = jwtAudience,
        IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(jwtKey))
    };
});

builder.Services.AddAuthorization();
```

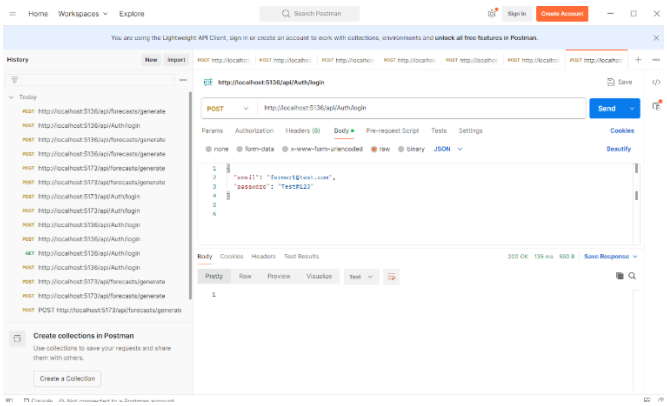
Program.cs

W26_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Program.cs

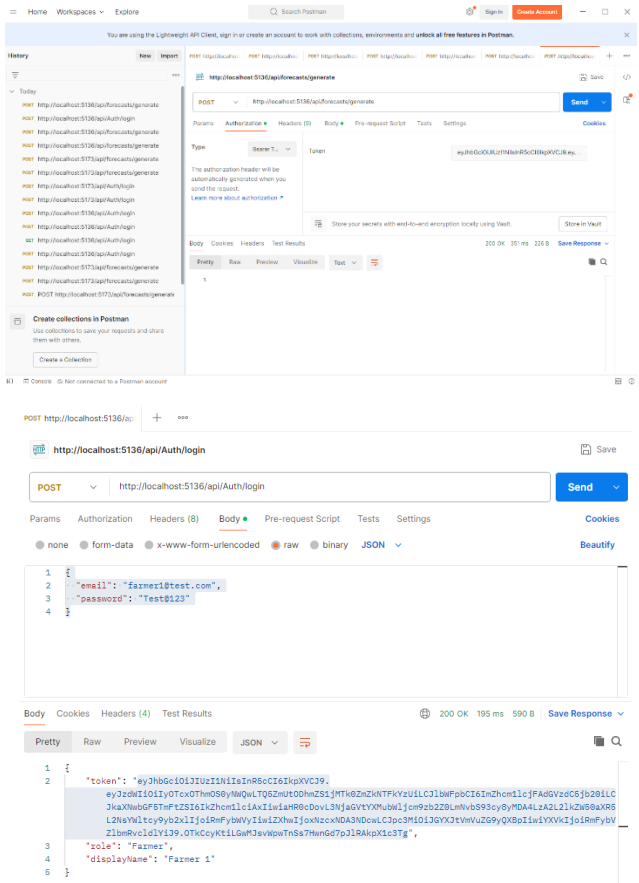
```
38  {
40      options.DefaultChallengeScheme = JwtBearerDefaults.AuthenticationScheme;
41  })
42  .AddJwtBearer(options =>
43  {
44      options.TokenValidationParameters = new TokenValidationParameters
45      {
46          ValidateIssuer = true,
47          ValidateAudience = true,
48          ValidateLifetime = true,
49          ValidateIssuerSigningKey = true,
50          ValidIssuer = jwtIssuer,
51          ValidAudience = jwtAudience,
52          IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(jwtKey))
53      };
54  });
55
56  builder.Services.AddAuthorization();
57
58  // CORS (React + SignalR)
59  builder.Services.AddCors(options =>
60  {
61      options.AddPolicy("ReactApp", policy =>
62          policy.WithOrigins("http://localhost:5173")
63                .AllowAnyHeader()
64                .AllowAnyMethod()
65                .AllowCredentials());
66  });
67
68  // register services
69  builder.Services.AddScoped<DemandForecastService>();
70  builder.Services.AddScoped<DispatchOptimizationService>();
71  builder.Services.AddScoped<RelationshipScoreService>();
72  builder.Services.AddScoped<ChatService>();
73  builder.Services.AddSignalR();
74  builder.Services.AddScoped<FarmerRecommendationService>();
75
76  var app = builder.Build();
77
78  app.UseCors("ReactApp");
79
80  if (app.Environment.IsDevelopment())
81  {
82      app.UseSwagger();
83      app.UseSwaggerUI();
84  }
85
86  // app.UseHttpsRedirection();
87
88  app.UseAuthentication();
89  app.UseAuthorization();
90
91  app.MapControllers();
92  app.MapHub<ChatHub>("/hubs/chat");
93
94  // seed
95  await SeedData.InitializeAsync(app.Services);
96
97  app.Run();
```

- Postman: JWT token generation logic using issuer, audience, and signing key

POST : http://localhost:5136/api/Auth/login

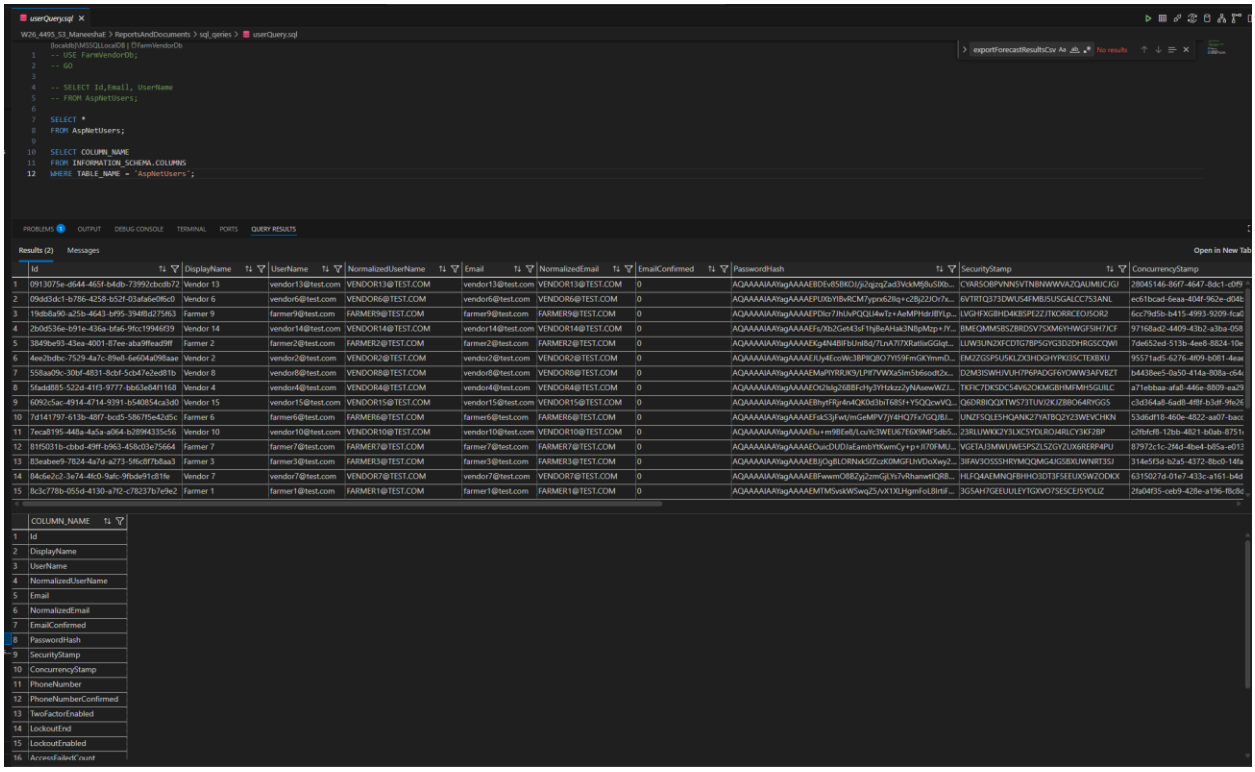


Bearer token



Database

- AspNetUsers Table



Feature 5: Farmer Inventory Management

This feature enables farmers to add, manage, and monitor product inventory lots with quantity and expiry tracking. It supports operational planning by providing clear visibility into available stock.

In the final implementation, this module was significantly enhanced by integrating product image selection using the Unsplash API, improving usability and visual interaction for non-technical users.

This feature includes:

Add product lot form

Inventory table view

Expiry date tracking

Expiry status badge logic

Product image selection using Unsplash API

Visual product identification instead of text-only selection

The design consists of:

Modal-based inventory lot entry

Real-time table updates after submission

Status labels such as *Fresh / Near Expiry / Expired*

Image-based product selection modal

Preview of selected product image before saving

Implementation

Frontend

- FarmerDashboard.jsx
- Add Product Lot modal
- Inventory table UI

- Dynamic status badge rendering
- Image selection modal integrated with Unsplash API
- Image preview component for selected product

Backend

- InventoryLotsController.cs
- ProductsController.cs
- CreateInventoryLotDto.cs
- InventoryLot.cs
- Validation for:
    - Quantity
    - Expiry date
    - Product selection

Database

InventoryLot table storing:

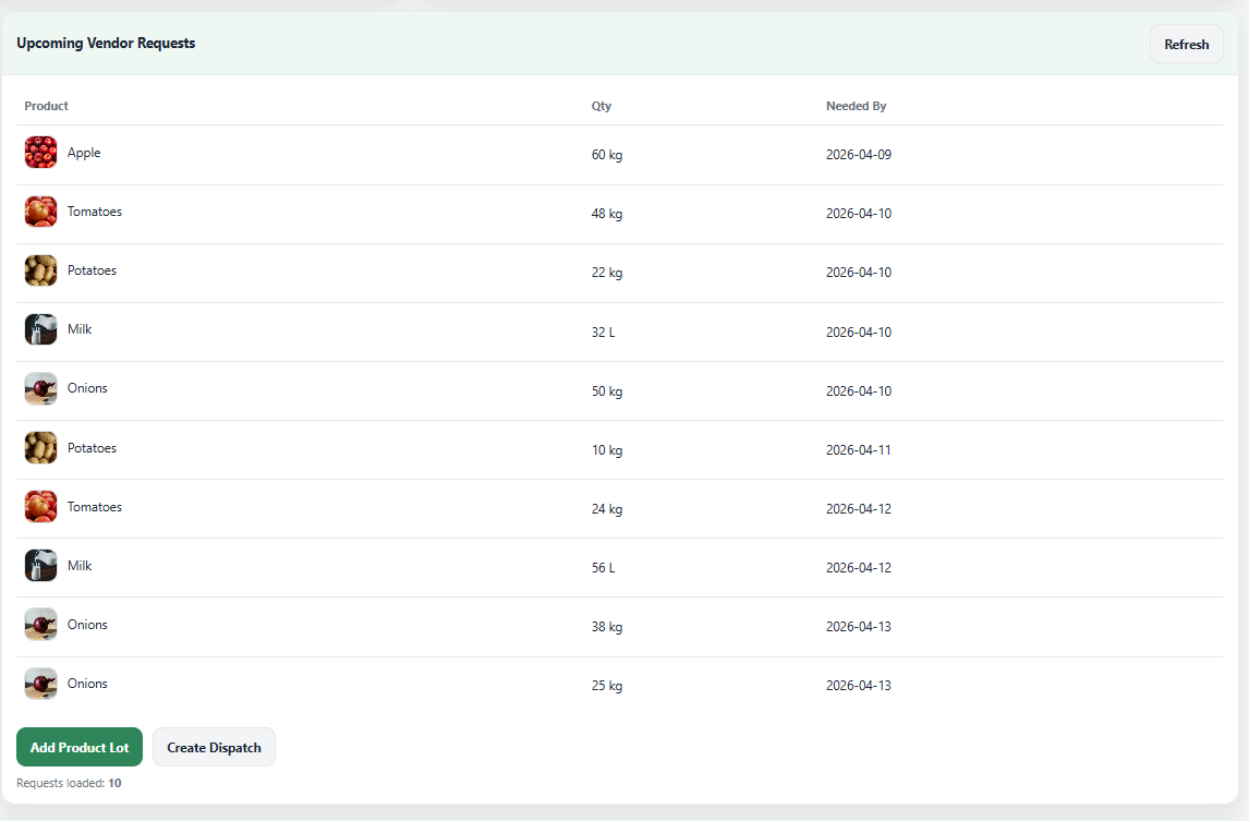
- FarmerId
- ProductId
- QuantityAvailable
- Unit
- ExpiryDate
- CreatedAt

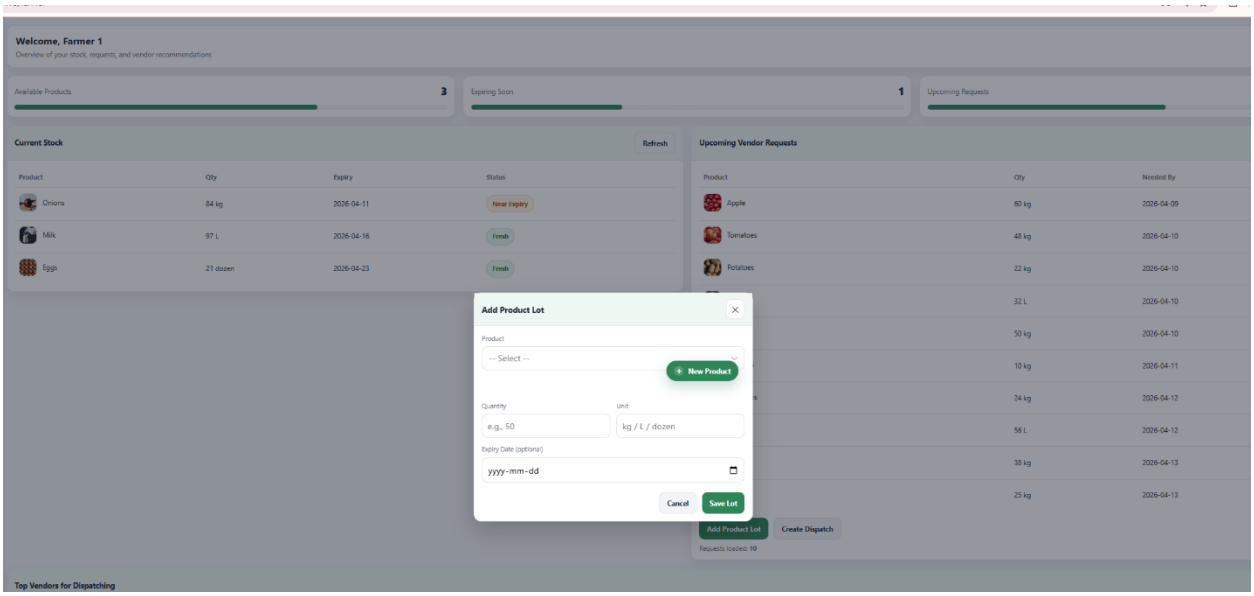
Product table enhanced with image fields

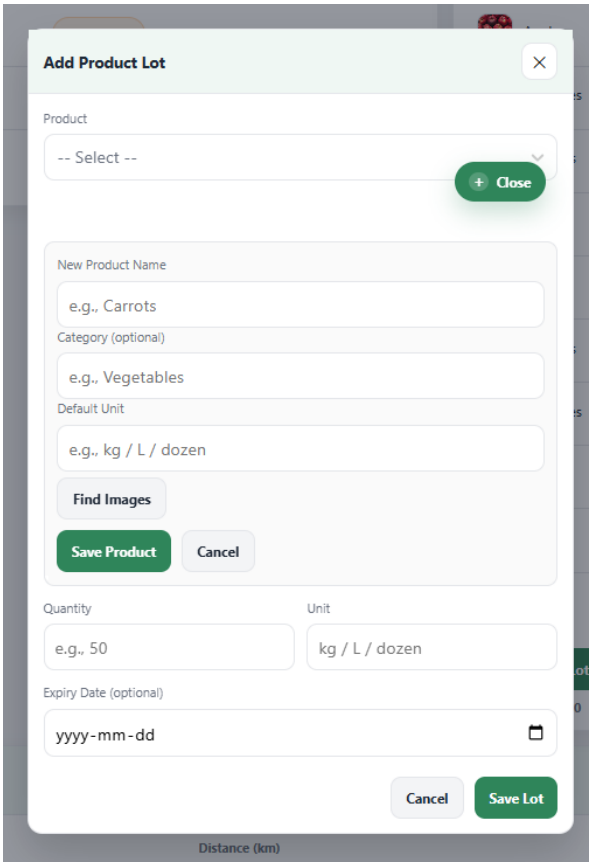
- ImageUrl
- ImageThumbUrl
- ImageSource
- PhotographerName
- PhotographerProfile

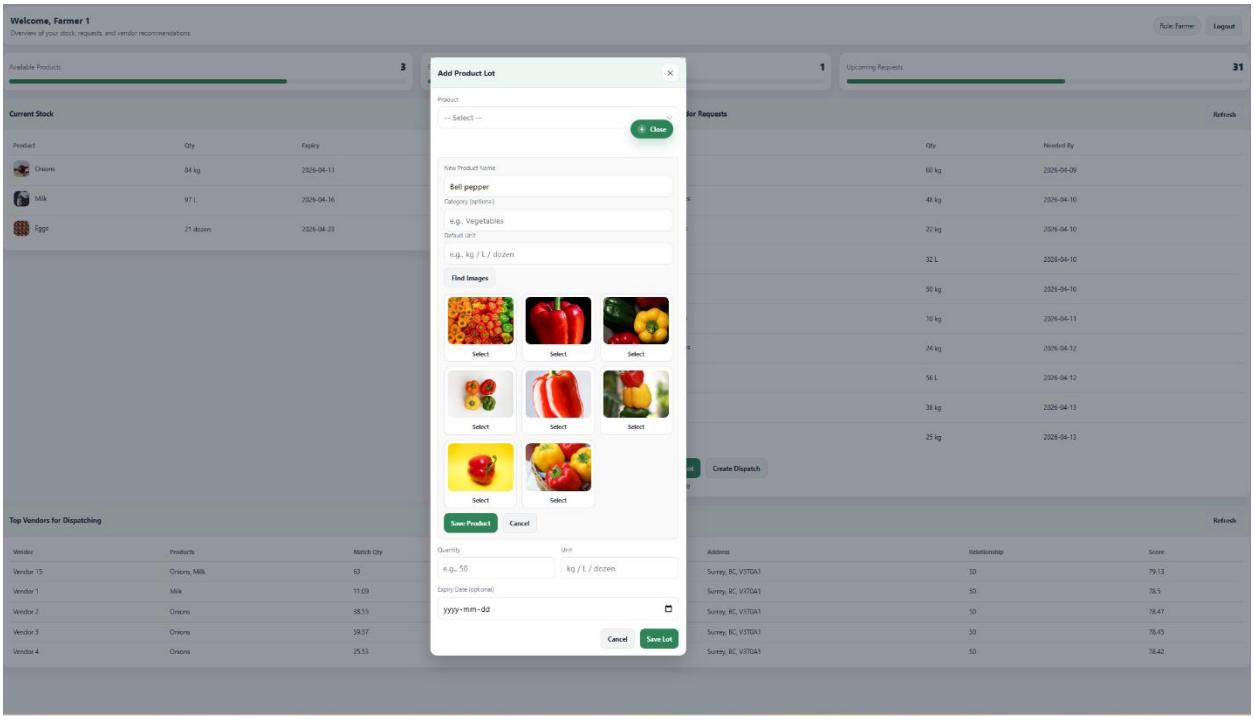
App

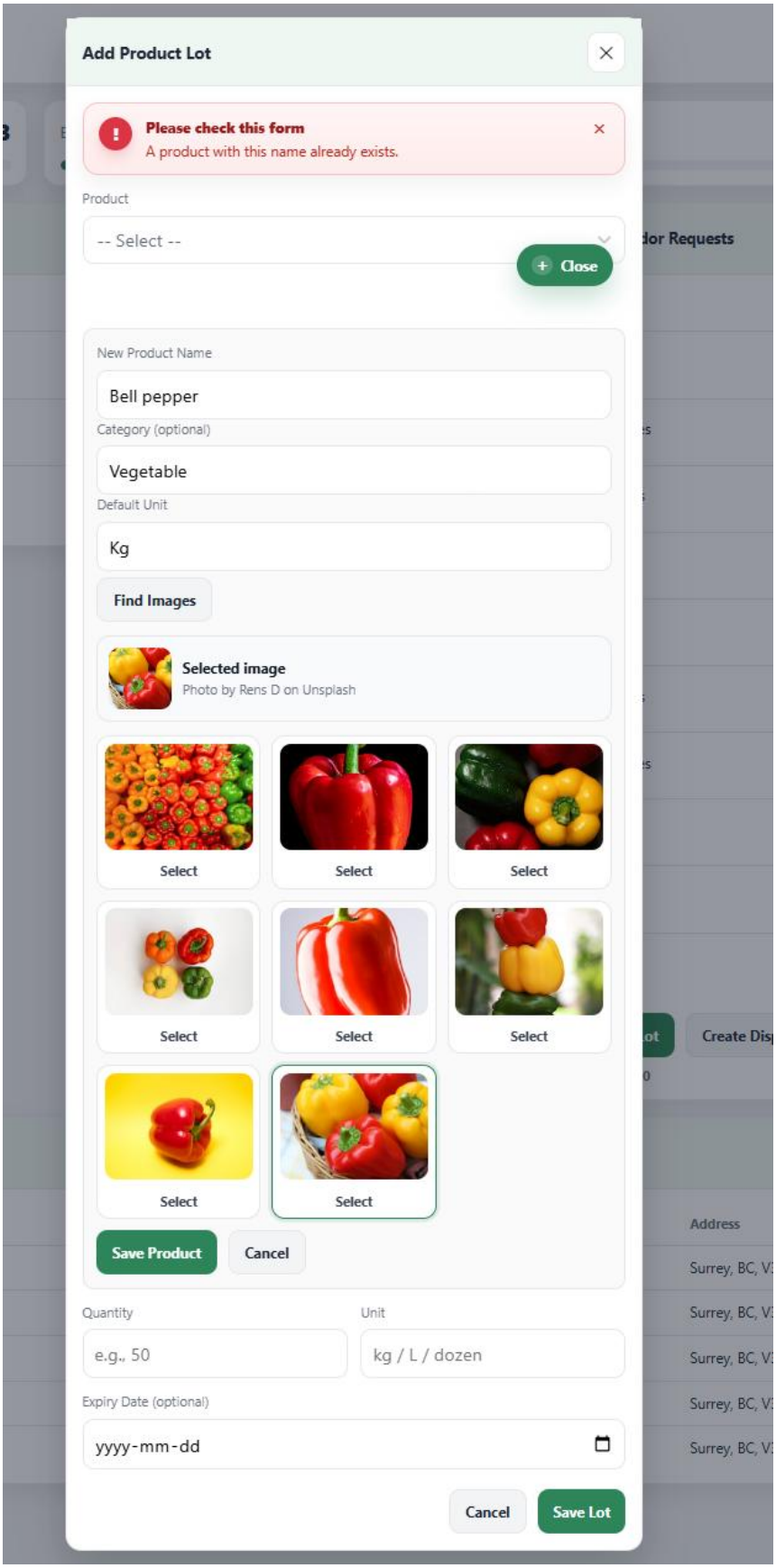
- FarmerDashboard

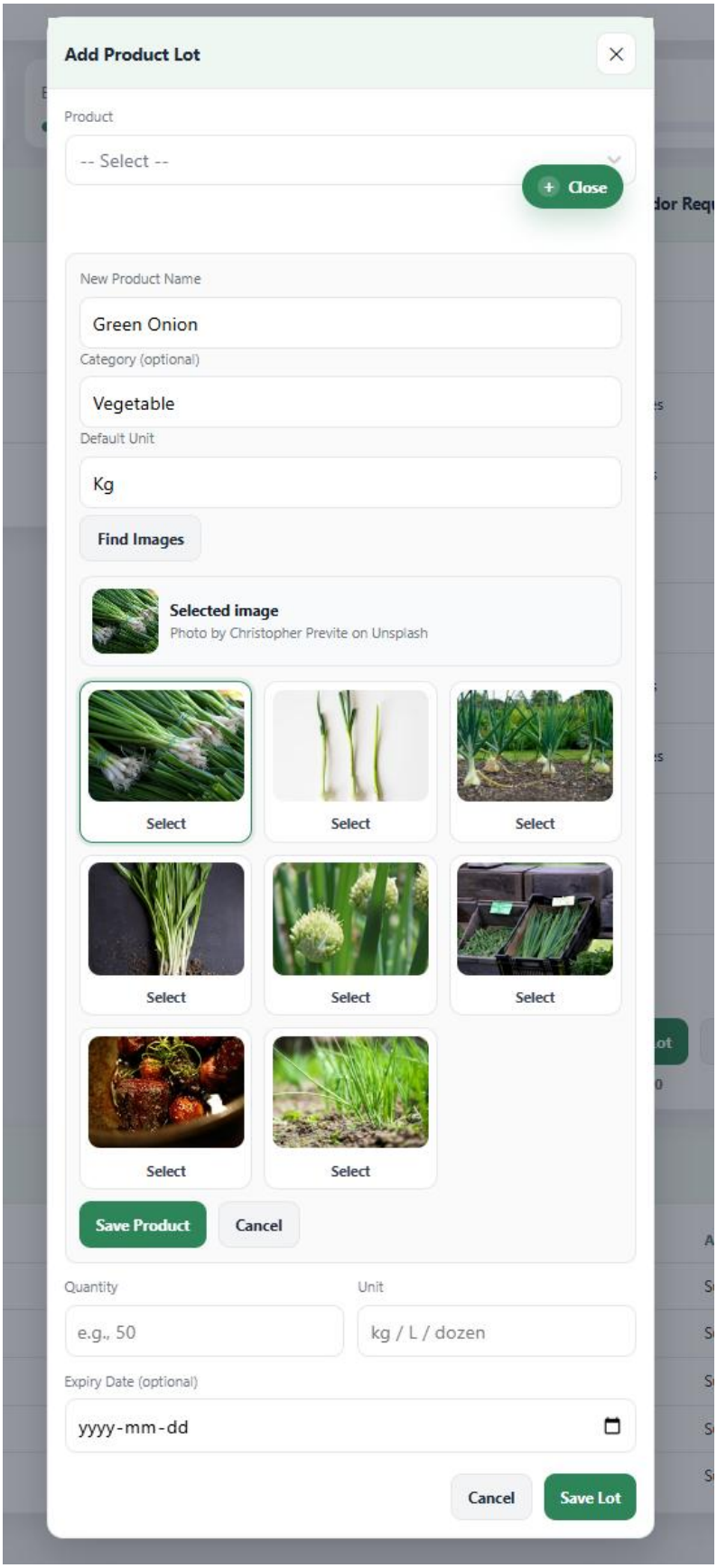


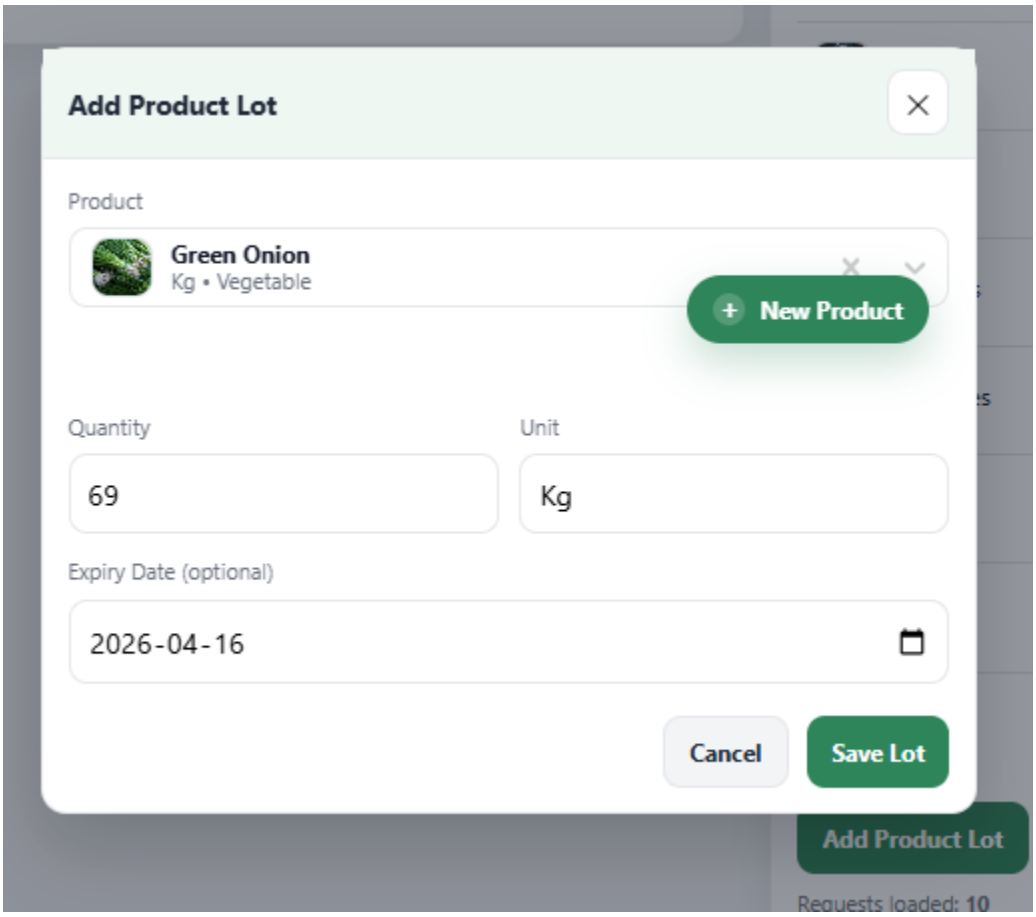


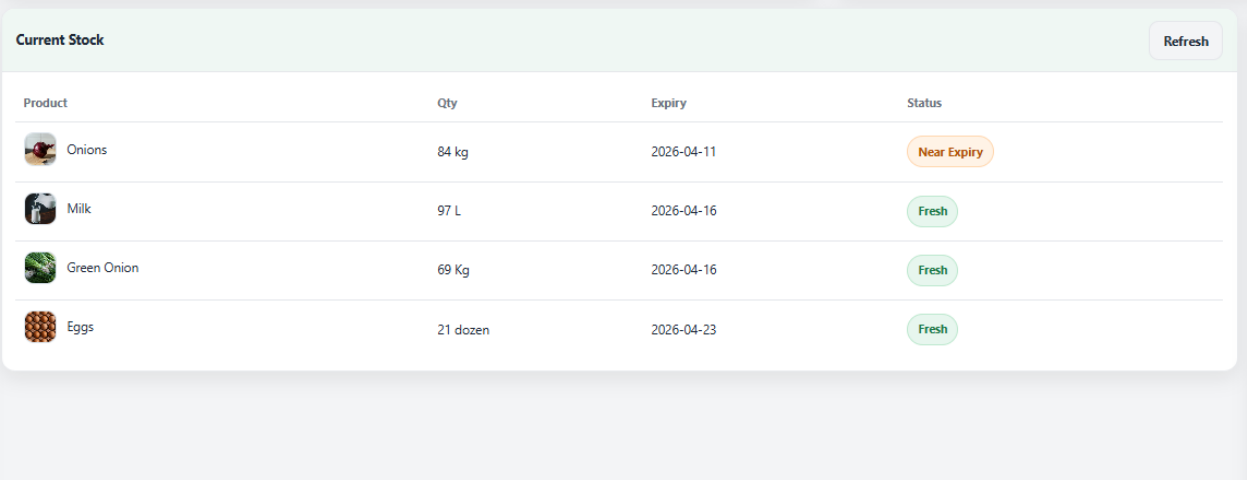

Add Product Lot
Product
-- Select --
Close
New Product Name
e.g., Carrots
Category (optional)
e.g., Vegetables
Default Unit
e.g., kg / L / dozen
Find Images
Save Product
Cancel
Quantity
e.g., 50
Unit
kg / L / dozen
Expiry Date (optional)
yyyy-mm-dd
Cancel
Save Lot
Distance (km)


Welcome, Farmer 1
Add Product Lot
Bell pepper
Find Images
Save Product
Cancel
Save Lot

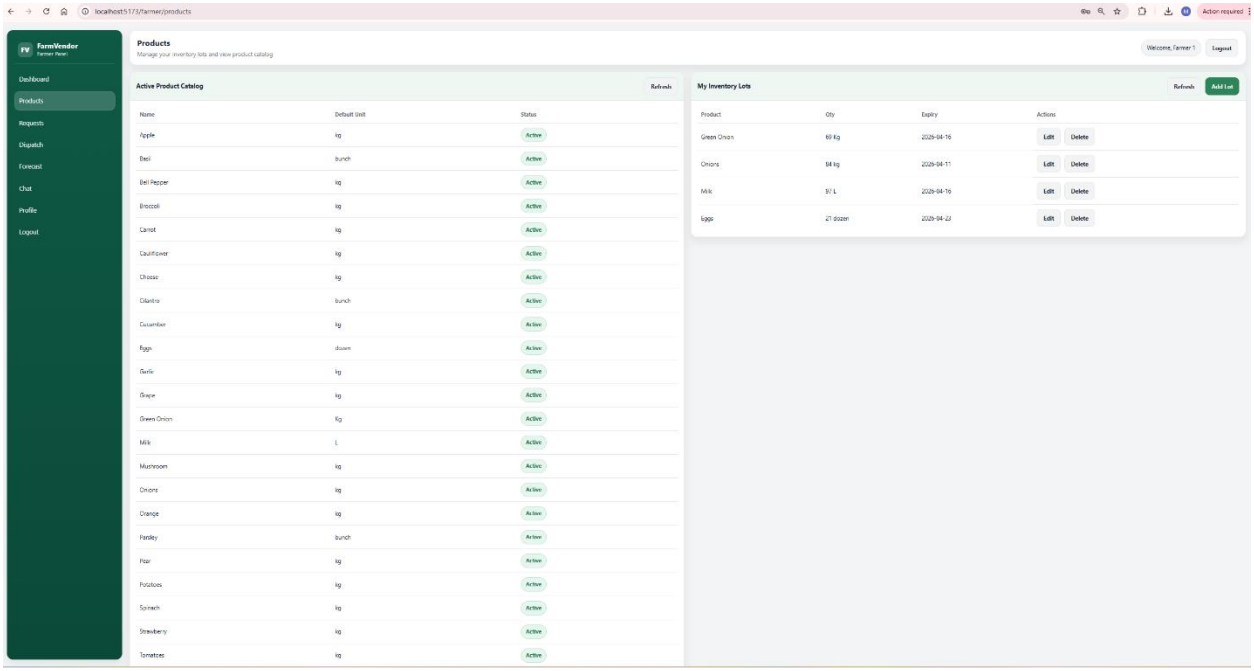

Add Product Lot
Please check this form
A product with this name already exists.
Product
-- Select --
Close
New Product Name
Bell pepper
Category (optional)
Vegetable
Default Unit
Kg
Find Images
Selected image
Photo by Rens D on Unsplash
Select
Select
Select
Select
Select
Select
Select
Select
Select
Save Product
Cancel
Quantity
e.g., 50
Unit
kg / L / dozen
Expiry Date (optional)
yyyy-mm-dd
Cancel
Save Lot
Address
Surrey, BC, V
Surrey, BC, V
Surrey, BC, V
Surrey, BC, V
Surrey, BC, V
Create Disp


Add Product Lot
Product
-- Select --
+ Close
New Product Name
Green Onion
Category (optional)
Vegetable
Default Unit
Kg
Find Images
Selected image
Photo by Christopher Previte on Unsplash
Select
Select
Select
Select
Select
Select
Select
Select
Save Product
Cancel
Quantity
e.g., 50
Unit
kg / L / dozen
Expiry Date (optional)
yyyy-mm-dd
Cancel
Save Lot


Add Product Lot
Product
Green Onion
Kg • Vegetable
New Product
Quantity
69
Unit
Kg
Expiry Date (optional)
2026-04-16
Cancel
Save Lot
Add Product Lot
Requests loaded: 10


Current Stock
Refresh
Product
Qty
Expiry
Status
Onions
84 kg
2026-04-11
Near Expiry
Milk
97 L
2026-04-16
Fresh
Green Onion
69 Kg
2026-04-16
Fresh
Eggs
21 dozen
2026-04-23
Fresh

- Product interface



Frontend

- FarmerDashboard.jsx

```
FarmerDashboard.jsx
W26_4495_S3_ManeeshaE > implementations > FarmVendor > farmvendor-web > src > pages > FarmerDashboard.jsx > FarmerDashboard
11  export default function FarmerDashboard() {
746                         requests.map((row, idx) => (
827                                             <div>
828                                               <span className="request-hover-label">Address:</span>{" "}
829                                               {[
830                                                 hoveredRequest.vendorCity,
831                                                 hoveredRequest.vendorProvince,
832                                                 hoveredRequest.vendorPostalCode,
833                                               ]
834                                                 .filter(Boolean)
835                                                 .join(", ") || "N/A"}
836                                             </div>
837
838                                             <div>
839                                               <span className="request-hover-label">Needed By:</span>{" "}
840                                               {new Date(hoveredRequest.neededBy).toISOString().slice(0, 10)}
841                                             </div>
842                                           </div>
843                                         </div>
844                                       </div>
845                                     </td>
846                                   </tr>
847                                 )}
848                               </React.Fragment>
849                             ))
850                           )}
851                         </tbody>
852                       </table>
853
854                       <div style={{ marginTop: 12, display: "flex", gap: 10, flexWrap: "wrap" }}>
855                         <button className="btn btn-primary" onClick={openAddLot}>
856                           Add Product Lot
857                         </button>
858                         <button className="btn btn-secondary" onClick={openCreateDispatch}>
859                           Create Dispatch
860                         </button>
861                       </div>
862
863                       <div style={{ marginTop: 8, fontSize: 12, color: "#6b7280" }}>
864                         Requests loaded: <b>{requests.length}</b>
865                       </div>
866                     </div>
867                   </div>
868                 </section>
869
870                 <section style={{ marginTop: 14 }}>
871                   <div className="card">
872                     <div className="card-header">
873                       <h2>Top Vendors for Dispatching</h2>
874                       <button
875                         className="btn btn-secondary"
876                         onClick={loadRecommendedVendors}
877                         disabled={loadingRecommendations}
878                       >
879                         {loadingRecommendations ? "Loading..." : "Refresh"}
880                       </button>
881                     </div>
882
883                     <div className="card-body">
884                       <table className="table">
885                         <thead>
886                           <tr>
887                             <th>Vendor</th>
```


FarmerDashboard.jsx M
W26_4495_S3_ManeeshaE > implementations > FarmVendor > farmvendor-web > src > pages > FarmerDashboard.jsx
export default function FarmerDashboard() {
//add product
const openAddLot = async () => {
setPageError("");
setLotError("");
setShowAddLot(true);
setShowNewProductFields(false);
setNewProductForm({
name: "",
category: "",
defaultUnit: "",
});
setLotForm({
productId: "",
quantityAvailable: "",
unit: "",
expiryDate: "",
});
setImageSearchResults([]);
setSelectedImage(null);
if (products.length === 0) {
try {
await loadProducts();
} catch (e) {
setLotError(e?.message || "Failed to load products for lot form");
}
}
};
const submitNewProduct = async () => {
setLotError("");
if (!newProductForm.name.trim()) {
return setLotError("Product name is required.");
}
if (!newProductForm.defaultUnit.trim()) {
return setLotError("Default unit is required.");
}
setSavingProduct(true);
try {
const res = await fetch(`${API_BASE}/api/products`, {
method: "POST",
headers: authHeaders,
body: JSON.stringify({
name: newProductForm.name.trim(),
category: newProductForm.category.trim()
? newProductForm.category.trim()
: null,
defaultUnit: newProductForm.defaultUnit.trim(),
imageUrl: selectedImage?.imageUrl || null,
imageThumbUrl: selectedImage?.thumbnailUrl || null,
imageSource: selectedImage?.source || null,
photographerName: selectedImage?.photographerName || null,
photographerProfile: selectedImage?.photographerProfile || null,
}),

FarmerDashboard.jsx M

W26_4495_S3_ManeeshaE > implementations > FarmVendor > farmvendor-web > src > pages > FarmerDashbo

```
11  export default function FarmerDashboard() {
289
290   const submitNewProduct = async () => {
291     setLotError("");
292
293     if (!newProductForm.name.trim()) {
294       return setLotError("Product name is required.");
295     }
296
297     if (!newProductForm.defaultUnit.trim()) {
298       return setLotError("Default unit is required.");
299     }
300
301     setSavingProduct(true);
302
303     try {
304       const res = await fetch(`${API_BASE}/api/products`, {
305         method: "POST",
306         headers: authHeaders,
307         body: JSON.stringify({
308           name: newProductForm.name.trim(),
309           category: newProductForm.category.trim()
310             ? newProductForm.category.trim()
311             : null,
312           defaultUnit: newProductForm.defaultUnit.trim(),
313           imageUrl: selectedImage?.imageUrl || null,
314           imageThumbUrl: selectedImage?.thumbnailUrl || null,
315           imageSource: selectedImage?.source || null,
316           photographerName: selectedImage?.photographerName || null,
317           photographerProfile: selectedImage?.photographerProfile || null,
318         }),
319       });
320
321       const text = await res.text();
322       if (!res.ok) throw new Error(text || "Failed to create product.");
323
324       const createdProduct = JSON.parse(text);
325
326       await loadProducts();
327
328       setLotForm((prev) => ({
329         ...prev,
330         productId: String(createdProduct.productId),
331         unit: createdProduct.defaultUnit || prev.unit,
332       }));
333
334       setNewProductForm({
335         name: "",
336         category: "",
337         defaultUnit: "",
338       });
339
340       setImageSearchResults([]);
341       setSelectedImage(null);
342       setShowNewProductFields(false);
343       setLotError("");
344     } catch (err) {
345       setLotError(err?.message || "Failed to create product.");
346     } finally {
347       setSavingProduct(false);
348     }
349   };
350
```

FarmerDashboard.jsx M

W26_4495_S3_ManeeshaE > implementations > FarmVendor > farmvendor-web > src > pages > FarmerDashboard.j

```
11   export default function FarmerDashboard() {
290    const submitNewProduct = async () => {
349    };
350
351    const submitLot = async (e) => {
352      e.preventDefault();
353      setLotError("");
354
355      if (!lotForm.productId) return setLotError("Please select a product.");
356      if (!lotForm.quantityAvailable || Number(lotForm.quantityAvailable) <= 0) {
357        return setLotError("Quantity must be greater than 0.");
358      }
359
360      setSavingLot(true);
361      try {
362        const payload = {
363          productId: Number(lotForm.productId),
364          quantityAvailable: Number(lotForm.quantityAvailable),
365          unit: lotForm.unit?.trim() ? lotForm.unit.trim() : null,
366          expiryDate: lotForm.expiryDate
367            ? new Date(lotForm.expiryDate).toISOString()
368            : null,
369        };
370
371        const res = await fetch(`${API_BASE}/api/InventoryLots`, {
372          method: "POST",
373          headers: authHeaders,
374          body: JSON.stringify(payload),
375        });
376
377        const text = await res.text();
378        if (!res.ok) throw new Error(text || "Failed to create inventory lot.");
379
380        setShowAddLot(false);
381        setLotError("");
382        setLotForm({
383          productId: "",
384          quantityAvailable: "",
385          unit: "",
386          expiryDate: "",
387        });
388
389        await loadDashboard();
390        await loadRecommendedVendors();
391      } catch (err) {
392        setLotError(err?.message || "Failed to save inventory lot");
393      } finally {
394        setSavingLot(false);
395      }
396    };
397
398    const openCreateDispatch = async () => {
399      setPageError("");
400      setDispatchError("");
401
402      try {
403        if (requests.length === 0) {
404          await loadDashboard();
405        }
406      } catch (e) {
407        setDispatchError(e?.message || "Failed to load requests");
408      }
409
```

FarmerDashboard.jsx M

V26_4495_S3_ManeeshaE > implementations > FarmVendor > farmvendor-web > src > pages > FarmerDashboard.jsx > Fa

```
11  export default function FarmerDashboard() {
928
929       {showAddLot && (
930         <div className="modal-backdrop" onClick={() => setShowAddLot(false)}>
931           <div className="modal" onClick={(e) => e.stopPropagation()}>
932             <div className="modal-header">
933               <h3>Add Product Lot</h3>
934               <button className="icon-btn" onClick={() => setShowAddLot(false)}>
935                 ×
936               </button>
937             </div>
938
939             <form onSubmit={submitLot} className="modal-body">
940               {lotError && (
941                 <div className="modal-notice modal-notice-error" role="alert">
942                   <div className="modal-notice-icon">!</div>
943                   <div className="modal-notice-content">
944                     <div className="modal-notice-title">Please check this form</div>
945                     <div className="modal-notice-text">{lotError}</div>
946                   </div>
947                   <button
948                     type="button"
949                     className="modal-notice-close"
950                     onClick={() => setLotError("")}
951                     aria-label="Dismiss error"
952                   >
953                     ×
954                   </button>
955                 </div>
956               )}
957
958               <div className="product-picker-wrap">
959                 <label className="field">
960                   <span>Product</span>
961
962                   <Select
963                     className="product-image-select"
964                     classNamePrefix="product-image-select"
965                     options={productOptions}
966                     value={selectedProductOption}
967                     onChange={(selected) => {
968                       setLotForm((prev) => ({
969                         ...prev,
970                         productId: selected ? String(selected.value) : "",
971                         unit: selected?.defaultUnit ?? "",
972                       }));
973
974                       if (lotError) setLotError("");
975                     }}
976                     placeholder="-- Select --"
977                     isClearable
978                     menuPortalTarget={document.body}
979                     styles={selectStyles}
980                     formatOptionLabel={renderProductOptionLabel}
981                     noOptionsMessage={() => "No products found"}
982                   />
983                 </label>
984
985                 <button
986                   type="button"
987                   className="fab-add-product"
988                   onClick={() => {
989                     setShowNewProductFields((prev) => !prev);
```

FarmerDashboard.jsx M

W26_4495_S3_ManeeshaE > implementations > FarmVendor > farmvendor-web > src > pages > FarmerDashboard.jsx

```jsx
export default function FarmerDashboard() {
                </label>

                <button
                  type="button"
                  className="fab-add-product"
                  onClick={() => {
                    setShowNewProductFields((prev) => !prev);
                    setLotError("");
                  }}
                  title="Add a new product"
                >
                  <span className="fab-plus">+</span>
                  <span>{showNewProductFields ? "Close" : "New Product"}</span>
                </button>
              </div>

              <div className="new-product-section">
                {showNewProductFields && (
                  <div className="new-product-card">
                    <label className="field">
                      <span>New Product Name</span>
                      <input
                        value={newProductForm.name}
                        onChange={(e) =>
                          setNewProductForm((prev) => ({
                            ...prev,
                            name: e.target.value,
                          }))
                        }
                        placeholder="e.g., Carrots"
                      />
                    </label>

                    <label className="field">
                      <span>Category (optional)</span>
                      <input
                        value={newProductForm.category}
                        onChange={(e) =>
                          setNewProductForm((prev) => ({
                            ...prev,
                            category: e.target.value,
                          }))
                        }
                        placeholder="e.g., Vegetables"
                      />
                    </label>

                    <label className="field">
                      <span>Default Unit</span>
                      <input
                        value={newProductForm.defaultUnit}
                        onChange={(e) =>
                          setNewProductForm((prev) => ({
                            ...prev,
                            defaultUnit: e.target.value,
                          }))
                        }
                        placeholder="e.g., kg / L / dozen"
                      />
                    </label>

                    <div className="image-search-toolbar">
```


FarmerDashboard.jsx M
W26_4495_S3_ManeeshaE > implementations > FarmVendor > farmvendor-web > src > pages > FarmerDashboard.jsx > Farme
11 export default function FarmerDashboard() {
1044
1045 <button
1046 type="button"
1047 className="btn btn-secondary"
1048 onClick={searchProductImages}
1049 disabled={searchingImages}
1050 >
1051 {searchingImages ? "Searching..." : "Find Images"}
1052 </button>
1053
1054
1055 {selectedImage && (
1056
1057 <img
1058 src={selectedImage.thumbnailUrl || selectedImage.imageUrl}
1059 alt={selectedImage.alt || newProductForm.name}
1060 />
1061
1062 Selected image
1063
1064 {selectedImage.photographerName
1065 ? `Photo by ${selectedImage.photographerName} on Unsplash`
1066 : "Unsplash image selected"}
1067
1068
1069
1070)}
1071
1072 {imageSearchResults.length > 0 && (
1073
1074 {imageSearchResults.map((img, idx) => (
1075 <button
1076 key={`${img.imageUrl}-${idx}`}
1077 type="button"
1078 className={`image-card ${
1079 selectedImage?.imageUrl === img.imageUrl ? "selected" : ""
1080 }`}
1081 onClick={() => setSelectedImage(img)}
1082 >
1083 <img
1084 src={img.thumbnailUrl || img.imageUrl}
1085 alt={img.alt || "product"}
1086 />
1087 Select
1088 </button>
1089))}
1090
1091)}
1092
1093
1094 <button
1095 type="button"
1096 className="btn btn-primary"
1097 onClick={submitNewProduct}
1098 disabled={savingProduct}
1099 >
1100 {savingProduct ? "Saving..." : "Save Product"}
1101 </button>
1102
1103 <button
1104 type="button"
1105 className="btn btn-secondary"


FarmerDashboard.jsx M
W26_4495_S3_ManeeshaE > implementations > FarmVendor > farmvendor-web > src > pages > FarmerDashboard.jsx > FarmerDash
11 export default function FarmerDashboard() {
1091)}
1092
1093
1094 <button
1095 type="button"
1096 className="btn btn-primary"
1097 onClick={submitNewProduct}
1098 disabled={savingProduct}
1099 >
1100 {savingProduct ? "Saving..." : "Save Product"}
1101 </button>
1102
1103 <button
1104 type="button"
1105 className="btn btn-secondary"
1106 onClick={() => {
1107 setShowNewProductFields(false);
1108 setLotError("");
1109 setImageSearchResults([]);
1110 setSelectedImage(null);
1111 }}
1112 >
1113 Cancel
1114 </button>
1115
1116
1117)}
1118
1119
1120
1121 <label className="field">
1122 Quantity
1123 <input
1124 type="number"
1125 step="0.01"
1126 value={lotForm.quantityAvailable}
1127 onChange={(e) =>
1128 setLotForm((prev) => ({ ...prev, quantityAvailable: e.target.value }))
1129 }
1130 placeholder="e.g., 50"
1131 />
1132 </label>
1133
1134 <label className="field">
1135 Unit
1136 <input
1137 value={lotForm.unit}
1138 onChange={(e) =>
1139 setLotForm((prev) => ({ ...prev, unit: e.target.value }))
1140 }
1141 placeholder="kg / L / dozen"
1142 />
1143 </label>
1144
1145
1146 <label className="field">
1147 Expiry Date (optional)
1148 <input
1149 type="date"
1150 value={lotForm.expiryDate}
1151 onChange={(e) =>
1152 setLotForm((prev) => ({ ...prev, expiryDate: e.target.value }))


FarmerDashboard.jsx M
W26_4495_S3_ManeeshaE > implementations > FarmVendor > farmvendor-web > src > pages > FarmerDashboard.jsx > FarmerDashboard
11 export default function FarmerDashboard() {
1118
1119
1120
1121 <label className="field">
1122 Quantity
1123 <input
1124 type="number"
1125 step="0.01"
1126 value={lotForm.quantityAvailable}
1127 onChange={(e) =>
1128 setLotForm((prev) => ({ ...prev, quantityAvailable: e.target.value }))
1129 }
1130 placeholder="e.g., 50"
1131 />
1132 </label>
1133
1134 <label className="field">
1135 Unit
1136 <input
1137 value={lotForm.unit}
1138 onChange={(e) =>
1139 setLotForm((prev) => ({ ...prev, unit: e.target.value }))
1140 }
1141 placeholder="kg / L / dozen"
1142 />
1143 </label>
1144
1145
1146 <label className="field">
1147 Expiry Date (optional)
1148 <input
1149 type="date"
1150 value={lotForm.expiryDate}
1151 onChange={(e) =>
1152 setLotForm((prev) => ({ ...prev, expiryDate: e.target.value }))
1153 }
1154 />
1155 </label>
1156
1157
1158 <button
1159 type="button"
1160 className="btn btn-secondary"
1161 onClick={() => {
1162 setShowAddLot(false);
1163 setLotError("");
1164 setImageSearchResults([]);
1165 setSelectedImage(null);
1166 }}
1167 >
1168 Cancel
1169 </button>
1170 <button type="submit" className="btn btn-primary" disabled={savingLot}>
1171 {savingLot ? "Saving..." : "Save Lot"}
1172 </button>
1173
1174 </form>
1175
1176
1177)}
1178
1179 {showCreateDispatch && (
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

- FarmerProduct.jsx

FarmerDashboard.jsx M | FarmerProducts.jsx

W26_4495_S3_ManeeshaE > implementations > FarmVendor > farmvendor-web > src > pages > FarmerProducts.jsx > ...

```jsx
import React, { useEffect, useMemo, useState } from "react";
import { useNavigate } from "react-router-dom";
import "../styles/dashboard.css";
import FarmerSidebar from "../assets/components/FarmerSidebar";

const API_BASE = import.meta.env.VITE_API_URL; // e.g., http://localhost:5136

export default function FarmerProducts() {
  const navigate = useNavigate();
  const token = localStorage.getItem("token");
  const displayName = localStorage.getItem("displayName") || "Farmer";

  const authHeaders = useMemo(() => {
    return {
      "Content-Type": "application/json",
      Authorization: `Bearer ${token}`,
    };
  }, [token]);

  const [loading, setLoading] = useState(true);
  const [error, setError] = useState("");

  // Catalog
  const [products, setProducts] = useState([]);

  // My lots
  const [lots, setLots] = useState([]);

  // Add Lot modal
  const [showAddLot, setShowAddLot] = useState(false);
  const [lotForm, setLotForm] = useState({
    productId: "",
    quantityAvailable: "",
    unit: "",
    expiryDate: "",
  });
  const [savingLot, setSavingLot] = useState(false);

  // Edit Lot modal
  const [showEditLot, setShowEditLot] = useState(false);
  const [editLot, setEditLot] = useState(null);
  const [savingEdit, setSavingEdit] = useState(false);

  const logout = () => {
    localStorage.clear();
    navigate("/login");
  };

  const loadCatalog = async () => {
    const res = await fetch(`${API_BASE}/api/products/active`, { headers: authHeaders });
    const text = await res.text();
    if (!res.ok) throw new Error(text || "Failed to load products");
    setProducts(JSON.parse(text));
  };

  const loadMyLots = async () => {
    const res = await fetch(`${API_BASE}/api/farmer/inventorylots`, { headers: authHeaders });
    const text = await res.text();
    if (!res.ok) throw new Error(text || "Failed to load inventory lots");
    setLots(JSON.parse(text));
  };

  const loadAll = async () => {
```

Backend

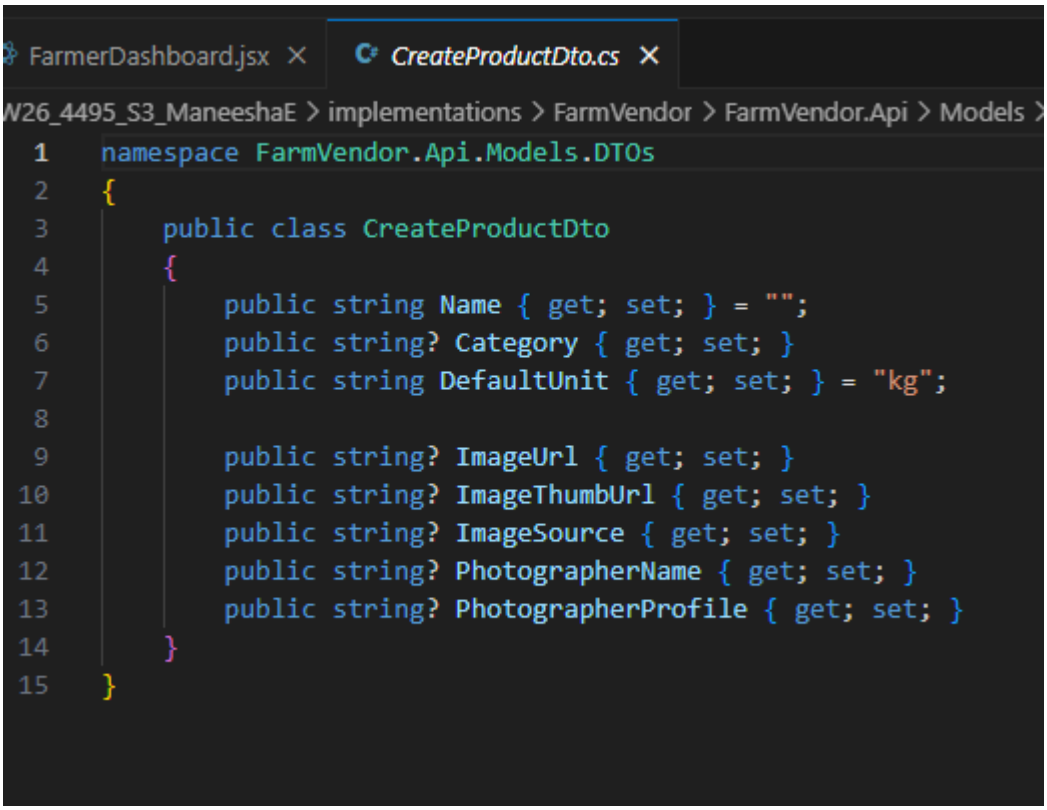
- ProductController.cs

FarmerDashboard.jsx | ProductsController.cs

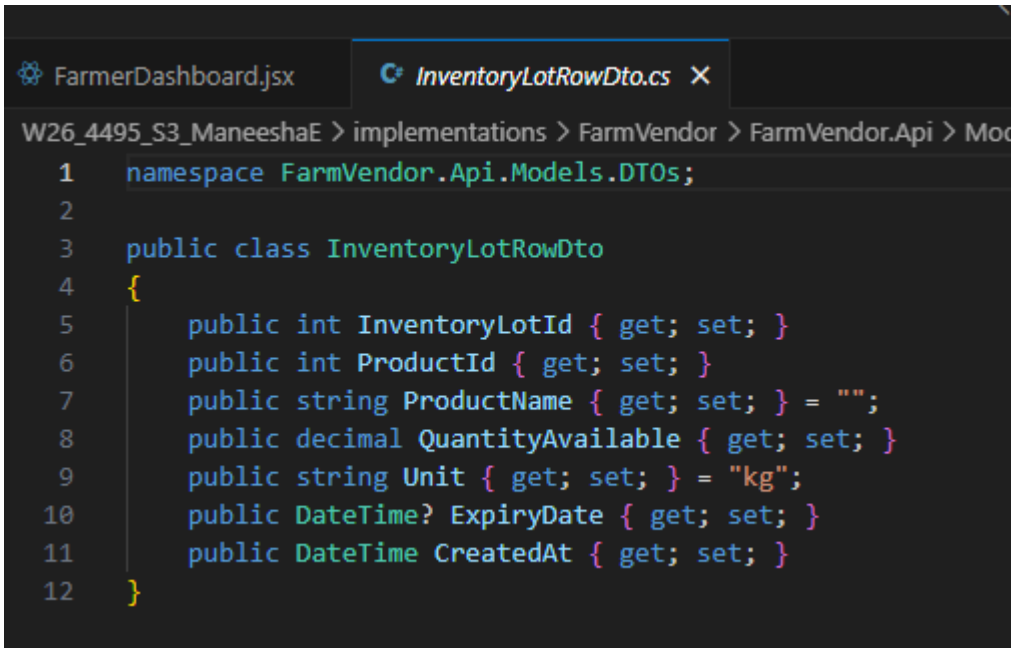
V26_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Controllers > ProductsController.cs

```
using System.Net.Http.Headers;
using System.Text.Json;
using FarmVendor.Api.Data;
using FarmVendor.Api.Models;
using FarmVendor.Api.Models.DTOs;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

namespace FarmVendor.Api.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    [Authorize]
    public class ProductsController : ControllerBase
    {
        private readonly AppDbContext _db;
        private readonly IConfiguration _config;

        public ProductsController(AppDbContext db, IConfiguration config)
        {
            _db = db;
            _config = config;
        }

        [HttpGet("active")]
        public async Task<IActionResult> GetActive()
        {
            var items = await _db.Product
                .Where(p => p.IsActive)
                .OrderBy(p => p.Name)
                .Select(p => new
                {
                    p.ProductId,
                    p.Name,
                    p.Category,
                    p.DefaultUnit,
                    p.ImageUrl,
                    p.ImageThumbUrl,
                    p.ImageSource,
                    p.PhotographerName,
                    p.PhotographerProfile
                })
                .ToListAsync();

            return Ok(items);
        }

        [HttpGet("search-images")]
        public async Task<IActionResult> SearchImages([FromQuery] string query)
        {
            if (string.IsNullOrWhiteSpace(query))
                return BadRequest("Query is required.");

            var accessKey = _config["Unsplash:AccessKey"];
            if (string.IsNullOrWhiteSpace(accessKey))
                return StatusCode(500, "Unsplash access key is not configured.");

            using var httpClient = new HttpClient();
            httpClient.DefaultRequestHeaders.Authorization =
                new AuthenticationHeaderValue("Client-ID", accessKey);
```

- CreateProductDTO.cs



- InventoryLotDTO.cs



- UpdateInventoryLotDTO.cs

FarmerDashboard.jsx | UpdateInventoryLotDto.cs

W26_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Models > DTOs

```
1   namespace FarmVendor.Api.Models.DTOs;
2
3   public class UpdateInventoryLotDto
4   {
5       public decimal QuantityAvailable { get; set; }
6       public string? Unit { get; set; }
7       public DateTime? ExpiryDate { get; set; }
8   }
```

- InventoryLot.cs

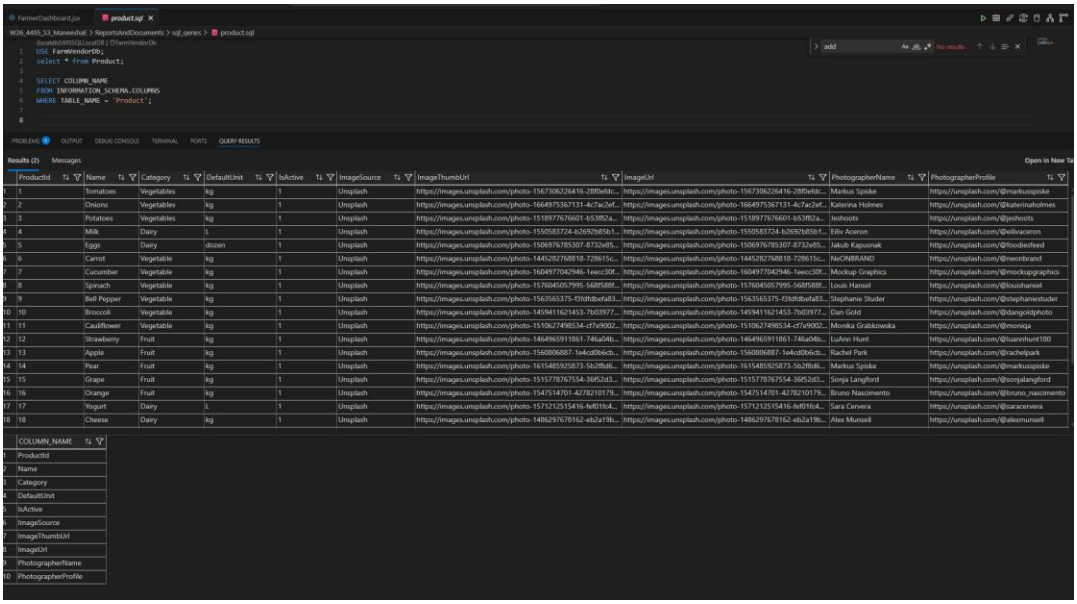
FarmerDashboard.jsx | InventoryLot.cs

W26_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Models

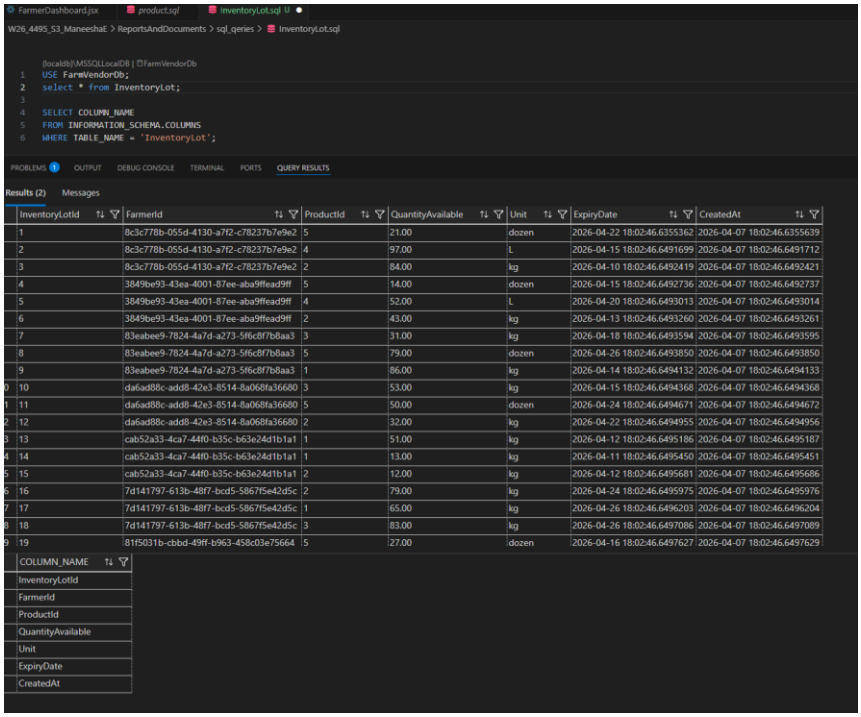
```
1   namespace FarmVendor.Api.Models;
2
3   public class InventoryLot
4   {
5       public int InventoryLotId { get; set; }
6
7       public string FarmerId { get; set; } = "";
8       public ApplicationUser Farmer { get; set; } = null!;
9
10      public int ProductId { get; set; }
11      public Product Product { get; set; } = null!;
12
13      public decimal QuantityAvailable { get; set; }
14
15      public string Unit { get; set; } = "kg";
16
17      public DateTime? ExpiryDate { get; set; }
18
19      public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
20  }
```

Database

- Product table



- InventoryLot table



Feature 6: Vendor Demand Request System

This feature allows vendors to create demand requests for products when stock is low. It supports structured communication of vendor needs and acts as a core input to forecasting and dispatch planning.

This feature includes:

- Low stock alert table
- Demand request creation
- Open / Fulfilled / Cancelled lifecycle tracking
- Product-based request creation

The design consists of:

- Low stock visibility
- Demand request creation form
- Request status lifecycle management

Implementation

Frontend

- VendorDashboard.jsx
- Create demand request button
- Low stock alert UI
- Open request list

Backend

- VendorDemandRequestsController.cs
- DemandRequest.cs
- CreateDemandRequestDto.cs
- Request status handling logic

Database

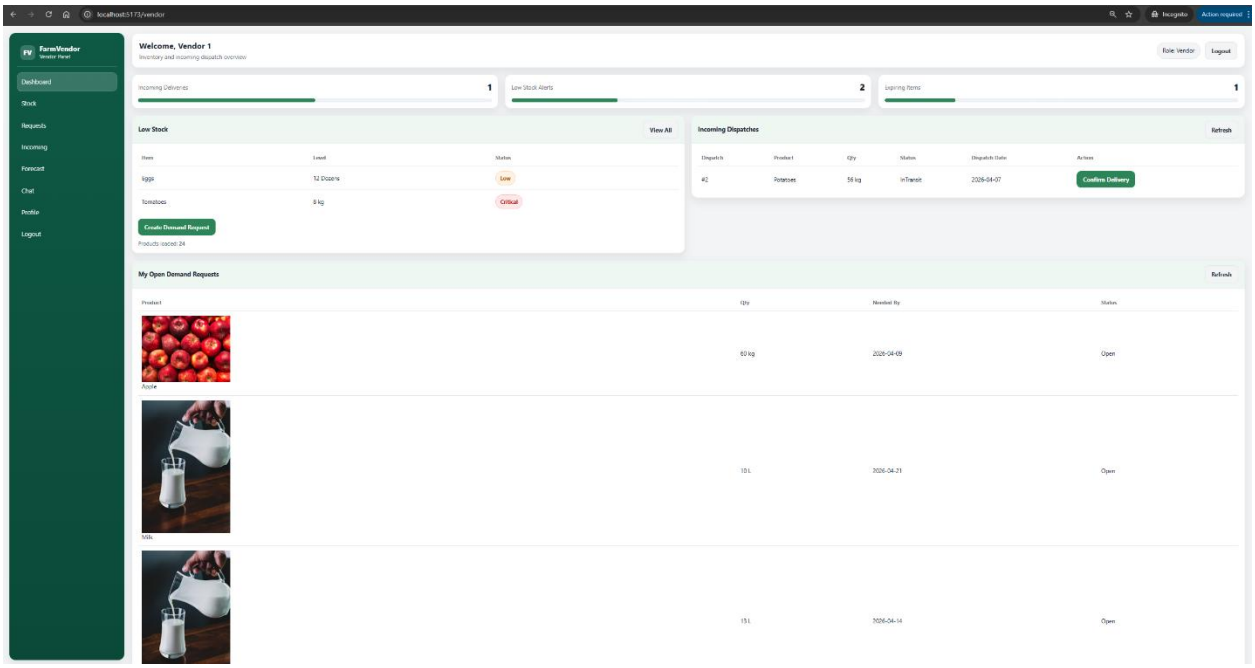
DemandRequest table storing:

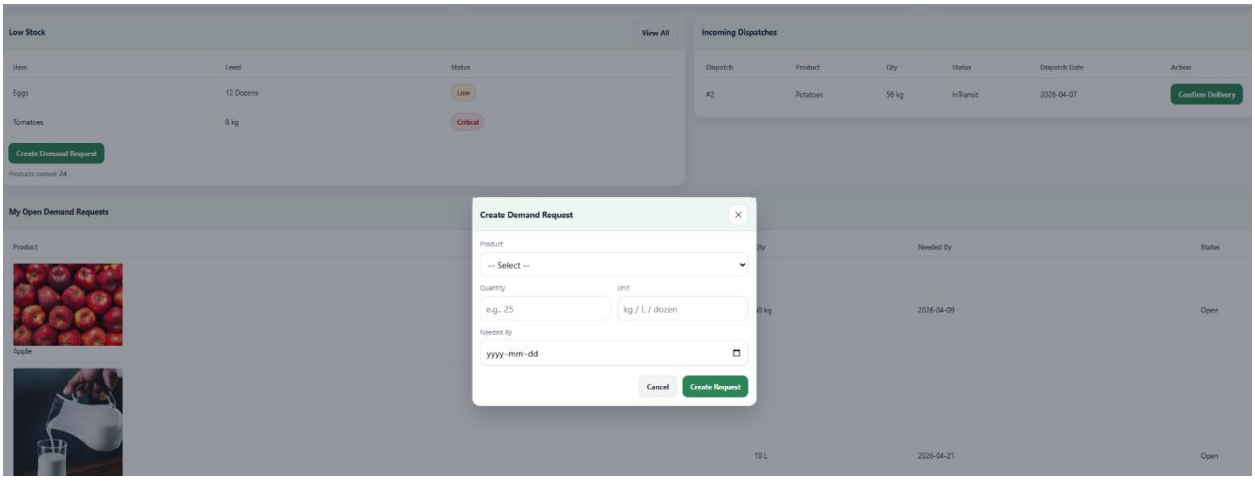
- VendorId
- ProductId
- QuantityRequested
- Unit

- NeededBy
- Status
- CreatedAt

App

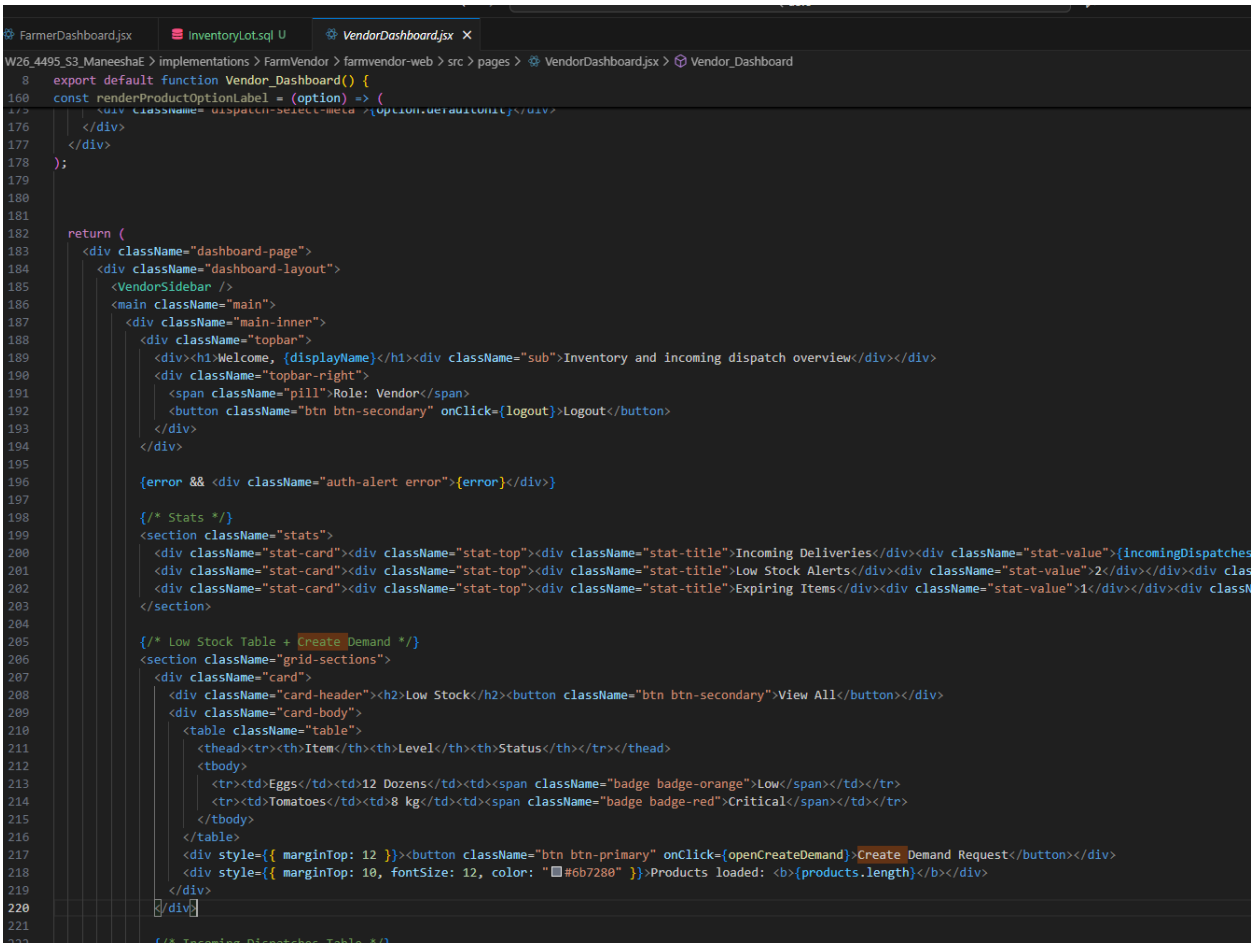
- VendorDashboard





Frontend

- VendorDashboard.jsx




armerDashboard.jsx
InventoryLot.sql U
VendorDashboard.jsx
_4495_S3_ManeeshaE > implementations > FarmVendor > farmvendor-web > src > pages > VendorDashboard.jsx > Vendor_Dashboard > openCreateDemand
export default function Vendor_Dashboard() {
// Products for dropdown
const [products, setProducts] = useState([]);
const [myRequests, setMyRequests] = useState([]);
const [incomingDispatches, setIncomingDispatches] = useState([]);
const [confirmingId, setConfirmingId] = useState(null);
const [showCreateDemand, setShowCreateDemand] = useState(false);
const [demandForm, setDemandForm] = useState({
productId: "",
quantityRequested: "",
unit: "",
neededBy: "",
});
const [savingDemand, setSavingDemand] = useState(false);
const authHeaders = useMemo(() => ({
"Content-Type": "application/json",
Authorization: `Bearer ${token}`,
}), [token]);
const logout = () => {
localStorage.removeItem("token");
localStorage.removeItem("role");
localStorage.removeItem("displayName");
navigate("/login");
};
// ---------- API LOADERS ----------
const loadProducts = async () => {
const res = await fetch(`${API_BASE}/api/products/active`, { headers: authHeaders });
const text = await res.text();
if (!res.ok) throw new Error(text || "Failed to load products");
setProducts(JSON.parse(text));
};
const loadMyRequests = async () => {
const res = await fetch(`${API_BASE}/api/vendor/demandrequests?status=Open`, { headers: authHeaders });
const text = await res.text();
if (!res.ok) throw new Error(text || "Failed to load vendor requests");
setMyRequests(JSON.parse(text));
};
const loadIncomingDispatches = async () => {
const res = await fetch(`${API_BASE}/api/vendor/dispatches/incoming`, { headers: authHeaders });
const text = await res.text();
if (!res.ok) throw new Error(text || "Failed to load incoming dispatches");
setIncomingDispatches(JSON.parse(text));
};
const loadDashboard = async () => {
await Promise.all([loadProducts(), loadMyRequests(), loadIncomingDispatches()]);
};
const openCreateDemand = async () => {
setError("");
try { if (products.length === 0) await loadProducts(); } catch (e) { setError(e?.message || "Failed to load products"); return; }
setDemandForm({ productId: "", quantityRequested: "", unit: "", neededBy: "" });
setShowCreateDemand(true);
};

FarmerDashboard.jsx | InventoryLot.sql U | VendorDashboard.jsx

26_4495_S3_ManeeshaE > implementations > FarmVendor > farmvendor-web > src > pages > VendorDashboard.jsx > Vendor_Dashboard > submitDemandRequest

```
export default function Vendor_Dashboard() {
  const loadMyRequests = async () => {
    if (!res.ok) throw new Error(text || "Failed to load vendor requests");
    setMyRequests(JSON.parse(text));
  };

  const loadIncomingDispatches = async () => {
    const res = await fetch(`${API_BASE}/api/vendor/dispatches/incoming`, { headers: authHeaders });
    const text = await res.text();
    if (!res.ok) throw new Error(text || "Failed to load incoming dispatches");
    setIncomingDispatches(JSON.parse(text));
  };

  const loadDashboard = async () => {
    await Promise.all([loadProducts(), loadMyRequests(), loadIncomingDispatches()]);
  };

  const openCreateDemand = async () => {
    setError("");
    try { if (products.length === 0) await loadProducts(); } catch (e) { setError(e?.message || "Failed to load products"); return; }
    setDemandForm({ productId: "", quantityRequested: "", unit: "", neededBy: "" });
    setShowCreateDemand(true);
  };

  const submitDemandRequest = async (e) => {
    e.preventDefault();
    setError("");

    if (!demandForm.productId) return setError("Please select a product.");
    if (!demandForm.quantityRequested || Number(demandForm.quantityRequested) <= 0)
      return setError("Quantity must be greater than 0.");
    if (!demandForm.neededBy) return setError("Please select Needed By date.");

    setSavingDemand(true);
    try {
      const payload = {
        productId: Number(demandForm.productId),
        quantityRequested: Number(demandForm.quantityRequested),
        unit: demandForm.unit?.trim() || null,
        neededBy: new Date(demandForm.neededBy).toISOString(),
      };

      const res = await fetch(`${API_BASE}/api/vendor/demandrequests`, {
        method: "POST",
        headers: authHeaders,
        body: JSON.stringify(payload),
      });

      const text = await res.text();
      if (!res.ok) throw new Error(text || "Failed to create demand request.");

      setShowCreateDemand(false);
      await loadMyRequests();
    } catch (err) {
      setError(err?.message || "Failed to create demand request.");
    } finally { setSavingDemand(false); }
  };

  const confirmDelivery = async (dispatchId) => {
```

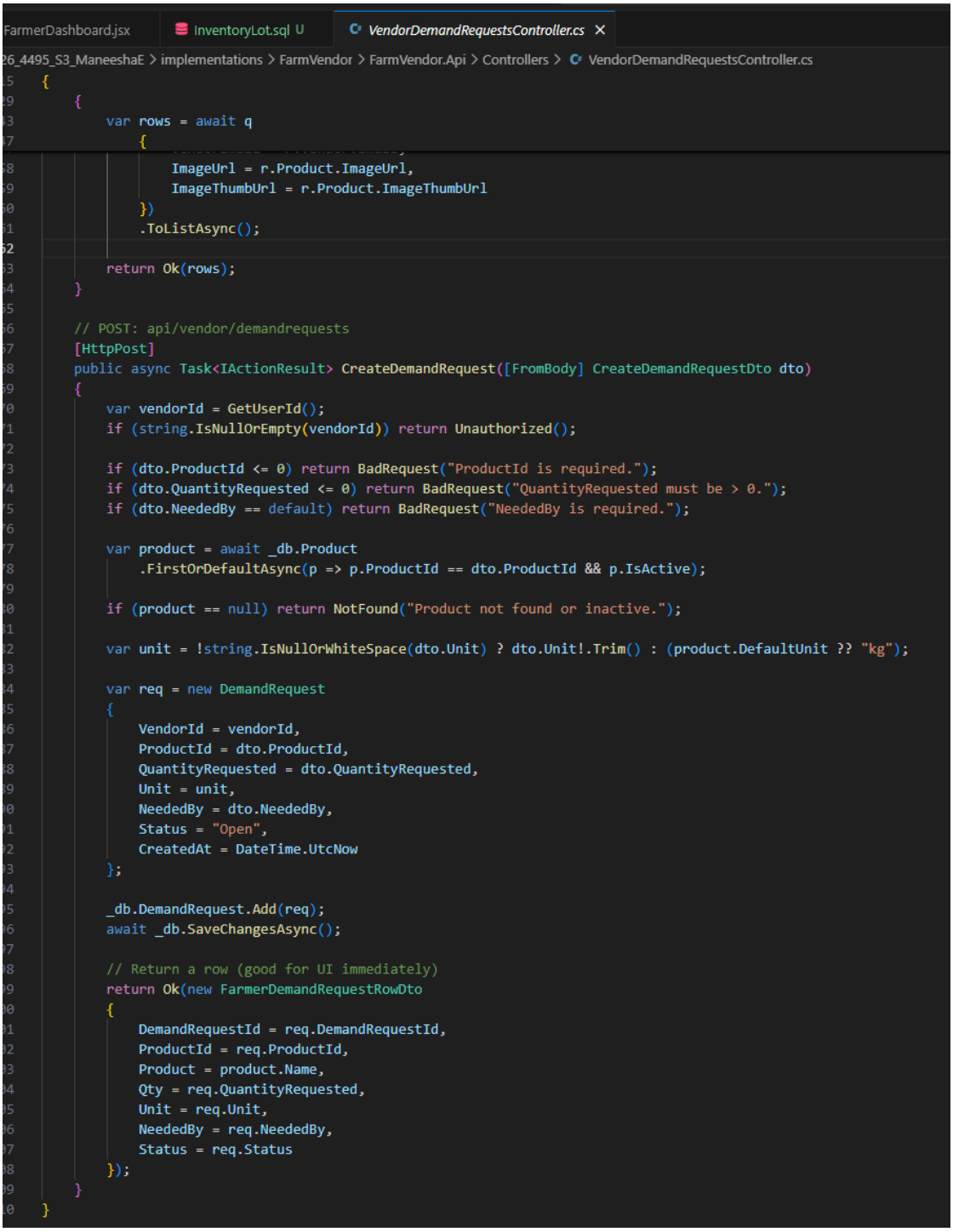
Backend

- VendorDemandRequestController.cs

FarmerDashboard.jsx | InventoryLot.sql U | VendorDemandRequestsController.cs

W26_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Controllers > VendorDemandRequestsController.cs

```
using FarmVendor.Api.Data;
using FarmVendor.Api.Models;
using FarmVendor.Api.Models.DTOs;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using System.Security.Claims;

namespace FarmVendor.Api.Controllers;

[ApiController]
[Route("api/vendor/demandrequests")]
[Authorize] // JWT required
public class VendorDemandRequestsController : ControllerBase
{
    private readonly AppDbContext _db;

    public VendorDemandRequestsController(AppDbContext db)
    {
        _db = db;
    }

    private string GetUserId()
        => User.FindFirstValue(ClaimTypes.NameIdentifier) ?? "";

    // GET: api/vendor/demandrequests?status=Open
    [HttpGet]
    public async Task<IActionResult> GetMyDemandRequests([FromQuery] string? status = null)
    {
        var vendorId = GetUserId();
        if (string.IsNullOrEmpty(vendorId)) return Unauthorized();

        var q = _db.DemandRequest
            .AsNoTracking()
            .Where(r => r.VendorId == vendorId)
            .Include(r => r.Product)
            .Include(r => r.Vendor)
            .AsQueryable();

        if (!string.IsNullOrWhiteSpace(status))
            q = q.Where(r => r.Status == status);

        var rows = await q
            .OrderByDescending(r => r.CreatedAt)
            .Take(100)
            .Select(r => new FarmerDemandRequestRowDto
            {
                DemandRequestId = r.DemandRequestId,
                ProductId = r.ProductId,
                Product = r.Product.Name,
                Qty = r.QuantityRequested,
                Unit = r.Unit,
                NeededBy = r.NeededBy,
                Status = r.Status,
                VendorId = r.VendorId,
                VendorName = r.Vendor.UserName,
                VendorEmail = r.Vendor.Email,
                ImageUrl = r.Product.ImageUrl,
                ImageThumbUrl = r.Product.ImageThumbUrl
            })
            .ToListAsync();

        return Ok(rows);
```


FarmerDashboard.jsx
InventoryLot.sql U
VendorDemandRequestsController.cs
26_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Controllers > VendorDemandRequestsController.cs
{
{
var rows = await q
{
ImageUrl = r.Product.ImageUrl,
ImageThumbUrl = r.Product.ImageThumbUrl
})
.ToListAsync();
return Ok(rows);
}
// POST: api/vendor/demandrequests
[HttpPost]
public async Task<IActionResult> CreateDemandRequest([FromBody] CreateDemandRequestDto dto)
{
var vendorId = GetUserId();
if (string.IsNullOrEmpty(vendorId)) return Unauthorized();
if (dto.ProductId <= 0) return BadRequest("ProductId is required.");
if (dto.QuantityRequested <= 0) return BadRequest("QuantityRequested must be > 0.");
if (dto.NeededBy == default) return BadRequest("NeededBy is required.");
var product = await _db.Product
.FirstOrDefaultAsync(p => p.ProductId == dto.ProductId && p.IsActive);
if (product == null) return NotFound("Product not found or inactive.");
var unit = !string.IsNullOrWhiteSpace(dto.Unit) ? dto.Unit!.Trim() : (product.DefaultUnit ?? "kg");
var req = new DemandRequest
{
VendorId = vendorId,
ProductId = dto.ProductId,
QuantityRequested = dto.QuantityRequested,
Unit = unit,
NeededBy = dto.NeededBy,
Status = "Open",
CreatedAt = DateTime.UtcNow
};
_db.DemandRequest.Add(req);
await _db.SaveChangesAsync();
// Return a row (good for UI immediately)
return Ok(new FarmerDemandRequestRowDto
{
DemandRequestId = req.DemandRequestId,
ProductId = req.ProductId,
Product = product.Name,
Qty = req.QuantityRequested,
Unit = req.Unit,
NeededBy = req.NeededBy,
Status = req.Status
});
}
}

- DemandRequest.cs

FarmerDashboard.jsx | InventoryLot.sql U | *DemandRequest.cs* ×

W26_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Models > DemandRequest.cs

```
1   namespace FarmVendor.Api.Models;
2
3   public class DemandRequest
4   {
5       public int DemandRequestId { get; set; }
6
7       public string VendorId { get; set; } = "";
8       public ApplicationUser Vendor { get; set; } = null!;
9
10      public int ProductId { get; set; }
11      public Product Product { get; set; } = null!;
12
13      public decimal QuantityRequested { get; set; }
14
15      public string Unit { get; set; } = "kg";
16
17      public DateTime NeededBy { get; set; }
18
19      public string Status { get; set; } = "Open";
20      // Open, Accepted, Fulfilled, Cancelled
21
22      public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
23  }
24
```

- CreateDemandRequestDTO.cs

FarmerDashboard.jsx | InventoryLot.sql U | *CreateDemandRequestDto.cs* ×

W26_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Models > DTOs > CreateDemandRequestDto.cs

```
1   namespace FarmVendor.Api.Models.DTOs;
2
3   public class CreateDemandRequestDto
4   {
5       public int ProductId { get; set; }
6       public decimal QuantityRequested { get; set; }
7       public string? Unit { get; set; }         // optional; can default from product
8       public DateTime NeededBy { get; set; }    // required
9   }
```

Feature 6: Dispatch Management Module

This feature enables farmers to create dispatches based on vendor demand requests and allows vendors to confirm delivery. It supports operational coordination between both roles.

This feature includes:

- Create Dispatch action from Farmer dashboard
- Request selection for dispatch
- Incoming dispatch list on Vendor dashboard
- Delivery confirmation
- Dispatch status tracking

The design consists of:

- Request-to-dispatch workflow
- Vendor-side delivery confirmation
- Status updates such as Planned / InTransit / Delivered

Implementation

Frontend

- Create Dispatch action in FarmerDashboard.jsx
- Incoming dispatch list in VendorDashboard.jsx
- Confirm Delivery button on vendor side
- Product image shown with requests in dropdown

Backend

- FarmerDispatchController.cs
- VendorDispatchesController.cs
- CreateDispatchDto.cs
- VendorConfirmDeliveryDto.cs
- Delivery status update logic

Database

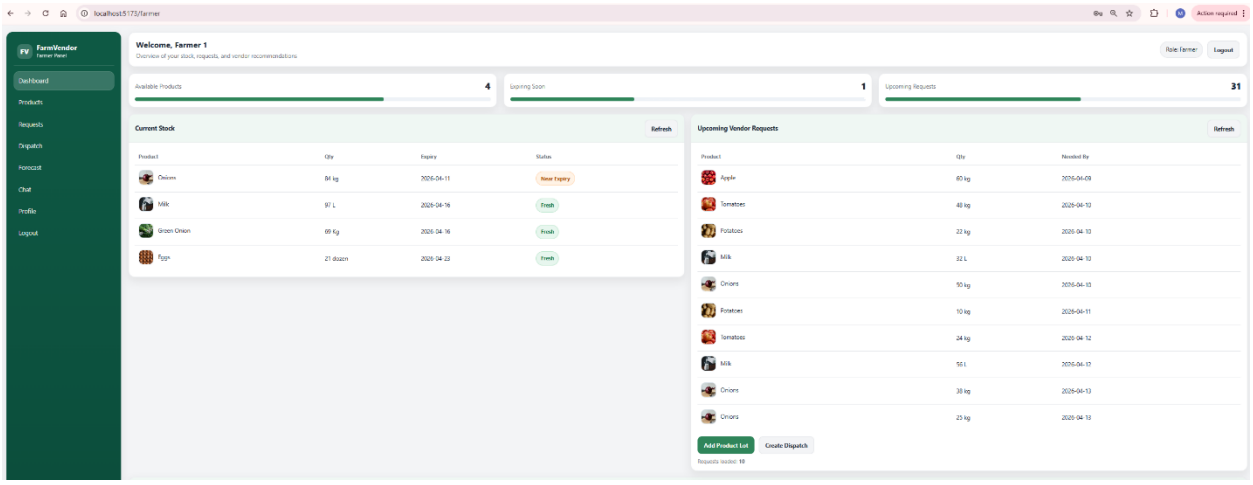
Dispatch table storing:

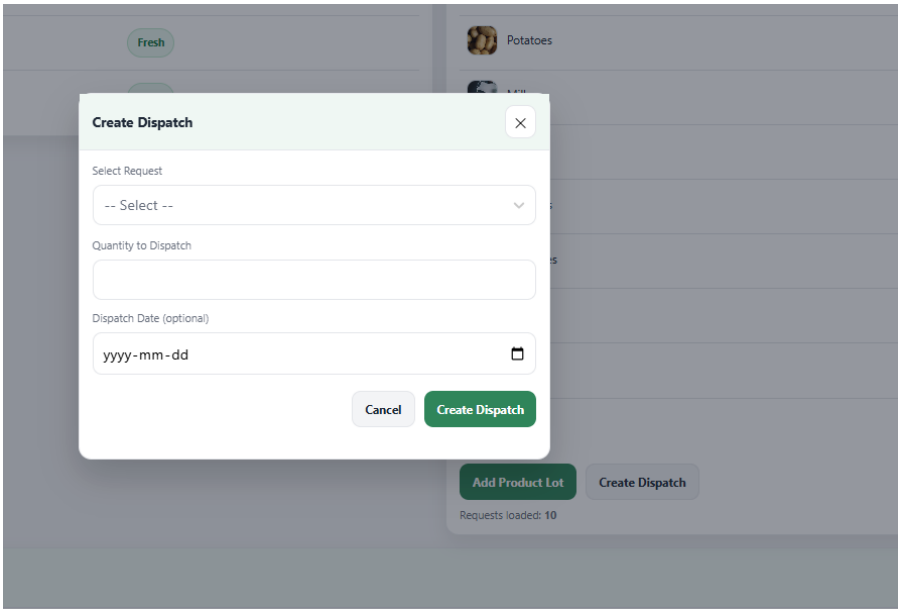
- FarmerId
- VendorId
- ProductId
- DemandRequestId

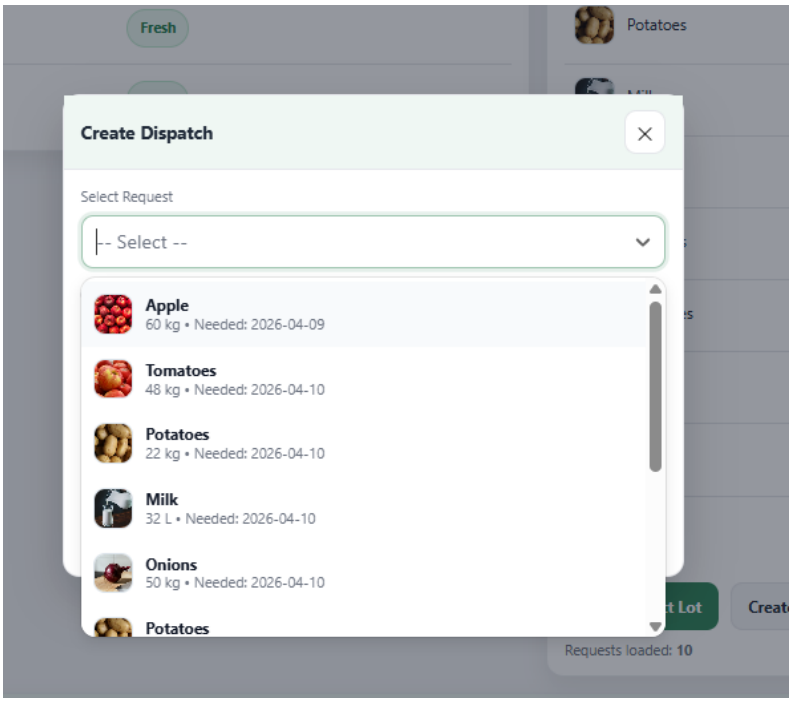
- QuantityDispatched
- Unit
- DispatchDate
- DeliveryStatus
- CreatedAt

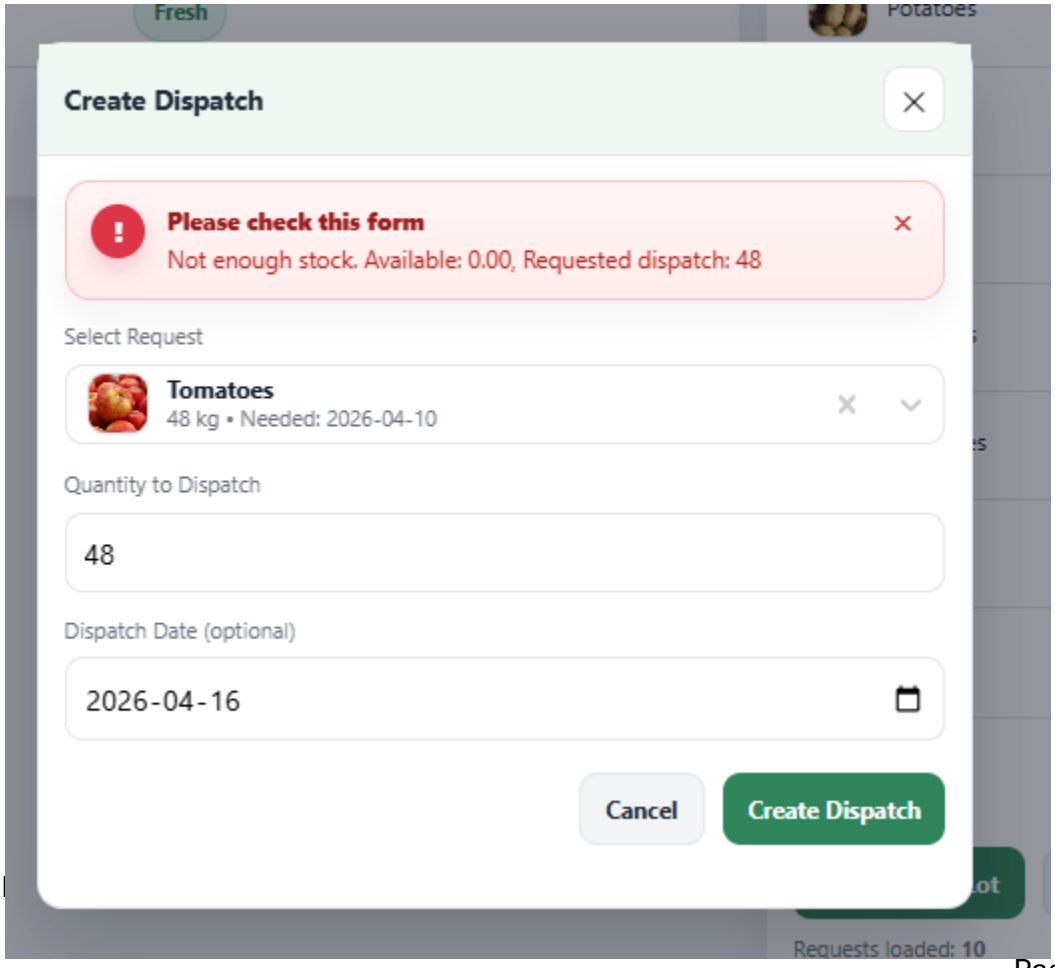
App

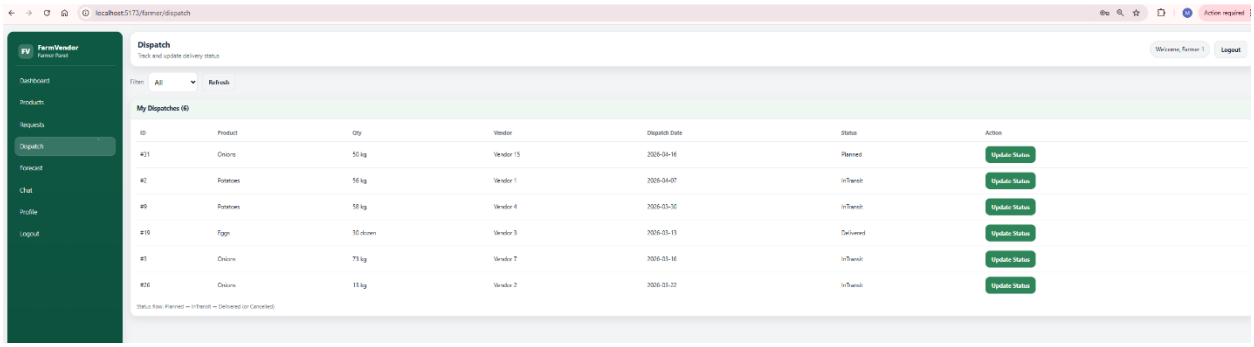
- FarmerDashboard

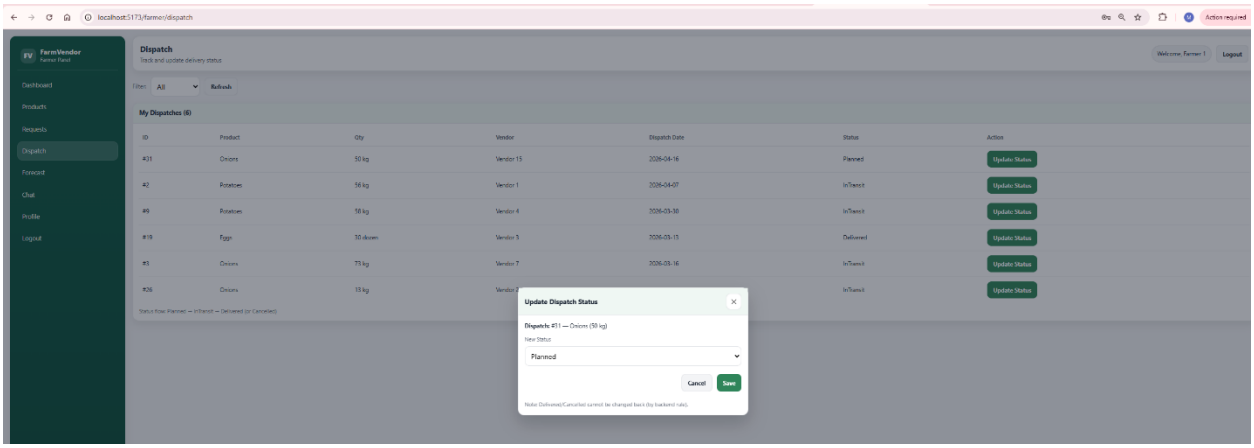


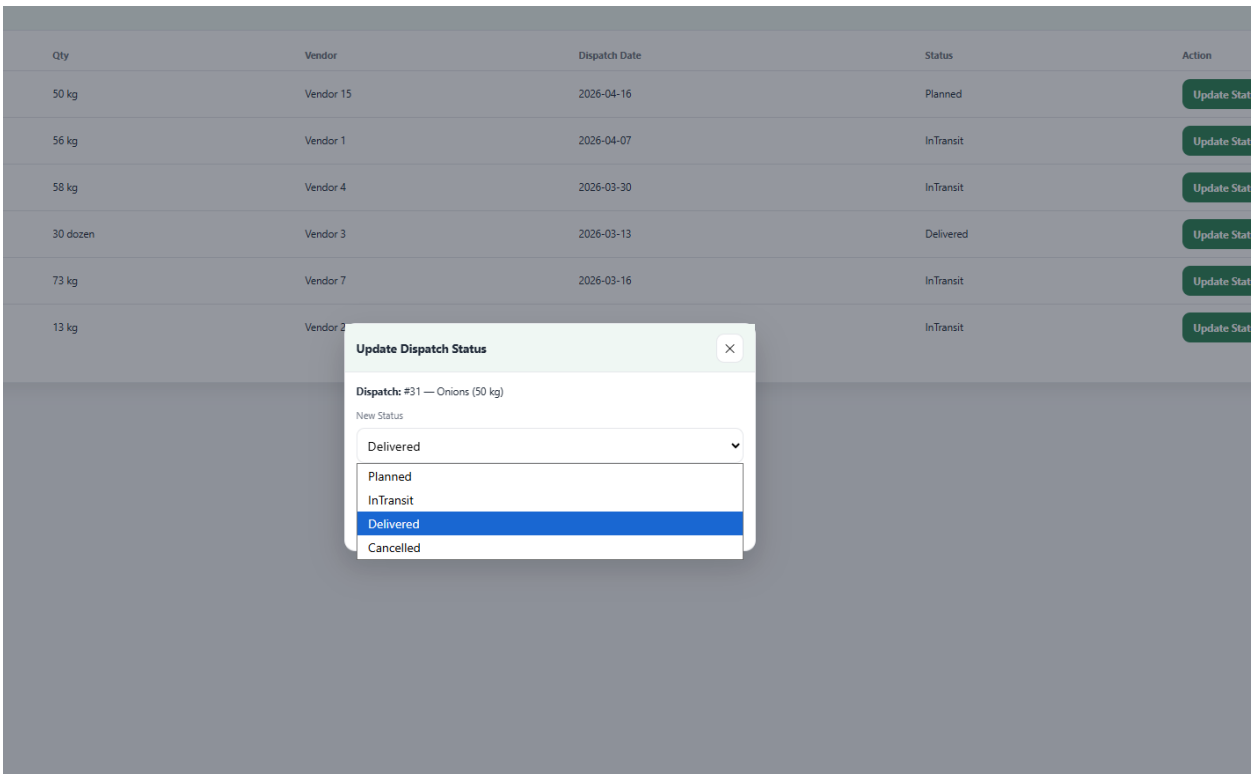


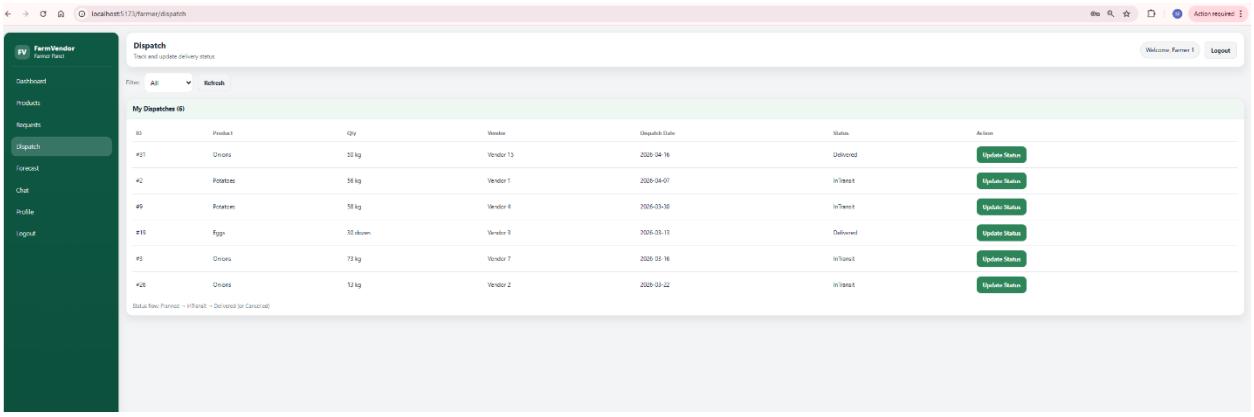

Fresh
Potatoes
Create Dispatch
Select Request
-- Select --
Apple
60 kg • Needed: 2026-04-09
Tomatoes
48 kg • Needed: 2026-04-10
Potatoes
22 kg • Needed: 2026-04-10
Milk
32 L • Needed: 2026-04-10
Onions
50 kg • Needed: 2026-04-10
Potatoes
Requests loaded: 10

- 
Fresh
Potatoes
Create Dispatch
Please check this form
Not enough stock. Available: 0.00, Requested dispatch: 48
Select Request
Tomatoes
48 kg • Needed: 2026-04-10
Quantity to Dispatch
48
Dispatch Date (optional)
2026-04-16
Cancel
Create Dispatch
Requests loaded: 10

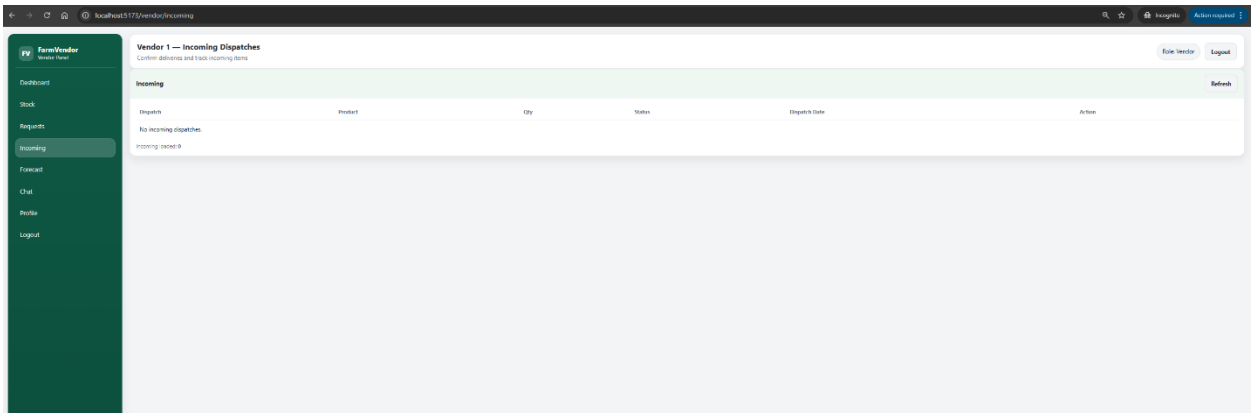

localhost:5173/farmer/dispatch
FarmVendor
Farmer Panel
Dashboard
Products
Requests
Dispatch
Forecast
Chat
Profile
Logout
Dispatch
Track and update delivery status
Welcome, Farmer 1
Logout
Filter:
All
Refresh
My Dispatches (6)
ID
Product
Qty
Vendor
Dispatch Date
Status
Action
#31
Onions
50 kg
Vendor 15
2026-04-16
Planned
Update Status
#2
Potatoes
56 kg
Vendor 1
2026-04-07
InTransit
Update Status
#9
Potatoes
58 kg
Vendor 4
2026-03-30
InTransit
Update Status
#19
Eggs
30 dozen
Vendor 3
2026-03-13
Delivered
Update Status
#3
Onions
73 kg
Vendor 7
2026-03-16
InTransit
Update Status
#26
Onions
13 kg
InTransit
Update Status
Status flow: Planned → InTransit → Delivered (or Cancelled)


localhost:5173/farmer/dispatch
Action required
FarmVendor
Farmer Panel
Dashboard
Products
Requests
Dispatch
Forecast
Chat
Profile
Logout
Dispatch
Track and update delivery status
Welcome, Farmer 1
Logout
Filter:
All
Refresh
My Dispatches (6)
ID
Product
Qty
Vendor
Dispatch Date
Status
Action
#31
Onions
50 kg
Vendor 15
2026-04-16
Planned
Update Status
#2
Potatoes
56 kg
Vendor 1
2026-04-07
InTransit
Update Status
#9
Potatoes
58 kg
Vendor 4
2026-03-30
InTransit
Update Status
#19
Eggs
30 dozen
Vendor 3
2026-03-13
Delivered
Update Status
#3
Onions
73 kg
Vendor 7
2026-03-16
InTransit
Update Status
#26
Onions
13 kg
InTransit
Update Status
Status flow: Planned → InTransit → Delivered (or Cancelled)
Update Dispatch Status
Dispatch: #31 — Onions (50 kg)
New Status
Planned
Cancel
Save
Note: Delivered/Cancelled cannot be changed back (by backend rule).


Qty
Vendor
Dispatch Date
Status
Action
50 kg
Vendor 15
2026-04-16
Planned
Update Stat
56 kg
Vendor 1
2026-04-07
InTransit
Update Stat
58 kg
Vendor 4
2026-03-30
InTransit
Update Stat
30 dozen
Vendor 3
2026-03-13
Delivered
Update Stat
73 kg
Vendor 7
2026-03-16
InTransit
Update Stat
13 kg
InTransit
Update Stat
Update Dispatch Status
Dispatch: #31 — Onions (50 kg)
New Status
Delivered
Planned
InTransit
Delivered
Cancelled



Vendor-dispatch incoming



Frontend

FarmerDashboard.jsx | InventoryLot.sql U | FarmerDispatch.jsx

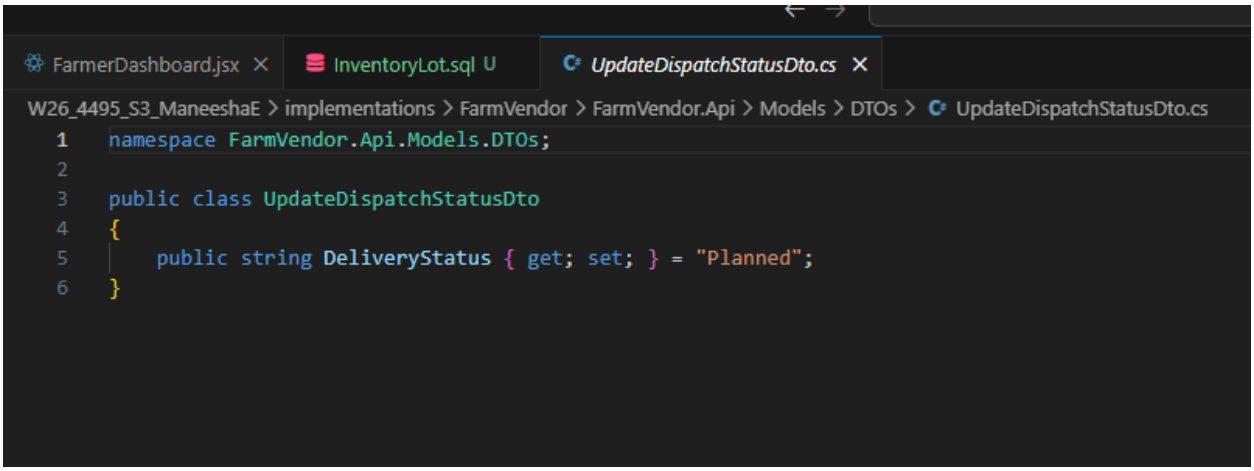
W26_4495_S3_ManeeshaE > implementations > FarmVendor > farmvendor-web > src > pages > FarmerDispatch.jsx > ...

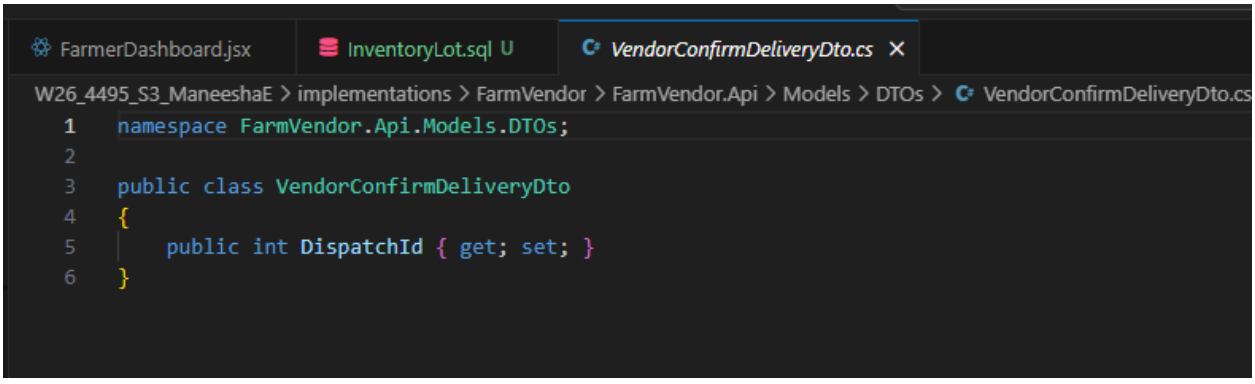
```jsx
import React, { useEffect, useMemo, useState } from "react";
import { useNavigate } from "react-router-dom";
import "../styles/dashboard.css";
import FarmerSidebar from "../assets/components/FarmerSidebar";

const API_BASE = import.meta.env.VITE_API_URL;

const STATUS_OPTIONS = ["All", "Planned", "InTransit", "Delivered", "Cancelled"];

export default function FarmerDispatch() {
  const navigate = useNavigate();
  const token = localStorage.getItem("token");
  const displayName = localStorage.getItem("displayName") || "Farmer";

  const authHeaders = useMemo(
    () => ({
      "Content-Type": "application/json",
      Authorization: `Bearer ${token}`,
    }),
    [token]
  );

  const logout = () => {
    localStorage.clear();
    navigate("/login");
  };

  const [statusFilter, setStatusFilter] = useState("All");
  const [dispatches, setDispatches] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState("");

  // Status modal
  const [showStatusModal, setShowStatusModal] = useState(false);
  const [selectedDispatch, setSelectedDispatch] = useState(null);
  const [newStatus, setNewStatus] = useState("Planned");
  const [saving, setSaving] = useState(false);

  const loadDispatches = async () => {
    setLoading(true);
    setError("");
    try {
      const qs = statusFilter === "All" ? "" : `?status=${encodeURIComponent(statusFilter)}`;
      const res = await fetch(`${API_BASE}/api/farmer/dispatches${qs}`, { headers: authHeaders });
      const text = await res.text();
      if (!res.ok) throw new Error(text || "Failed to load dispatches");
      setDispatches(JSON.parse(text));
    } catch (e) {
      setError(e?.message || "Failed to load dispatches");
    } finally {
      setLoading(false);
    }
  };

  const openStatusModal = (d) => {
    setError("");
    setSelectedDispatch(d);
    setNewStatus(d.deliveryStatus || "Planned");
    setShowStatusModal(true);
  };

  const saveStatus = async (e) => {
    e.preventDefault();
```

Backend


FarmerDashboard.jsx
InventoryLot.sql U
DispatchesController.cs
6_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Controllers > DispatchesController.cs
using FarmVendor.Api.Data;
using FarmVendor.Api.Models;
using FarmVendor.Api.Models.DTOs;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using System.Security.Claims;
namespace FarmVendor.Api.Controllers;
[ApiController]
[Route("api/dispatches")]
[Authorize]
public class DispatchesController : ControllerBase
{
private readonly AppDbContext _db;
public DispatchesController(AppDbContext db)
{
_db = db;
}
private string GetUserId()
=> User.FindFirstValue(ClaimTypes.NameIdentifier) ?? "";
[HttpPost]
public async Task<IActionResult> CreateDispatch([FromBody] CreateDispatchDto dto)
{
var farmerId = GetUserId();
if (string.IsNullOrEmpty(farmerId)) return Unauthorized();
if (dto.DemandRequestId <= 0) return BadRequest("DemandRequestId is required.");
if (dto.QuantityDispatched <= 0) return BadRequest("QuantityDispatched must be greater than 0.");
// Load demand request + product + vendor
var req = await _db.DemandRequest
.Include(r => r.Product)
.FirstOrDefaultAsync(r => r.DemandRequestId == dto.DemandRequestId);
if (req == null) return NotFound("Demand request not found.");
if (req.Status != "Open")
return BadRequest($"Demand request is not Open. Current status: {req.Status}");
// (Optional but recommended) ensure farmer has inventory for this product
var availableQty = await _db.InventoryLot
.Where(l => l.FarmerId == farmerId && l.ProductId == req.ProductId)
.SumAsync(l => l.QuantityAvailable);
if (availableQty < dto.QuantityDispatched)
return BadRequest($"Not enough stock. Available: {availableQty}, Requested dispatch: {dto.QuantityDispatched}");
var dispatch = new Dispatch
{
DemandRequestId = req.DemandRequestId,
FarmerId = farmerId,
VendorId = req.VendorId,
ProductId = req.ProductId,
QuantityDispatched = dto.QuantityDispatched,
Unit = string.IsNullOrWhiteSpace(req.Unit) ? "kg" : req.Unit,
DispatchDate = dto.DispatchDate ?? DateTime.UtcNow,
DeliveryStatus = "Planned",
CreatedAt = DateTime.UtcNow
OBLEMS 1
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
QUERY RESULTS


FarmerDashboard.jsx
InventoryLot.sql U
CreateDispatchDto.cs
W26_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Models > DTOs > CreateDisp
namespace FarmVendor.Api.Models.DTOs;
public class CreateDispatchDto
{
public int DemandRequestId { get; set; }
public decimal QuantityDispatched { get; set; }
public DateTime? DispatchDate { get; set; } // this is optional
}


FarmerDashboard.jsx
InventoryLot.sql U
UpdateDispatchStatusDto.cs
W26_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Models > DTOs > UpdateDispatchStatusDto.cs
namespace FarmVendor.Api.Models.DTOs;
public class UpdateDispatchStatusDto
{
public string DeliveryStatus { get; set; } = "Planned";
}

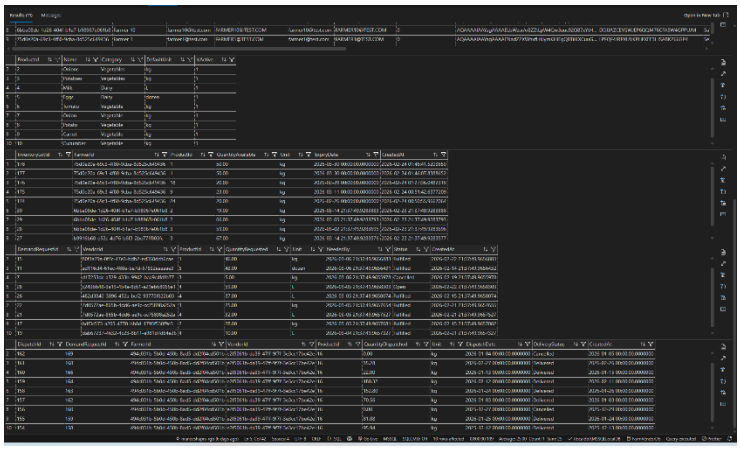

FarmerDashboard.jsx
InventoryLot.sql U
VendorConfirmDeliveryDto.cs
W26_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Models > DTOs > VendorConfirmDeliveryDto.cs
namespace FarmVendor.Api.Models.DTOs;
public class VendorConfirmDeliveryDto
{
public int DispatchId { get; set; }
}

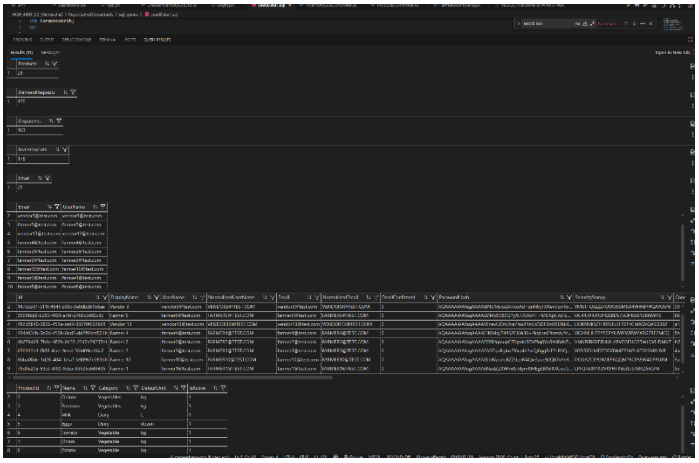
FarmerDashboard.jsx | InventoryLot.sql U | *Dispatch.cs* ×

W26_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Models > Dispatch.cs

```
1   namespace FarmVendor.Api.Models;
2
3   public class Dispatch
4   {
5       public int DispatchId { get; set; }
6
7       public int? DemandRequestId { get; set; }
8       public DemandRequest? DemandRequest { get; set; }
9
10      public string FarmerId { get; set; } = "";
11      public ApplicationUser Farmer { get; set; } = null!;
12
13      public string VendorId { get; set; } = "";
14      public ApplicationUser Vendor { get; set; } = null!;
15
16      public int ProductId { get; set; }
17      public Product Product { get; set; } = null!;
18
19      public decimal QuantityDispatched { get; set; }
20
21      public string Unit { get; set; } = "kg";
22
23      public DateTime DispatchDate { get; set; }
24
25      public string DeliveryStatus { get; set; } = "Planned";
26      // Planned, InTransit, Delivered, Cancelled
27
28      public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
29  }
30
```

Database





Feature 7: Dashboard Analytics and Status Badge System

This feature improves usability by showing summarized operational insights and visual status indicators for both farmer and vendor dashboards.

Design includes:

- Analytics summary cards
- Expiry status badges
- Low stock alerts
- Dynamic UI rendering based on live API data

Logic implemented using:

- Expiry date difference calculation
- Quantity threshold rules
- React conditional rendering
- API-driven dashboard refresh

Implementation

Frontend

- FarmerDashboard.jsx
- VendorDashboard.jsx
- dashboard.css
- Dynamic summary cards and badges

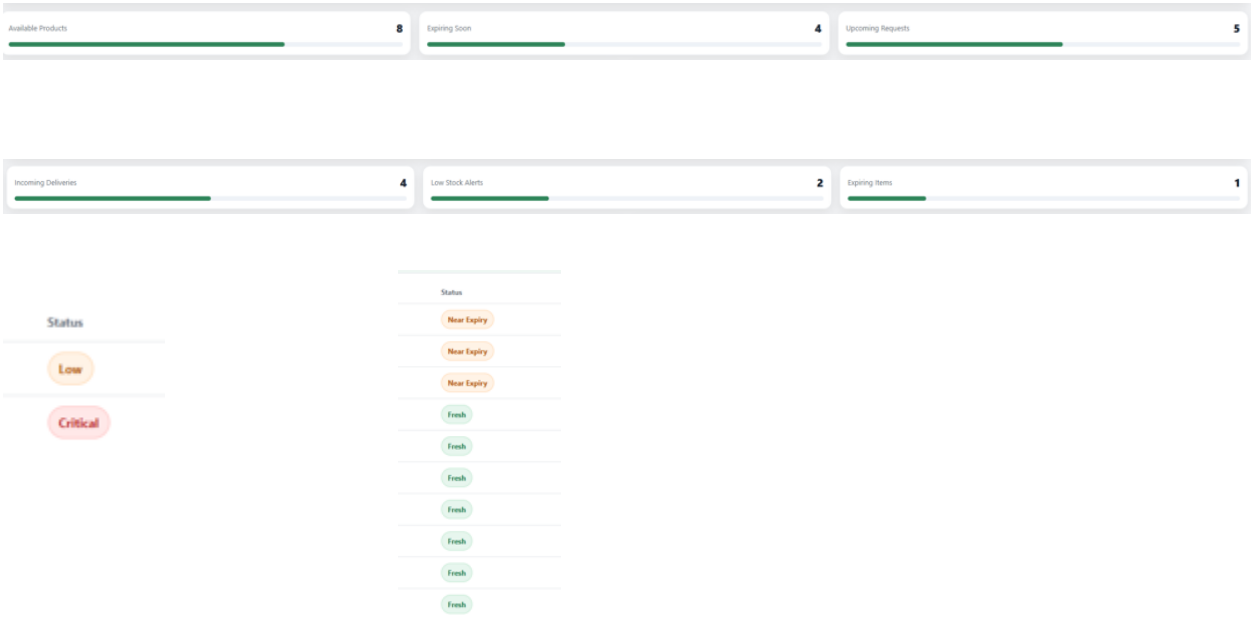
Backend

- FarmerController.cs
- Vendor dashboard API endpoints
- Aggregation logic for counts and summary metrics

Database

- Data derived from:
    - InventoryLot

- DemandRequest
- Dispatch



6. Evaluation Technique

The FarmVendor system was evaluated using a combination of different algorithm comparison, functional and integration testing, scenario-based evaluation, and usability assessment. These evaluation techniques were selected to validate system correctness, decision quality, and practical usability in real-world agricultural supply-chain scenarios. The outcomes of these evaluations directly influenced several design and implementation decisions in the final system.

Forecasting model evaluation

The demand forecasting component was evaluated by comparing multiple approaches before finalizing the forecasting algorithm. Initially, simple logic-based methods, including basic averages and Moving Average techniques, were considered due to their ease of implementation. However, preliminary testing revealed that these approaches were unable to capture temporal trends, seasonal variations, and fluctuations present in historical demand request data.

As an alternative, the ML.NET Singular Spectrum Analysis (SSA) time-series forecasting algorithm was implemented and evaluated using multi-period historical demand data. SSA demonstrated superior performance in identifying trends and generating smoother, forward-looking demand predictions compared to traditional statistical methods.

Design Decision Outcome:
Based on this evaluation, the logic-based forecasting approach was replaced with ML.NET SSA. Forecasting results were fully integrated into the system and used as a core input for vendor recommendation and dispatch planning, enabling proactive and data-driven decision-making.

Comparison between ML.NET SSA model and Moving Average model

To evaluate the effectiveness of the demand forecasting component in the FarmVendor system, two forecasting approaches were implemented and compared: a Moving Average model and a machine-learning-based ML.NET Singular Spectrum Analysis (SSA) model. Both models were applied to the same historical demand dataset across multiple vendors and products, allowing a direct comparison of forecast behavior and suitability for decision support.

Vendor	Product	Forecast Date	Qty	Model	Lookback	Created At
Vendor 8	Eggs	2026-04-08	27.57	MLNET_SSA		2026-04-09
Vendor 6	Milk	2026-04-08	7.4	MLNET_SSA		2026-04-09
Vendor 6	Tomatoes	2026-04-08	50.59	MLNET_SSA		2026-04-09
Vendor 12	Eggs	2026-04-08	30.58	MLNET_SSA		2026-04-09
Vendor 13	Milk	2026-04-08	25.57	MLNET_SSA		2026-04-09
Vendor 1	Tomatoes	2026-04-08	14.52	MLNET_SSA		2026-04-09
Vendor 5	Milk	2026-04-08	39.59	MLNET_SSA		2026-04-09
Vendor 5	Tomatoes	2026-04-08	29.57	MLNET_SSA		2026-04-09
Vendor 3	Milk	2026-04-08	42.26	MLNET_SSA		2026-04-09
Vendor 9	Milk	2026-04-08	36.58	MLNET_SSA		2026-04-09
Vendor 9	Tomatoes	2026-04-08	26.57	MLNET_SSA		2026-04-09
Vendor 7	Eggs	2026-04-08	52.86	MLNET_SSA		2026-04-09
Vendor 2	Tomatoes	2026-04-08	21.56	MLNET_SSA		2026-04-09
Vendor 10	Tomatoes	2026-04-08	36.55	MLNET_SSA		2026-04-09
Vendor 4	Eggs	2026-04-08	34.58	MLNET_SSA		2026-04-09
Vendor 15	Eggs	2026-04-08	17.54	MLNET_SSA		2026-04-09
Vendor 11	Tomatoes	2026-04-08	56.6	MLNET_SSA		2026-04-09
Vendor 14	Tomatoes	2026-04-08	30.31	MLNET_SSA		2026-04-09

Vendor	Product	Forecast Date	Qty	Model	Lookback	Created At
Vendor 8	Eggs	2026-04-08	28	MovingAverage	3	2026-04-09
Vendor 6	Milk	2026-04-08	8	MovingAverage	3	2026-04-09
Vendor 6	Tomatoes	2026-04-08	51	MovingAverage	3	2026-04-09
Vendor 12	Eggs	2026-04-08	31	MovingAverage	3	2026-04-09
Vendor 13	Milk	2026-04-08	26	MovingAverage	3	2026-04-09
Vendor 1	Tomatoes	2026-04-08	15	MovingAverage	3	2026-04-09
Vendor 5	Milk	2026-04-08	40	MovingAverage	3	2026-04-09
Vendor 5	Tomatoes	2026-04-08	30	MovingAverage	3	2026-04-09
Vendor 3	Milk	2026-04-08	42.67	MovingAverage	3	2026-04-09
Vendor 9	Milk	2026-04-08	37	MovingAverage	3	2026-04-09
Vendor 9	Tomatoes	2026-04-08	27	MovingAverage	3	2026-04-09
Vendor 7	Eggs	2026-04-08	35.67	MovingAverage	3	2026-04-09
Vendor 2	Tomatoes	2026-04-08	22	MovingAverage	3	2026-04-09
Vendor 10	Tomatoes	2026-04-08	37	MovingAverage	3	2026-04-09
Vendor 4	Eggs	2026-04-08	35	MovingAverage	3	2026-04-09
Vendor 15	Eggs	2026-04-08	18	MovingAverage	3	2026-04-09
Vendor 11	Tomatoes	2026-04-08	57	MovingAverage	3	2026-04-09
Vendor 14	Tomatoes	2026-04-08	30.67	MovingAverage	3	2026-04-09

Moving Average Model Evaluation

The Moving Average model was implemented using a fixed lookback window of **three periods**, producing forecasts based on recent historical demand values. As shown in the dataset, the Moving Average model generated reasonable baseline estimates for short-term demand. For example, Vendor 6's milk demand was forecast at **8 units**, while Vendor 11's tomato demand was forecast at **57 units**.

However, the Moving Average model exhibited several limitations. Because it relies strictly on recent averages, it is **highly sensitive to short-term fluctuations** and does not capture longer-term trends or demand patterns. In cases where historical demand varied significantly, the model produced abrupt changes or plateau-like forecasts. Additionally, the Moving Average approach treats all past observations equally within the lookback window, limiting its ability to adapt to evolving demand behavior.

ML.NET SSA Model Evaluation

The ML.NET SSA model was implemented to address the limitations of simple statistical forecasting. SSA is a time-series decomposition technique that identifies underlying trends

and patterns in historical data. When applied to the same dataset, the SSA model produced forecasts that were smoother and more trend-aware.

For instance, Vendor 6's milk demand forecast shifted from 8 units (Moving Average) to 7.4 units (SSA), reflecting a slight adjustment based on learned patterns rather than a direct average. Similarly, Vendor 7's egg demand increased from 35.67 units under the Moving Average model to 52.86 units under the SSA model, indicating that SSA detected an upward demand trend that the Moving Average approach failed to capture.

Across most vendor–product combinations, SSA forecasts differed moderately from Moving Average values rather than drastically, suggesting that SSA refines forecasts by accounting for trend continuity rather than simply smoothing past values.

Quantitative and Behavioral Comparison

Overall, both models produced forecasts within a comparable numerical range; however, their forecasting behavior differed significantly:

- The Moving Average model provided quick, interpretable baseline forecasts but struggled with trend detection and adaptability.
- The ML.NET SSA model produced more stable and forward-looking forecasts by learning from historical patterns, reducing sensitivity to short-term noise.

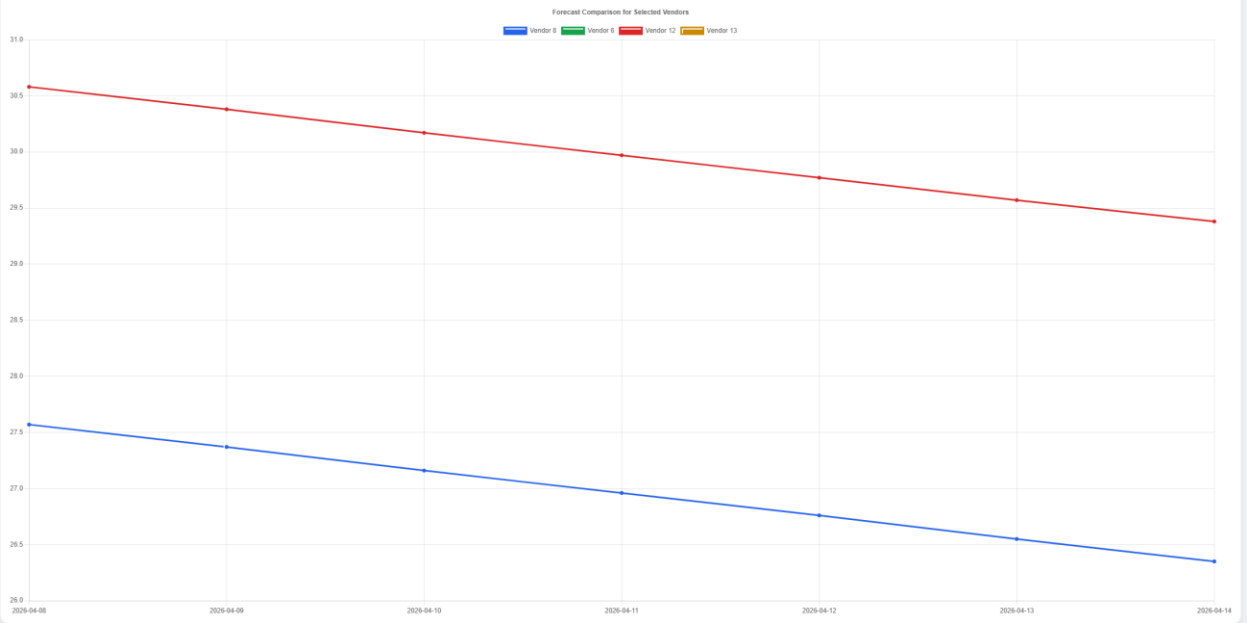
SSA forecasts also demonstrated greater consistency across similar products and vendors, which is particularly important for downstream processes such as vendor recommendation and dispatch planning.

Based on this evaluation, the Moving Average model was retained as a baseline comparison, while the ML.NET SSA model was selected as the primary forecasting mechanism for the FarmVendor platform. The SSA model's ability to capture trends and produce smoother predictions made it more suitable for integration into the vendor recommendation system and overall decision-support workflow.

By incorporating SSA-based forecasts, the system shifted from reactive forecasting toward proactive demand planning, enabling farmers to make better-informed inventory and dispatch decisions.

Aspect	Moving Average	ML.NET SSA
Model Type	Statistical	Machine-Learning (Time-Series)
Trend Detection	Limited	Strong
Sensitivity to Noise	High	Lower
Forecast Smoothness	Moderate	High
Interpretability	High	Moderate
Suitability for Decision Support	Limited	High

- Graph generated for **ML.NET SSA** Model



Recommendation Logic Evaluation

The vendor recommendation feature was evaluated by comparing different decision-making strategies. An optimization-based approach was initially explored; however, early experimentation showed that such models were difficult to interpret and explain to non-technical users, limiting their practical usability.

To address this, a weighted scoring-based recommendation system was implemented and evaluated through scenario-based testing. Multiple test scenarios were executed by varying key inputs such as forecasted demand, available inventory, geographic distance (calculated using the Haversine formula), and historical relationship scores. The resulting rankings were reviewed to ensure consistency, transparency, and logical behavior.

Design Decision Outcome:
The optimization-based approach was replaced with a scoring-based model. This change improved interpretability, transparency, and user trust, making the recommendation system more suitable for real-world agricultural users.

Functional and Integration Testing

Functional testing was conducted across all major system modules, including:

- Role-based authentication and authorization
- Farmer inventory management
- Vendor demand request lifecycle
- Dispatch creation and delivery confirmation
- Real-time chat communication

Integration testing focused on validating seamless interaction between the frontend (React) and backend (ASP.NET Core APIs), as well as ensuring proper enforcement of JWT-based security. SignalR-based real-time messaging was tested to confirm message delivery, role isolation, and conversation persistence.

Design Decision Outcome:
Testing results led to improvements in API validation, error handling, and frontend state management, ensuring system stability and secure role-based access.

Usability Evaluation

Usability evaluation was performed through developer testing, instructor testing and informal peer testing, focusing on common user workflows such as adding inventory, submitting demand requests, creating dispatches, and initiating chat conversations, adding products. Observations were made regarding ease of navigation, clarity of information, and error frequency.

Based on this evaluation, several UI/UX enhancements were implemented, including searchable dropdowns, improved input validation, shared sidebar navigation, and clearer status indicators.

Design Decision Outcome:
These changes reduced manual input errors, improved task completion efficiency, and enhanced the overall user experience for both farmers and vendors.

Overall, the evaluation techniques confirmed that the FarmVendor system functions correctly, produces meaningful decision-support outputs, and remains usable for non-technical users. The evaluation process directly informed key design choices, including the adoption of machine-learning-based forecasting, the use of a transparent scoring-based recommendation model, and multiple usability improvements. This iterative evaluation-driven approach ensured alignment between the project's research objectives and the final implemented solution.

Evaluation Method	What Was Evaluated	Outcome
Model comparison	Moving Average vs ML.NET SSA	SSA selected as primary model
Functional testing	End-to-end workflows	Main features worked correctly
Usability evaluation	Dashboard clarity and ease of use	UI improved through redesign

7. Reflections/Discussions

The development of FarmVendor provided valuable insights into full-stack application design, machine learning integration, and user-centered system development. The project involved combining ASP.NET Core, React, SQL Server, ML.NET, SignalR, and external APIs into one integrated platform.

Challenges Faced

One of the most significant challenges was implementing machine learning forecasting in a realistic way. The ML.NET SSA model required sufficient historical demand data, which meant that data preparation and simulation became an essential part of the project. Other challenges included synchronizing frontend and backend changes, handling JWT and SignalR authentication issues, and ensuring that dashboard visualizations correctly reflected saved forecast data. Integrating product images and updating the schema without breaking existing workflows was also challenging.

Lessons Learned

This project emphasized the importance of iterative development, testing, and refinement. I learned that the success of a machine learning feature depends not only on the algorithm, but also on the quality and structure of the historical data. I also learned that usability is a major part of system effectiveness; visual improvements such as product images, cleaner dashboards, and better interaction patterns can significantly improve how users understand and use a system.

Most Satisfying Parts

The most satisfying aspect of the project was successfully integrating multiple advanced features into one working system. In particular, generating ML-based forecasts and visualizing them in charts was highly rewarding. It was also satisfying to enhance the dashboard with image-based product identification and real-time chat communication, making the application feel practical and complete.

Future Improvements

Future work could include deploying the system to a cloud platform, collecting real agricultural data for more accurate model training, and performing formal user testing with farmers and vendors. Additional enhancements could include push notifications, mobile

access, more advanced recommendation models, and integration with external logistics or weather data sources.

8. AI Use Section

AI Tool Name	Version / Account Type	Specific Feature for Which the AI Tool Was Used	Value Addition (What I Added Beyond AI Output)
ChatGPT	Free Version	• Research on existing farm–vendor management tools • Identification of gaps and missing features in existing solutions • High-level backend system design guidance	I conducted a critical review of existing platforms and refined the project idea by identifying missing functionalities. I independently designed and implemented all backend workflows, APIs, and database schemas specific to the FarmVendor system.
ChatGPT	Premium Version (GPT-4)	• Guidance on ASP.NET Core Identity and JWT authentication • Debugging JWT authorization and Postman API testing issues • React component structure and dashboard UI design suggestions	I reviewed and selectively applied AI suggestions, manually implemented authentication and authorization logic, resolved environment-specific issues independently, and validated functionality through end-to-end testing and debugging.
ChatGPT	Premium Version (GPT-4)	• Backend query debugging and EF Core issues • DTO structure design guidance • Dispatch status update flow guidance	I adapted suggestions to match my project's namespace, DbSet naming, Identity setup, and business rules. All DTOs, controllers, and service logic were designed and implemented manually and tested against real scenarios.

ChatGPT	Premium Version (GPT-4)	• Research brainstorming for forecasting and optimization modules • Comparison between Moving Average and ML-based forecasting approaches	I designed my own research novelty by implementing both baseline Moving Average and ML.NET SSA forecasting models, conducting comparisons using real dataset outputs, and integrating the chosen model into decision-support workflows.
ChatGPT	Premium Version (GPT-4)	• ML.NET SSA forecasting pipeline explanation and integration guidance	I independently implemented the ML.NET SSA forecasting engine, configured training windows and prediction horizons, integrated it with EF Core persistence, and validated forecasts using historical demand data.
ChatGPT	Premium Version (GPT-4)	• Optimization research and design guidance for dispatch planning	I designed and implemented a greedy allocation optimization technique to match farmer inventory with forecasted vendor demand, created custom services and APIs, and validated optimization results through scenario testing.
ChatGPT	Premium Version (GPT-4)	• Relationship scoring logic brainstorming	I implemented a custom relationship-scoring model based on fulfillment rate, on-time delivery, and cancellation rate. I integrated automatic score updates into dispatch confirmation workflows and exposed APIs for analytics.
ChatGPT	Premium Version (GPT-4)	• Full-stack integration and troubleshooting guidance	I resolved backend–frontend mismatches, corrected DTO binding errors, aligned API contracts, and stabilized forecasting, optimization, recommendation, and chat modules for final evaluation.

ChatGPT	Premium Version (GPT-4)	• Chat module design and SignalR integration guidance	I independently designed and implemented a real-time farmer–vendor chat system using SignalR, ensured secure JWT-based communication, fixed CORS and foreign-key issues, and tested full conversation workflows.
ChatGPT	Premium Version (GPT-4)	• UI/UX improvement suggestions (searchable dropdowns, navigation simplification)	I redesigned the frontend UX by replacing manual ID entry with searchable dropdowns, implemented a common navigation bar to reduce React code complexity, and improved responsiveness and usability across dashboards.
ChatGPT	Premium Version (GPT-4)	• Documentation, work-log structuring, and report organization guidance	I rewrote and refined all documentation using my actual implementation details, validated every section against the rubric, and ensured accuracy, consistency, and academic integrity throughout the final report.
ChatGPT	Premium Version (GPT-4)	• Guidance on structuring the final report sections, including Evaluation Techniques, Reflections, AI Use, and Work-Log consolidation • Assistance in refining academic language and ensuring rubric compliance	I independently authored all report content using my actual implementation details, validated technical accuracy against the codebase, ensured consistency across sections, and aligned the final document strictly with course requirements and grading criteria.
ChatGPT	Premium Version (GPT-4)	• Debugging support for forecast analytics charts not rendering correctly • Assistance identifying API endpoint mismatches and missing parameters	I manually traced backend APIs, verified database records, corrected frontend API calls, and fixed chart-loading logic. All fixes were implemented and tested independently to ensure accurate data visualization.

ChatGPT	Premium Version (GPT-4)	• Assistance in validating vendor recommendation logic, including forecast date alignment and inventory overlap issues	I analyzed recommendation outputs, refined scoring logic in FarmerRecommendationService.cs, validated results using real forecast data, and ensured recommendations change correctly based on farmer inventory and forecast inputs.
Mermaid (Live Editor)	Free Web Tool	• Creation of system architecture diagrams, database ERD, and workflow flowcharts for final report documentation	I manually defined the complete folder structure, entity relationships, and process flows based on the implemented system. All diagrams were verified against the actual codebase and database schema to ensure correctness and consistency.
ChatGPT	Premium Version (GPT-4)	• Assistance in generating Mermaid syntax for architecture, ERD, and flow diagrams	I reviewed and edited all Mermaid diagrams, ensured accurate mapping to controllers, services, entities, and frontend components, and adjusted diagram structure to match the real system design.
ChatGPT	Premium Version (GPT-4)	• Guidance on organizing work-log tables, consolidating Progress Reports 1–5, and ensuring realistic daily granularity	I reconstructed the full work-log using actual development activities, verified time distribution, mapped logs to implemented features, and ensured compliance with academic logging standards.
ChatGPT	Premium Version (GPT-4)	• Support in validating database architecture documentation and ERD explanations	I independently reviewed all entities, relationships, and foreign-key mappings, ensured normalization, and confirmed that the ERD accurately represents the implemented SQL Server schema.

Summary of AI use

Artificial Intelligence tools, primarily ChatGPT (Free and GPT-4 Premium) and Mermaid Live Editor, were utilized throughout the development of the FarmVendor system to support research, design, implementation, debugging, and documentation activities.

ChatGPT was initially used to explore existing farm–vendor management systems and identify gaps, which helped refine the project scope. It provided high-level guidance on system architecture, authentication (ASP.NET Identity with JWT), frontend design, and API structuring. Additionally, AI-assisted suggestions were used for troubleshooting issues related to backend queries, EF Core, API integration, and frontend rendering.

For advanced features, AI supported conceptual understanding of forecasting and optimization techniques. However, the Moving Average and ML.NET SSA forecasting models were independently implemented, configured, and validated using real historical data. Similarly, optimization logic, relationship scoring models, and recommendation systems were designed and developed manually based on domain-specific requirements.

In full-stack development, AI contributed to resolving integration issues, but all critical implementations—including DTO design, service-layer logic, API contracts, and database schema—were independently built, tested, and refined. The real-time chat module using SignalR was also fully implemented with manual handling of authentication, CORS, and communication workflows.

AI tools also supported UI/UX improvements and documentation structuring. However, all final designs, usability enhancements, and report content were carefully reviewed, rewritten, and aligned with actual implementation details to ensure accuracy and academic integrity.

Mermaid Live Editor was used for generating system diagrams, but all architectural flows, ERDs, and diagrams were manually defined and verified against the implemented codebase and database schema.

Overall, AI served as a supportive assistant for ideation and guidance, while the core system design, development, debugging, evaluation, and documentation were independently executed, ensuring originality, correctness, and practical applicability of the FarmVendor system

Appendix – Prompt History

Appendix X.1: Backend Development & Debugging Prompts

Prompt ID	AI Prompt Description
P-B1	Fix IQueryable implicit conversion error in EF Core Include query
P-B2	Why am I getting 400 error when posting dispatch
P-B3	How to design DTO for dispatch status update
P-B4	How to return demandRequestId in projection query
P-B5	Fix alignment issue in styling auth.css
P-B6	Fix dependency injection error when using RelationshipScoreService in ASP.NET Core controller

Appendix X.2: Forecasting, Optimization & Research Design Prompts

Prompt ID	AI Prompt Description
P-F1	How to implement a moving-average demand forecasting module in ASP.NET Core using EF Core data
P-F2	Design the DemandForecast model and API endpoints for generating and retrieving forecast results
P-F3	How to implement time-series forecasting using ML.NET SSA algorithm
P-F4	Create ML.NET pipeline for predicting future demand values from historical data
P-F5	How to integrate ML.NET forecasting into ASP.NET Core service layer

P-F6	Store forecast results generated by ML.NET in SQL Server using Entity Framework Core
P-F7	Design forecasting comparison between Moving Average and ML.NET SSA models
P-F8	How to export database data to CSV for research experiments and evaluation metrics

Appendix X.3: Frontend Analytics & Visualization Prompts

Prompt ID	AI Prompt Description
P-UI1	Create React chart component to display demand history and forecast predictions
P-UI2	Integrate forecast visualization charts into React analytics dashboard
P-UI3	Create React frontend views to display forecast results, optimization plans, and relationship statistics
P-UI4	How to integrate backend analytics endpoints with React dashboard pages using secure API calls
P-UI5	Check the endpoint error for chart forecast error that is not populating in forecastApi.js

Appendix X.4: Optimization & Recommendation System Prompts

Prompt ID	AI Prompt Description
P-O1	How to create a greedy optimization algorithm to allocate farmer inventory to vendor demand requests

P-O2	How to design RecommendedDispatchPlan table and DispatchOptimizationService for dispatch recommendation logic
P-O3	Optimization research and design for demand-driven allocation strategies
P-O4	Design recommendation system to suggest best vendors for farmers based on forecast and inventory data
P-O5	Implement ML-based or rule-based vendor recommendation logic using forecast demand and available inventory
P-O6	Create API endpoint to return top recommended vendors for a farmer dashboard
P-O7	Integrate vendor recommendation API into React dashboard and display results in a table UI

Appendix X.5: Real-Time Chat & System Integration Prompts

Prompt ID	AI Prompt Description
P-C1	How to implement real-time chat functionality using SignalR in ASP.NET Core with React frontend
P-C2	Fix foreign key constraint error when creating chat conversation between two users in EF Core
P-C3	How to correctly store and retrieve authenticated userId from JWT/localStorage in React application
P-C4	Resolve SignalR connection issues including CORS and WebSocket negotiation errors in ASP.NET Core
P-C5	Design chat system with conversation creation, message storage, and real-time updates using SignalR

P-C6	Replace manual user ID input with searchable dropdown using name/email in React UI
P-C7	Create backend API to search users (vendors/farmers) by name or email using Entity Framework Core
P-C8	Fix React module import/export errors and ensure proper API integration structure
P-C9	Create API endpoints for vendor and product lookup with search functionality in ASP.NET Core
P-C10	How to handle validation errors and payload structure issues in React form submission to backend APIs

Appendix X.6: Routing, Navigation & System Refinement Prompts

Prompt ID	AI Prompt Description
P-R1	How to implement nested routes with Outlet in React Router v6
P-R2	Fix routing and authentication issues causing unexpected redirects to login page in React Router
P-R3	Refactor React navigation into reusable sidebar components to reduce code duplication
P-R4	Improve overall system integration between frontend and backend by aligning API contracts and data flow

9. Work Date/Hours logs for student

Date	No. of Hours	Description of Work Done
13-Jan	2	Finalized the research idea for the FarmVendor application by clearly defining the problem statement and project scope.
13-Jan	1	Installed basic development tools including .NET SDK, Visual Studio Code, and SQL Server LocalDB.
14-Jan	4	Created a local database and initial React web application. Tested application execution in the browser and resolved multiple dependency installation issues.
15-Jan	4	Integrated ASP.NET Core Web API (.NET 8.0). Configured SQL Server LocalDB (FarmVendorDb), implemented ASP.NET Identity, defined Farmer and Vendor roles, and set up JWT authentication.
17-Jan	2	Tested authentication endpoints using Postman, including POST /api/auth/register and POST /api/auth/login.
20-Jan	0.5	Attended project idea re-evaluation meeting with instructor (Madam Priya) and refined the research direction.
21-Jan	2	Conducted initial research on existing mobile and web-based agricultural supply-chain applications.
22-Jan	2	Performed background research and literature review related to demand forecasting and supply-chain systems.
24-Jan	2	Conducted research on React integration with ASP.NET backend architecture.
25-Jan	2	Continued research proposal development, focusing on feature scope and system design.
26-Jan	1	Finalized the research proposal and prepared it for submission.
27-Jan	3	Reviewed submitted proposal feedback with instructor and made modifications to the data-collection and forecasting strategy.
29-Jan	4	Implemented JWT authorization configuration and secured API endpoints with role-based access control.

02-Feb	2	Moved GitHub repository to the root directory and cleaned unused project artifacts.
04-Feb	3	Created initial frontend sketches for the Farmer Dashboard and planned React component structure. Downloaded and configured required React dependencies.
05-Feb	2	Resolved installation and configuration issues related to Node.js and npm.
08-Feb	3	Completed login and registration UI design. Added dashboard CSS styling and modified application routing for Farmer and Vendor dashboards.
09-Feb	3	Uploaded modified proposal to GitHub. Updated progress report 1, linked working code screenshots, and verified GitHub repository structure.
23-Feb	8	Implemented Add Product Lot feature (React form + API POST)
		Created API endpoint POST /api/farmer/inventorylots using CreateInventoryLotDto and InventoryLotsController
		Final Midterm video recording
		Final Midterm report completion and submission
24-Feb	5	Implemented API endpoint POST /api/farmer/inventorylots using CreateInventoryLotDto and InventoryLotsController
28-Feb	3	Developed Dispatch module (frontend) and updated UI styles (FarmerDashboard.jsx, dashboard.css)
01-Mar	5	Implemented Dispatch module (backend) (DispatchController.cs, CreateDispatchDto.cs)
02-Mar	6	Fixed Create Dispatch - select request issue by aligning backend dispatch initialization and frontend handling
		Fixed Dispatched items update issue by adding/returning demandRequestId correctly in FarmerController.cs
		Implemented nested routing support by adding Outlet in RequireAuth and RequireRole.jsx
		Completed Farmer Products page frontend + backend (FarmerProducts.jsx, FarmerController.cs / ProductsController.cs, InventoryLotDto.cs, UpdateInventoryLotDto.cs)
		Completed Farmer Requests page frontend + backend (FarmerController.cs, FarmerDemandRequestRowDto.cs, FarmerRequests.jsx)
		Completed Farmer Dispatch page frontend + backend (DispatchRowDto.cs, UpdateDispatchStatusDto.cs, FarmerDispatchController.cs, FarmerDispatch.jsx)

03-Mar	6	Implemented inventory lot creation and stock validation logic to support dispatch planning from farmer inventory against vendor demand requests.
		Developed Vendor Dashboard functionality for Create Demand Request and integrated frontend actions with backend API endpoints.
		Implemented Incoming Dispatches view and Confirm Delivery workflow, including updates to dispatch delivery status and related demand request status.
05-Mar	5	Resolved full-stack integration issues involving DTO mismatches, dropdown binding problems, missing response fields, and EF Core query typing issues.
		Updated VendorDashboard.jsx and integrated modal-based demand request creation with VendorDemandRequestsController.cs.
		Added backend support for CSV/data extraction to prepare for forecasting and experimental analysis.
07-Mar	6	Began implementation of a moving-average demand forecasting baseline in .NET as the first forecasting approach for the research module.
		Created the DemandForecast model, supporting DTOs, and required database updates for forecast storage.
		Completed baseline forecasting workflow and tested forecast generation successfully through API endpoints.
		Designed the optimization module structure by defining the optimization result table and dispatch recommendation flow.
		Implemented DispatchOptimizationService.cs using greedy baseline allocation logic based on forecast demand and available farmer inventory.
		Created optimization API endpoints for generating and retrieving recommended dispatch plans.
08-Mar	3	Conducted research and design for the optimization component, focusing on demand-driven farmer-to-vendor dispatch decision logic using forecast and inventory data.
		Refined the optimization formulation to support the research novelty component of the project.
09-Mar	7	Implemented the optimization module and connected it with forecast and inventory data to generate recommended dispatch plans.
		Implemented relationship score tracking between Farmer and Vendor pairs based on dispatch outcomes.

		Added automatic score updates for Delivered and Cancelled dispatch events.
		Computed relationship score using measurable factors including on-time delivery rate, fulfillment rate, and cancellation rate.
		Exposed backend API endpoints for viewing relationship statistics.
		Added and updated files required for relationship scoring, including RelationshipStat, RelationshipScoreRowDto, RelationshipScoreService, and RelationshipStatsController.
		Integrated score update logic into VendorDispatchesController confirm delivery flow and outlined remaining frontend analytics views for forecast, optimization, and relationship endpoints.
		Developed Progress report3
10-Mar	6	Individual checking
		changing research baseline model to ML related model which focus on ML.NET's forecasting task
		Researched machine learning–based forecasting approaches and identified ML.NET SSA (Singular Spectrum Analysis) time series forecasting as a more suitable ML-driven model for demand prediction.
		Updated research direction by replacing the baseline-only approach with an ML-focused forecasting implementation using ML.NET TimeSeries API and refactored the forecasting workflow in DemandForecastService.cs
		Studied ML.NET forecasting pipeline structure including SSA training window, horizon prediction, model retraining, and demand sequence preparation from historical demand request data, preparing the implementation structure for MLDemandForecastEngine.cs
		Studied ML.NET forecasting pipeline structure including SSA training window, horizon prediction, model retraining, and demand sequence preparation from historical demand request data.

		Refactored forecasting architecture to support multiple forecasting models (Moving Average + ML.NET SSA) and updated related models including DemandForecast.cs, GenerateForecastDto.cs, and DemandForecastRowDto.cs
11-Mar	6	Implemented the ML.NET forecasting engine inside the backend service layer.
		Created MLDemandForecastEngine.cs to train ML.NET models using historical demand request quantities stored in the database.
		Implemented time-series training pipeline using MLContext and SSA forecasting trainer to predict future vendor demand quantities inside MLDemandForecastEngine.cs
		Integrated ML forecast generation into the existing forecasting workflow by updating DemandForecastService.cs so prediction results can be stored in the DemandForecast table
12-Mar	5	Tested the ML.NET forecasting pipeline using historical demand request data inserted into the database and validated forecasting output.
		Verified training data extraction, forecast horizon configuration, and prediction output structure in MLDemandForecastEngine.cs
		Added model identification field (ModelName) to distinguish forecast outputs from MovingAverage vs MLNET_SSA models and updated related structures in DemandForecast.cs and DemandForecastRowDto.cs
13-Mar	4	Developed backend API endpoints to support ML forecasting execution and retrieval through the forecasting module
		Implemented API logic in ForecastController.cs for generating ML forecasts and storing prediction results in the database.
		Ensured compatibility between backend forecasting APIs and frontend analytics components by updating request/response DTOs such as GenerateForecastDto.cs and DemandForecastRowDto.cs

		Developed frontend analytics dashboard for displaying forecasting results and model comparison functionality.
16-Mar	6	Implemented Vendor Forecast Analytics page in VendorForecast.jsx including forecast generation form and model comparison module between Moving Average and ML.NET models.
		Integrated forecasting APIs with React frontend components using secure token-based authentication through forecastApi.js and existing route protection logic
		Added forecast visualization charts to the analytics dashboard to represent demand history and forecast trends visually.
		Implemented chart component ForecastLineChart.jsx to display historical demand data alongside forecast predictions.
		Integrated chart component into both analytics pages VendorForecast.jsx and FarmerForecast.jsx to provide graphical interpretation of forecasting results.
		Began implementation of a real-time communication feature between farmers and vendors within the FarmVendor system.
		Designed the chat module architecture and developed frontend messaging components FarmerChat.jsx and VendorChat.jsx.
		Updated authentication logic in Login.jsx to store userId in localStorage to support message sender identification for chat communication.
		Prepared frontend system structure for integrating backend chat APIs and enabling future real-time messaging functionality between farmers and vendors.
		Completed Progress report 4 and uploaded to blackboard and checked in to github repository.
18-Mar	4	Implemented and tested the farmer–vendor chat module in the FarmVendor system. Added chat conversation creation and message flow between farmer and vendor users.
		Fixed login-related user identification issues by updating Login.jsx to store userId in localStorage correctly.
		Resolved chat database and foreign key issues by verifying valid farmer and vendor IDs from the AspNetUsers table and testing conversation creation successfully.

		Fixed SignalR/CORS connection issues by updating backend configuration in Program.cs and retesting real-time chat connection.
		Improved chat usability by replacing manual user ID entry with searchable selection of existing farmers and vendors using name/email search. Updated ChatController.cs, chatApi.js, FarmerChat.jsx, and VendorChat.jsx.
		Tested the full chat workflow from both farmer and vendor sides, including conversation opening, message display, and sending messages in a more user-friendly interface.
19-Mar	3	Enhanced user experience of forecasting interface by replacing manual userId inputs with searchable dropdowns for vendor and product selection in FarmerForecast.jsx. Developed backend APIs for vendor and product lookup with search functionality in DemandForecastController.cs (endpoints: /vendors, /products). Updated API integration in forecastApi.js to support dynamic search queries and data fetching. Improved validation handling and request payload structure in forecasting workflow to correctly include vendorId and prevent API errors.
		Updated the navigation bar by creating a separate common component file. Refactored FarmerSidebar.jsx, VendorSidebar.jsx, and App.jsx to import the shared component. All related .jsx files were updated to include the new import structure and integrate the sidebar elements.
20-Mar	3	Implemented ML-based vendor recommendation feature to identify top vendors for dispatching based on forecast demand, inventory availability, and historical request data. Developed backend logic in FarmerRecommendationService.cs and exposed API via FarmerController.cs (/recommended-vendors). Integrated frontend in FarmerDashboard.jsx to display top recommended vendors dynamically.
22-Mar	3	Populated the system with additional historical demand data to support forecasting experiments and verified that ML.NET SSA forecast rows were generated successfully in the DemandForecast table.

23-Mar	5	Debugged why recommended vendors and charts were not appearing by checking forecast dates, inventory overlap, API responses, and endpoint usage. Fixed forecasting chart integration by correcting the chart API endpoint in forecastApi.js, updating chart-loading logic to use the selected forecasting model, and refining DemandForecastService.cs chart data retrieval to return recent historical values together with matching forecast values from the selected date onward. Continued testing of farmer-specific recommended vendors and confirmed that recommendation results change based on the logged-in farmer's inventory and forecast matching data.
		Progress report 5 submission
24-Mar	5	Enhanced forecasting dashboard UI with advanced settings panel including vendor, product, date, model, and horizon selection.
		Implemented default parameter auto-loading for vendor, product, and forecast date to improve usability.
		Improved error handling for forecast generation and API failures in FarmerForecast.jsx.
25-Mar	4	Implemented CSV export functionality for forecast results.
		Added Export button to forecasting dashboard and generated downloadable CSV file including vendor name, product name, forecast date, quantity, model, and created date.
		Ensured removal of internal IDs from export and replaced with user-friendly names.
26-Mar	6	Upgraded forecast visualization to support multi-vendor comparison charts.
		Developed MultiVendorForecastLineChart.jsx to display up to 4 vendors in a single graph with distinct colors and legends.
		Integrated vendor selection logic allowing dynamic selection of top vendors from forecast results.
27-Mar	5	Implemented backend API enhancement for chart data retrieval per vendor and product.
		Added filtering logic in DemandForecastController.cs to support vendorId, productId, and forecastDate.
		Improved chart API response to include both historical demand and forecast data series.

28-Mar	4	Enhanced frontend chart rendering logic by fixing dataset mapping and handling missing/null values.
		Resolved chart data alignment issues between historical and forecast series.
		Improved chart readability with legend labels and consistent formatting.
29-Mar	6	Implemented vendor filtering logic based on farmer inventory.
		Restricted vendors to only those having demand requests matching farmer's available products.
		Updated backend logic in DemandForecastService.cs and FarmerController.cs to support real-world filtering conditions.
30-Mar	5	Improved forecasting data realism by generating additional historical demand data.
		Enhanced SQL scripts using CTE logic to simulate multi-period demand patterns.
		Validated ML.NET SSA forecasting output with expanded dataset.
31-Mar	4	Enhanced dashboard UX by displaying selected vendor name and product name directly on the chart section.
		Removed redundant UI elements such as "Available Model Names" for cleaner interface.
01-Apr	7	Improved layout alignment and readability.
		Integrated Unsplash API for product image selection in inventory management.
		Implemented image search and selection within Add Product Lot modal.
02-Apr	6	Stored image metadata (ImageUrl, ImageThumbUrl, PhotographerName) in Product entity and displayed images in UI.
		Enhanced product selection UX by displaying images alongside product names in dropdowns.
		Improved user experience by allowing visual identification of products.
		Added fallback handling for missing images.
03-Apr	8	Improved real-time chat module stability and authentication handling.

		Resolved SignalR WebSocket connection issues caused by missing JWT token.
		Updated ChatHub configuration and frontend connection logic to include access token.
04-Apr	6	Enhanced chat UI by replacing user IDs with display names in conversation lists.
		Implemented backend join queries in ChatController.cs to return vendor and farmer names.
		Improved conversation selection usability with searchable dropdowns.
05-Apr	7	Optimized API performance by reducing redundant API calls in forecasting and chat modules.
		Implemented caching and minimized repeated fetch requests in React components.
		Improved loading times for dashboard and analytics pages.
06-Apr	8	Performed full system integration testing across all modules including forecasting, dispatch, recommendation, chat, and inventory.
		Verified role-based access control for Farmer and Vendor workflows.
		Fixed minor bugs related to DTO mismatches and API responses.
07-Apr	7	Conducted final UI/UX refinements across the application.
		Improved dashboard layout to resemble Power BI style with better spacing and card alignment.
		Enhanced consistency of buttons, dropdowns, and tables.
		Final report modifications
08-Apr	8	Final testing and validation of ML.NET SSA forecasting accuracy and system outputs.
		Verified correctness of forecast results, recommendation outputs, and chart visualizations.
		Prepared final screenshots, documentation, and report content.
		Final report modifications
		Completed final report writing including architecture diagrams, implemented features, evaluation techniques, and AI usage section.

		Prepared appendix with prompt history and finalized all submission materials.
		Prepared presentation file

10.Concluding Remarks

The FarmVendor project successfully achieved its objective of designing and implementing a data-driven agricultural supply coordination system that connects farmers and vendors through an intelligent, user-friendly platform.

The system evolved from a basic CRUD-based application into a comprehensive decision-support system incorporating advanced features such as demand forecasting, vendor recommendation, real-time communication, and interactive analytics dashboards. A key contribution of this project is the integration of machine learning–based forecasting using ML.NET Singular Spectrum Analysis (SSA), which enables more accurate and future-oriented demand predictions compared to traditional statistical methods.

The project also demonstrates the practical application of full-stack development using ASP.NET Core, React, SQL Server, and modern APIs such as SignalR and Unsplash. The inclusion of role-based authentication, dynamic dashboards, and real-time chat further enhances usability and system effectiveness in real-world agricultural scenarios.

Through iterative development, testing, and evaluation, several design improvements were made, including enhanced data generation strategies, optimized API integration, and improved user experience through intuitive UI components and visualization tools. The implementation of multi-vendor comparison charts and exportable analytics further strengthens the system's analytical capabilities.

Overall, the FarmVendor system demonstrates a scalable and extensible architecture that can be further expanded to support real-world agricultural supply chain optimization. Future enhancements may include advanced machine learning models, mobile application support, and integration with external market data sources.

11.References

AgriWebb. (2024). *AgriWebb farm management software* [Mobile application]. https://www.agriwebb.com

FarmLogs. (2024). *FarmLogs: Farm management and record keeping* [Mobile application]. https://farmlogs.com

CropIn Technology Solutions. (2024). *CropIn smart farm management platform* [Mobile application]. https://www.cropin.com

AgriDigital. (2024). *AgriDigital grain supply chain platform* [Web-based platform]. https://www.agridigital.io

Microsoft. (2024). *ML.NET Documentation*. https://learn.microsoft.com/en-us/dotnet/machine-learning/

Microsoft. (2024). *ASP.NET Core Documentation*. https://learn.microsoft.com/en-us/aspnet/core/

React Documentation. (2024). https://react.dev/

Chart.js Documentation. (2024). https://www.chartjs.org/docs/latest/

Unsplash API Documentation. (2024). https://unsplash.com/documentation

Microsoft. (2024). *SignalR Documentation*. https://learn.microsoft.com/en-us/aspnet/core/signalr/

SQL Server LocalDB Documentation. (2024). https://learn.microsoft.com/en-us/sql/database-engine/configure-windows/sql-server-express-localdb

12.Appendix A: Installation Guide

This section provides step-by-step instructions to set up and run the FarmVendor application locally.

1. Clone the Repository

git clone https://github.com/maneeshaprs-rgb/W26_4495_S3_ManeeshaE.git
cd W26_4495_S3_ManeeshaE

2. Backend Setup (ASP.NET Core API)

cd FarmVendor/FarmVendor.Api

Install Dependencies

dotnet restore

Install EF Core Tool (if not installed)

dotnet tool install --global dotnet-ef

Install ML.NET Packages

dotnet add package Microsoft.ML
dotnet add package Microsoft.ML.TimeSeries

3. Database Setup

Run Migrations

dotnet ef migrations add AddProductImageFields
dotnet ef database update

(If migrations already exist, only run database update)

4. Run Backend Server

dotnet build
dotnet run

Backend will run at:

http://localhost:5136

5. Frontend Setup (React + Vite)

cd ../../farmvendor-web
npm install

Install Required Packages

npm install react-chartjs-2 chart.js
npm install @microsoft/signalr
npm install react-select

6. Configure Environment Variables

Create .env file inside farmvendor-web:

VITE_API_URL=http://localhost:5136

7. Run Frontend

npm run dev

Application will run at:

http://localhost:5173/

8. Database Connection (Optional View)

- Install SQL Server Extension in VS Code
- Connect using:
    - Server: (localdb)\MSSQLLocalDB
    - Authentication: Windows
    - Trust Certificate: Yes

9. Test Credentials

Farmer

- Email: farmer1@test.com
- Password: Test@123

Vendor

- Email: vendor1@test.com
- Password: Test@123

10. Sample Test Parameters (Forecasting)

- Vendor ID: 2660cd91-4a85-4083-b925-ceceab6b8529
- Product ID: 4
- Forecast Date: 2026-03-12

11. Run Sample SQL Scripts

Location:

ReportsAndDocuments/sql_queries/

Run:

- Product.sql
- Product2.sql

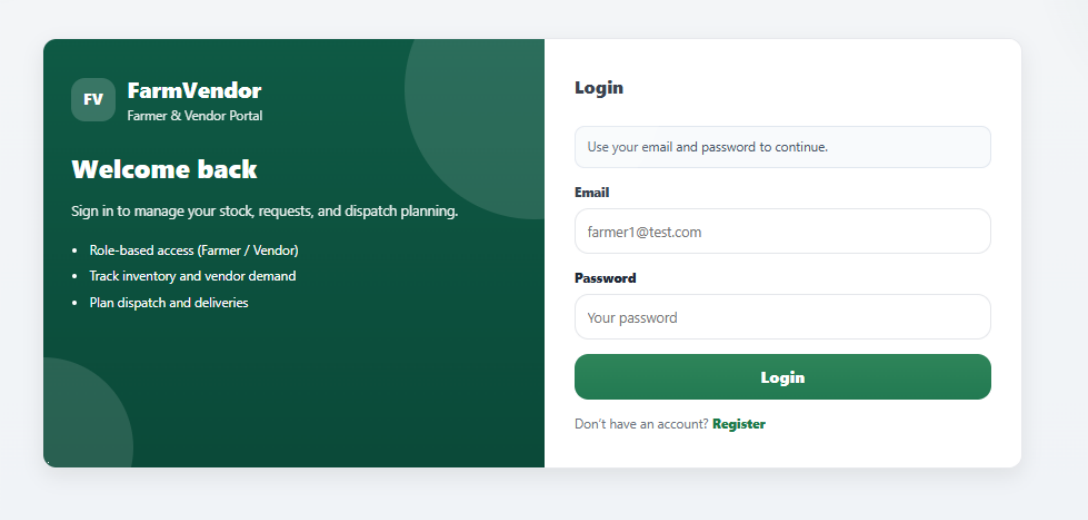
13.Appendix B: User Guide

This section explains how end users (Farmers and Vendors) can use the FarmVendor system.

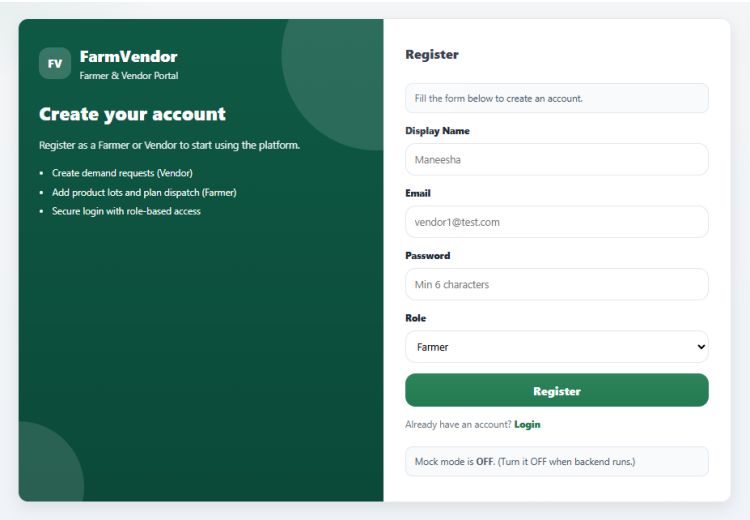
1. Login / Registration

- Open: http://localhost:5173
- Register as:
- Farmer OR Vendor
- Login using credentials

- Register interface



- Login interface

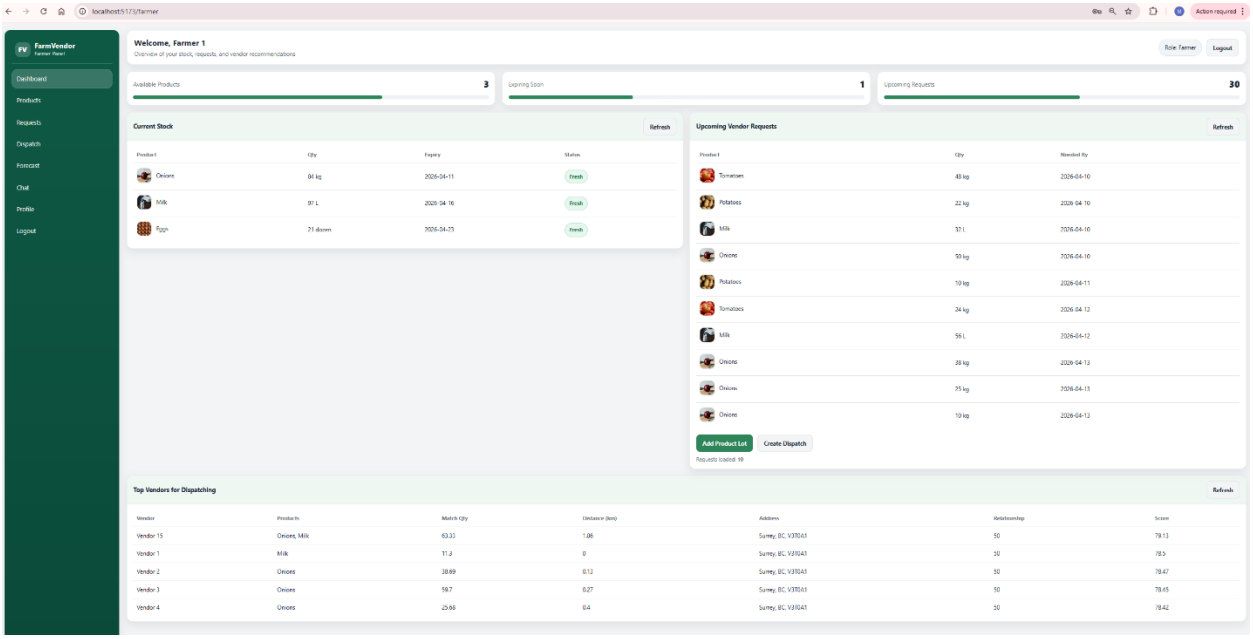


2. Farmer Functionalities

Dashboard

- View inventory summary
- View recommended vendors

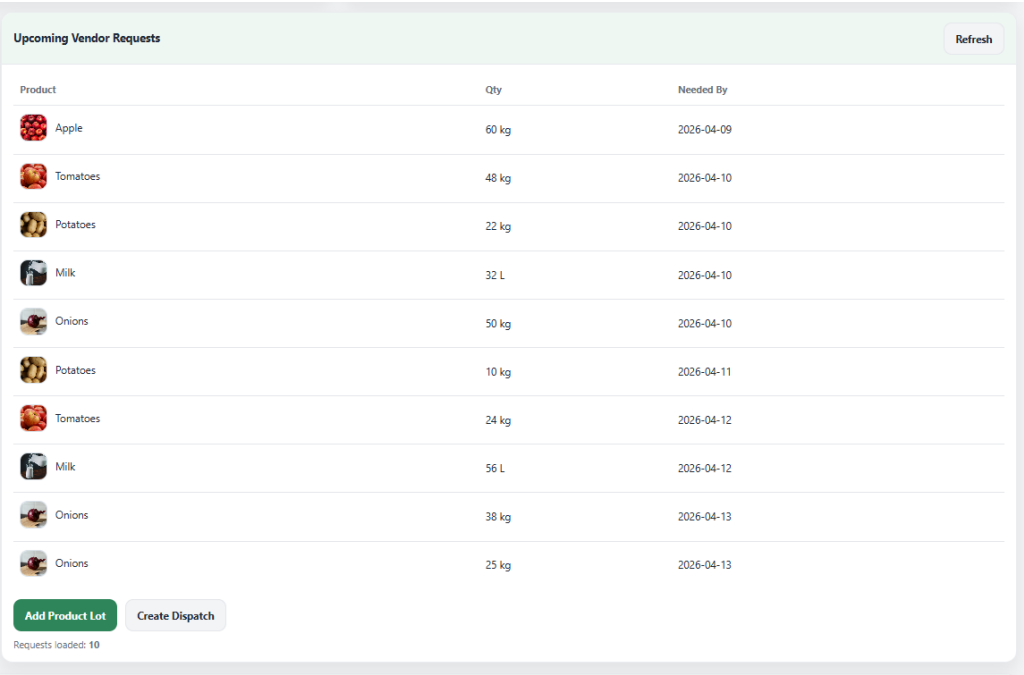
- FarmerDashboard

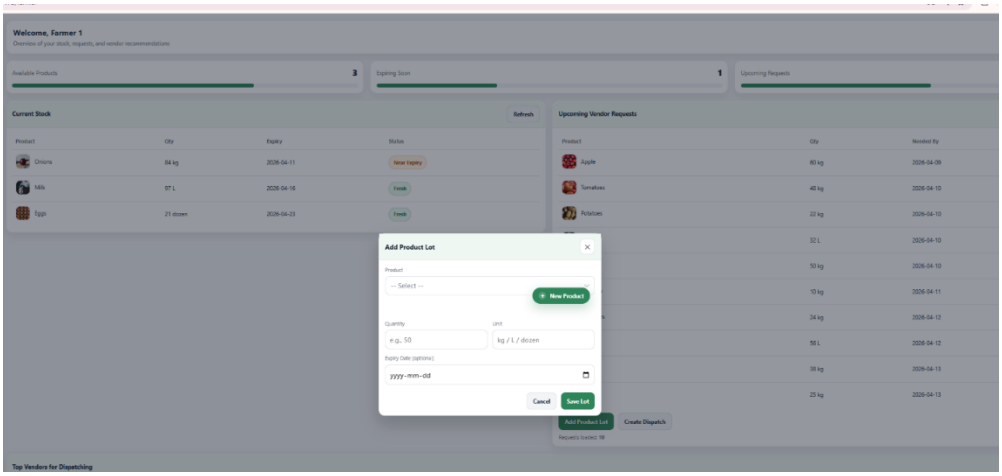


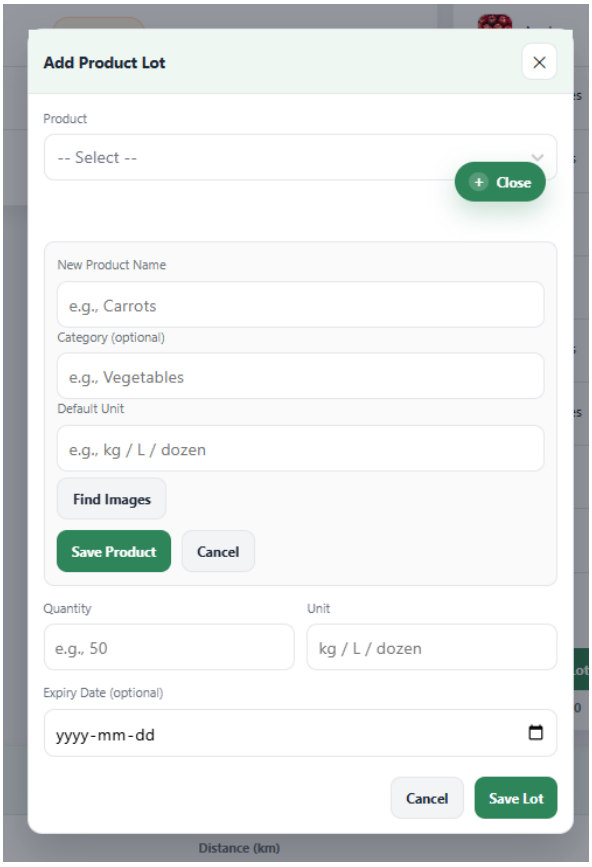
Inventory Management

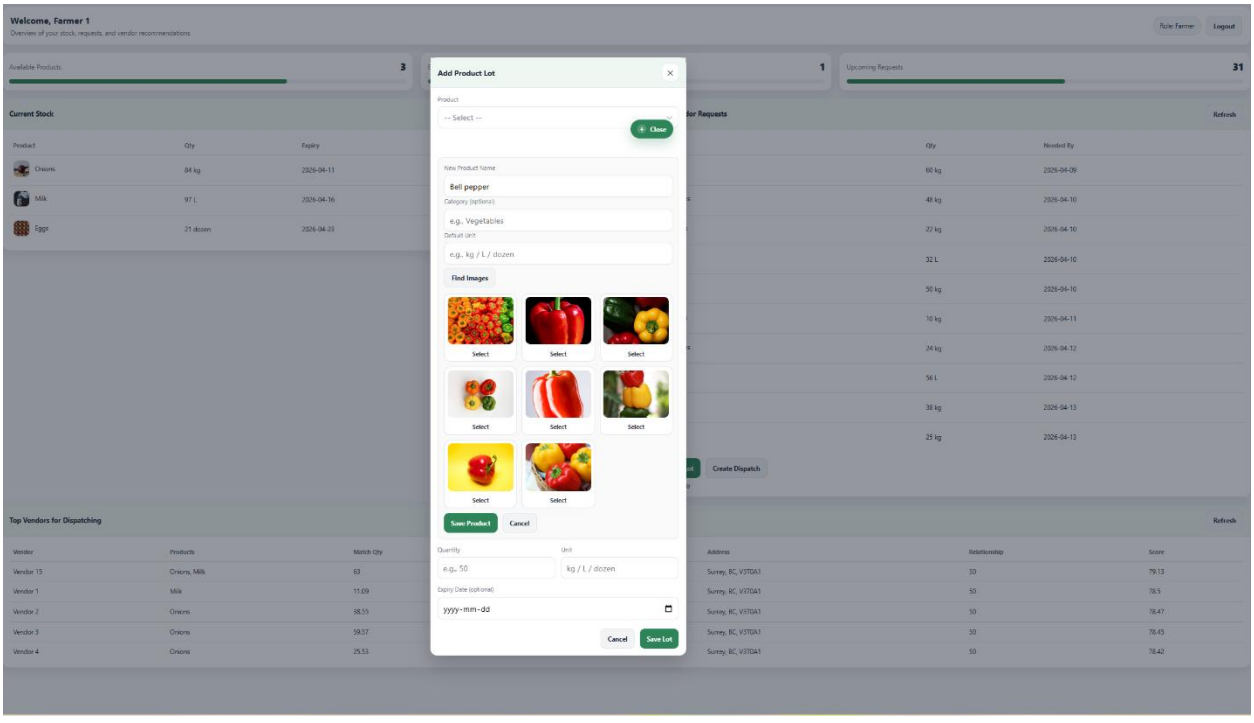
- Add product lot
- Select product (with image from Unsplash)
- Enter quantity and expiry date

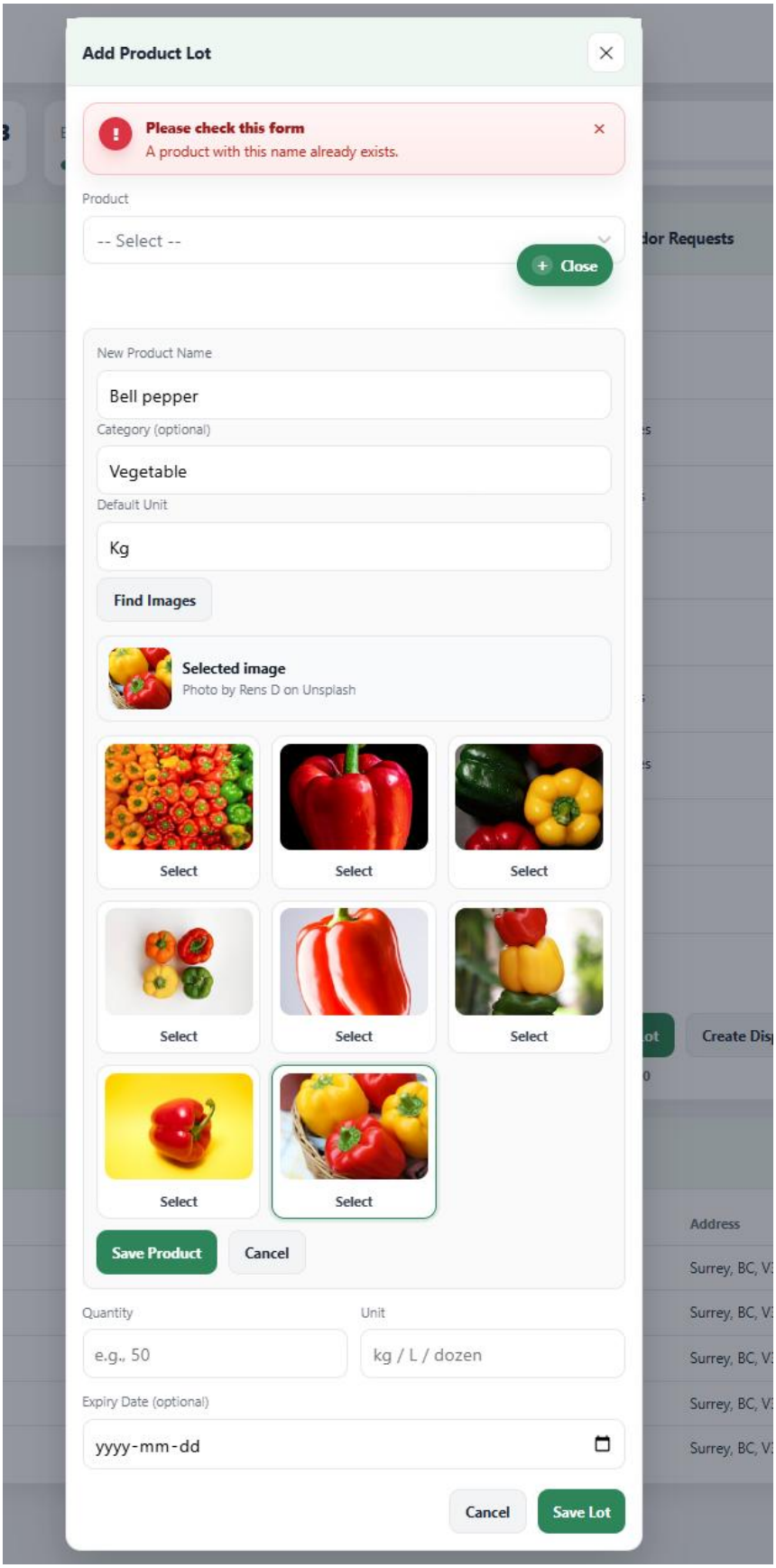
- FarmerDashboard

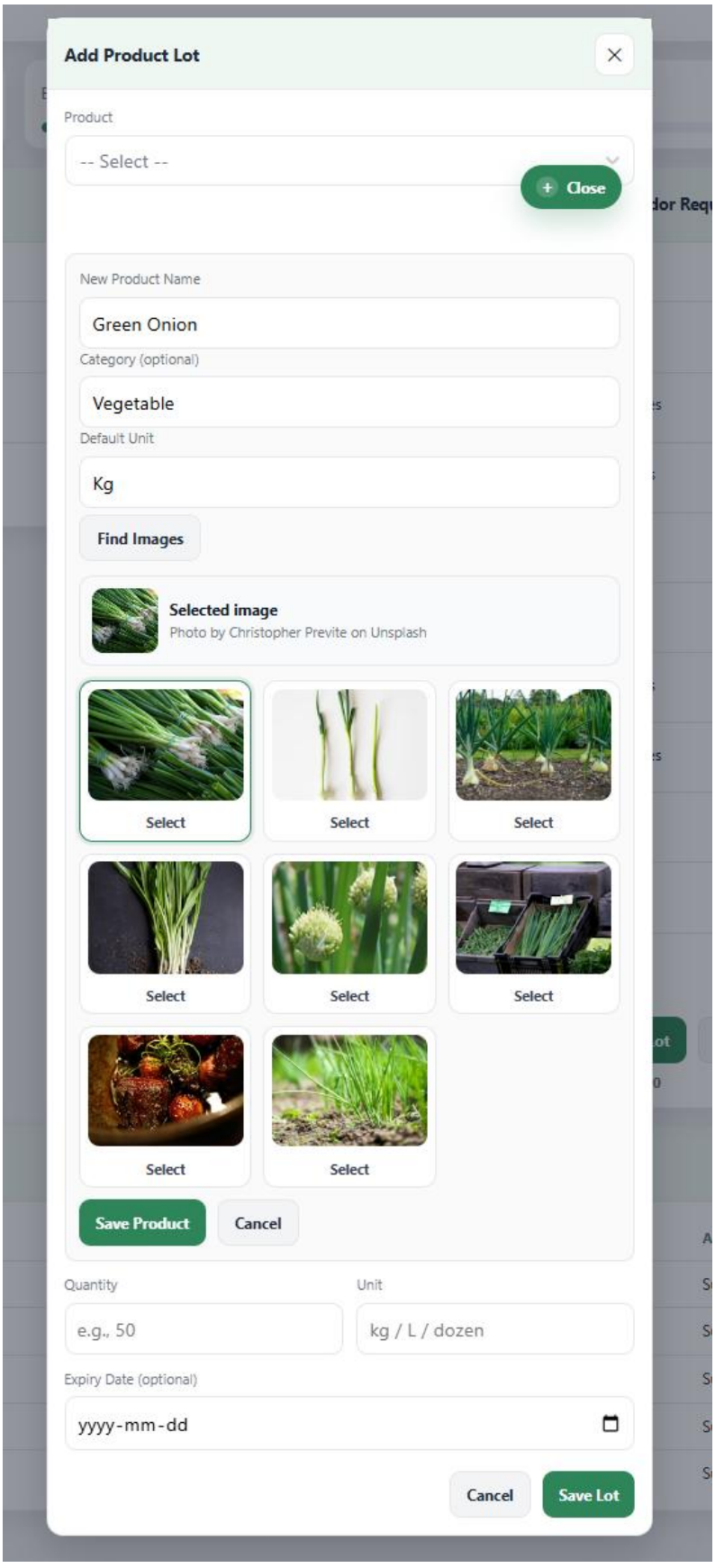


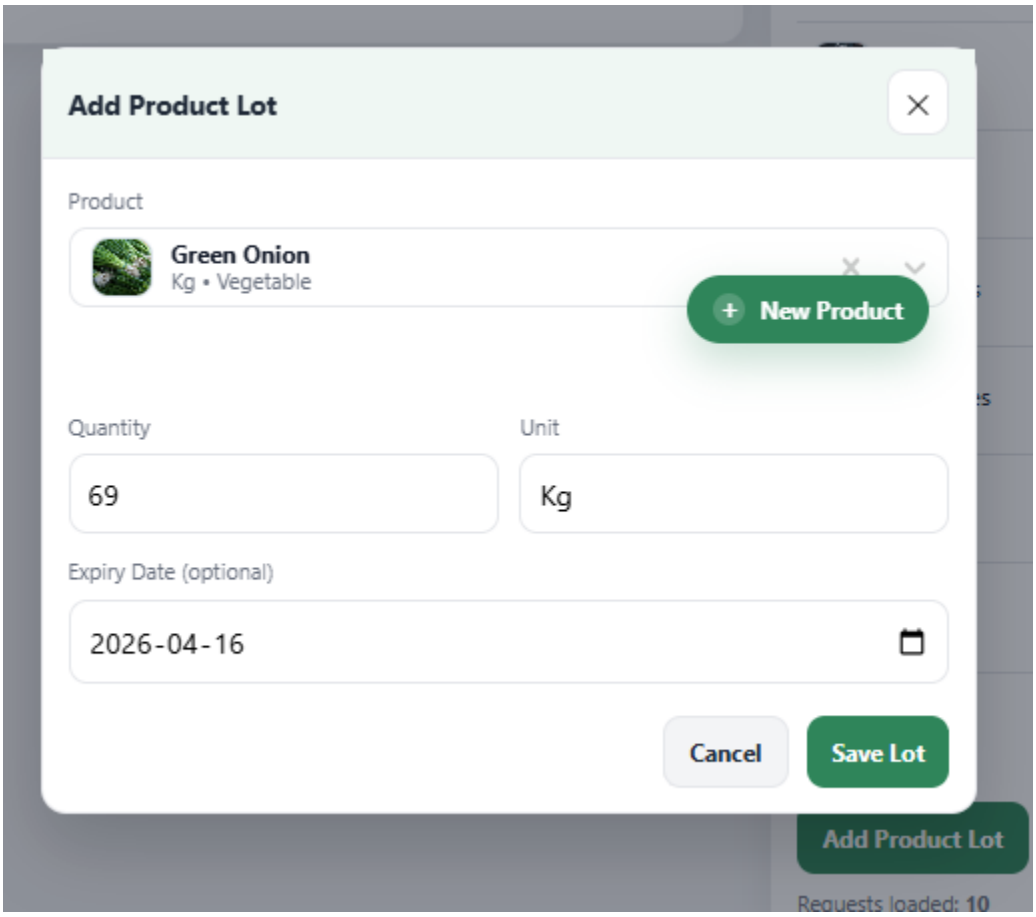


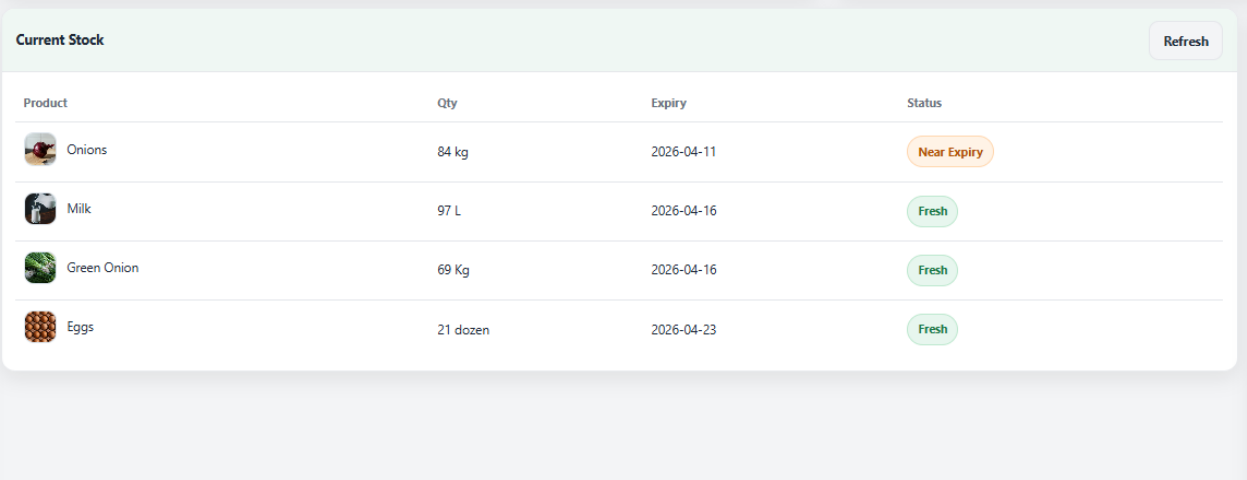

Add Product Lot
Product
-- Select --
Close
New Product Name
e.g., Carrots
Category (optional)
e.g., Vegetables
Default Unit
e.g., kg / L / dozen
Find Images
Save Product
Cancel
Quantity
e.g., 50
Unit
kg / L / dozen
Expiry Date (optional)
yyyy-mm-dd
Cancel
Save Lot
Distance (km)


Welcome, Farmer 1
Add Product Lot
Bell pepper
Find Images
Save Product
Cancel
Save Lot

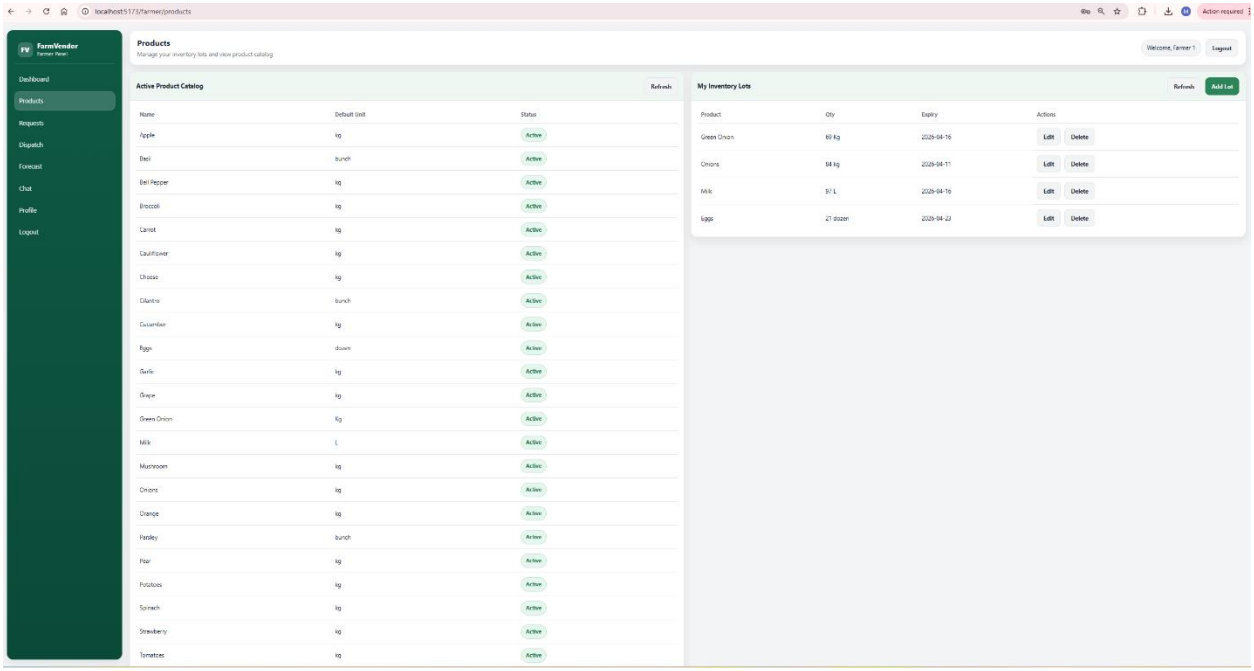

Add Product Lot
Please check this form
A product with this name already exists.
Product
-- Select --
Close
New Product Name
Bell pepper
Category (optional)
Vegetable
Default Unit
Kg
Find Images
Selected image
Photo by Rens D on Unsplash
Select
Select
Select
Select
Select
Select
Select
Select
Select
Save Product
Cancel
Quantity
e.g., 50
Unit
kg / L / dozen
Expiry Date (optional)
yyyy-mm-dd
Cancel
Save Lot
Address
Surrey, BC, V
Surrey, BC, V
Surrey, BC, V
Surrey, BC, V
Surrey, BC, V
Create Disp


Add Product Lot
Product
-- Select --
+ Close
New Product Name
Green Onion
Category (optional)
Vegetable
Default Unit
Kg
Find Images
Selected image
Photo by Christopher Previte on Unsplash
Select
Select
Select
Select
Select
Select
Select
Select
Save Product
Cancel
Quantity
e.g., 50
Unit
kg / L / dozen
Expiry Date (optional)
yyyy-mm-dd
Cancel
Save Lot


Add Product Lot
Product
Green Onion
Kg • Vegetable
New Product
Quantity
69
Unit
Kg
Expiry Date (optional)
2026-04-16
Cancel
Save Lot
Add Product Lot
Requests loaded: 10


Current Stock
Refresh
Product
Qty
Expiry
Status
Onions
84 kg
2026-04-11
Near Expiry
Milk
97 L
2026-04-16
Fresh
Green Onion
69 Kg
2026-04-16
Fresh
Eggs
21 dozen
2026-04-23
Fresh

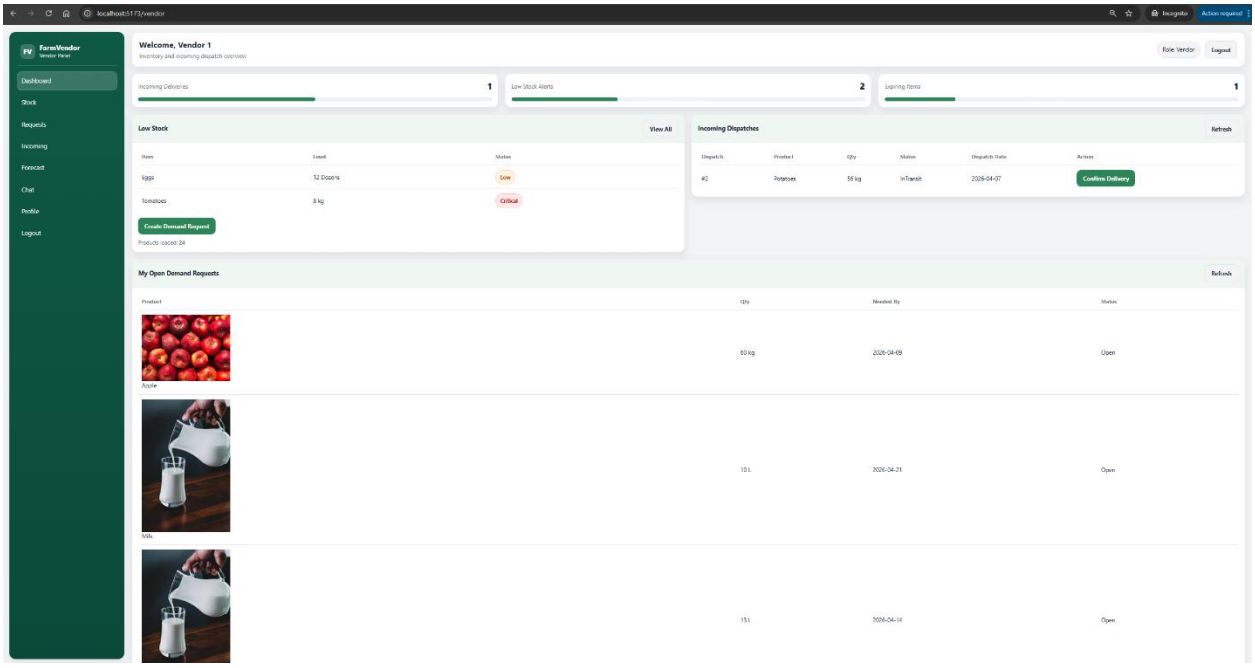
- Product interface

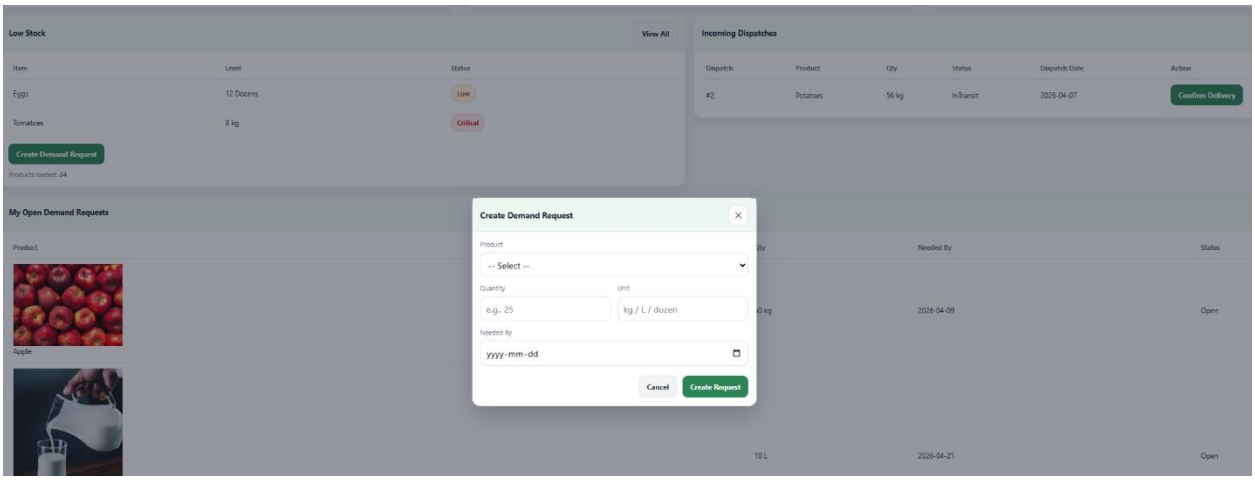


Demand Requests

- View vendor requests
- Track status

- VendorDashboard

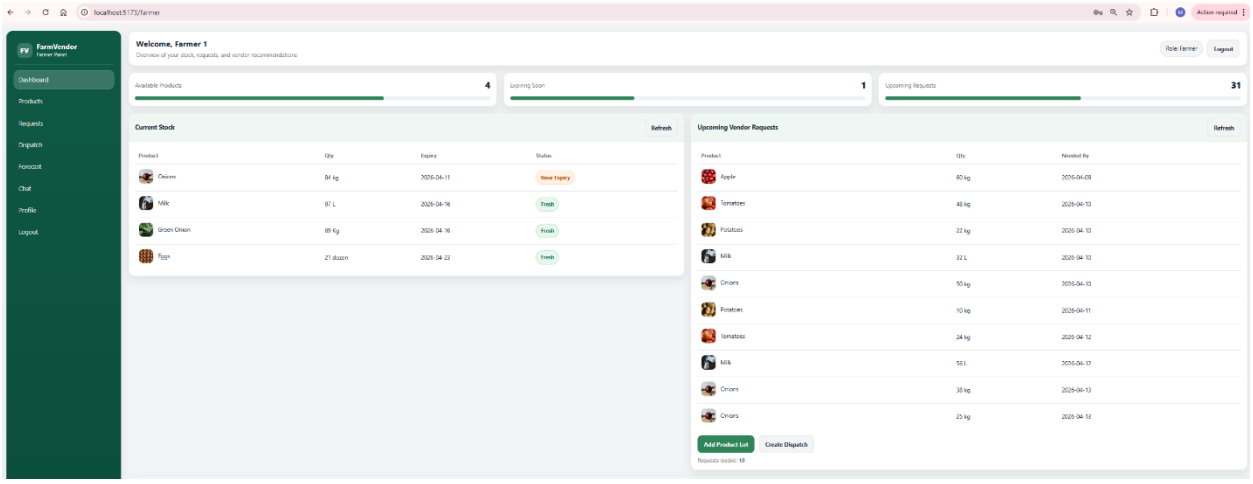


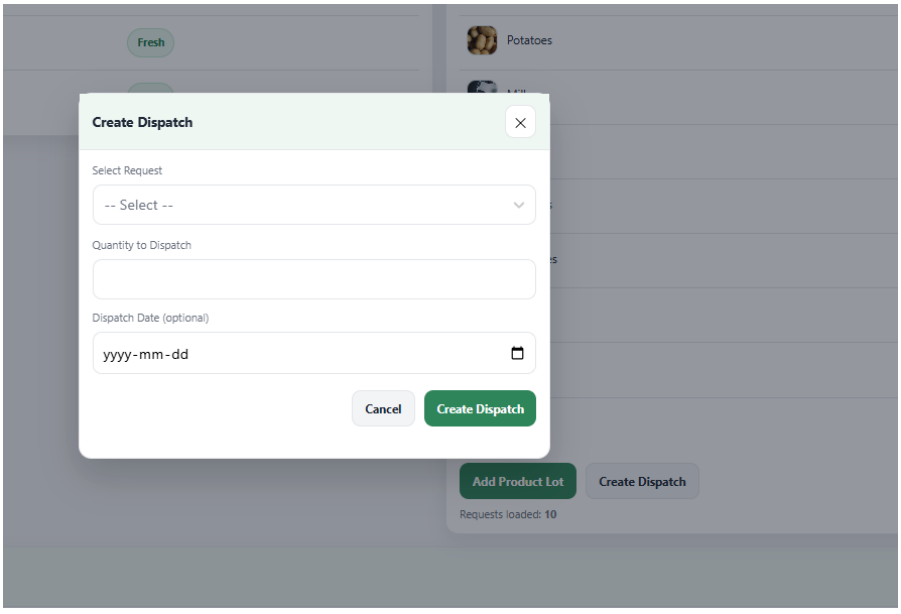


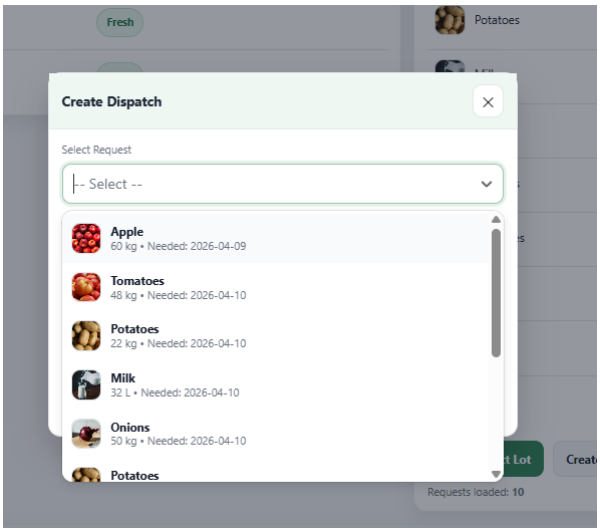
Dispatch Module

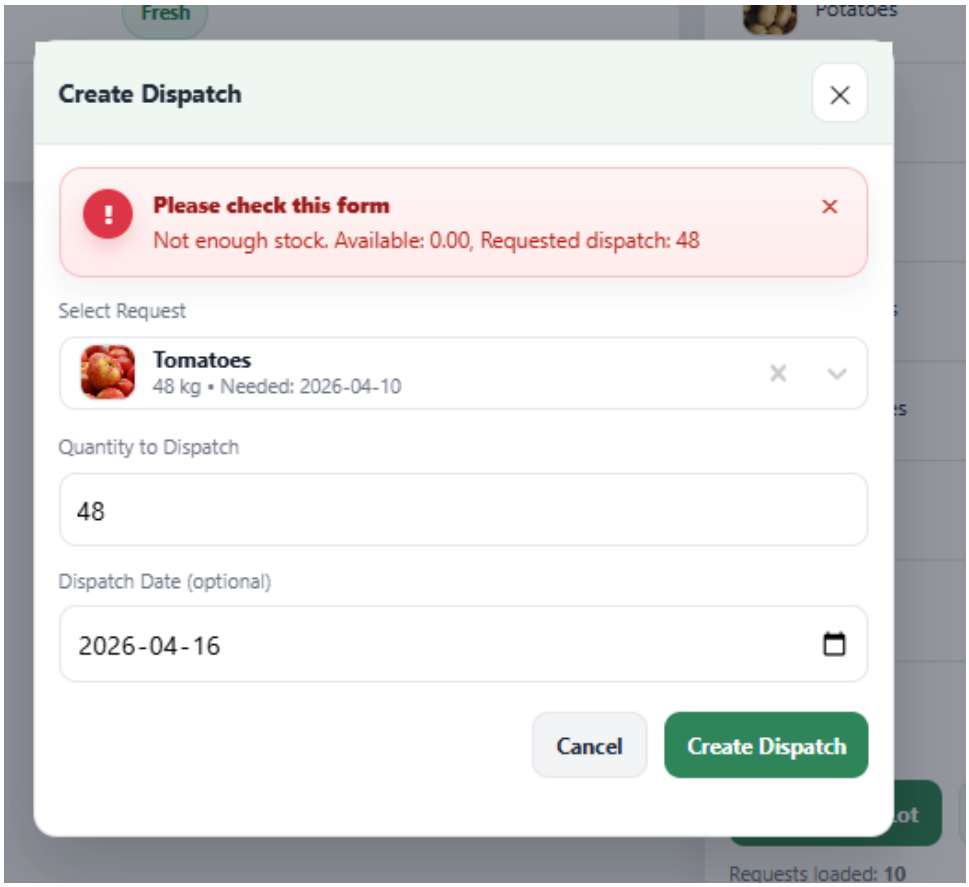
- Create dispatch for vendor requests
- Update delivery status

- FarmerDashboard

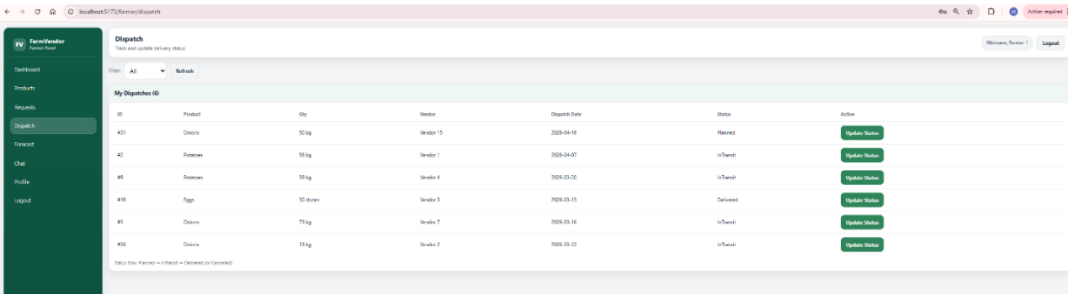


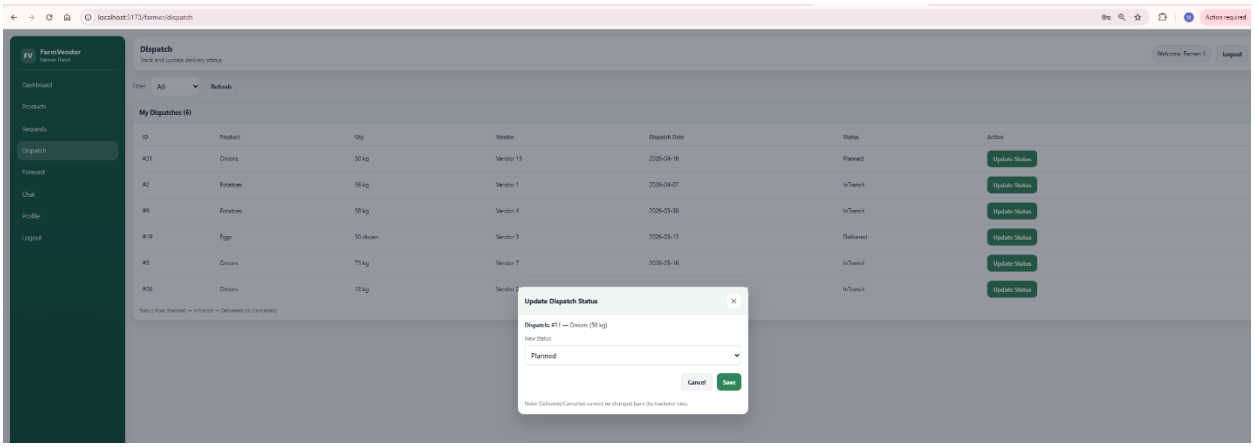


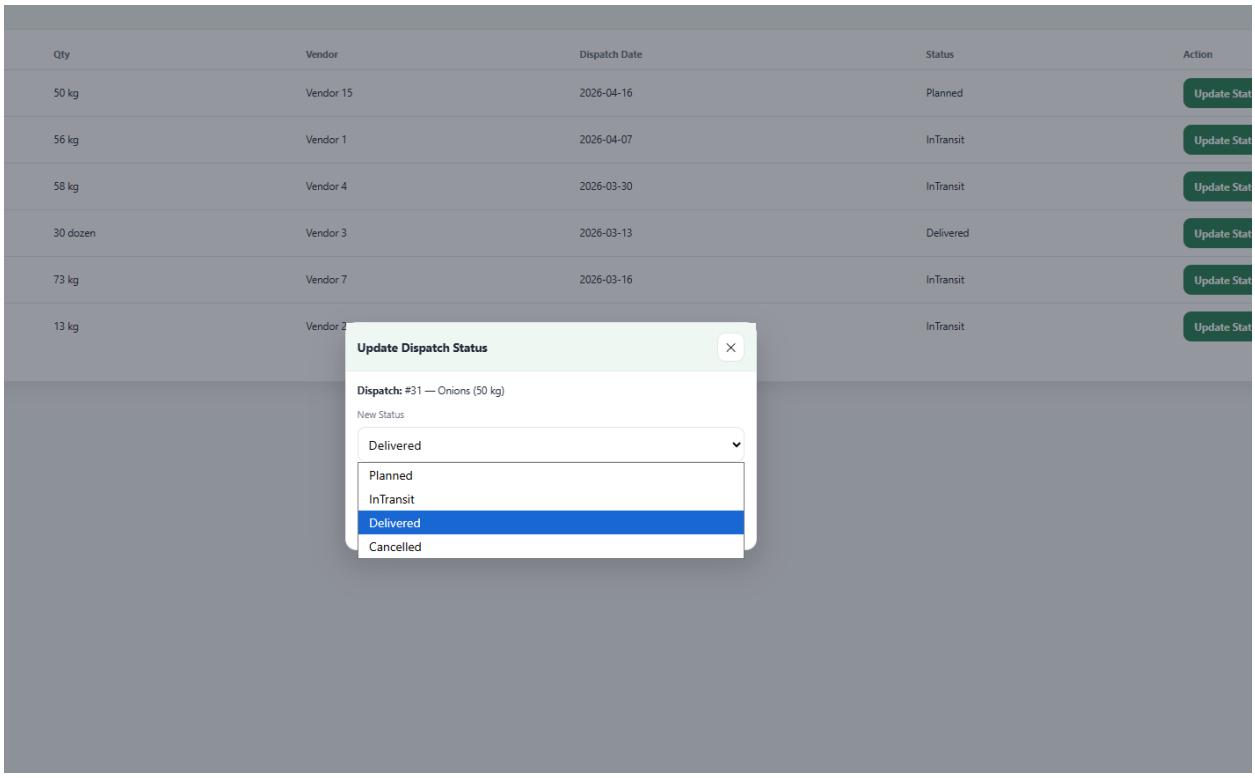

Fresh
Potatoes
Create Dispatch
Select Request
-- Select --
Apple
60 kg • Needed: 2026-04-09
Tomatoes
48 kg • Needed: 2026-04-10
Potatoes
22 kg • Needed: 2026-04-10
Milk
32 L • Needed: 2026-04-10
Onions
50 kg • Needed: 2026-04-10
Potatoes
Requests loaded: 10

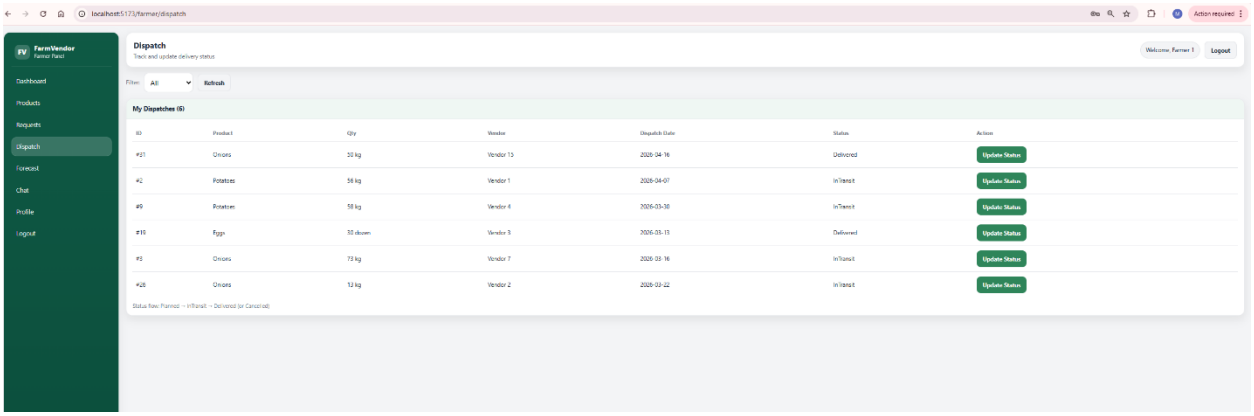

Fresh
Potatoes
Create Dispatch
Please check this form
Not enough stock. Available: 0.00, Requested dispatch: 48
Select Request
Tomatoes
48 kg • Needed: 2026-04-10
Quantity to Dispatch
48
Dispatch Date (optional)
2026-04-16
Cancel
Create Dispatch
Requests loaded: 10

- Dispatch interface

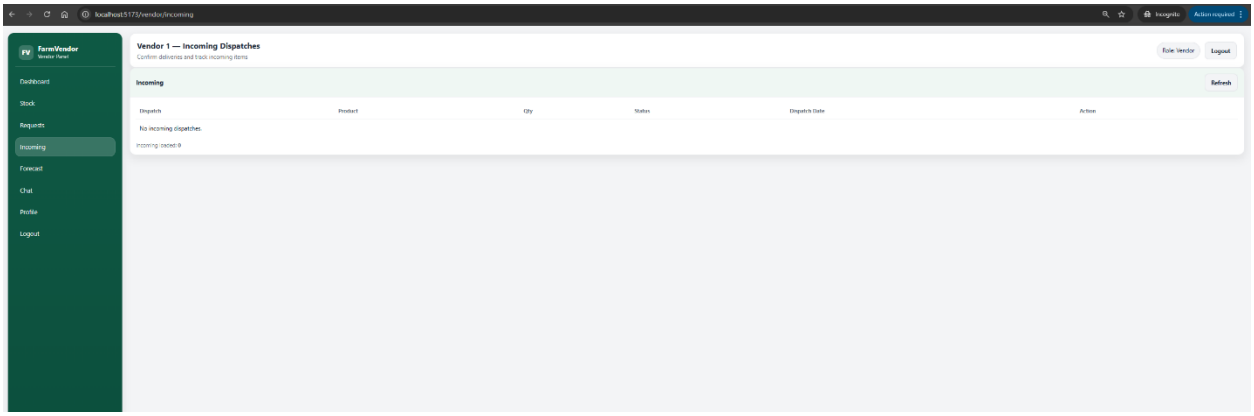






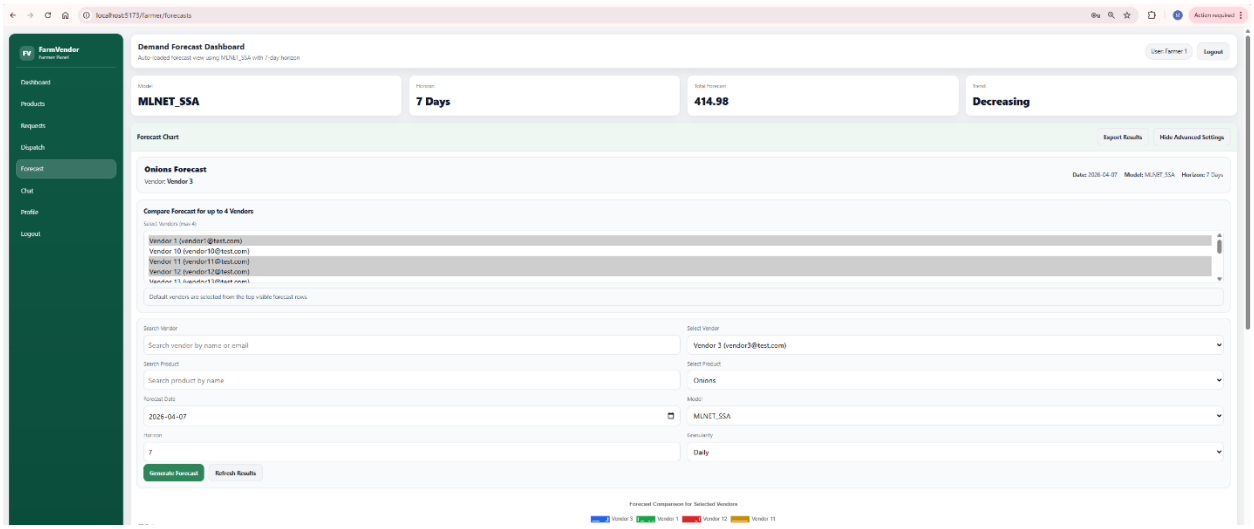


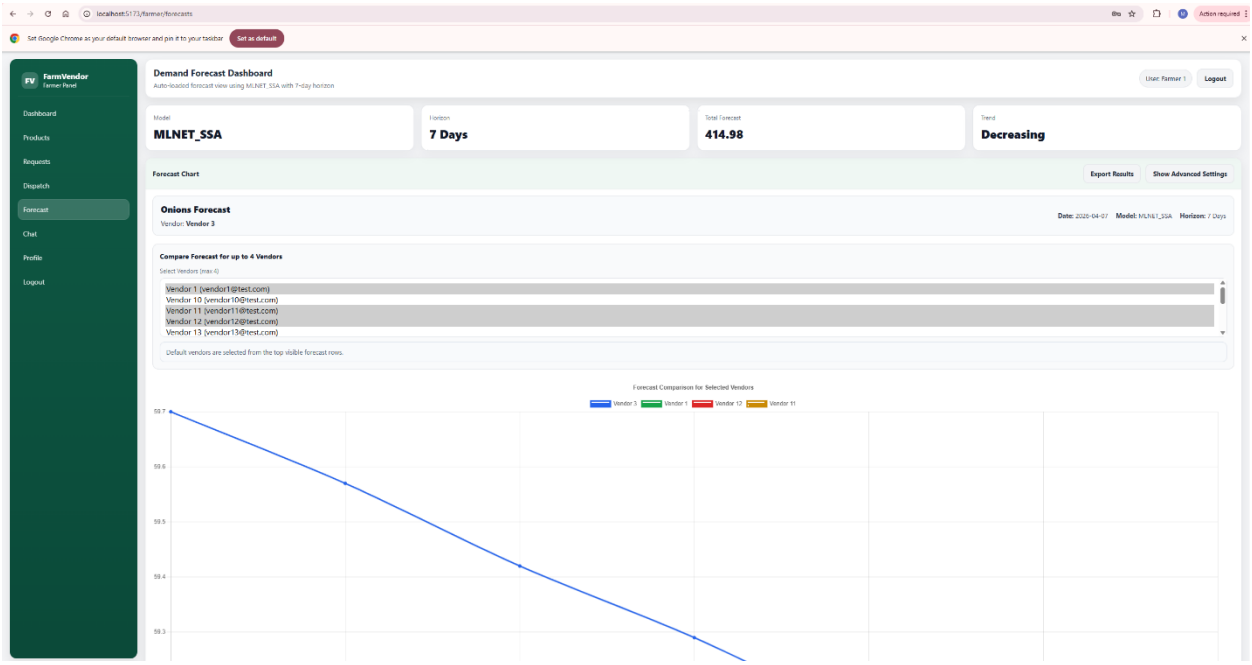
Vendor-dispatch incoming

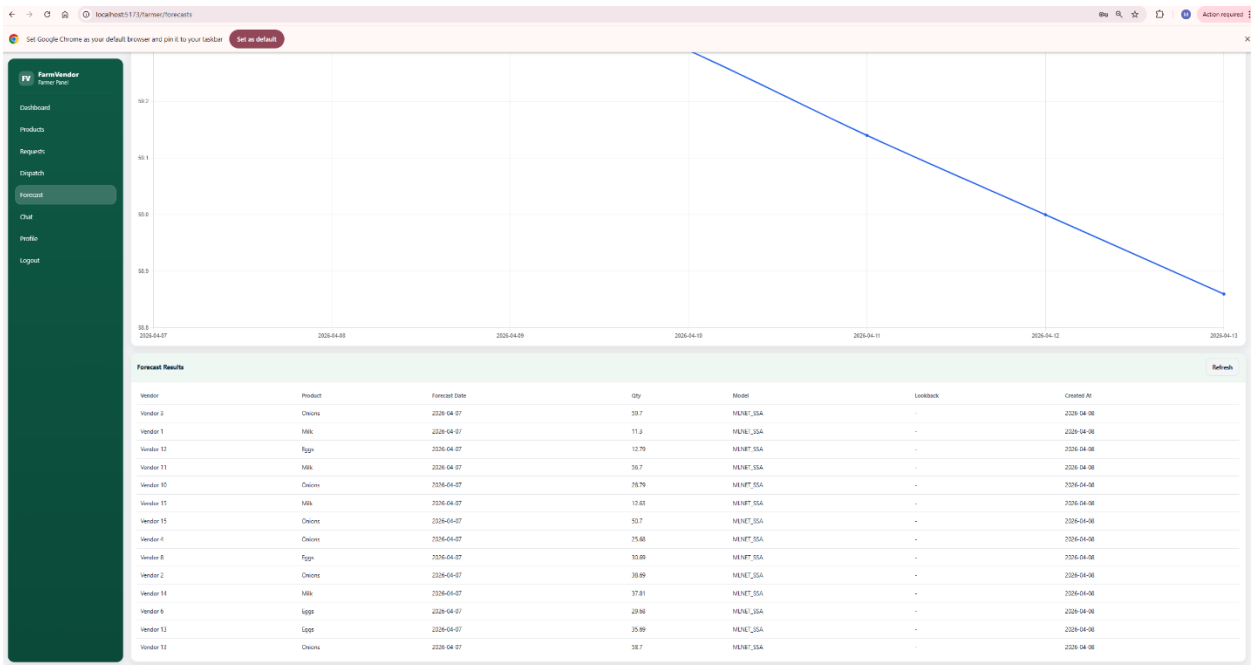


Forecast Dashboard

- Select vendor, product, and date
- Choose model (Moving Average / ML.NET SSA)
- View:
    - Forecast chart
    - Multi-vendor comparison
- Export results as CSV



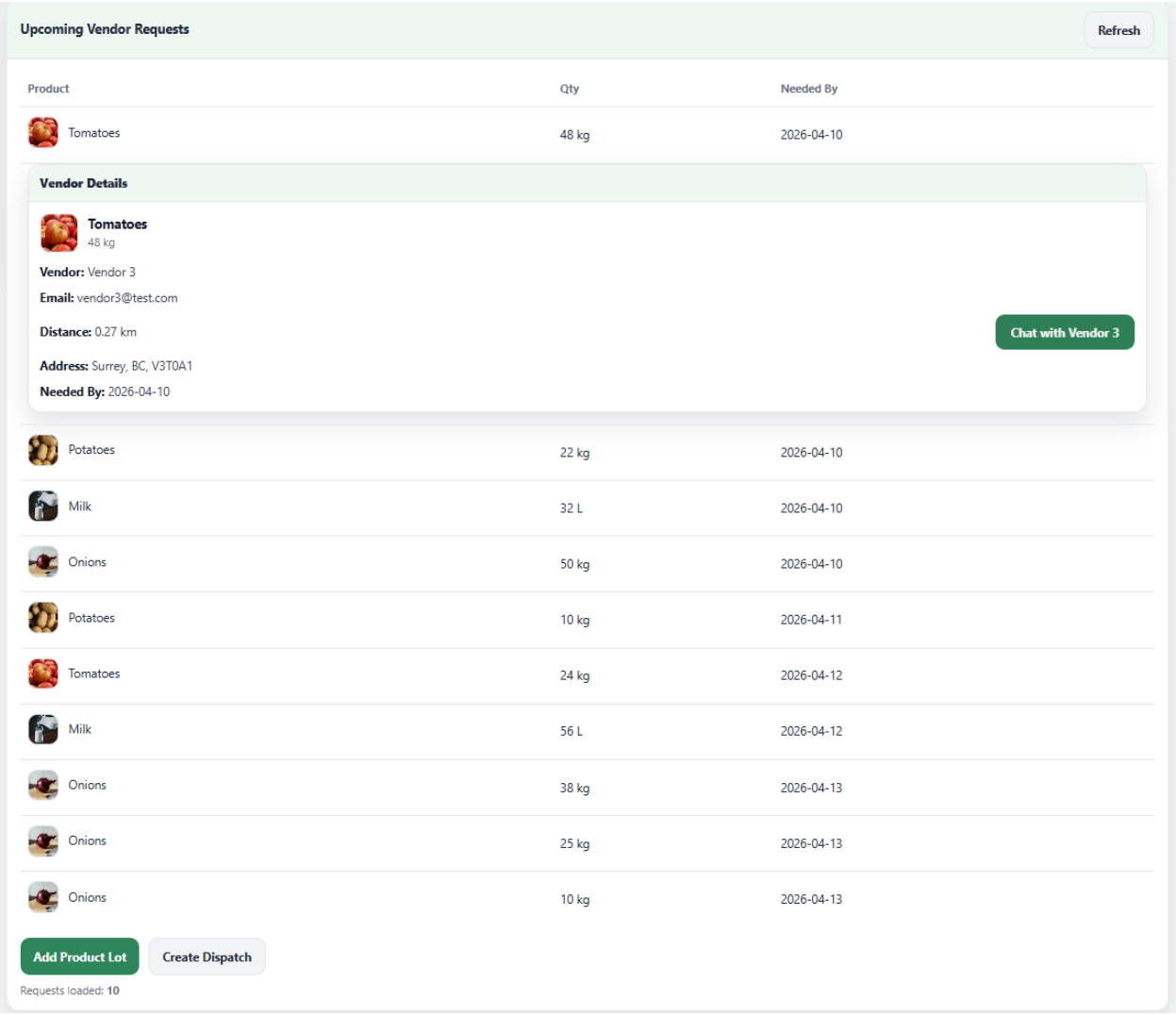

Demand Forecast Dashboard
Auto-loaded forecast view using MLNET_SSA with 7-day horizon
MLNET_SSA
7 Days
414.98
Decreasing
Onions Forecast
Vendor 3
Compare Forecast for up to 4 Vendors
Vendor 1 (vendor1@test.com)
Vendor 10 (vendor10@test.com)
Vendor 11 (vendor11@test.com)
Vendor 12 (vendor12@test.com)
Vendor 13 (vendor13@test.com)
Forecast Comparison for Selected Vendors


Forecast Results
Refresh

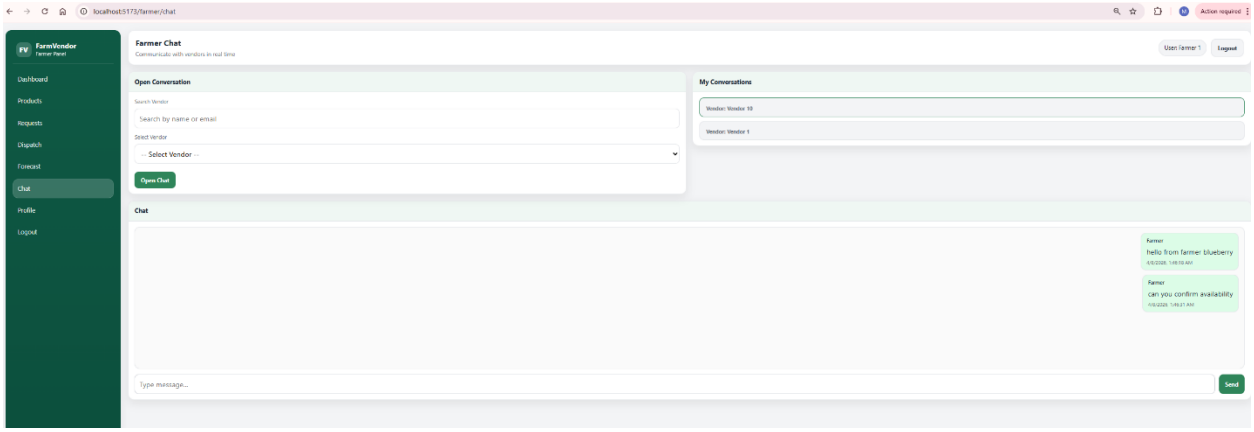
Chat Module

- Select vendor
- Start conversation
- Send and receive real-time messages

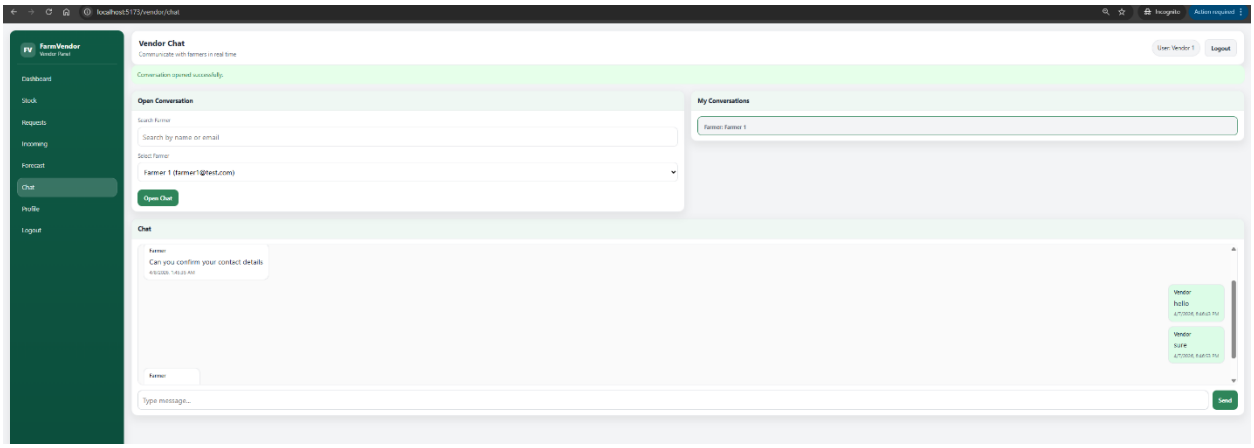
  - farmerDashboard



- Farmer chat panel



- Vendor chat panel

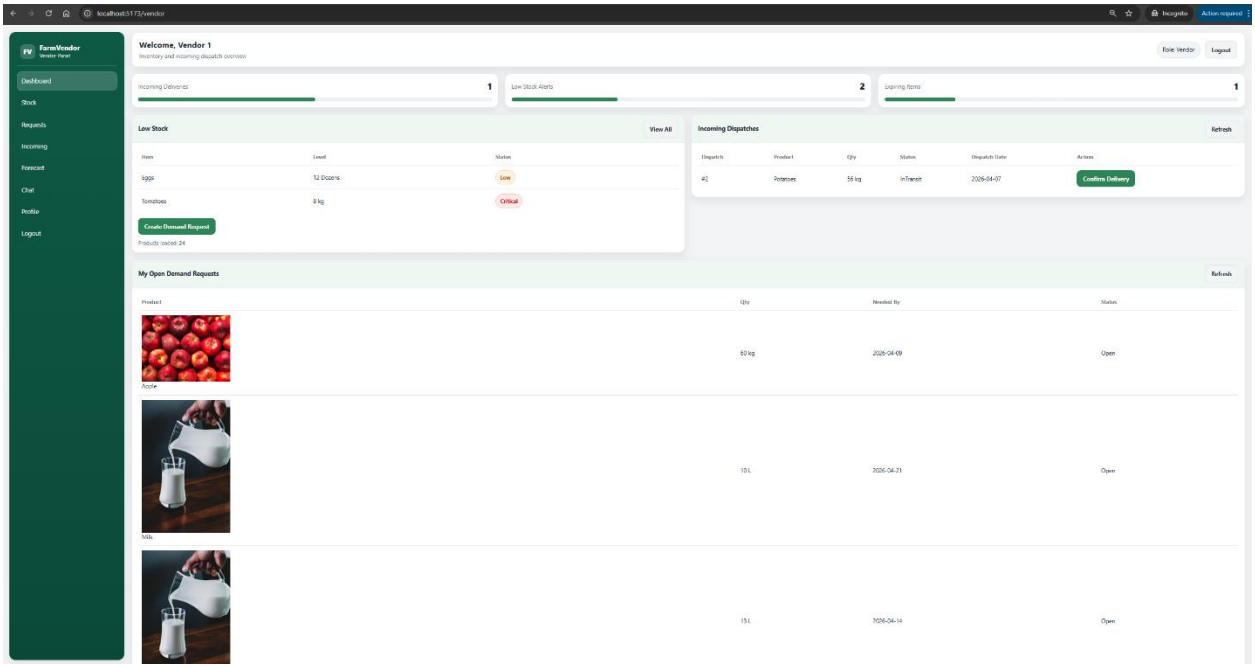


3. Vendor Functionalities

Dashboard

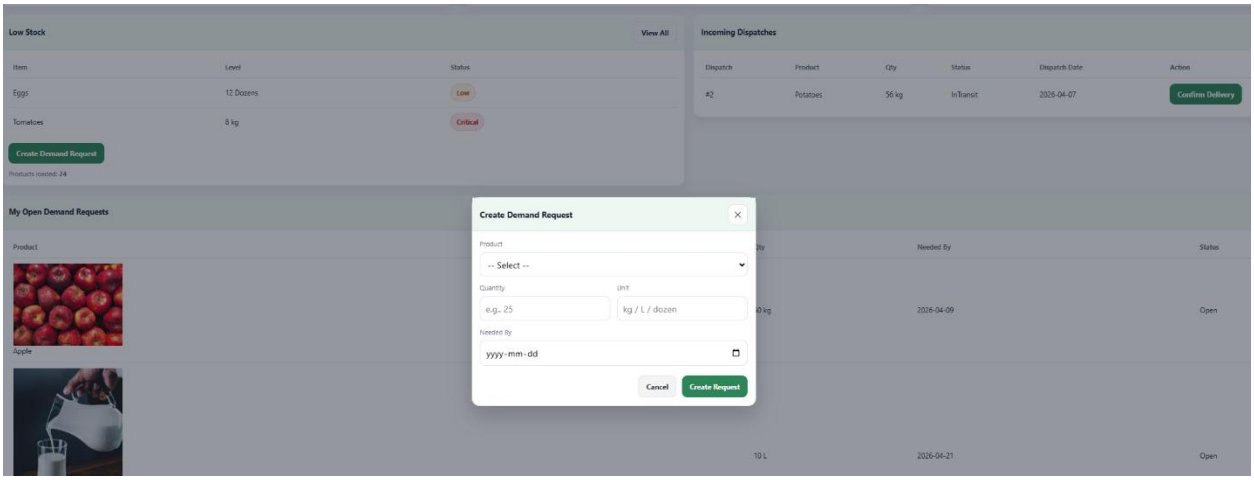
- View low stock alerts

- VendorDashboard



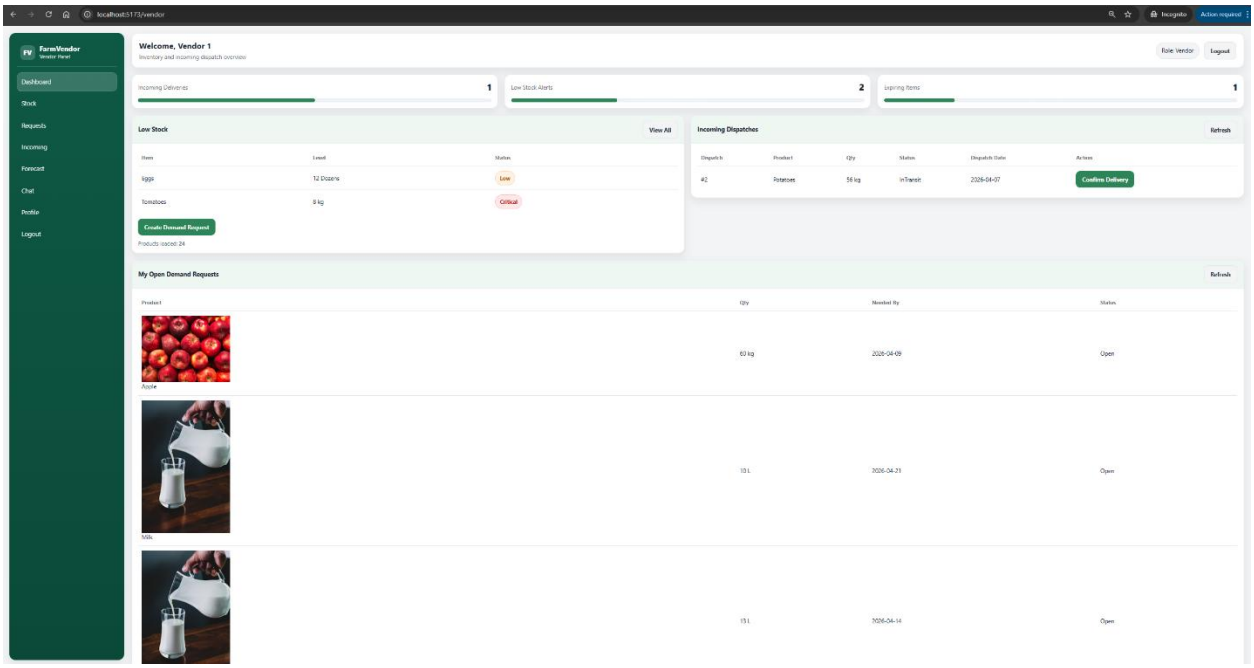
Demand Requests

- Create demand requests
- Track request status



Incoming Dispatches

- View deliveries from farmers
- Confirm delivery



Forecast Analytics

- View demand trends
- Compare forecasting models

Chat

- Communicate with farmers in real time

    - Vendor chat panel